

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

AWS Cloud Foundations

(CLF-C02)

Mohamed Sami

بسم الله الرحمن الرحيم

الحمد لله الذي علّم بالقلم، علّم الإنسان ما لم يعلم، وجعل طلب العلم من أشرف العبادات وأعظم القربات، والصلاة والسلام على سيدنا محمد ﷺ، وعلى آله وصحبه ومن سار على دربه إلى يوم الدين.

أما بعد، فإن هذا العمل لم يكن في يوم من الأيام مشروعًا ربحيًا، ولا محاولة للظهور أو التميز، وإنما هو جهد بسيط أرجو به وجه الله تعالى، ومحاولة لنشر علم تعلمته خلال رحلتي في مجال الشبكات وأمن المعلومات، لعل الله أن يجعل فيه نفعًا لكل من يقرأه أو يتعلم منه.

هذا الكتاب هو خلاصة تجربة تعليم وتعلم، وخلاصة ساعات طويلة من القراءة والتجربة والخطأ والمحاولة. ولذلك فهو ليس كتابًا معصومًا من الخطأ، بل عمل بشري قابل للنقص، والكمال لله وحده.

فإن وجدت فيه صوابًا فبفضل الله، وإن وجدت خطأ فأرجو تنبيهي إليه مشكورًا، فالدال على الخير كفاعله، وتصحيح العلم من أعظم أبواب الأجر.

أضع هذا العمل صدقة جارية عني، وأسأل الله أن ينفع به كل من قرأه أو علّمه أو نقله أو استفاد منه، وأن يجعله في ميزان حسناتي وحسنات والديّ ومن له حق عليّ، وأن يكتب أجره لكل من ساهم في نشره أو تيسيره للناس.

فلعل كلمة فيه تفتح باب فهم لطالب علم، أو تختصر طريقًا على مجتهد، أو تكون سببًا في رزق أو علم نافع لأحد المسلمين.

وما كان لله دام واتصل، وما كان لغير الله انقطع وانفصل.

أسأل الله أن يبارك في علمكم، وأن يجعلكم من أهل النفع، وأن يرزقنا وإياكم الإخلاص في القول والعمل، وأن يجعل هذا العمل خالصًا لوجهه الكريم لا نبتغي به إلا رضاه.

والله ولي التوفيق.

Table of Contents

♦ Module 1: Cloud Concepts Overview	5
1. Introduction to cloud computing.....	5
2. Advantages of cloud computing	17
3. Introduction to Amazon Web Services (AWS).....	23
4. Moving to the AWS Cloud – The AWS Cloud Adoption Framework (AWS CAF)	31
♦ Module 2: Cloud Economics and Billing	38
1. Fundamentals of pricing.....	38
2. Total Cost of Ownership.....	46
3. AWS Organizations.....	53
4. AWS Billing and Cost Management	61
5. Technical support	71
♦ Module 3: AWS Global Infrastructure Overview	78
1. AWS Global Infrastructure	78
2. AWS services and service category overview	81
♦ Module 4: AWS Cloud Security	86
1. AWS shared responsibility model.....	86
2. AWS Identity and Access Management (IAM)	89
3. Securing a new AWS account.....	95
4. Securing accounts.....	97
5. Securing data on AWS.....	99
6. Working to ensure compliance	101
♦ Module 5: Networking and Content Delivery	103
1. Networking Basics	103
2. Amazon VPC	106
3. VPC Networking.....	110
4. VPC Security.....	118
5. Amazon Route S3	123
8. Amazon CloudFront.....	131
♦ Module 6: Compute	138
1. Compute Services Overview	138
2. Amazon EC2	143
3. Amazon EC2 Cost Optimization	176

4. Container services	186
5. Introduction to AWS Lambda	200
6. Introduction to AWS Elastic Beanstalk	210
♦ Module 7: Storage	216
1. Amazon Elastic Block Store (Amazon EBS)	216
2. Amazon Simple Storage Service (Amazon S3)	229
3. Amazon Elastic File System (Amazon EFS)	248
4. Amazon S3 Glacier	257
♦ Module 8: Databases	271
1. Amazon Relational Database Service (Amazon RDS)	271
2. Amazon DynamoDB	304
3. Amazon Redshift	316
4. Amazon Aurora	327
♦ Module 9: Cloud Architecture	334
1. AWS Well-Architected Framework	334
2. Reliability and availability	347
3. AWS Trusted Advisor	354
♦ Module 10: Automatic Scaling and Monitoring	356
1. Elastic Load Balancing	356
2. Amazon CloudWatch	365
3. Amazon EC2 Auto Scaling	372

♦ **Module 1: Cloud Concepts Overview**

1. Introduction to cloud computing

❖ What is cloud computing?

قبل ما AWS تشرح Cloud Computing، بتسألك: What does cloud computing mean to you? يعني إيه مفهوم ال Cloud Computing بالنسبة لك؟، وده سؤال للتفكير فقط، مفيش إجابة واحدة صح.

ال Cloud Computing هو توفير موارد وخدمات ال IT عند الطلب (On-Demand) عبر الإنترنت، مع الدفع مُقابل الإستخدام فقط (Pay-as-you-go).

يعني بدل ما تشتري أجهزة وسيرفرات بنفسك، تقدر تستخدم موارد موجودة عند AWS وقت ما تحتاجها، وتدفع فقط على قد استخدامك.

ال Cloud Computing بتوفر لك العديد من الموارد، زي:

- **Compute Power**: قوة المعالجة (Servers / Virtual Machines)
- **Database**: قواعد البيانات
- **Storage**: التخزين
- **Applications**: التطبيقات
- **Other IT Resources**: أي موارد تقنية أخرى زي الشبكات والخدمات المُختلفة

كل الموارد دي مُتاحة من خلال الإنترنت.

يعني ببساطة بدل ما يكون عندك Data Center أو Servers في شركتك، تقدر تستخدم موارد موجودة عند مزود خدمة زي AWS، وتدفع فقط مُقابل اللي استخدمته.

بدل ما:

- تشتري Servers
- تركيبها
- تصيبتها
- تدفع كهرباء وتبريد

AWS بتعمل كل ده بدل منك، وأنت بتستخدم الموارد عن طريق الإنترنت وقت ما تحتاجها.

♦ المقصود بـ On-Demand Delivery

ال On-Demand يعني عند الطلب، يعني مش لازم تشتري Server أو Storage قبل ما تحتاجه.

أول ما تحتاج سيرفر تعمله في دقائق بسرعة، وتستخدمه.

ولما تخلص تقدر تقفله أو تمسحه في أي وقت.

يعني الموارد بتكون مُتاحة في أي وقت.

♦ Pay-as-you-go Pricing

دي من أهم مميزات ال Cloud.

معناها بتدفع على قد اللي استخدمته فقط.

مثلاً:

- لو شغلت السيرفر لمدة ساعتين، هتدفع تمن ساعتين.
 - ولو خزنت 100 GB، هتدفع تمن ال 100 GB فقط.
- يعني مفيش شراء نهائي ولا تدفع على حاجة مش بتستخدمها.

♦ الموارد دي موجودة فين؟

كل الموارد دي شغالة على Servers موجودة داخل Data Centers ضخمة.

ال Data Centers دي موجودة في أماكن مختلفة حول العالم.

لما تستخدم AWS:

- AWS هي اللي مالكة ال Servers
- AWS هي اللي بتصينها
- AWS هي اللي بتديرها

وأنت مجرد مُستخدم بتستخدمها عن طريق الإنترنت.

❖ Infrastructure as software

♦ يعني إيه Infrastructure as Software؟

قبل ال Cloud، كنا بنفكر في ال Infrastructure على إنها أجهزة (Hardware).

لكن مع ال Cloud، بقينا بنفكر فيها على إنها Software.

يعني بدل ما تشتري سيرفر حقيقي، تقدر تعمله بضغطة زر من AWS.

❖ Traditional Computing Model (Infrastructure as Hardware)

في النظام التقليدي (Traditional)، ال Infrastructure عبارة عن Hardware حقيقي.

يعني:

- Servers
- Storage
- Network Devices

كلها أجهزة موجودة عندك في الشركة أو ال Data Center.

◆ مشاكل ال Hardware Solutions

1. تحتاج Space

لازم يكون عندك مكان تحط فيه الأجهزة.
وده ممكن يكون Server Room، أو Data Center.
يعني لازم توفر مكان مُخصص للأجهزة.

2. هتحتاج Staff

لازم يكون عندك فريق مسؤول عن:

- تركيب الأجهزة
- تشغيلها
- صيانتها
- متابعة الأعطال

يعني محتاج موظفين مُتخصصين يديروا ال Infrastructure.

3. بتحتاج Physical Security

لإن الأجهزة موجودة عندك، فلازم تحميها.
زي مثلاً:

- كاميرات مراقبة
- أبواب مُؤمنة
- بصمة أو كارت دخول
- أنظمة إنذار

علشان محدش يوصل للأجهزة أو يسرقها.

4. بتحتاج Planning

قبل ما تشتري أي أجهزة، لازم تخطط كويس.
زي مثلاً:

- هتحتاج كام Server
- كام Storage
- كام مُستخدم هيسخدم النظام

لإن أي قرار غلط هيكلفك فلوس.

5. هتحتاج (CapEx) Capital Expenditure

وده معناه إنك تدفع مبلغ كبير من البداية.

يعني لازم تشتري:

- Servers
- Storage
- Networking
- Rack
- UPS
- وكل المعدات المطلوبة

قبل حتى ما تبدأ تستخدم النظام.

6. Hardware Procurement Cycle طويل

ال Procurement Cycle يعني رحلة شراء وتجهيز الأجهزة، ودي بتأخذ وقت طويل.

مثلاً لو احتجت Server جديد:

1. تطلبه من الشركة
2. تستنى يوصل
3. تركيبه في ال Rack
4. توصله بالكهرباء
5. توصله بالشبكة
6. تثبت عليه ال Operating System
7. تجهزه للإستخدام

كل الخطوات دي ممكن تأخذ أيام أو حتى أسابيع.

من أكبر مشاكل النظام التقليدي إنك لازم تتوقع احتياجاتك قبل ما تشتري الأجهزة.

يعني لازم تخمن أقصى ضغط (Maximum Peak) ممكن يحصل على النظام.

مثلاً لو متوقع إن هيبقى عندك 1000 مُستخدم في نفس الوقت، لازم تشتري أجهزة تستحمل ال 1000 مُستخدم حتى لو العدد ده بيحصل مرة واحدة في السنة.

ولو اشتريت أجهزة أكبر من احتياجك الحقيقي، النتيجة إن جزء كبير من الأجهزة هيفضل شغال لكنه مش مُستخدم.

وده معناه إنك دفعت فلوس على Resources مش مُستفيد بيها.

ولو اشتريت أجهزة أقل من احتياجك، لما الضغط يزيد، الأجهزة مش هتستحمل.

وده ممكن يخلي ال Website يبقى بطيء، أو ال Application يقع.

ولو احتياجاتك هتحتاج:

- تشتري أجهزة جديدة
- تركيبها
- تدفع فلوس زيادة
- وتستنى لحد ما تشتغل

يعني أي تغيير بياخد وقت ومجهود وتكلفة.

❖ Cloud Computing Model (Infrastructure as Software)

في ال Cloud Computing بقينا نبص لل Infrastructure على إنها Software مش Hardware. يعني بدل ما تشتري سيرفر حقيقي، تقدر تنشئ سيرفر من AWS في دقائق، ولما تخلص منه تمسحه. يعني الموارد بقت بتتعمل وتتحدف بسهولة، وإيها Software. ال Cloud بيعتمد على Software Solutions بدل ال Hardware، وده بيديك مميزات كتير.

1. Flexible

يعني مرنة جداً، تقدر تختار الخدمة اللي تناسب احتياجاتك، ولو احتياجاتك اتغيرت، تقدر تغير الموارد بسهولة. يعني مش مُرتبط بأجهزة ثابتة.

2. Provision Resources On-Demand

تقدر تنشئ أي Resource وقت ما تحتاجه. مثلاً:

- تعمل EC2 Instance
- تعمل Database
- تعمل Storage

كل ده في دقائق، ولما تخلص منهم تقدر تمسحهم. يعني الموارد بتكون On-Demand.

3. Pay for What You Use

في ال Cloud أنت بتدفع على قد الإستخدام فقط. يعني:

- استخدمت السيرفر لمدة ساعتين، هتدفع ثمن الساعتين فقط
- استخدمت 100 GB Storage، هتدفع ثمن ال 100 GB فقط

مش محتاج تدفع مبلغ كبير في البداية.

4. Elastic Scaling

تقدر تزود أو تقلل الموارد بسهولة حسب الضغط.

ولو بتستخدم ال Auto Scaling:

- الضغط زاد، AWS تزود عدد السيرفرات تلقائياً
- الضغط قل، AWS تقلل عدد السيرفرات تلقائياً

وده اسمه Elastic Scaling.

5. Temporary and Disposable Resources

في ال Cloud، الموارد تعتبر مؤقتة.

يعني بتعمل سيرفر وتستخدمه وبعد ماتخلص شغلك عليه بتقدر تمسحه.

مش لازم يفضل موجود طول الوقت.

6. Low Upfront Cost

مش محتاج تشتري أجهزة أو تدفع تكلفة كبيرة في البداية.

تقدر تبدأ بتكلفة بسيطة جداً، وتدفع حسب الاستخدام.

ال Software Solutions تُعتبر أفضل من ال Hardware Solutions لأنها:

- أسرع
- أسهل
- أقل تكلفة

يعني بدل ما تستنى أيام أو أسابيع علشان تجهز سيرفر، تقدر تعمله في دقائق.

◆ Eliminate the Undifferentiated Heavy-Lifting Tasks

دي من أهم مميزات ال Cloud، المقصود بيها إن AWS بتشيل عنك الشغل الروتيني.

زي:

- شراء الأجهزة (Procurement)
- صيانة الأجهزة (Maintenance)
- التخطيط للسعة (Capacity Planning)

بدل ما تضيع وقتك في الحاجات دي، تقدر تركز على:

- تطوير ال Application
- تحسين الخدمة
- إضافة Features جديدة

Service Models & Deployment Strategies ♦

مع انتشار ال Cloud، ظهرت طرق مُختلفة لإستخدامه.

زي:

- Service Models
- Deployment Strategies

كل طريقة بتديك مستوى مُختلف من:

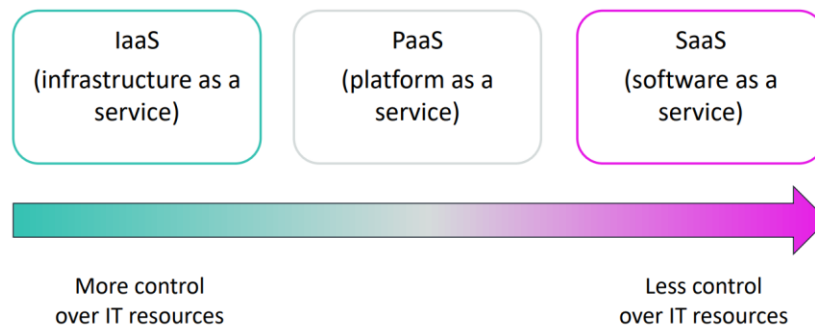
- Control
- Flexibility
- Management

وهنشرحهم بالتفصيل بعد كده.

Cloud service models ❖

لما تستخدم ال Cloud، مش لازم كل الناس تستخدمه بنفس الطريقة.

- في ناس بتحب تدير كل حاجة بنفسها.
 - وفي ناس عايزة تركز على تطوير ال Application فقط.
 - وفي ناس عايزة تستخدم برنامج جاهز بدون ما تدير أي حاجة.
- علشان كده AWS وال Cloud Providers وفروا ثلاثة Cloud Service Models.
- كل Model بيديك مستوى مختلف من التحكم (Control) ومسؤوليات مُختلفة.



1. Infrastructure as a Service (IaaS)

في ال IaaS، ال Cloud Provider بيوفرلك البنية التحتية الأساسية.

زي:

- Virtual Machines
- Storage
- Networking

وأنت اللي بتدير باقي الحاجات.

أنت مسؤول عن:

- تثبيت الـ Operating System
- تحديث الـ Operating System
- تثبيت البرامج
- إعداد الـ Application
- إدارة البيانات

الـ Cloud Provider مسؤول عن:

- Servers
- Storage
- Networking
- Data Center
- Hardware

الميزة إن الـ IaaS بيدك أعلى مستوى من التحكم (Highest Level of Control)، ولذلك هو أكثر Service Model يشبه الـ Traditional IT.

2. Platform as a Service (PaaS)

في الـ PaaS، الـ Cloud Provider يدير البنية التحتية والـ Operating System.

وأنت بتركز فقط على تشغيل وتطوير الـ Application.

أنت مسؤول عن

- كتابة الـ Code
- نشر الـ Application
- إدارة بيانات التطبيق

الـ Cloud Provider مسؤول عن

- Hardware
- Networking
- Storage
- Operating System
- Runtime Environment

الميزة إنك مش هتشغل بالك بإدارة الـ Server أو الـ Operating System، وتقدر تركز بالكامل على تطوير التطبيق.

3. Software as a Service (SaaS)

في ال SaaS، ال Cloud Provider بيوفرلك برنامج جاهز للإستخدام.

أنت كل اللي عليك تستخدم البرنامج.

أنت مسؤول عن استخدام البرنامج فقط

ال Cloud Provider مسؤول عن

- Hardware
- Networking
- Storage
- Operating System
- Updates
- Maintenance
- Security
- تشغيل البرنامج بالكامل

الميزة إنك مش محتاج تدير أي Infrastructure أو Servers، كل حاجة بتكون جاهزة.

♦ ترتيب مستوى التحكم

من أعلى Control إلى أقل Control:

- IaaS: أعلى تحكم
- PaaS: تحكم متوسط
- SaaS: أقل تحكم

كل ما تنزل من IaaS إلى SaaS مسؤولياتك تقل، والإدارة اللي بيعملها ال Cloud Provider تزيد.

❖ Cloud computing deployment models

بعد ما عرفنا إن في 3 Cloud Service Models (IaaS - PaaS - SaaS)، في سؤال ثاني مهم التطبيق بتاعي هيشغل فين؟

وده اللي بيجاوب عليه Deployment Model.

يعني ال Deployment Model بيحدد مكان تشغيل ال Application وال Resources.

وفيه 3 أنواع:

- Cloud
- Hybrid
- On-Premises

1. Cloud Deployment

في النوع ده كل حاجة بتكون موجودة على ال Cloud.

يعني:

- ال Servers
- ال Database
- ال Storage
- ال Networking

كلهم موجودين على AWS.

مش محتاج يكون عندك أي سيرفر داخل الشركة.

التطبيق ممكن يكون معمول من البداية على ال Cloud، أو كان شغال داخل الشركة، وبعد كده نقلته إلى AWS للإستفادة من مميزات ال Cloud.

الشركات بتستخدمه لأنها بتستفيد من مميزات ال Cloud مثل:

- Scalability
- High Availability
- Pay-as-you-go
- عدم شراء Hardware

2. Hybrid Deployment

كلمة Hybrid معناها هجين أو مزيج.

يعني الشركة مش بتنقل كل حاجة لل Cloud، ولا بتسيب كل حاجة داخل الشركة.

لكن بتقسم الشغل بين الإثنين، يعني جزء يفضل داخل الشركة وجزء يتنقل لل Cloud.

والإثنين يكون بينهم اتصال.

بنستخدمه لإن أوقات بيكون عندك Systems قديمة داخل الشركة وصعب تنقلها مرة واحدة.

فتبدأ تنقل الخدمات الجديدة لل Cloud وتسيب القديمة شغالة داخل الشركة.

وبكده تستفيد من مميزات ال Cloud بدون ما تغير كل ال Infrastructure مرة واحدة.

3. On-Premises Deployment

في النوع ده كل حاجة موجودة داخل الشركة.

الشركة هي اللي بتشتري:

- Servers
- Storage
- Networking

وهي اللي تديرهم بنفسها، ومغيش أي حاجة موجودة على AWS.

بعض الشركات بتستخدمه لإن الشركات بتحب يكون كل شيء تحت سيطرتها، أو عندها قوانين تمنع خروج البيانات خارج الشركة. لكن في المقابل هتتحمل تكلفة شراء الأجهزة وصيانتها وإدارتها.

ملاحظة:

أحياناً هتسمع مُصطلح Private Cloud.

وده بيكون نفس الـ On-Premises، لكن الشركة بتستخدم Virtualization وأدوات لإدارة الموارد بطريقة شبه الـ Cloud.

مثال للتوضيح:

شركة عندها Data Center خاص بيها، وكل الـ Servers والـ Databases موجودين داخل المبنى.

مفيش أي Service شغالة على AWS، يبقى ده On-Premises Deployment.

الفرق بينهم

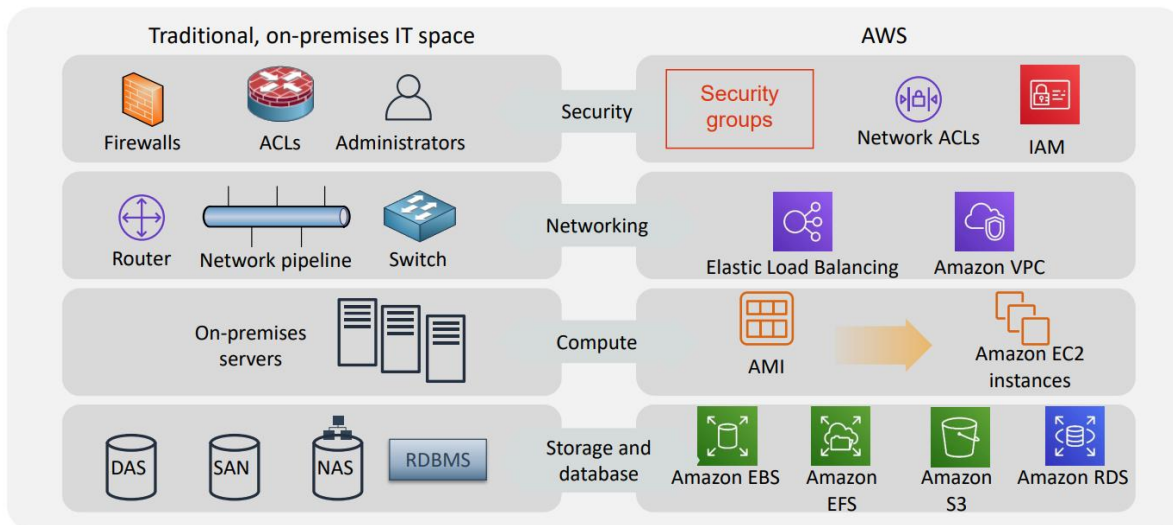
- **Cloud**: كل الموارد موجودة على AWS.
- **Hybrid**: جزء على AWS، وجزء داخل الشركة.
- **On-Premises**: كل الموارد داخل الشركة.

❖ Similarities between AWS and traditional IT

كثير من الناس أول ما تبدأ AWS بتفتكر إنها حاجة مُختلفة تماماً عن الـ IT التقليدي.

لكن الحقيقة، AWS مبني على نفس مفاهيم الـ IT التقليدي، بس بيقدمها كخدمات على الـ Cloud.

يعني لو أنت فاهم Networking و Servers و Storage، هتلاقي إن AWS بيقدم نفس الحاجات، لكن بشكل أسهل وأسرع.



Security ♦

في ال On-Premises كنا بنستخدم:

- Firewalls
- ACLs
- Administrator

علشان نحمي الشبكة ونتحكم في صلاحيات المُستخدمين.

في AWS هنستخدم:

- Security Groups
- Network ACLs (NACLs)
- IAM (Identity and Access Management)

يعني بدل ما تحط Firewall جهاز حقيقي، AWS بيديك Security Groups و NACLs يعملوا نفس الوظيفة.

وبدل ال Administrator اللي بيدى صلاحيات للمستخدمين، AWS بيستخدم IAM لإدارة المُستخدمين والصلاحيات.

Networking ♦

في ال Data Center كنا بنستخدم:

- Routers
- Switches
- Network Cables

علشان نوصل الأجهزة ببعض.

في AWS هنستخدم:

- Amazon VPC
- Elastic Load Balancer (ELB)

ال VPC يعتبر الشبكة الخاصة بتاعتك داخل AWS.

وال Load Balancer ببوزع الترافيك على أكثر من Server بدل ما كله يروح لسيرفر واحد.

Compute (Servers) ♦

في ال On-Premises كنت بتشتري سيرفر حقيقي وتحط عليه ال Operating System.

في AWS بدل السيرفر الحقيقي هتستخدم Amazon EC2 Instance.

ولما تحب تعمل نسخة جاهزة من السيرفر علشان تنشئ منها Servers جديدة بتستخدم Amazon Machine Image (AMI).

Storage ♦

في ال Data Center كان عندك أكثر من نوع Storage مثل:

- DAS (Direct Attached Storage)
- SAN (Storage Area Network)
- NAS (Network Attached Storage)
- Database Server

في AWS هتلاقي خدمات تؤدي نفس الوظائف:

- Amazon EBS Storage: مرتبط بسيرفر EC2
- Amazon EFS File Storage: تقدر أكثر من EC2 يستخدمه
- Amazon S3 Object Storage: لتخزين الملفات
- Amazon RDS Relational Database Service

يعني AWS وفر كل أنواع ال Storage الموجودة في ال Data Center لكن كخدمات جاهزة.

AWS مش ببيخترع مفاهيم جديدة هو بياخد نفس مكونات ال IT التقليدي ويحولها إلى Cloud Services. فلو أنت فاهم ال On-Premises هيكون تعلم AWS أسهل بكتير.

2. Advantages of cloud computing

في الجزء ده هنتكلم عن مميزات ال Cloud Computing وليه الشركات بقت تعتمد عليه بدل النظام التقليدي.

1. Trade capital expense for variable expense

الميزة الأولى هي إنك بدل ما تدفع مبلغ كبير جداً عشان تشتري سيرفرات وأجهزة وتبني ال Data Center، بتدفع فقط على قد استخدامك.

يعني لو احتجت Server لفترة معينة، تشغله وتدفع مقابل الفترة دي فقط، ولما تخلص تقفله، ومش هتدفع أي حاجة بعد كده.

وده هو نظام Pay-as-you-go.

زمان، لو شركة كانت عايزة تبدأ مشروع جديد، كانت لازم تشتري:

- Servers
- Storage
- Networking Devices
- وتجهز ال Data Center بالكامل

وده كله كان بيحتاج مبلغ كبير جداً من البداية، سواء استخدمت الأجهزة دي أو لا.

وده بنسميه Capital Expense (CapEx)، يعني مصاريف رأسمالية بتدفعها مقدماً لشراء وتجهيز البنية التحتية.

المُشكلة إنك بعد ما تشتري الأجهزة هتفضل تدفع تكلفة تشغيلها وصيانتها، سواء كانت شغالة أو واقفة.

أما في AWS أنت مش بتشتري أي أجهزة، ومش بتبني Data Center، أنت بتستخدم الموارد وقت ما تحتاجها فقط، وبتدفع مقابل الاستخدام الفعلي.

وده بنسميه Variable Expense، لأن التكلفة بتتغير حسب استخدامك.

كمان لو احتجت موارد أكثر لتشغيل Application جديد، تقدر تزودها في دقائق، بدل ما تستنى أيام أو أسابيع عشان تشتري أجهزة جديدة.

وكمان AWS هو اللي بيتولى صيانة وتشغيل البنية التحتية، فمش هتضيق وقت أو فلوس في إدارة الـ Data Center، وتقدر تركز على تطوير مشروعك.

ملحوظة مُهمّة للإمتحان

لما تشوف Trade Capital Expense for Variable Expense افتكر إن بدل ما تشتري Servers و Data Center وتدفع مبلغ كبير مُقدماً (CapEx)، AWS بيخليك تدفع فقط على قد استخدامك (Variable Expense / Pay-as-you-go).

2. Massive economies of scale



الميزة الثانية هي إن AWS بيستفيد من إنه بيخدم ملايين العملاء حول العالم، فبيشتري سيرفرات وهاردات وأجهزة شبكات وكهرباء وإنترنت بكميات ضخمة جداً.

ولأنه بيشتري بالكميات دي، الشركات المصنعة بتديله خصومات كبيرة جداً، فتكون تكلفة تشغيل AWS أقل من أي شركة هتبني Data Center خاص بيها.

وبالتالي AWS يقدر يقدم خدماته بأسعار أقل للعملاء، وأنت تستفيد من الأسعار دي بدون ما تحتاج تشتري أي أجهزة بنفسك.

ببساطة كل ما عدد عملاء AWS يزيد، كل ما تكلفة التشغيل تقل، وكل ما يقدر يقلل أسعار خدماته أكثر.

يعني بدل ما كل شركة تبني Data Center لوحدها وتدفع تكلفة كبيرة، AWS بيبني بنية تحتية واحدة ضخمة، وكل العملاء بيشتركوا فيها، وده بيخلي تكلفة كل عميل أقل.

وده هو المقصود بـ Benefit from Massive Economies of Scale.

ملحوظة مُهمّة للإمتحان

لما تشوف Massive Economies of Scale افتكر إن AWS بيخدم ملايين العملاء، فيشتري موارد بكميات ضخمة وبالتالي تكلفة التشغيل تقل، وساعتها يقدم الخدمة بأسعار أقل للعملاء.

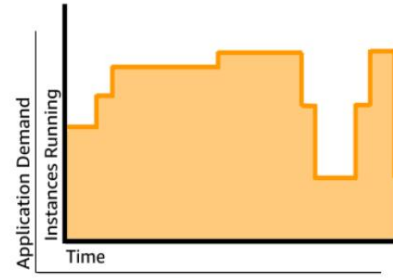
3. Stop guessing capacity



Overestimated server capacity



Underestimated server capacity



Scaling on demand

الميزة الثالثة هي إنك تبطل تخمن هتحتاج كام Server أو كام Resource قبل ما تشغل مشروعك.

زمان، قبل ما تعمل أي Application، كنت لازم تتوقع عدد المُستخدمين وتشتري الأجهزة على أساس التوقع ده.

ولو توقعت غلط، هيحصل واحد من حاجتين:

- لو اشتريت أجهزة أكثر من احتياجك، هتلاقي جزء كبير منها شغال بدون استخدام، وبالتالي تكون دفعت فلوس على Resources فاضية (Idle Resources).
- ولو اشتريت أجهزة أقل من احتياجك، أول ما عدد المُستخدمين يزيد، ال Servers مش هتستحمل، وهيحصل بطاء أو ممكن الخدمة تقف بسبب نقص ال Capacity.

في AWS، المُشكلة دي مش موجودة، أنت مش محتاج تخمن من البداية.

ابدأ بالموارد اللي محتاجها دلوقتي فقط.

- ولو عدد المُستخدمين زاد، تقدر تزود ال Resources في دقائق.
- ولو الإستخدام قل، تقدر تقلل ال Resources برضه في دقائق.

يعني هتستخدم على قد احتياجك الحقيقي، مش على قد توقعاتك.

وده هو المقصود بـ Stop Guessing Capacity.

ملحوظة مُهمّة للإمتحان

لما تشوف Stop Guessing Capacity افكر إن بدل ما تتوقع احتياجك من الموارد وتشتريها مقدّمًا، AWS يخليك تزود أو تقلل ال Resources وقت ما تحتاج، بدون تخمين.

4. Increase speed and agility



Weeks between wanting resources and having resources



Minutes between wanting resources and having resources

الميزة الرابعة هي إنك تقدر تجهز وتشغل أي Resource جديد في دقائق بدل ما تستنى أيام أو أسابيع. زمان، لو Developer احتاج Server جديد عشان يجرب Feature أو يشغل Application، كان لازم الشركة:

- تشترى Server
- تركيبه
- تثبت عليه نظام التشغيل
- تضبط الشبكة
- تجهزه للإستخدام

وكل ده ممكن ياخد أيام أو حتى أسابيع.

أما في AWS كل اللي عليك إنك تدخل على ال Console، وتختار نوع ال Server، وتضغط Launch.

وخلال دقائق هيكون ال Server جاهز للإستخدام.

وده بيخلي ال Developers يشتغلوا أسرع بكثير، ويجربوا أفكار جديدة بدون انتظار.

- ولو التجربة نجحت، يكملوا عليها.
- ولو منجحتش، يوقفوا ال Resource فوراً وميدفعوش غير تكلفة الفترة اللي استخدموه فيها.

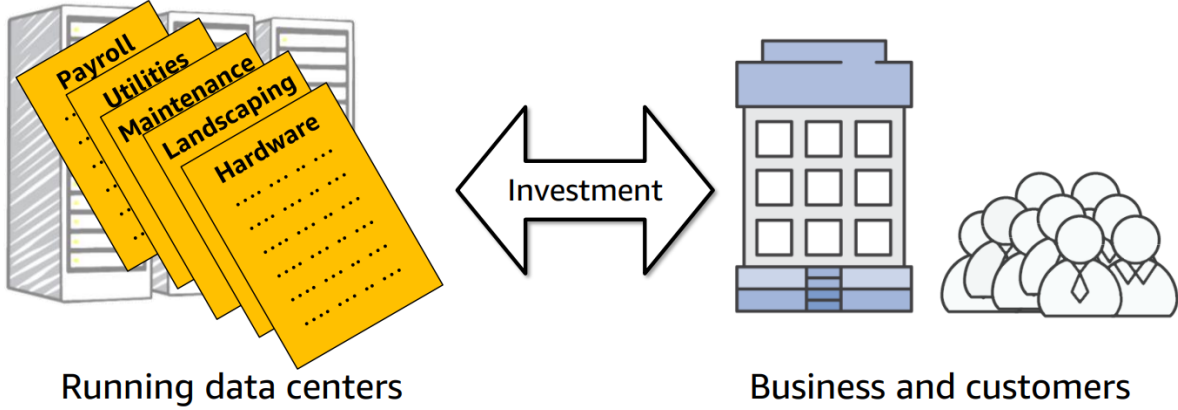
وده بيخلي الشركة تبقى أسرع وأكثر مرونة (Agile) في تطوير التطبيقات وإطلاق الخدمات الجديدة.

وده هو المقصود بـ Increase Speed and Agility.

ملحوظة مُهمّة للإمتحان

لما تشوف Increase Speed and Agility افكر، إن في AWS تقدر تنشئ أو تحذف أي Resource خلال دقائق، بدل ما تستنى أيام أو أسابيع، وده يسرع التطوير ويزود مرونة الشركة (Agility).

5. Stop spending money on running and maintaining data centers



الميزة الخامسة هي إنك تبطل تضيع وقت وفلوس في تشغيل وصيانة الـ Data Center، وتركز على تطوير مشروعك أو شغلك. زمان، لو عندك Data Center، كنت مسؤول عن حاجات كثير زي:

- شراء Servers
- تركيب الأجهزة (Rack & Stack)
- توصيل الكهرباء والإنترنت
- التبريد (Cooling)
- الصيانة
- تغيير الأجهزة لو حصل فيها عطل

كل الحاجات دي كانت بتستهلك وقت ومجهود وفلوس، مع إنها مش هي اللي بتميز مشروعك.

في AWS كل مسؤوليات تشغيل وصيانة الـ Data Center بتكون على AWS.

AWS هو اللي بيهتم بالأجهزة، والكهرباء، والتبريد، والصيانة، واستبدال أي جهاز يتعطل.

وأنت تقدر تركز على اللي يهمك فعلاً، زي:

- تطوير التطبيق
- إضافة Features جديد
- تحسين تجربة المستخدم.
- تنمية البيزنس

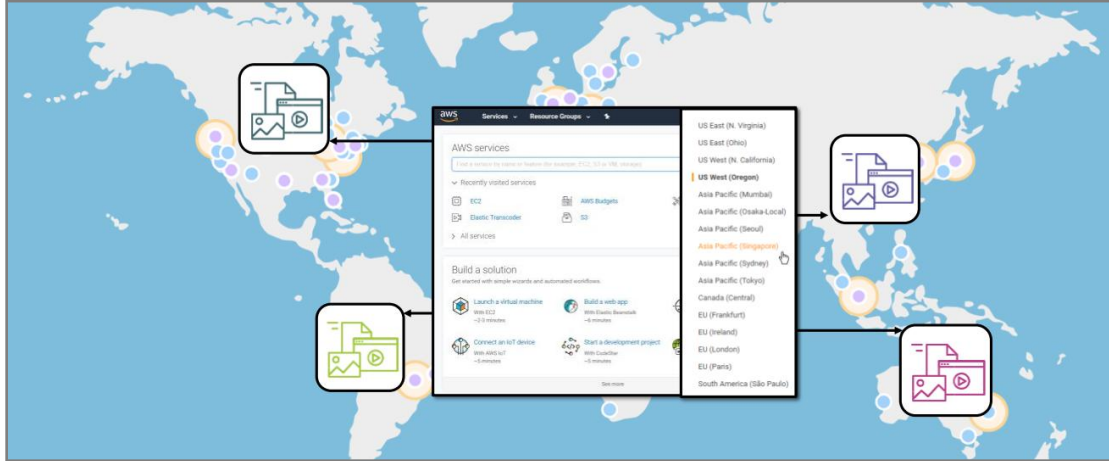
يعني بدل ما تقضي وقتك في إدارة الـ Infrastructure، هتقضي وقتك في تطوير مشروعك.

وده هو المقصود بـ Stop Spending Money on Running and Maintaining Data Centers.

ملحوظة مهمة للإمتحان

لما تشوف Stop Spending Money on Running and Maintaining Data Centers افكر، إن AWS بيتولى تشغيل وصيانة الـ Data Center، وأنت بتركز فقط على تطوير تطبيقك أو البيزنس بدل إدارة البنية التحتية.

Go global in minutes .6



الميزة السادسة هي إنك تقدر تنشر (Deploy) تطبيقك في أي مكان في العالم خلال دقائق.

زمان، لو شركة عايزة تخدم عملاء في دولة ثانية، كانت محتاجة تبني Data Center جديد أو تستأجر Servers في الدولة دي، وده كان بياخد وقت طويل وتكلفة كبيرة.

أما في AWS كل اللي عليك إنك تختار الـ AWS Region اللي عايز تشغل فيها التطبيق، وبكام ضغطة تقدر تنشره هناك.

وده معناه إنك تقدر توصل لعملاء في أي دولة بسرعة، بدون ما تبني أي بنية تحتية جديدة.

كمان لما يكون التطبيق قريب من المُستخدمين، البيانات هتوصل وتترجع في وقت أقل، وبالتالي يحصل Latency أقل ويكون أداء التطبيق أفضل.

يعني المُستخدم هيحس إن التطبيق أسرع وتجربته هتكون أحسن.

وده كله بيتم بسهولة وبتكلفة أقل مُقارنةً بالطريقة التقليدية.

وده هو المقصود بـ Go Global in Minutes.

ملحوظة مُهمة للإمتحان

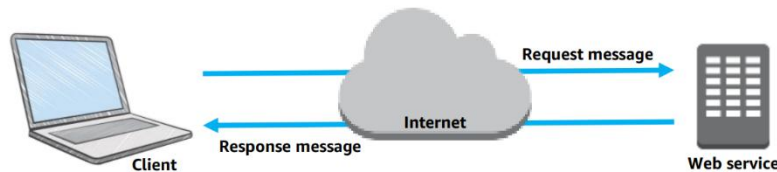
لما تشوف Go Global in Minutes افتكر إن AWS يخليك تنشر تطبيقك في أي AWS Region حول العالم خلال دقائق، وبالتالي توصل للمُستخدمين بسرعة وتقلل الـ Latency وتحسن تجربة المُستخدم.

3. Introduction to Amazon Web Services (AWS)

What are web services? ❖

ال Web Service هي أي خدمة أو برنامج سيكون متاح على الإنترنت، بحيث أي Application يقدر يتواصل معاه ويستخدم الوظائف أو البيانات اللي بيقدمها.

ببساطة بدل ما كل برنامج يشتغل لوحده، البرامج تقدر تتكلم مع بعض عن طريق Web Service.



أي Application لما يحتاج بيانات أو خدمة بيبعت Request لل Web Service عن طريق API. وال Web Service تستقبل ال Request، تنفذه، وبعد كده ترجع Response بالنتيجة.

علشان أي برنامج يقدر يفهم الثاني، لازم يكون فيه طريقة موحدة لتبادل البيانات.

لذلك ال Web Services بتستخدم Formats قياسية (Standardized Formats) أشهرها:

- JSON (JavaScript Object Notation)
- XML (Extensible Markup Language)

وده يخلي أي Application يقدر يقرأ البيانات بسهولة مهما كانت لغة البرمجة اللي مكتوب بيها.

♦ هل ال Web Service مرتبطة بلغة برمجة معينة؟

لا، ال Web Service مش مرتبطة بأي:

- Operating System
- Programming Language

يعني ممكن تكون مكتوبة بـ Java، و Application ثاني مكتوب بـ Python أو C#، وكلهم بقدروا يتواصلوا مع بعض من خلال ال API.

ال Web Service بتوفر ملف أو وصف (Interface Definition) يوضح:

- إيه ال APIs الموجودة
- كل API بيبستقبل إيه
- وكل API بيرجع إيه

وده يساعد أي Developer يفهم إزاي يستخدم ال Web Service بدون ما يحتاج يعرف تفاصيل تنفيذها.

ال Web Service ممكن تكون Discoverable، يعني أي Developer أو Application يقدر يعرف الخدمات وال APIs اللي بتوفرها ويبدأ يستخدمها بسهولة.

What is AWS? ❖

Amazon Web Services (AWS) هي منصة Cloud Computing من شركة Amazon، بتوفر عدد كبير جداً من خدمات الـ Cloud عن طريق الإنترنت.

بدل ما تشتري Servers أو تبني Data Center خاص بـك، تقدر تستخدم موارد AWS وقت ما تحتاجها، وتدفع فقط مقابل استخدامها.

AWS بتوفر تقريباً كل الموارد اللي ممكن تحتاجها، زي:

- Compute (Servers)
- Storage
- Networking
- Databases
- Security
- وغيرهم من مئات الخدمات

كل الخدمات دي بتكون متاحة أونلاين، وتقدر تستخدمها في أي وقت.

On-Demand Resources ♦

من أهم مميزات AWS إن الموارد بتكون On-Demand.

يعني أول ما تحتاج Server أو Database أو Storage، تقدر تنشئه في دقائق، وهيكون جاهز للإستخدام فوراً. مش هتستني أيام أو أسابيع عشان تشتري أجهزة أو تجهزها.

Flexible •

AWS مرنة جداً (Flexible).

يعني تقدر:

- تزود الموارد (Scale Up) لما عدد المستخدمين يزد
- تقلل الموارد (Scale Down) لما الإستخدام يقل
- توقف الموارد مؤقتاً أو تحذفها نهائياً لو مش محتاجها

وده كله بيتم بسهولة وفي أي وقت.

Pay-as-you-go ♦

في AWS أنت مش بتشتري الأجهزة.

أنت بتستخدم الموارد وتدفع فقط على قد استخدامها.

وده بيحول تكلفة الـ IT من Capital Expense (CapEx) (شراء أجهزة وبنية تحتية)، إلى Operational Expense (OpEx) (تدفع مقابل الإستخدام الفعلي فقط).

Building Blocks ♦

AWS بتوفر خدمات كثير مُصممة إنها تشتغل مع بعض.

فكر فيها كأنها مكعبات LEGO، كل Service تعتبر قطعة.

وأنت تختار القطع اللي محتاجها، وتركبها مع بعض علشان تبني أي Application أو Solution.

ولو احتياجك اتغيرت بعد كده، تقدر تضيف Services جديدة أو تشيل Services بسهولة.

Categories of AWS services ❖

AWS فيها عدد كبير جداً من الخدمات.

علشان يكون سهل تلاقي الخدمة اللي محتاجها، AWS قسمت الخدمات دي إلى Categories (فئات).

كل Category بتجمع الخدمات اللي ليها نفس الوظيفة أو بتخدم نفس الغرض.



ولإن عدد خدمات AWS كبير جداً.

فبدل ما كل الخدمات تكون في قائمة واحدة، AWS بتقسمها لمجموعات علشان يسهل عليك:

- تعرف وظيفة كل خدمة
- تلاقي الخدمة المناسبة بسرعة
- تبني ال Solution بطريقة منظمة

كل Category بتحتوي على خدمة واحدة أو أكثر.

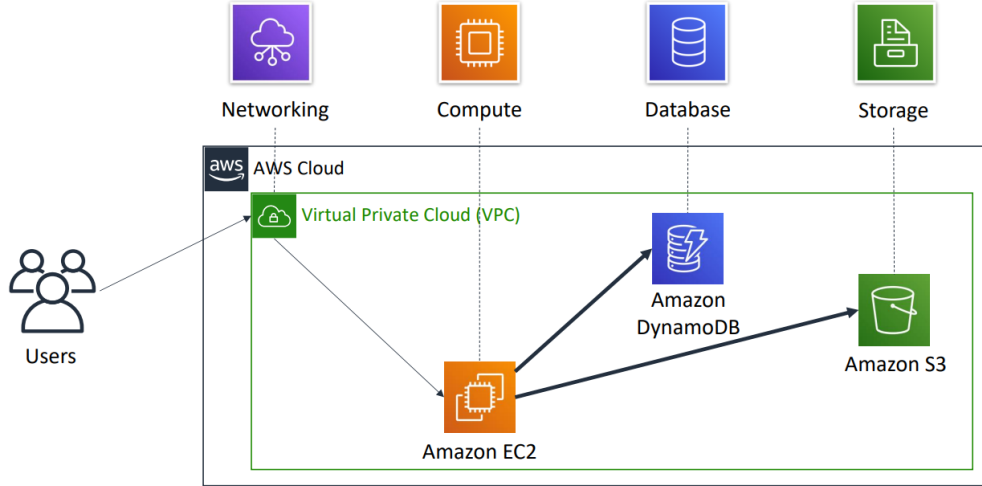
يعني مثلاً:

- Compute خاصة بال Category
- Storage خاصة بال Category
- Networking خاصة بال Category
- Database خاصة بال Category
- Security خاصة بال Category

وكل Category بيكون فيها مجموعة من الخدمات الخاصة بالمجال ده.

❖ Simple solution example

علشان تفهم فكرة AWS Services Categories أكثر، AWS جايه مثال بسيط لتطبيق بيجمع بيانات العملاء. الفكرة إنك بتستخدم أكثر من خدمة من أكثر من Category، وكل خدمة ليها دور مُعين.



◆ إزاي ال Solution ده بيشغل؟

1. العميل يرسل البيانات

العملاء بيبعتوا البيانات أو الطلبات الخاصة بيهم إلى Amazon EC2. وده يُعتبر السيرفر اللي التطبيق شغال عليه.

2. ال EC2 بيعالج البيانات

ال EC2 يستقبل البيانات، وبديل ما يخزن كل Request لوحده، بيجمع البيانات لمدة دقيقة (Batch)، وبعد كده يجهزها للتخزين.

3. تخزين البيانات

بعد ما ال EC2 يخلص معالجة البيانات، يحفظها داخل Amazon S3. وده مكان تخزين الملفات أو ال Objects. يعني كل عميل بيكون ليه Object خاص بيه داخل S3.

4. البحث داخل البيانات

بعد كده لو التطبيق احتاج يبحث عن بيانات عميل مُعين، هiestخدم Amazon DynamoDB. وده Database سريع بيخزن معلومات تساعد التطبيق يوصل لبيانات العميل بسهولة. مثلاً لو عايز تعرف كل الملفات اللي العميل رفعها خلال آخر أسبوع، ال DynamoDB يساعدك تلاقيها بسرعة.

5. تشغيل كل الخدمات داخل شبكة خاصة

كل الخدمات السابقة مُمكن تشتغل داخل Amazon VPC
علشان تكون كلها موجودة داخل شبكة خاصة وآمنة.

♦ الفكرة اللي AWS عايز يوصلها

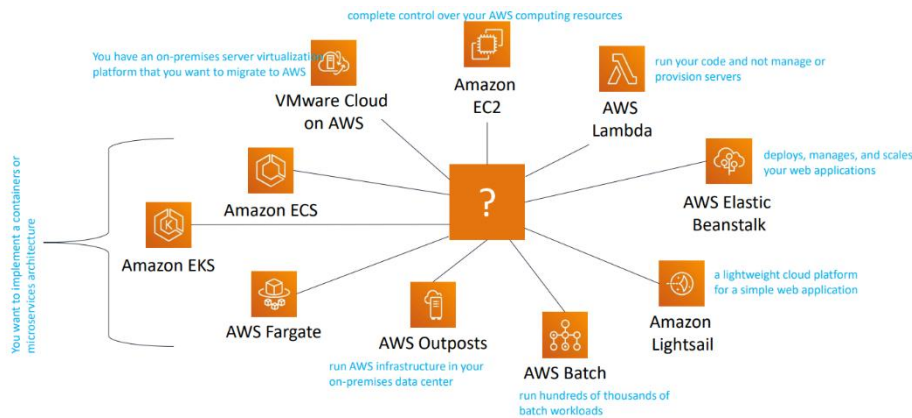
الهدف من المثال ده مش تحفظ أسماء الخدمات، الهدف إنك تفهم إن أي Solution على AWS بيتكون من مجموعة Services، وكل Service ليها وظيفة مُعينة.
وفي النهاية كل الخدمات دي بتشتغل مع بعض علشان تشتغل التطبيق.

❖ Choosing a service

في AWS غالباً هتلاقي أكثر من Service تقدر تعمل نفس المهمة، لكن مش كل Service مناسبة لكل مشروع.
علشان كده اختيار ال Service بيعتمد على:

- Business Goals: إيه الهدف من مشروعك؟
- Technology Requirements: إيه المتطلبات التقنية للمشروع؟

يعني أنت بتختار الخدمة اللي تناسب احتياجاتك، مش مُجرد أي خدمة.



♦ أمثلة على خدمات ال Compute

AWS فيها أكثر من خدمة خاصة بال Compute، وكل واحدة معمولية لإستخدام مُعين.

• Amazon EC2

استخدم EC2 لو أنت عايز:

- تتحكم بالكامل في ال Server
- تختار نظام التشغيل
- تثبت البرامج بنفسك
- تدير ال Server بنفسك

يعني EC2 مناسب لو عايز Full Control.

• AWS Lambd

استخدم Lambda لو عايز تشغل الكود فقط، ومش عايز تدير أو تنشئ Servers. كل اللي عليك ترفع الكود، وAWS يشغله عند الحاجة.

• AWS Elastic Beanstalk

استخدم Elastic Beanstalk لو عندك Web Application.

وعايز AWS هو اللي يعمل:

- Deploy
- Management
- Scaling

بدل ما تعمل كل ده بنفسك.

• Amazon Lightsail

استخدم Lightsail لو عندك Website أو Application بسيط، وعايز خدمة سهلة وسريعة.

• AWS Batch

استخدم AWS Batch لو، عندك عدد ضخيم جداً من ال Batch Jobs، وعايز تشغل مئات الآلاف منها بشكل تلقائي.

• AWS Outposts

استخدم AWS Outposts لو عايز خدمات AWS تشتغل داخل ال On-Premises Data Center بتاعتك.

يعني البنية التحتية موجودة عندك، لكن بتستخدم تكنولوجيا AWS.

• Amazon ECS / Amazon EKS / AWS Fargate

استخدم الخدمات دي لو شغال ب Containers أو Microservices.

كلها خدمات مُخصصة لتشغيل وإدارة ال Containers.

• VMware Cloud on AWS

استخدم VMware Cloud on AWS لو عندك بيئة VMware داخل شركتك، وعايز تنقلها إلى AWS بسهولة.

♦ الفكرة اللي AWS عايز يوصلها

AWS مش بيقدم Service واحدة لكل حاجة، هو بيوفر أكثر من Service لنفس المجال، وكل Service مناسبة لاستخدام معين. وأنت بتختار الخدمة اللي تحقق احتياجات مشروعك، ونفس الفكرة موجودة في باقي ال Categories، مش ال Compute بس.

Services covered in this course ❖

دي الخدمات اللي هندرسها في الكورس.

Compute services – - Amazon EC2 - AWS Lambda - AWS Elastic Beanstalk - Amazon EC2 Auto Scaling - Amazon ECS - Amazon EKS - Amazon ECR - AWS Fargate 	Storage services – - Amazon S3 - Amazon S3 Glacier - Amazon EFS - Amazon EBS 	Management and Governance services – - AWS Trusted Advisor - AWS CloudWatch - AWS CloudTrail - AWS Well-Architected Tool - AWS Auto Scaling - AWS Command Line Interface - AWS Config - AWS Management Console - AWS Organizations 
Security, Identity, and Compliance services – - AWS IAM - Amazon Cognito - AWS Shield - AWS Artifact - AWS KMS 	Database services – - Amazon RDS - Amazon DynamoDB - Amazon Redshift - Amazon Aurora 	AWS Cost Management services – - AWS Cost & Usage Report - AWS Budgets - AWS Cost Explorer 
	Networking and Content Delivery services – - Amazon VPC - Amazon Route 53 - Amazon CloudFront - Elastic Load Balancing 	

Three ways to interact with AWS ❖

علشان تستخدم خدمات AWS، لازم يكون فيه طريقة تتواصل بيها مع AWS.

AWS بيوفر 3 طرق تقدر من خلالها تنشئ وتدير ال Resources الخاصة ببيك.



AWS Management Console

Easy-to-use graphical interface



Command Line Interface (AWS CLI)

Access to services by discrete commands or scripts



Software Development Kits (SDKs)

Access services directly from your code (such as Java, Python, and others)

1. AWS Management Console

دي أسهل طريقة لإستخدام AWS.

وهي عبارة عن واجهة رسومية (GUI) بتفتحها من خلال المتصفح.

يعني بدل ما تكتب أوامر كل اللي عليك تضغط بالماوس، وتختار الخدمة، وتنشئ ال Resource بسهولة.

ودي أكثر طريقة بيستخدمها المبتدئين.

2. AWS Command Line Interface (AWS CLI)

لو مش عايز تستخدم ال Console ممكن تستخدم AWS CLI. ودي عبارة عن برنامج بتثبته على جهازك، وبعد كده تكتب أوامر (Commands) من خلال ال Terminal أو ال Command Prompt. الميزة هنا إنك تقدر تنفذ الأوامر بسرعة، وكمان تعمل Scripts تنفذ أكثر من عملية مرة واحدة.

3. Software Development Kits (SDKs)

لو أنت Developer وتعمل Application تقدر تستخدم AWS SDK. ال SDK عبارة عن مكتبات (Libraries) جاهزة للغات برمجة مختلفة زي:

- Python
- Java
- #C
- JavaScript
- وغيرها

بدل ما تدخل على ال Console أو تكتب أوامر، برنامجك نفسه يقدر يتواصل مع AWS ويعمل كل العمليات تلقائياً عن طريق الكود.

♦ هل الطرق دي مختلفة؟

من ناحية الاستخدام آه، لكن من ناحية التنفيذ، كلهم بيعملوا نفس الحاجة. سواء استخدمت:

- AWS Console
- AWS CLI
- AWS SDK

في النهاية كلهم بيعتوا Requests إلى AWS بإستخدام نفس ال API. يعني الاختلاف في طريقة الإستخدام فقط، لكن التواصل مع AWS واحد.

4. Moving to the AWS Cloud – The AWS Cloud Adoption Framework (AWS CAF)

AWS Cloud Adoption Framework (AWS CAF) ❖

كل شركة لديها طريقة مُختلفة في الإنتقال إلى الـ Cloud.

لكن علشان أي شركة تنجح في استخدام الـ Cloud، لازم يكون فيه توافق بين 3 عناصر أساسية:

- **People:** الأشخاص والموظفين.
- **Process:** العمليات وطريقة الشغل.
- **Technology:** التكنولوجيا المُستخدمة.

يعني مش كفاية إن الشركة تشتري خدمات AWS، لازم كمان الموظفين يكون عندهم المهارات المُناسبة، وطريقة العمل تكون مُناسبة للـ Cloud.

❖ ليه AWS عملت AWS CAF؟

كثير من الشركات وهي بتنقل أنظمتها إلى الـ Cloud بتواجه مشاكل زي:

- الموظفين معندهم الخبرة الكافية
- العمليات الحالية مش مُناسبة للـ Cloud
- مفيش خطة واضحة للإنتقال

علشان كده AWS قدمت (CAF) Cloud Adoption Framework.

❖ إيه هو AWS CAF؟

هو عبارة عن Framework (إطار عمل) فيه مجموعة من الإرشادات (Guidance) وأفضل الممارسات (Best Practices).

بيساعد الشركات إنها:

- تعرف هي واقفة فين دلوقتي (Current State)
- تحدد هي عايزة توصل لإيه (Target State)
- تحط خطة واضحة للإنتقال إلى الـ Cloud

❖ الـ AWS CAF بيساعد الشركات في إيه؟

بيساعدها على:

- اكتشاف أي نقص في مهارات الموظفين
 - اكتشاف المشاكل في العمليات الحالية
 - وضع خطة مُناسبة للإنتقال إلى الـ Cloud
- تسريع عملية الـ Cloud Adoption بطريقة صحيحة.

◆ AWS CAF يعتمد على إيه؟

ال AWS CAF يقسم عملية الانتقال إلى 6 Perspectives (وجهات نظر أو مجالات تركيز).

كل Perspective يتركز على جزء مُعين داخل الشركة.

وال Perspectives دي بتغطي العناصر الثلاثة الأساسية:

 BUSINESS	 PLATFORM
 PEOPLE	 SECURITY
 GOVERNANCE	 OPERATIONS

• People

• Process

• Technology

AWS CAF perspectives

كل Perspective بتحتوي على مجموعة من Capabilities.

وال Capability هي مسؤولية أو وظيفة مُعينة داخل الشركة.

AWS بيستخدم ال Capabilities دي علشان يعرف إيه الأجزاء اللي محتاجة تطوير، وإيه النقص الموجود داخل الشركة.

وبعدها الشركة تقدر تحط خطة لمعالجة النقص ده قبل الانتقال إلى ال Cloud.

❖ Six core perspectives

بعد ما عرفنا إن AWS CAF يقسم عملية الانتقال إلى ال Cloud إلى 6 وجهات نظر، AWS قسمهم إلى مجموعتين رئيسيتين.

 BUSINESS
 PEOPLE
 GOVERNANCE

Focus on **business** capabilities

 PLATFORM
 SECURITY
 OPERATIONS

Focus on **technical** capabilities

أول مجموعة: Business Perspectives

ودي بتركز على جانب البنزس والإدارة داخل الشركة.

وبتشمل:

• Business

• People

• Governance

المجموعة دي هدفها إنها تتأكد إن الشركة جاهزة من ناحية:

• أهداف البنزس

• الموظفين

• الإدارة والسياسات

يعني مش بتركز على التكنولوجيا نفسها، لكنها بتركز على إدارة الشركة وكيفية تبني ال Cloud.

ثاني مجموعة: Technical Perspectives

ودي بتركز على الجانب التقني.

وبتشمل:

- Platform
- Security
- Operations

المجموعة دي هدفها إنها تتأكد إن البنية التحتية والتكنولوجيا جاهزة للعمل على الـ Cloud.

يعني بتركز على:

- بناء وتشغيل البيئة على AWS
- تأمينها
- إدارتها وتشغيلها بشكل صحيح

♦ الفكرة اللي AWS عايز يوصلها

علشان تنجح في الـ Cloud مش كفاية تهتم بالتكنولوجيا بس.

لازم كمان تهتم بالبنس، والموظفين، والإدارة.

لذلك AWS قسم الـ 6 Perspectives إلى:

- Business Perspectives 3
- Technical Perspectives 3

علشان يغطي كل جوانب الشركة.

❖ Business perspective

الـ Business Perspective بتركز على البنس نفسه، يعني تتأكد إن استخدام الـ Cloud هيحقق أهداف الشركة، مش مجرد استخدام تكنولوجيا جديدة.

فيه أشخاص داخل الشركة مسؤولين عن الجزء ده، زي:

- Business Managers
- Finance Managers
- Budget Owners
- Strategy Stakeholders

دول بيستخدموا AWS CAF علشان يعرفوا:

- هل نقل الشركة إلى الـ Cloud هيكون مفيد فعلاً؟
- هل هيقلل التكاليف؟
- هل هيحقق أهداف البنس؟
- وياه أول المشاريع اللي المفروض تنتقل للـ Cloud؟

يعني قبل ما الشركة تبدأ تستخدم AWS، لازم يكون عندها Business Case، أو سبب واضح يوضح ليه هنتقل لل Cloud وإيه الفائدة اللي هترجع عليها.

كمان لازم يتأكدوا إن أهداف البرنس (Business Goals)، وأهداف قسم ال IT (IT Goals) ماشيين في نفس الإتجاه.

يعني مينفعش البرنس يكون هدفه تقليل التكاليف وزيادة سرعة إطلاق الخدمات، وفي نفس الوقت قسم ال IT يشتغل بطريقة تعطل تحقيق الأهداف دي.

لازم يكون فيه توافق بين الإثنين.

❖ People perspective

ال People Perspective بتركز على الأشخاص اللي هيشغلوا على ال Cloud.

لإن نجاح الانتقال إلى AWS مش بيعتمد على التكنولوجيا بس لازم كمان يكون الموظفين عندهم المهارات المناسبة ويعرفوا يشتغلوا على الخدمات الجديدة.

الأشخاص المسؤولين عن الجزء ده بيكونوا زي:

- Human Resources (HR)
- Staffing Managers
- People Managers

دول بيستخدموا AWS CAF علشان يعرفوا:

- هل الموظفين عندهم المهارات المطلوبة؟
- هل محتاجين تدريب على AWS؟
- هل فيه وظائف أو أدوار جديدة لازم تتعمل؟
- هل هيكل الشركة الحالي مناسب لل Cloud؟

يعني قبل الانتقال إلى AWS الشركة بتعمل Gap Analysis.

وده معناه إنها تقارن بين المهارات الموجودة حالياً، والمهارات المطلوبة للعمل على ال Cloud.

ولو اكتشفوا إن فيه نقص، يبدأوا يحلوا المشكلة عن طريق:

- تدريب الموظفين
- تعيين موظفين جدد
- أو تعديل هيكل الفريق

والهدف من كل ده إن الشركة تقدر تتأقلم بسرعة مع أي تغيير أو تكنولوجيا جديدة.

❖ Governance perspective

الـ Governance Perspective يتركز على إدارة الـ IT واتخاذ القرارات والسياسات، بحيث تتأكد إن استخدام AWS ماضي مع أهداف الشركة.

الأشخاص المسؤولين عن الجزء ده بيكونوا زي:

- Chief Information Officer (CIO)
- Program Managers
- Enterprise Architects
- Business Analysts
- Portfolio Managers

دول بيستخدموا AWS CAF علشان يتأكدوا إن:

- استراتيجية الـ IT متوافقة مع استراتيجية البنس.
- فيه سياسات واضحة لإدارة الـ Cloud.
- القرارات الخاصة بالـ IT بتخدم أهداف الشركة.

يعني مش أي فريق يستخدم AWS بالطريقة اللي تعجبه، لازم يكون فيه Governance، يعني قواعد وسياسات تنظم استخدام الـ Cloud.

كمان الـ Governance بيساعد الشركة إنها تستفيد بأكبر قيمة من استثماراتها في الـ IT، وتقلل المخاطر (Risks) اللي ممكن تحصل أثناء أو بعد الانتقال إلى الـ Cloud.

❖ Platform perspective

الـ Platform Perspective يتركز على البنية التحتية (Infrastructure) والتصميم الفني (Architecture) للأنظمة اللي هتشتغل على AWS.

الأشخاص المسؤولين عن الجزء ده بيكونوا زي:

- Chief Technology Officer (CTO)
- IT Managers
- Solutions Architects

دول بيستخدموا AWS CAF علشان يحددوا:

- إيه الخدمات اللي هيسخدموها على AWS
- إزاي هيبنوا الـ Architecture
- وإزاي هيتنقل الأنظمة الحالية إلى الـ Cloud

يعني دورهم الأساسي هو تصميم البيئة الجديدة على AWS.

كمان لازم يكون عندهم تصور واضح للشكل النهائي للبيئة بعد الانتقال، واللي بنسميه Target State Environment.

يعني يعرفوا بالتفصيل:

- هيسخدموا إيه من خدمات AWS
- الخدمات هتتصل ببعض إزاي
- والتطبيق هيشغل إزاي بعد ما ينتقل لل Cloud
- علشان كده AWS CAF بيوفر لهم:
- Best Practices لبناء تطبيقات جديدة على AWS
- Architectural Patterns تساعدكم يصمموا ال Architecture بطريقة صحيحة
- إرشادات لنقل ال On-Premises Workloads إلى ال Cloud

❖ Security perspective

ال Security Perspective بتركز على تأمين بيئة ال AWS وحماية البيانات والأنظمة.

الأشخاص المسؤولين عن الجزء ده بيكونوا زي:

- Chief Information Security Officer (CISO)
- IT Security Managers
- IT Security Analysts

دول بيسخدموا AWS CAF علشان يتأكدوا إن بيئة ال Cloud آمنة، وإنها تحقق مُتطلبات الأمان الخاصة بالشركة.

لازم يتأكدوا إن الشركة تقدر:

- تراقب كل اللي بيحصل داخل البيئة (Visibility)
- تراجع العمليات والأنشطة عند الحاجة (Auditability)
- تتحكم في صلاحيات المُستخدمين والخدمات (Control)
- تطبق سياسات الأمان بسرعة مع أي تغيير جديد (Agility)

كمان AWS CAF بيساعدكم في:

- اختيار أدوات الحماية المُناسبة
- تطبيق Security Controls المُناسبة لإحتياجات الشركة
- حماية البيانات والموارد الموجودة على AWS

يعني الهدف الأساسي هو إن الشركة تستخدم ال Cloud بشكل آمن بدون ما تعرض بياناتها أو أنظمتها للخطر.

❖ Operations perspective

ال Operations Perspective يتركز على تشغيل وإدارة بيئة الـ AWS بعد ما الشركة تنقل أنظمتها إلى الـ Cloud.

الأشخاص المسؤولين عن الجزء ده بيكونوا زي:

- IT Operations Managers

- IT Support Managers

دول بيستخدموا AWS CAF علشان يتأكدوا إن تشغيل الأنظمة اليومية ماشي بشكل مُنظم وسليم.

يعني بيحددوا:

- إزاي يتم تشغيل الخدمات على AWS
- إزاي يتم مُتابعة الأنظمة ومُراقبتها
- إزاي يتم التعامل مع الأعطال
- وياه الإجراءات اللي فريق التشغيل يمشي عليها كل يوم

كمان AWS CAF بيساعدهم في:

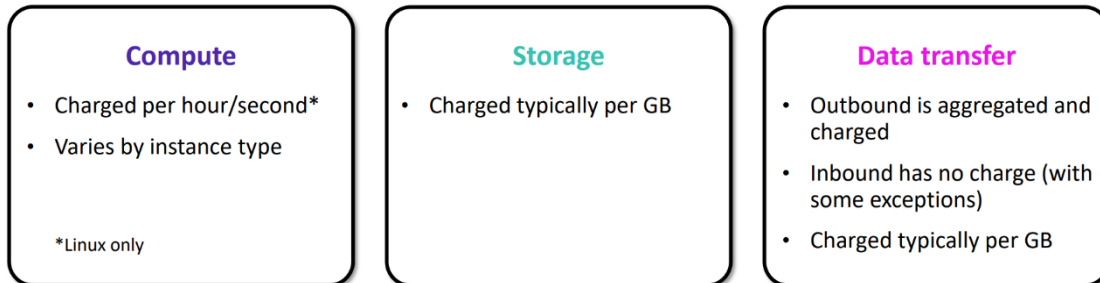
- معرفة إجراءات التشغيل الحالية (Current Operating Procedures)
- تحديد إيه اللي محتاج يتغير بعد الإنتقال إلى الـ Cloud
- معرفة التدريب اللي فريق التشغيل محتاجه علشان يشتغل على AWS بكفاءة

والهدف الأساسي إن تشغيل الأنظمة على AWS يكون مُستقر، ومُنظم، ويخدم احتياجات البنس بشكل مُستمر.

♦ **Module 2: Cloud Economics and Billing**

1. Fundamentals of pricing

AWS pricing model ❖



في AWS، تكلفة استخدام الخدمات تعتمد بشكل أساسي على 3 عوامل رئيسية:

1. الحوسبة (Compute):

يتم دفع الثابتة أو بالساعة طول ما السيرفر شغال، والسعر يتغير حسب قوة الجهاز (Instance Type).

2. التخزين (Storage):

يتم دفع إيجار للمساحة التي ملفاتك وأخذها، والحساب غالباً بالجيجابايت (GB).

3. نقل البيانات (Data Transfer):

المقصود بـ Data Transfer هو نقل البيانات من وإلى AWS.

• Inbound Data Transfer

هو نقل البيانات إلى AWS.

وفي أغلب الحالات مفيش رسوم على الـ Inbound Data Transfer.

لكن يوجد بعض الإستثناءات حسب الخدمة، لذلك لازم تراجع أسعار الخدمة قبل استخدامها.

• Outbound Data Transfer

هو نقل البيانات من AWS إلى خارجها.

يتم تجميع (Aggregate) كل الـ Outbound Data Transfer من مختلف الخدمات.

وبعد كده يتم حساب التكلفة عليه، التكلفة غالباً بتكون لكل GB.

رسوم الـ Outbound Data Transfer هتظهر في الفاتورة الشهرية باسم AWS Data Transfer Out.

❖ How do you pay for AWS?

فلسفة التسعير في AWS بسيطة، وهي إنك بتدفع مُقابل اللي استخدمته فقط. يعني في نهاية كل شهر، AWS بتحسب الموارد اللي استخدمتها، وبعدها تدفع قيمتها. كمان تقدر تبدأ أو توقف استخدام أي خدمة في أي وقت، بدون الحاجة لعقود طويلة المدة.

بتعتمد AWS على نموذج تسعير يشبه Utility-Style Pricing، يعني تدفع حسب استهلاكك الفعلي للموارد. وبيعتمد على أربع أفكار أساسية:

1. Pay for what you use

بتدفع فقط مُقابل الموارد اللي استخدمتها فعلاً، يعني لو استخدمت خدمة لفترة معينة، هتدفع تكلفة الفترة دي فقط.

2. Pay less when you reserve

لو عملت Reservation لبعض الموارد، هتدفع تكلفة أقل مقارنة بالدفع العادي.

3. Pay less when you use more

كلما زاد استخدامك لبعض الخدمات، مُمكن تحصل على أسعار أقل.

4. Pay even less as AWS grows

مع استمرار AWS في النمو وتحسين بنيتها التحتية، بتعمل تخفيضات على أسعار العديد من الخدمات، وبالتالي العملاء بيستفيدوا من انخفاض الأسعار.

❖ Pay for what you use

مع AWS، بتدفع فقط مُقابل الخدمات والموارد اللي استخدمتها، بدون الحاجة لدفع مبالغ كبيرة في البداية. الفكرة في الـ On-Premises، قبل ما تبدأ تشغيل أي Application، لازم تشتري:

- Servers
- Software Licenses
- أو تستأجر مكان (Data Center)

وده بيحتاج تكلفة كبيرة مُقدماً (Upfront Cost) حتى قبل ما تبدأ تستخدم الموارد. أما في AWS، مش محتاج تعمل كل ده، بتستخدم الخدمة وقت ما تحتاجها، وبتدفع فقط مُقابل استخدامها.

مُميزات الـ Pay for what you use:

- لا توجد مصاريف كبيرة في البداية (No Large Upfront Expenses).
- بتقلل التكاليف، لإنك مش محتاج تشتري Infrastructure بنفسك.
- تقدر تزود أو تقلل استخدام الخدمات حسب احتياجاتك.
- كل خدمات AWS مُتاحة On Demand.
- لا توجد عقود طويلة المدة (Long-Term Contracts).
- لا توجد تراخيص مُعقدة (Complex Licensing Dependencies).

❖ Pay less when you reserve

في بعض خدمات AWS، زي:

- Amazon EC2
- Amazon RDS

تقدر تحجز الموارد مُسبقاً باستخدام Reserved Instances (RIs).

وده بيديك خصم مُمكن يوصل إلى 75% مقارنةً باستخدام On-Demand.

فكرة الـ Reserved Instances إنك بدل ما تدفع بالسعر العادي كل مرة تستخدم فيها الـ Instance، تقدر تعمل Reservation لفترة، وفي المُقابل AWS بتديك خصم.

وكل ما دفعت مُقدماً أكثر، كلما كان الخصم أكبر.

♦ أنواع الـ Reserved Instances:

1. All Upfront Reserved Instance (AURI)

في النوع ده بتدفع المبلغ بالكامل مُقدماً، وبتحصل على أكبر خصم.

2. Partial Upfront Reserved Instance (PURI)

في النوع ده بتدفع جزء من المبلغ مُقدماً، وبتحصل على خصم أقل من AURI.

يعني بتوفر جزء من التكلفة، لكن مش بنفس نسبة الخصم الكبيرة.

3. No Upfront Reserved Instance (NURI)

في النوع ده مش بتدفع أي مبلغ مُقدماً، لكن بتحصل على أقل خصم بين الأنواع الثلاثة.

وده بيساعدك تحتفظ برأس المال لإستخدامه في مشاريع ثانية.

♦ فوائد الـ Reserved Instances:

ممکن توفر حتى 75% مُقارنةً بالـ On-Demand.

بتساعد في إدارة الميزانية بشكل أفضل.

بتقلل المخاطر لأن تكلفة الموارد بتكون معروفة مُسبقاً.

مُناسبة للشركات اللي عندها استخدام ثابت ولفترات طويلة.

❖ Pay less by using more

في AWS، كلما زاد استخدامك لبعض الخدمات، سعر الوحدة ييقل.
يعني كل ما تستخدم أكثر، هتدفع أقل لكل GB، وده اسمه Volume-Based Discounts.
الفكرة إن بعض خدمات AWS بتستخدم Tiered Pricing، وده معناه إن السعر بيتقسم إلى شرائح.
كل ما دخلت في شريحة استخدام أكبر، تكلفة الـ GB بتقل.

♦ خدمات بتستخدم Tiered Pricing

من أمثلة الخدمات اللي بتطبق النظام ده:

- Amazon S3
- Amazon EBS
- Amazon EFS

في الخدمات دي، كلما زادت كمية البيانات اللي بتخزنها، سعر الـ GB ييقل.

♦ Data Transfer In

نقل البيانات إلى AWS (Data Transfer In) يكون مجاني دائماً.

♦ اختيار خدمة التخزين المناسبة

AWS بتوفر أكثر من خدمة تخزين، وكل خدمة ليها تكلفة مختلفة حسب احتياجاتك.

وبالتالي تقدر تختار الخدمة المناسبة حسب:

- مدى تكرار الوصول للبيانات.
- مستوى الأداء المطلوب.

وده بيساعدك تقلل التكلفة مع الحفاظ على:

- الأداء (Performance)
- الأمان (Security)
- الإعتماذية (Durability)

❖ Pay even less as AWS grows

كلما AWS يتكبر وتتوسع، بتشتغل باستمرار على تقليل تكلفة تشغيل خدماتها. والنتيجة إن جزء من التوفير ده ينتقل للعملاء في صورة خفض أسعار الخدمات.

♦ إزاي ده بيحصل؟

AWS بتركز على:

- تقليل تكلفة تشغيل مراكز البيانات (Data Centers)
 - تحسين كفاءة التشغيل (Operational Efficiency)
 - تقليل استهلاك الكهرباء (Power Consumption)
 - تقليل تكلفة إدارة وتشغيل الخدمات بشكل عام
- كل التحسينات دي بتساعد AWS تقلل تكلفة التشغيل.

♦ الإستفادة للعميل

لإن AWS حجمها كبير جداً (Economies of Scale)، فهي تقدر تقدم الخدمات بتكلفة أقل. ولما تحقق توفير في التشغيل، بتنقل جزء من التوفير ده للعملاء عن طريق خفض الأسعار. من سنة 2006 لحد 2019، AWS قللت أسعارها 75 مرة!

♦ تحسين الموارد

مع نمو AWS، بتوفر Resources أحدث وأعلى في الأداء. وفي كثير من الحالات، الموارد الجديدة دي بتحل محل الموارد القديمة بدون أي تكلفة إضافية على العميل.

❖ Custom pricing

AWS مش مُجرد أسعار ثابتة للكل، هي مرنة جداً مع المشاريع الضخمة.

1. تلبية الإحتياجات المُختلفة (Meet varying needs):

لو شركتك ليها طبيعة شغل خاصة جداً ومحتاجة خطة أسعار مُخصصة مش موجودة في الخطط العادية، بتبعت لـ AWS ويتقعد معاك وتفضّلك نظام تسعير مخصوص ليك.

2. للمشاريع العملاقة (High-volume projects):

الميزة دي مُتاحة غالباً للمشاريع اللي بتستهلك موارد مهولة (زي شركات البث المُباشرة العالمية أو الهيئات الحكومية الكبيرة). لو استهلاكك كبير وليك مُتطلبات تقنية فريدة، AWS بتقدم لك عروض أسعار خاصة تناسب حجم استهلاكك ده، وغالباً بتكون أوفر بكثير من الأسعار المُعلنة للجمهور.

AWS Free Tier ❖

علشان تساعد العملاء الجدد يجربوا خدمات AWS، بتوفر AWS برنامج اسمه AWS Free Tier. البرنامج ده بيديك فرصة تستخدم بعض خدمات AWS مجاناً لمدة سنة من تاريخ إنشاء الحساب الجديد. الفكرة إن لو أنت عميل جديد في AWS، تقدر تستخدم بعض الخدمات مجاناً خلال أول سنة، وده بييساعدك تتعلم وتجرب الخدمات عملياً بدون تكلفة.

❖ أمثلة على الخدمات المشمولة

خلال فترة AWS Free Tier تقدر تستخدم مجاناً (ضمن الحدود المسموح بها) خدمات مثل:

- Amazon EC2 (مثل T2 Micro Instance)
- Amazon S3
- Amazon EBS
- Elastic Load Balancing (ELB)
- AWS Data Transfer
- بالإضافة إلى خدمات AWS أخرى

كل خدمة لها حدود استخدام مجانية، وإذا تجاوزتها بيتم احتساب تكلفة الاستخدام الزائد.

❖ الهدف من AWS Free Tier

اكتساب خبرة عملية (Hands-on Experience) في استخدام خدمات AWS. تجربة خدمات AWS بدون تكلفة خلال أول سنة للعملاء الجدد.

Services with no charge ❖

في AWS، في بعض الخدمات مفيش رسوم على استخدامها نفسها، لكن ده لا يعني إن كل حاجة مُرتبطة بيها مجانية. لإن ممكن تستخدم خدمة مجانية، لكنها تشغل خدمات تانية مدفوعة.



Amazon VPC



Elastic Beanstalk**



Auto Scaling**



AWS CloudFormation**



AWS Identity and Access Management (IAM)

♦ الخدمات التي لا يوجد عليها رسوم مباشرة

1. Amazon VPC

بتستخدمه لإنشاء شبكة افتراضية خاصة (Virtual Network) داخل AWS.
استخدام خدمة VPC نفسها مجاني.

2. AWS Identity and Access Management (IAM)

خدمة لإدارة:

- المُستخدمين (Users)
 - الصلاحيات (Permissions)
 - التحكم في الوصول إلى خدمات وموارد AWS
- استخدام IAM مجاني.

3. AWS Elastic Beanstalk

خدمة بتساعدك تنشر (Deploy) وتدير التطبيقات على AWS بسهولة.
الخدمة نفسها بدون رسوم إضافية.

4. AWS CloudFormation

بتساعدك تنشئ وتجهز مجموعة من موارد AWS بطريقة مُنظمة وآلية.
استخدام CloudFormation مجاني.

5. Auto Scaling

بتضيف أو تحذف الموارد تلقائياً حسب الشروط التي بتحددها.
خدمة Auto Scaling نفسها مجانية.

6. AWS OpsWorks

خدمة لإدارة وتشغيل التطبيقات بسهولة.
الخدمة نفسها بدون رسوم إضافية.

◆ نقطة مهمة جداً

رغم إن الخدمات السابقة لا يوجد عليها رسوم مباشرة، إلا إن الخدمات التي تستخدمها من خلالها ممكن تكون مدفوعة.

مثال

لو استخدمت Auto Scaling وأضاف Instance جديدة من نوع EC2 بسبب زيادة الضغط:

- خدمة Auto Scaling مجانية.
 - لكن EC2 Instance التي اتعملت عليها تكلفة.
- يعني هتدفع ثمن ال EC2 فقط، وليس خدمة Auto Scaling.

◆ Consolidated Billing

هي ميزة داخل AWS Organizations بتسمح لك تجمع أكثر من حساب AWS في فاتورة واحدة.

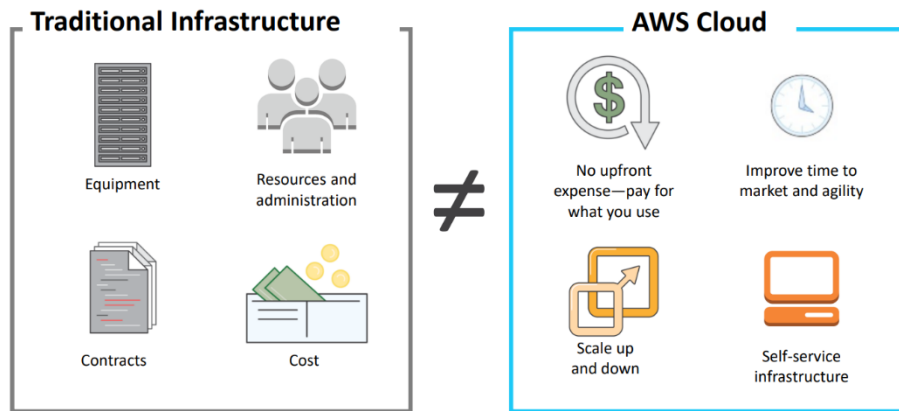
ومن فوائدها:

- فاتورة واحدة لكل الحسابات.
- متابعة تكلفة كل حساب بسهولة.

الإستفادة من Volume Pricing Discounts عند دمج استخدام جميع الحسابات.

2. Total Cost of Ownership

On-premises versus cloud ❖



عند مقارنة On-Premises مع Cloud، الفرق الأساسي بينهم سيكون في طريقة توفير وتشغيل البنية التحتية (Infrastructure).

أولاً: On-Premises

في ال On-Premises، الشركة هي التي تشتري وتدير كل حاجة بنفسها.

وده يشمل:

- الأجهزة (Hardware)
- المكان أو ال Data Center (Facilities)
- التراخيص (Licenses)
- فريق الصيانة والإدارة (Maintenance Staff)

كل دي تُعتبر تكاليف ثابتة (Capital Expenses - CapEx).

ولو احتجت تزود الموارد (Scale Up)، هيكون مكلف ويأخذ وقت.

ولو قللت استخدامك (Scale Down)، التكاليف الثابتة مش هتقل لأنك بالفعل اشتريت الأجهزة.

ثانياً: AWS Cloud

في ال AWS Cloud، AWS هي التي بتوفر وتدير:

- الأجهزة
- مراكز البيانات
- الصيانة

وأنت بتدفع فقط مقابل الموارد اللي استخدمتها، وتقدر تزود أو تقلل الموارد بسهولة.

التكلفة سهلة التقدير لأنها بتعتمد على الاستخدام الفعلي (Usage).

♦ الفرق بين النموذجين

On-Premises

- يحتاج تكلفة كبيرة في البداية (Upfront Cost).
- يعتمد على شراء الأجهزة وإدارتها.
- التوسع أصعب وأبطأ.
- تقليل الموارد مش يقلل التكاليف الثابتة.

AWS Cloud

- مفيض تكلفة كبيرة في البداية
- Pay as you use (ادفع مقابل ما تستخدمه فقط)
- التوسع أو تقليل الموارد سهل
- بيوفر مرونة (Flexibility) وسرعة (Agility)
- بيوفر Self-Service Infrastructure، يعني تقدر تنشئ الموارد بنفسك بدون الحاجة لشراء أجهزة أو انتظار تجهيزها.

❖ What is Total Cost of Ownership (TCO)?

ال Total Cost of Ownership (TCO) هو تقدير مالي (Financial Estimate) يُستخدم لحساب إجمالي تكلفة امتلاك وتشغيل نظام أو خدمة.

ال TCO مش يحسب فقط سعر الخدمة نفسها، لكنه يحسب أيضاً كل التكاليف المرتبطة بها سواء كانت مباشرة أو غير مباشرة. يُستخدم ال TCO في:

- مقارنة تكلفة تشغيل البنية التحتية (Infrastructure) أو Workload على On-Premises مع تشغيلها على AWS.
- المساعدة في إعداد الميزانية (Budgeting).
- بناء Business Case قبل اتخاذ قرار نقل الأنظمة إلى ال Cloud.

بدل ما تبص على سعر الخدمة فقط، ال TCO يحسب إجمالي التكلفة، وبالتالي سيساعدك تعرف أي الحلين أوفر:

- تشغيل النظام داخل الشركة (On-Premises)
- تشغيله على AWS

وده سيساعد الشركات تاخذ قرار مبني على التكلفة الحقيقية لكل اختيار.

TCO considerations ❖

عند حساب Total Cost of Ownership (TCO)، لازم تحسب كل التكاليف المرتبطة بتشغيل البنية التحتية، وليس سعر الأجهزة فقط.

1	Server Costs	Hardware: Server, rack chassis power distribution units (PDUs), top-of-rack (TOR) switches (and maintenance)	Software: Operating system (OS), virtualization licenses (and maintenance)	Facilities cost			
				Space	Power	Cooling	
2	Storage Costs	Hardware: Storage disks, storage area network (SAN) or Fibre Channel (FC) switches	Storage administration costs	Facilities cost			
				Space	Power	Cooling	
3	Network Costs	Network hardware: Local area network (LAN) switches, load balancer bandwidth costs	Network administration costs	Facilities cost			
				Space	Power	Cooling	
4	IT Labor Costs	Server administration costs					

القائمة الموجودة في المخطط مُختصرة (Abbreviated List)، والغرض منها هو توضيح أنواع التكاليف الأساسية المرتبطة بإدارة وتشغيل مركز البيانات (Data Center)، وليس عرض جميع التكاليف الممكنة.

التكاليف تنقسم إلى أربع فئات رئيسية:

1. Server Costs

تشمل تكاليف السيرفرات، مثل:

- Hardware
- Servers
- Rack Chassis
- Power Distribution Units (PDUs)
- Top-of-Rack (ToR) Switches
- وصيانة هذه الأجهزة

:Software

- نظام التشغيل (Operating System - OS)
- تراخيص الـ Virtualization
- وصيانتها

:Facilities Cost

- مساحة المكان (Space)
- الكهرباء (Power)
- التبريد (Cooling)

2. Storage Cost

تشمل تكاليف التخزين، مثل:

- أقراص التخزين (Storage Disks)
- Storage Area Network (SAN)
- Fibre Channel (FC) Switches
- تكلفة إدارة التخزين (Storage Administration)
- تكلفة المكان والكهرباء والتبريد

3. Network Costs

تشمل تكاليف الشبكة، مثل:

- أجهزة الشبكة (LAN Switches)
- Load Balancer
- Bandwidth

بالإضافة إلى:

- تكلفة إدارة الشبكة
- تكلفة المكان والكهرباء والتبريد

4. IT Labor Costs

وهي تكلفة فريق ال IT المسؤول عن:

- إدارة السيرفرات
- إدارة الشبكات
- إدارة التخزين
- تشغيل وصيانة البنية التحتية بالكامل

♦ عند المُقارنة بين On-Premises و AWS

عند مُقارنة الحلين، لازم تحسب التكلفة الحقيقية لكل واحد.

• في AWS

التكاليف بتكون أوضح وأسهل في الحساب، لأنها بتعتمد على الإستخدام الفعلي، مثل:

- RAM
- Storage
- Bandwidth

والأسعار بتكون مُعلنة، لذلك تقدر تحسب التكلفة بسهولة.

• في On-Premises

الحساب سيكون أصعب، لأنك لازم تأخذ في الاعتبار:
التكاليف المباشرة (Direct Costs) مثل:

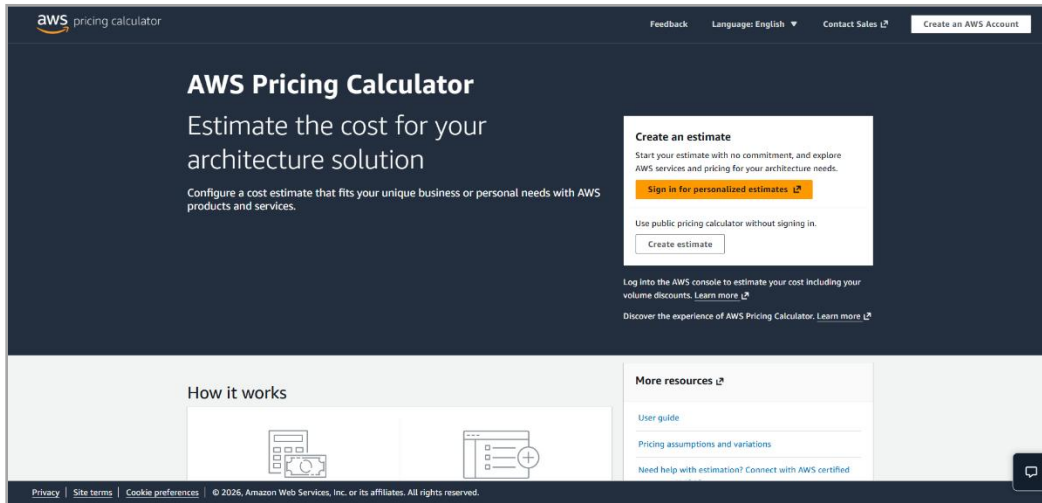
- الكهرباء
- مساحة الـ Data Center
- التخزين
- تشغيل وإدارة السيرفرات

التكاليف غير المباشرة (Indirect Costs) مثل:

- البنية التحتية للشبكات
- بنية التخزين
- أي تكاليف إضافية مرتبطة بتشغيل البيئة

❖ AWS Pricing Calculator

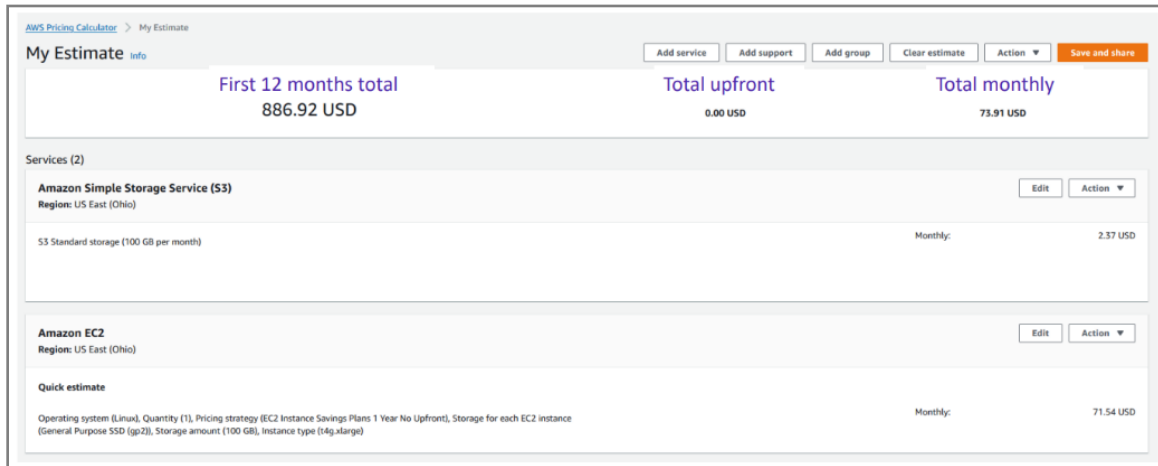
الـ AWS Pricing Calculator هي أداة من AWS بتساعدك على تقدير تكلفة استخدام خدمات AWS قبل تنفيذ المشروع.



♦ الأداة بتساعدك في إيه؟

- تحسب التكلفة الشهرية
- تقلل المصاريف
- تخطط للحل قبل التنفيذ
- تعرف تكلفة كل خدمة
- تختار الـ Instance المناسب
- تنظم الخدمات في مجموعات

Reading an estimate ❖



بعد ما تعمل Estimate في AWS Pricing Calculator، هتلاقي النتيجة متقسمة لـ 3 أجزاء رئيسية:

- First 12 months total
- Total upfront
- Total monthly

First 12 months total ❖

يعني إجمالي التكلفة خلال أول سنة.

وده بيكون مجموع:

- التكلفة اللي هتدفعها في البداية (Upfront)
- التكلفة الشهرية لمدة 12 شهر

Total upfront ❖

يعني المبلغ اللي هتدفعه في البداية عند إنشاء الـ AWS Resources.

Total monthly ❖

يعني المبلغ المُتوقع تدفعه كل شهر أثناء تشغيل الـ AWS Resources.

لو أنت عامل Groups، هتقدر تشوف تكلفة كل Service لوحدها، وكل Group تكلفته كام.

وده بيسهل تعرف كل جزء في الـ Solution بيكلف كام.

لو عندك أكثر من تصميم للـ Solution، تقدر تعمل Group لكل تصميم، وتقارن تكلفة كل Group.

وبعدها تختار التصميم الأنسب من ناحية التكلفة.

Additional benefit considerations ❖

لما تقارن بين On-Premises و AWS Cloud، مش هتنبص على التكلفة بس، لكن كمان على الفوائد اللي هتحققها. الفوائد دي نوعين:

- Hard Benefits
- Soft Benefits

1. Hard Benefits (فوائد مُباشرة وسهل تتقاس بالفلوس)

دي فوائد تقدر تشوفها وتحسبها بالأرقام لأنها بتقلل التكلفة بشكل مُباشر. بتشمل:

تقليل المصاريف على:

- Compute
- Storage
- Networking
- Security

تقليل تكلفة شراء:

- Hardware
- Software
- (Capital Expenses - CapEx)

تقليل تكاليف:

- التشغيل (Operational Costs)
- Backup
- Disaster Recovery

تقليل عدد موظفين التشغيل (Operations Personnel).

2. Soft Benefits (فوائد غير مُباشرة)

دي فوائد صعب تتحسب بالأرقام، لكنها ممكن تكون قيمتها أكبر من الـ Hard Benefits. بتشمل:

- إعادة استخدام الخدمات والتطبيقات بدل بناء كل حاجة من البداية
- زيادة إنتاجية المطورين (Developer Productivity)
- تحسين رضا العملاء (Customer Satisfaction)
- مرونة أكبر في البزنس للإستجابة بسرعة لأي فرص أو تغييرات جديدة
- زيادة الوصول للعملاء على مستوى العالم (Global Reach)

Cloud Total Cost of Ownership (TCO) ♦

ال TCO يحدد تكلفة تشغيل الحل بعد نقله إلى ال Cloud.

ويتم عن طريق مُقارنة:

- تكلفة البنية الحالية (On-Premises)
- تكلفة نفس البنية بعد نقلها إلى AWS

وده بيوضح فرق التكلفة بين الحلين.

لكن الفرق في التكلفة لوحده ممكن ميوضحش التأثير المالي الكامل للانتقال إلى ال Cloud.

Return on Investment (ROI) ♦

ال ROI يُستخدم علشان يعرف القيمة أو العائد اللي اتحقق بعد نقل النظام لل Cloud مقارنة بالمصاريف.

ويبدأ ب تحديد ال Hard Benefits لأنها فوائد مُباشرة وسهل قياسها.

بعد كده يتم تحديد ال Soft Benefits لأنها فوائد صعب قياسها، لكنها ممكن تكون أهم وأكبر في القيمة.

ولذلك لازم تفهم النوعين علشان تقدر تقيم القيمة الحقيقية للانتقال إلى ال Cloud.

3. AWS Organizations

Introduction to AWS Organizations ❖

هي خدمة مجانية (Free Account Management Service) بتسمحك تجمع أكثر من AWS Account داخل Organization واحدة وتديرهم كلهم من مكان واحد.



AWS
Organizations

بالتالي بدل ما تدير كل Account لوحده، تقدر تتحكم فيهم بشكل مركزي.

مميزات AWS Organizations ♦

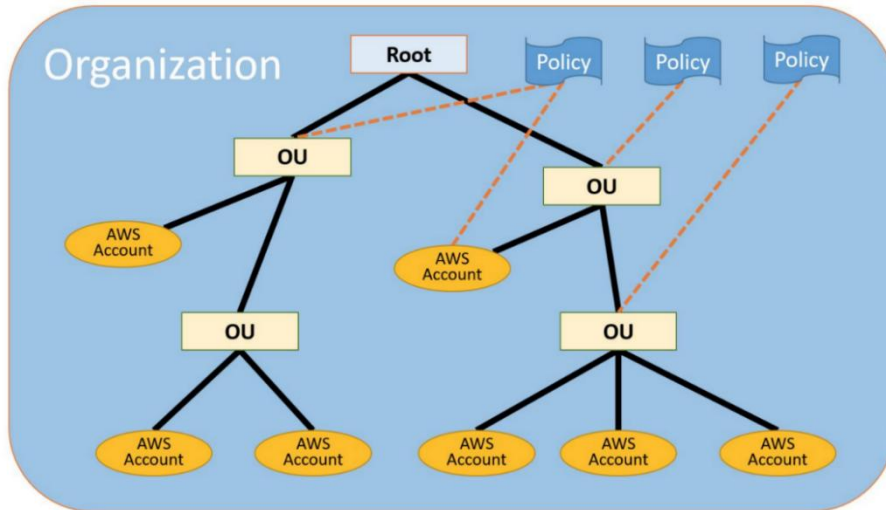
تقدر تطبق سياسات الوصول (Access Policies) على كل ال AWS Accounts من مكان واحد بدل ما تضيفها لكل Account بشكل منفصل.

تقدر تحدد مين يقدر يستخدم أي AWS Service، ومين ممنوع يستخدمها، يعني تقدر تمنع Accounts معينة من استخدام Services معينة.

تقدر تنشئ AWS Accounts جديدة وتديرها بشكل أوتوماتيك بدل ما تعمل كل حاجة يدوي.

كل ال AWS Accounts بيكون ليها فاتورة واحدة (Single Bill) بدل ما كل Account يبقى له فاتورة مُستقلة، وده ببسبيل مُتابعة المصاريف وإدارة الفواتير.

◆ مُصطلحات (AWS Terminology) AWS Organizations



• Root

ال Root هو أعلى مستوى في AWS Organizations.

كل ال AWS Accounts وال OUs سيكونوا موجودين تحته. تقدر تعتبره بداية الهيكل كله.

• Organizational Unit (OU)

ال OU هو عبارة عن Container (حاوية) يجمع مجموعة من AWS Accounts تحت بعض. وكمان ال OU ممكن يحتوي على OUs ثانية.

وده بيساعدك تعمل هيكل هرمي (Hierarchy) لتنظيم ال Accounts. الهيكل (Hierarchy) بيكون عامل زي شجرة مقلوبة:

- Root في الأعلى.
- تحته Organizational Units (OUs).
- وفي النهاية AWS Accounts.

• Policies

تقدر تعمل Attach Policy لأي Node في ال Hierarchy.

لما تعمل Attach Policy على:

- Root: هتتطبق على كل اللي تحته.
 - OU: هتتطبق على ال OU وكل ال OUs وال Accounts اللي تحتها.
 - Account: هتتطبق على ال Account ده فقط.
- يعني ال Policy بتنزل لتحت (Flows Down) في الهيكل.

◆ Child و Parent

كل OU ليها Parent واحد فقط.

وكل AWS Account بينتمي إلى OU واحدة فقط.

◆ AWS Account

ال AWS Account هو الحساب العادي اللي بيحتوي على ال AWS Resources.

ولو عملت Attach ل Policy على Account، هتأثر على الحساب ده فقط.

❖ Key features and benefits

AWS Organizations بيوفر مجموعة من المميزات المهمة:

1. Policy-based account management

تقدر تعمل Service Control Policies (SCPs) للتحكم في استخدام خدمات AWS عبر أكثر من AWS Account من مكان واحد.

2. Group-based account management

تقدر تجمع ال Accounts في (OUs) Groups، وبعد كده تعمل Attach ل Policies على ال Group، وبالتالي كل ال Accounts الموجودة فيه هتطبق عليها نفس ال Policies.

3. Application Programming Interfaces (APIs) that automate account management

تقدر تستخدم APIs علشان تعمل Automation لإنشاء وإدارة ال AWS Accounts الجديدة بدل ما تعمل كل حاجة يدوي.

4. Consolidated billing

تقدر تستخدم طريقة دفع واحدة (Single Payment Method) لكل ال AWS Accounts الموجودة في ال Organization.

ومع Consolidated Billing تقدر:

- تشوف فاتورة مُجمعة لكل ال Accounts
- تشوف كل المصاريف من مكان واحد
- تستفيد من Pricing Benefits الناتجة عن Aggregated Usage (استهلاك كل ال Accounts مع بعض)
- تدبر الفواتير لكل ال Accounts من مكان واحد
- تستفيد من Volume Discounts كلما زاد إجمالي الاستخدام

Security with AWS Organizations ❖

في AWS فيه طريقتين للتحكم في الصلاحيات:

- IAM Policies
- Service Control Policies (SCPs)

1. IAM Policies

بُستخدَم AWS Identity and Access Management (IAM) للتحكم في صلاحيات:

- Users
- Groups
- Roles

يعني تقدر تعمل Allow أو Deny للوصول إلى:

- AWS Services (زي Amazon S3)
- AWS Resources مُعينة (زي S3 Bucket مُعين)
- API Actions مُعينة (زي s3:CreateBucket)

ال IAM Policy ال بتتطبق فقط على:

- IAM Users
- IAM Groups
- IAM Roles

ولا يمكن إنها تمنع صلاحيات AWS Account Root User.

2. Service Control Policies (SCPs)

ال SCPs بُستخدَم مع AWS Organizations.

من خلالها تقدر تعمل Allow أو Deny لإستخدام خدمات AWS على:

- AWS Account مُعين.
- مجموعة Accounts داخل OU.

ولما تعمل Attach ل SCP هتأثر على كل:

- IAM Users
- IAM Groups
- IAM Roles

وكمان AWS Account Root User.

♦ الفرق بين IAM Policies و SCPs

IAM Policies

بالتطبيق على:

- Users
- Groups
- Roles

لا تؤثر على Root User.

SCPs

بالتطبيق على:

- AWS Account
- OU

وتؤثر على:

- Users
- Groups
- Roles
- Root User

❖ **Organizations setup**

علشان تعمل إعداد لـ AWS Organizations، بتمشي على 4 خطوات:

Step 1: Create Organization

أول خطوة هي إنشاء Organization.

الـ AWS Account الحالي هيبقى Primary (Management) Account.

بعد كده تعمل Invite لـ AWS Account موجود علشان ينضم لـ Organization.

وتقدر كمان تنشئ AWS Account جديد كـ Member Account.

Step 2: Create Organizational Units (OUs)

بعد إنشاء الـ Organization تعمل Organizational Units (OUs).

وبعدها توزع الـ Member Accounts داخل الـ OUs المناسبة.

Create Service Control Policies (SCPs) :Step 3

بعد تنظيم الـ Accounts تنشئ (SCPs) Service Control Policies.

الـ SCPs تُستخدم لتحديد القيود (Restrictions) على العمليات أو الخدمات التي يقدر المُستخدمين والـ Roles يستخدموها داخل الـ Member Accounts.

الـ SCP هي نوع من الـ Organization Control Policies.

Test Restrictions :Step 4

آخر خطوة هي اختبار الـ Policies.

تسجيل الدخول كمستخدم في كل OU وتشوف:

- هل الـ SCPs مُطبقة؟
 - وهل الوصول للخدمات بقي حسب القيود التي عملتها؟
- وممكن كمان تستخدم IAM Policy Simulator لإختبار وحل مشاكل:

- IAM Policies
- Resource-based Policies

الموجودة على:

- IAM Users
- IAM Groups
- IAM Roles

Limits of AWS Organizations ❖

في AWS Organizations فيه شوية Limits لازم تعرفها.

Limits		
Limits on Names	Names must be composed of Unicode characters.	
	Names must not exceed 250 characters in length.	
Maximum and Minimum Values	Number of AWS accounts	Varies. Note: An invitation sent to an account counts against this limit.
	Number of roots	1
	Number of OUs	1,000
	Number of policies	1,000
	Maximum size of a service control policy document	5,120 bytes
	Maximum nesting of OUs in a root	5 levels of OUs under a root
	Invitations sent per day	20
	Number of member accounts you can create concurrently	Only five can be in progress at one time
	Number of entities to which you can attach a policy	Unlimited

Limits on Names ♦

أي اسم تعمله سواء لـ:

- Account
- OU
- Root
- Policy

لازم يكون مُكوّن من Unicode Characters، وميزدش عن 250 Character.

Number of AWS Accounts ♦

Varies (بيختلف حسب الحالة).

مُهم: أي Invitation تبعتها لـ Account بتتسبب من ضمن الحد ده.

Number of Roots ♦

Root 1 فقط لكل Organization.

Number of OUs ♦

الحد الأقصى 1000 OU.

Number of Policies ♦

الحد الأقصى 1000 Policy.

Maximum size of a Service Control Policy (SCP) ♦

أقصى حجم لـ SCP Document هو 5120 Bytes.

Maximum Nesting of OUs ♦

تقدر تعمل 5 مستويات فقط من الـ OUs تحت الـ Root.

Invitations sent per day ♦

تقدر تبع 20 Invitation في اليوم.

Member Accounts created concurrently ♦

تقدر تنشئ 5 Member Accounts في نفس الوقت فقط.

◆ Number of entities to which you can attach a policy

مفیش حد أقصى لعدد ال Entities اللي تقدر تعملها Attach لل Policy.

❖ Accessing AWS Organizations

تقدر تدير AWS Organizations بأكثر من طريقة.

1. AWS Management Console

دي واجهة رسومية (Browser-based Interface).

من خلالها تقدر:

- تدير ال Organization
- تدير ال AWS Resources
- تعمل أي عملية خاصة بال AWS Organizations

2. AWS Command Line Interface (AWS CLI)

ال AWS CLI بيسمحك تنفذ أوامر من خلال Command Line.

تستخدمه علشان:

- تنفيذ أوامر خاصة بـ AWS Organizations
 - تنفيذ أوامر خاصة بخدمات AWS
- وده بيكون في بعض الأحيان أسرع وأسهل من استخدام ال Console.

3. Software Development Kits (SDKs)

ال AWS SDKs عبارة عن Libraries بتساعد المطورين في التعامل مع AWS.

بتقوم تلقائياً بـ:

- توقيع الطلبات (Signing Requests)
- إدارة الأخطاء (Managing Errors)
- إعادة إرسال الطلبات تلقائياً عند الحاجة (Retry Requests)

ومتوفرة للغات برمجة زي:

- Java
- Python
- Ruby
- NET.
- iOS
- Android

4. HTTPS Query API

الـ AWS Organizations HTTPS Query API بيديك وصول برمجي (Programmatic Access) إلى:

- AWS Organizations
- AWS Services

من خلال إرسال HTTPS Requests مباشرة.

وعند استخدامه لازم تعمل Digital Signing للطلبات باستخدام الـ Credentials الخاصة بيك.

4. AWS Billing and Cost Management

❖ Introducing AWS Billing and Cost Management

الـ AWS Billing and Cost Management هي الخدمة اللي بتستخدمها علشان:



- تدفع فاتورة AWS
- تتابع استخدامك (Usage)
- تدير وتحدد ميزانية التكاليف (Budget)

الخدمة كمان بتساعدك تعمل Forecast للتكاليف والإستخدام.

يعني تقدر تتوقع هتصرف كام في المستقبل، وهيكون استخدامك عامل إزاي.

وده بيساعدك تخطط لمصاريفك من بدري.

تقدر تختار الفترة الزمنية اللي عايز تشوف بياناتها.

- وكمان تقدر تعرض البيانات:
- Monthly (شهريًا)
- Daily (يوميًا)

تقدر تستخدم:

- Filtering لتصفية البيانات
- Grouping لتجميع البيانات

وده بيساعدك تحلل بيانات الاستخدام والتكاليف بطريقة أسهل.

الأداة دي بتساعدك:

- تفهم بيانات Cost و Usag
- تتابع اتجاهات (Trends) الاستخدام والتكاليف.
- تحدد فرص لتقليل وتحسين التكاليف (Optimization)
- تعرف إزاي بتستخدم خدمات AWS

AWS Billing Dashboard ❖



الـ AWS Billing Dashboard هي الصفحة الرئيسية التي تستخدمها علشان تتابع تكاليف واستخدام AWS. من خلالها تقدر تعرف بسرعة:

- صرفت كام لحد دلوقتي
- إيه أكثر الخدمات اللي بتكلفك
- هل مصروفاتك بتزيد ولا بتقل مع الوقت.

1. متابعة المصروفات الحالية (Month-to-Date Expenditure)

الـ Dashboard بتعرض المصاريف من أول الشهر لحد اللحظة الحالية. وده بييساعدك تتابع استهلاكك أول بأول بدل ما تستنى الفاتورة في آخر الشهر.

2. معرفة أكثر الخدمات تكلفة

الـ Dashboard بتوضحلك الخدمات اللي واخدة أكبر جزء من الفاتورة. يعني لو عندك مثلاً:

- Amazon EC2
- Amazon S3
- Amazon RDS

هتتعرف مين فيهم هو اللي بيصرف أكثر.

وده بييساعدك تحدد لو فيه خدمة محتاجة تقلل استهلاكها.

3. متابعة اتجاه التكاليف (Cost Trend)

ال Dashboard بتوضح إذا كانت التكاليف:

- بتزيد
- بتقل
- ولا ثابتة

وده بيديك فكرة عامة عن استهلاكك مع مرور الوقت.

◆ Spend Summary

ده جزء مهم جداً في ال Dashboard.

بيعرض 3 معلومات أساسية:

1. Last Month Spend

بيوضح صرفت كام الشهر اللي فات.

2. Month-to-Date Estimated Cost

التكلفة التقديرية من أول الشهر لحد دلوقتي.

يعني النظام بيحسب تقريباً أنت صرفت كام حتى الآن.

3. Forecast

وده توقع لمصاريفك بنهاية الشهر الحالي.

يعني AWS بتتوقع لو استمررت بنفس معدل الاستخدام، هتدفع كام آخر الشهر.

◆ Month-to-Date Spend by Service

الجراف ده بيقسم الفاتورة على الخدمات.

وبيوضح:

- أكثر خدمات AWS استخداماً
- تكلفة كل خدمة
- نسبة كل خدمة من إجمالي الفاتورة

Tools ❖

من خلال AWS Billing Dashboard تقدر تفتح مجموعة من الأدوات التي بتساعدك تتابع، تحلل، وتخطط لتكاليف AWS. أهم الأدوات هي:

- AWS Budgets
- AWS Cost Explorer
- AWS Cost and Usage Report



1. AWS Budgets

الأداة دي بتساعدك تحدد ميزانية (Budget) للتكاليف أو للإستخدام. يعني بدل ما تسبب المصاريف تزيد بدون مُتابعة، تقدر تحدد حد مُعين.



2. AWS Cost Explorer

الأداة دي بتساعدك تحلل التكاليف والإستخدام. من خلالها تقدر تعرف:

- صرفت كام
- إيه الخدمة اللي كلفتك أكثر
- استخدامك بيتغير إزاي مع الوقت

يعني هي أداة مُخصصة لتحليل بيانات التكلفة وعرضها بشكل واضح.



3. AWS Cost and Usage Report (CUR)

دي أكثر أداة فيها تفاصيل.

بتطلع تقرير كامل عن:

- كل التكاليف
- كل الإستخدام
- اتجاهات (Trends) الإستخدام
- إزاي بتستخدم خدمات AWS

ومن خلال التقرير ده تقدر تحدد أماكن تقدر تقلل فيها التكلفة (Cost Optimization Opportunities).

❖ الفرق بينهم

AWS Budgets: لتحديد ميزانية، وإرسال تنبيهات عند الإقتراب أو تجاوزها.

AWS Cost Explorer: لتحليل التكاليف والإستخدام، ومعرفة الخدمات الأكثر تكلفة.

AWS Cost and Usage Report: تقرير تفصيلي جداً عن جميع بيانات التكلفة والإستخدام، بتساعدك في تحليل الإستخدام واكتشاف فرص تقليل التكاليف.

Monthly bills ❖

ال Monthly Bills هي الصفحة التي بتعرض فاتورة AWS الشهرية بالتفصيل.
من خلالها تقدر تعرف بالظبط صرفت كام، وعلى إيه.
الصفحة بتعرض التكاليف اللي اتحسبت عليك خلال الشهر الحالي أو الشهر السابق لكل خدمة من خدمات AWS.
يعني بدل ما تشوف رقم إجمالي بس، هتتعرف تكلفة كل Service لوحدها.

Cost Breakdown by Service ♦

الفاتورة بتقسم التكاليف حسب كل خدمة.

مثال:

- Amazon EC2: تكلفة تشغيل ال Instances.
- Amazon S3: تكلفة التخزين.
- Amazon RDS: تكلفة قواعد البيانات.

وده بيساعدك تعرف كل خدمة كلفتك كام.

Breakdown by AWS Region ♦

مش بس بتعرف تكلفة كل خدمة، لكن كمان تعرف الخدمة دي كانت مُستخدمة في أي Region.

مثال:

EC2 في us-east-1 = 30\$

EC2 في eu-west-1 = 15\$

فبتقدر تعرف تكلفة كل Region بشكل مُنفصل.

Breakdown by Linked Account ♦

لو بتستخدم AWS Organizations وعندك أكثر من AWS Account مربوطين مع بعض، الفاتورة كمان بتقسم التكاليف حسب كل Linked Account.

يعني تعرف كل Account صرف كام من إجمالي الفاتورة.

Up-to-Date Cost Information ♦

الصفحة بتوفر أحدث معلومات عن:

- التكاليف (Costs).
- الاستخدام (Usage).

وده معناه إنك تقدر تتابع مصروفاتك بشكل مُستمر.

Detailed Breakdown ♦

بتعرض كمان تفاصيل الفاتورة، يعني:

- كل خدمة استخدمتها
- تكلفة كل خدمة
- تفاصيل الاستخدام الخاص بيها

BILLS | COST EXPLORER | BUDGETS | REPORTS

Total		\$7,453.41 USD
AWS Marketplace Charges		\$15.00
▼ Usage Charges and Recurring Fees		\$15.00
Invoice 32342548 – AWS Service Charges: Usage charge for this statement period	2017-10-10	\$15.00
AWS Service Charges		\$7,438.41
▼ Usage Charges and Recurring Fees		\$7,414.41
Invoice 32342513 – AWS Service Charges: Usage charge for this statement period	2017-10-10	\$7,414.41
▼ Usage Charges and Recurring Fees		\$24.00
Invoice 32342507 – AWS Service Charges: Subscription charge	2017-10-10	\$24.00

زي ما أنت شايف، الفاتورة متقسمة؛ مثلاً فيه جزء لـ AWS Marketplace (لو شاري برنامج من شركة ثانية عن طريق AWS) وجزء لخدمات AWS نفسها (Service Charges).

تقدر تفتح كل خدمة وتشوف تفاصيل استخدامك باليوم والساعة.

الصفحة دي هي كشف الحساب النهائي والمدقق لكل الفلوس اللي اتصرفت خلال الشهر.

Cost Explorer ❖

الـ AWS Cost Explorer هي أداة موجودة داخل AWS Billing and Cost Management.

وظيفتها إنها تعرض التكاليف والإستخدام في شكل Charts ورسوم بيانية، علشان يبقى سهل تتابع وتفهم مصروفاتك.

الأداة بتساعدك إنك:

- تشوف تكاليف AWS بشكل رسومي
- تفهم بتصرف فلوسك فين
- تتابع استخدامك مع مرور الوقت
- تدير التكاليف بشكل أفضل

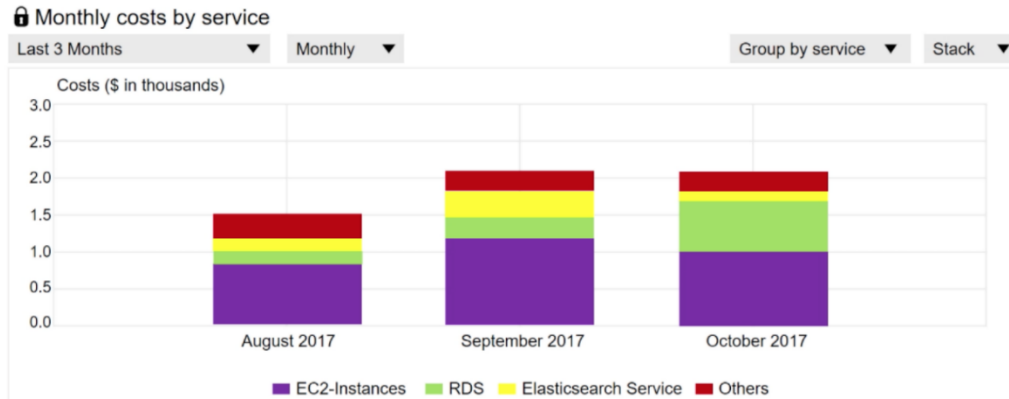
أول ما تفتح Cost Explorer هتلاقي تقرير افتراضي (Default Report).

التقرير ده بيعرض:

- الخدمات اللي صرفت عليها أكبر تكلفة.
- تكلفة كل خدمة مقارنةً بباقي الخدمات.

يعني بسرعة تعرف إيه أكثر Service مستهلكة من ميزانيتك.

BILLS | COST EXPLORER | BUDGETS | REPORTS



Monthly Running Costs Report ♦

ده تقرير بيعرض:

- إجمالي تكاليفك خلال آخر 3 شهور.
- Forecast أو توقع للتكاليف خلال الشهر القادم.
- كمان بيعرض Confidence Interval، يعني النطاق المُتوقع الي غالباً هتقع فيه التكلفة.

مميزات AWS Cost Explorer ♦

بيعرض التكاليف في شكل Charts و Graphs بدل الأرقام فقط، وده ببسهل تحليلها. تقدر تشوف بيانات التكاليف الخاصة بآخر 13 شهر، وده بيساعدك تقارن بين الشهور المختلفة. يقدر يتوقع هتصرف كام خلال الـ 3 شهور الجاية بُناءً على استخدامك الحالي. بيساعدك تكتشف نمط الإنفاق (Spending Patterns)، يعني تعرف:

- هل المصروفات بتزيد؟
- ولا بتقل؟
- وهل فيه خدمات بتستهلك تكلفة بشكل غير طبيعي؟

وده بيساعدك تحدد أماكن فيها مُشكلة في التكاليف.

بيوضحلك أكثر خدمات AWS استخداماً، وبالتالي تعرف مين السبب في أكبر جزء من الفاتورة.

تقدر تشوف Metrics إضافية، زي:

- أي Availability Zone عليها أكبر Traffic
- أي Linked AWS Account هو الأكثر استخداماً

الفرق بين Cost Explorer و Monthly Bills ♦

Monthly Bills: بتعرض الفاتورة بالتفصيل (كل Service، Region، Linked Account).

Cost Explorer: بيعرض نفس البيانات لكن في شكل Charts وتحليلات و Forecast علشان يسهل مُتابعة التكاليف.

Forecast and track costs ❖

الـ AWS Budgets هي أداة بتساعدك تتابع التكاليف والإستخدام، وكمات تتوقع (Forecast) هتصرف كام في المُستقبل. الأداة بتعتمد على البيانات اللي بيعرضها AWS Cost Explorer علشان تعمل التوقعات.

العلاقة بين AWS Budgets و Cost Explorer ♦

Cost Explorer بيجمع ويحلل بيانات التكاليف والإستخدام.

AWS Budgets بتستخدم البيانات دي علشان:

- تتابع الميزانية.
- تعرض التكاليف الحالية.
- تتوقع المصروفات المُستقبلية

يعني Cost Explorer بيحلل البيانات، و AWS Budgets تستخدم التحليل ده لمُراقبة الميزانية.

Track Your Budget ♦

تقدر تحدد Budget مُعينة، وبعد كده AWS Budgets تتابعها باستمرار.

مثال لو حددت ميزانية 100 دولار للشهر.

الأداة هتفضل تقارن:

- المصروفات الحالية
- الميزانية اللي أنت حددتها

علشان تعرف إذا كنت ماشي كويس أو قربت تتجاوزها.

BILLS | COST EXPLORER | **BUDGETS** | REPORTS

Create budget

Copy

Edit

Delete

Download CSV

Filter by budget name

	Budget name	Current	Forecasted	Budgeted	Current vs. budgeted	Forecasted vs. budgeted
☐	▶ Total Monthly Cost	\$760.27	\$787.44	\$1,000.00		
☐	▼ S3 Usage Bucket	2978.00 Req	3650.16 Req	3000.00 Req		

Budget details

Start date

10/01/17

End date -

Budget Period

Monthly

Variance analysis

الـ AWS Budgets مش بتعرض المصروفات الحالية بس.

هي كمان بتعمل Forecast أو توقع للتكاليف.

يعني لو استمررت بنفس مُعدل الإستخدام الحالي، هتتوقع هتصرف كام في نهاية الفترة.

Budget Notifications ♦

من أهم مميزات AWS Budgets إنها بتبعثلك تنبيهات (Alerts).

التنبيه يوصلك في حالتين:

الحالة الأولى

لو المصاريف الحالية عدت الميزانية.

مثال:

لو ال Budget = \$20

وانت صرفت \$105

هيوصلك Alert إنك تجاوزت الميزانية.

الحالة الثانية

حتى لو لسه متجاوزتش الميزانية، لكن النظام مُتوقع إنك هتتجاوزها قبل نهاية الشهر.

مثال:

لو ال Budget = \$100

وانت صرفت \$80

لكن ال Forecast بيقول إنك هتوصل لـ \$120

AWS هتبعثلك Alert بدري علشان تلحق تقلل المصروفات.

تقدر تختار الميزانية تكون:

- Monthly: شهرية
- Quarterly: كل 3 شهور
- Yearly: سنوية

وكمان تقدر تحدد بنفسك:

- تاريخ البداية
- تاريخ النهاية

التنبيهات ممكن توصلك عن طريق:

- Email
- Amazon SNS (Simple Notification Service)

Cost and usage reporting ❖

الـ AWS Cost and Usage Report (CUR) هو أكثر تقرير تفصيلاً في AWS بالنسبة للتكاليف والإستخدام.

يجمع كل معلومات التكاليف (Costs) والإستخدام (Usage) في مكان واحد.

هو تقرير شامل يعرض كل التفاصيل الخاصة بإستخدامك لخدمات AWS.

BILLS | COST EXPLORER | BUDGETS | **REPORTS**

Product Code	Usage Type	Operation	Availability Zone	Usage Amount	Currency Code	Line Item Description
Amazon S3	Requests – Tier 1	ListAllMyBuckets		2	USD	\$0.00 per request – PUT, COPY, POST, LIST under the global free tier
Amazon EC2	USW2-Boxusage:t2.micro	Runinstnaces:0002	us-west-2a	1	USD	\$0.00 per Windows t2.micro instance-hour under monthly free tier
Amazon S3	Requests – Tier 1	ListAllMyBuckets		2	USD	\$0.00 per request – PUT, COPY, POST, LIST under the global free tier
Amazon EC2	USW2-Boxusage:t2.micro	Runinstnaces:0002	us-west-2a	1	USD	\$0.00 per Windows t2.micro instance-hour under monthly free tier
Amazon S3	Requests – Tier 1	ListAllMyBuckets		2	USD	\$0.00 per request – PUT, COPY, POST, LIST under the global free tier
Amazon S3	Requests – Tier 1	ListAllMyBuckets		2	USD	\$0.00 per request – PUT, COPY, POST, LIST under the global free tier

التقرير يعرض:

- تكلفة كل خدمة
- كمية استخدام كل خدمة
- تفاصيل الإستهلاك لكل Account أو المُستخدمين الموجودين فيه

يعني تقدر تعرف بالضبط مين استخدم إيه، واستهلك قد إيه.

التقرير يعرض الإستهلاك لكل Service Category.

مثال:

- Amazon EC2
- Amazon S3
- Amazon RDS

لكل خدمة هتلاقي تفاصيل الإستهلاك الخاصة بيها.

تقدر تختار شكل التقرير يكون:

- Hourly: يعرض البيانات كل ساعة.
- Daily: يعرض البيانات يوم بيوم.

وده بيساعدك تختار مستوى التفاصيل اللي يناسبك.

لو فعلت إعدادات Tax Allocation، التقرير هيعرض كمان الضرائب (Taxes) الخاصة بإستخدامك. تقدر تخلي AWS تنشر التقارير تلقائياً داخل Amazon S3 Bucket، وبالتالي التقارير تبقى محفوظة عندك وتقدر ترجع لها في أي وقت.

التقرير بيتحدث مرة واحدة يومياً (Once a Day)، يعني كل يوم هيتم إضافة أحدث بيانات التكاليف والإستخدام.

♦ الفرق بين أدوات إدارة التكاليف

AWS Billing Dashboard: يعرض ملخص سريع للتكاليف الحالية

AWS Cost Explorer: يعرض Charts وتحليلات و Forecast للتكاليف.

AWS Budgets: لمتابعة الميزانية وإرسال Alerts عند الاقتراب أو تجاوزها.

AWS Cost and Usage Report (CUR): أكثر تقرير تفصيلاً، ويعرض جميع بيانات Usage و Costs ويمكن حفظه داخل ال Amazon S3.

5. Technical support

❖ AWS support

AWS Support هي خدمة من AWS هدفها إنها تساعد العملاء في استخدام خدمات AWS وحل أي مشاكل ممكن تواجههم. الخدمة بتوفر أدوات (Tools) وخبرات (Expertise) تناسب احتياجات كل عميل.

AWS Support بتقدم:

- أدوات تساعدك في إدارة واستخدام خدمات AWS.
 - خبراء من AWS يساعدوك في حل المشاكل والإجابة على الأسئلة.
- يعني بدل ما تعتمد على نفسك فقط، تقدر تستفيد من دعم AWS مباشرةً.

AWS بتوفر أكثر من Support Plan.

كل Plan بيقدم مستوى مختلف من الدعم حسب احتياج العميل.

يعني العميل يختار الخطة المناسبة له سواء استخدامه بسيط أو كبير.

♦ من يمكنه استخدام AWS Support؟

AWS Support مُصممة لكل أنواع العملاء، سواء كانوا:

1. Experimenting with AWS

لو أنت لسه بتتعلم AWS أو بتجرب الخدمات لأول مرة.

AWS توفر لك الدعم المناسب لمرحلة التعلم والتجربة.

2. Production Use of AWS

لو عندك Application شغال فعلياً على AWS وبيخدم المُستخدمين. هتحتاج دعم يساعدك تحافظ على تشغيل الخدمة وتحل أي مشاكل بسرعة.

3. Business-Critical Use of AWS

لو شركتك مُعتمدة بالكامل على AWS، وأي توقف في الخدمة هياثر على البنزنس. في الحالة دي AWS بتوفر مستوى أعلى من الدعم يناسب الأنظمة الحساسة.

♦ نوع الدعم يختلف حسب احتياج العميل

AWS مش بتقدم نفس الدعم لكل العملاء.

لكن مستوى الدعم بيتحدد حسب:

- احتياجات العميل
- طريقة استخدامه لـ AWS
- أهدافه من استخدام الخدمات

يعني كلما كان استخدامك أكبر أو أكثر أهمية، تقدر تختار Support Plan توفرلك دعم أقوى.

الـ AWS Support مش بس بيحل المشاكل، لكنه كمان بيساعدك تخطط، وتنشر (Deploy)، وتحسن (Optimize) بيئة AWS بتاعتك بثقة.

1. Proactive Guidance

الـ Proactive Guidance يعني AWS تساعدك قبل ما تحصل المُشكلة، مش بعد ما تحصل.

وده بيتم عن طريق (Technical Account Manager (TAM.

وده عبارة عن خبير من AWS بيكون نقطة التواصل الأساسية (Primary Point of Contact) معاك.

دور الـ TAM:

- يديك إرشادات (Guidance)
- يعمل مُراجعة لـ Architecture (Architectural Review)
- يتابع معاك بشكل مُستمر

يساعدك أثناء:

- Planning
- Deployment
- Optimization

يعني الـ TAM بيشتغل معاك علشان يمنع المشاكل قبل ما تحصل ويحسن تصميم بيئة AWS.

2. Best Practices

لو عاينز تتأكد إنك شغال طبقاً لـ AWS Best Practices، AWS Support بتوفر AWS Trusted Advisor. الـ AWS Trusted Advisor يعتبر زي خير AWS بيحلل بيئتك. بيفحص الـ AWS Environment ويقولك فين التحسينات اللي ممكن تعملها. بيساعدك في:

- تقليل التكاليف الشهرية
- تحسين الأداء (Performance)
- زيادة Fault Tolerance
- اتباع أفضل ممارسات AWS

يعني بدل ما تراجع كل حاجة بنفسك، Trusted Advisor يعمل Check ويطلعلك Recommendations.

3. Account Assistance

لو عندك مُشكلة خاصة بالحساب أو الفواتير، AWS بتوفر AWS Support Concierge. الـ Support Concierge هو مُتخصص في:

- Billing
- Account Issues

مسؤول عن:

- تحليل مشاكل الفواتير بسرعة
- المساعدة في مشاكل الحساب
- الرد على أي استفسارات خاصة بالحساب أو الدفع

ملاحظة، الـ Support Concierge لا يتعامل مع المشاكل التقنية، وإنما يتعامل فقط مع مشاكل Billing و Account.

❖ AWS Support Plans

AWS بتوفر 4 Support Plans، وكل Plan مناسبة لنوع معين من العملاء حسب احتياجاتهم. الخطط هي:

- Basic Support
- Developer Support
- Business Support
- Enterprise Support

1. Basic Support

دي الخطة الأساسية، ومُتاحة لكل عملاء AWS مجاناً.

بتوفر:

- Customer Service 24/7 (خدمة العملاء)
- Documentation
- Whitepapers
- Support Forums
- الوصول إلى 6 Core Trusted Advisor Checks
- Personal Health Dashboard لمُتابعة حالة خدمات AWS

مُناسبة لأي شخص لسه بدأ يستخدم AWS أو استخدامه بسيط ومش محتاج دعم فني مُباشر.

2. Developer Support

دي مُناسبة للمطورين أو الناس اللي لسه بتجرب AWS.

مُناسبة للـ

- اللي بيعمل Testing أو Early Development
- اللي لسه بيستكشف خدمات AWS
- اللي بيستخدم AWS في Non-Production Workloads

الـ Non-Production يعني البيئة مش شغالة مع العملاء الحقيقيين، زي بيئة التطوير أو الإختبار.

بتوفر:

- Guidance
- Technical Support
- مساعدة أثناء مرحلة التطوير

3. Business Support

دي مُناسبة للشركات اللي عندها Applications شغالة فعلياً على AWS.

مُناسبة للـ

- عنده Production Workloads
- بيشغل Application حقيقي بيستخدمه العملاء
- بيستخدم خدمات AWS بشكل كبير
- محتاج التطبيق يفضل:
 - Available
 - Scalable
 - Secure

Production Workload يعني التطبيق شغال فعلاً وبيخدم المُستخدمين الحقيقيين.

بتوفر:

- دعم فني أعلى من Developer
- مساعدة للحفاظ على تشغيل التطبيقات في بيئة الإنتاج

4. Enterprise Support

دي أعلى Support Plan في AWS.

مُخصصة للشركات الكبيرة التي أي توقف في الخدمة هيأثر على البنس.

مناسبة للـ

- Business-Critical Workloads
- Mission-Critical Workloads

يعني الأنظمة التي لازم تفضل شغالة باستمرار.

بتوفر:

Proactive Management: يعني مساعدة مستمرة لتحسين، الكفاءة (Efficiency)، والتوفر (Availability).

AWS Best Practices: مساعدة في تصميم وتشغيل الـ Workloads طبقاً لأفضل ممارسات AWS.

Launches and Migrations: مساعدة أثناء إطلاق المشاريع الجديدة، نقل الأنظمة إلى AWS.

Technical Account Manager (TAM): من أهم مميزات Enterprise Support.

الـ TAM:

- خبير تقني من AWS
- يفهم الـ Use Case بتاعك
- يفهم الـ Architecture الخاصة بـك
- يكون Primary Point of Contact لأي احتياج تقني
- يقدم Guidance ومتابعة مستمرة

❖ AWS Support Plans

مش كل المشاكل في AWS ليها نفس الأولوية.

علشان كده، AWS بتحدد Severity Level (درجة خطورة المشكلة)، وعلى أساسها بيتحدد سرعة استجابة فريق الدعم (Response Time).

كلما كانت المشكلة أخطر، كلما كان رد AWS أسرع.

♦ ما هو Case Severity؟

هو مستوى خطورة المشكلة التي بتبلغ عنها لـ AWS Support.

AWS قسمت المشاكل إلى 5 مستويات.

	Critical	Urgent	High	Normal	Low
Basic	No Case Support				
Developer Plan (Business hours)				12 hours or less	24 hours or less
Business Plan (24/7)		1 hour or less	4 hours or less	12 hours or less	24 hours or less
Enterprise Plan (24/7)	15 minutes or less	1 hour or less	4 hours or less	12 hours or less	24 hours or less

1. Critical

ده أعلى مستوى خطورة.

معناه:

- الوزن الكامل في خطر
 - الوظائف الأساسية (Critical Functions) في التطبيق متوقفة تماماً
- يعني التطبيق الأساسي مش شغال، والعملاء مش قادرين يستخدموه.

2. Urgent

درجة أقل من Critical.

معناه:

- الوزن متأثر بشكل كبير
 - وظائف مهمة في التطبيق غير متاحة
- يعني التطبيق شغال جزئياً، لكن جزء مهم منه واقع.

3. High

معناه:

- الوظائف المهمة مازالت شغالة، لكن أداؤها ضعيف أو فيه مشاكل.
- يعني الخدمة لم تتوقف، لكن الأداء متدهور.

4. Normal

معناه:

- وظائف غير أساسية (Non-Critical Functions) فيها مشكلة
 - أو عندك سؤال تقني متعلق بالتطوير ويحتاج رد سريع
- يعني المشكلة موجودة لكنها لا تؤثر على تشغيل التطبيق الأساسي.

.5 Low

أقل مستوى خطورة.

معناه:

- سؤال عام عن التطوير
 - أو طلب إضافة Feature جديدة
 - أو استفسار عادي
- وده لا يعتبر عطل في الخدمة.

◆ Response Time

ال Response Time هو الوقت المتوقع الي AWS هتبدأ خلاله ترد على ال Support Case.

سرعة الرد بتعتمد على عاملين:

- Support Plan الي مشترك فيها
- Case Severity

يعني:

ال Enterprise Support هيرد أسرع من Business، وال Business هيرد أسرع من Developer. وكلما زادت Severity، كلما كان الرد أسرع.

◆ Basic Support

مُهم جداً، ال Basic Support لا يوفر Case Support.

يعني لا يمكنك فتح Technical Support Case مع AWS.

ستحصل فقط على الخدمات الأساسية مثل:

- Documentation
- Forums
- Personal Health Dashboard
- Core Trusted Advisor Checks

◆ اختيار Support Plan

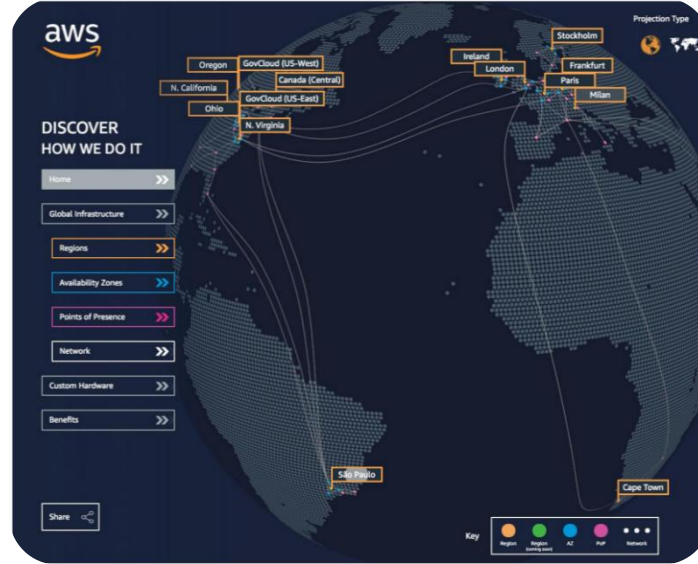
قبل ما تختار Support Plan، لازم تسأل نفسك:

- هل التطبيق بتاعي Production
 - هل أي توقف هياثر على البرنس
 - هل محتاج استجابة سريعة للمشاكل الحرجة
- الإجابة على الأسئلة دي هي الي تحدد الخطة المناسبة.

◆ Module 3: AWS Global Infrastructure Overview

1. AWS Global Infrastructure

في الشاتر ده، هنوضح ونشرح السؤال اللي بيمر في خيال أي حد، هي فين الكلاود دي بالضبط؟
AWS بانية بنية تحتية عالمية هدفها الأساسي إنها تكون مرنة، يُعتمد عليها، وقابلة للتوسع، والأهم إنها متآمنة بأعلى المستويات.



تقدر تشوف البنية التحتية لـ AWS من هنا <https://infrastructure.aws>

1. المناطق الجغرافية (AWS Regions)

الـ Region هو عبارة عن مساحة جغرافية حقيقية على الخريطة (زي منطقة لندن، باريس، أو شمال فيرجينيا).
أنت اللي بتحدد بياناتك تتخزن في أنهي منطقة، ومحدث غيرك يقدر ينقلها لمنطقة ثانية.
التواصل بين المناطق وبعضها بيتتم عن طريق شبكة فاير خاصة وعالمية تابعة لـ AWS، مش بيمشي في الإنترنت العام، وده بيضمن سرعة خرافية وأمان عالي.
كل منطقة مُصممة بحيث تكون مُستقلة تماماً ومؤمنة ضد الأعطال.

إزاي تختار الـ Region المناسب لمشروعك؟

ماتختار عشوائي، فيه 4 عوامل لازم تراعيهم في كتابك:

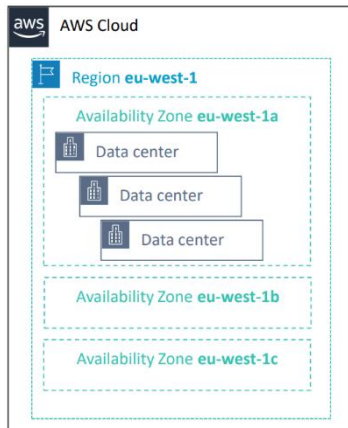
- الحوكمة والقوانين (Data Governance): فيه دول بتشترط إن بيانات مواطنيها ماتخرجش بره حدود البلد.
- القرب من العملاء (Latency): اختار أقرب منطقة للناس اللي هتستخدم موقعك عشان الموقع يفتح عندهم بدون زيادة في التأخير.
- توفر الخدمات: مش كل خدمات AWS موجودة في كل المناطق؛ اتأكد إن الخدمة اللي محتاجها متوفرة في المنطقة اللي اخترتها.
- التكلفة: الأسعار بتختلف من بلد للثانية حسب تكاليف التشغيل والضرائب هناك.

2. مناطق التوافر (Availability Zones - AZs)

كل Region يتكون من كذا AZ (على الأقل إثنين).

إيه هي AZ؟ هي عبارة عن واحد أو أكثر من مراكز البيانات (Data Centers) المنفصلة تماماً ومعزولة عن بعضها في الكهرباء والتبريد والشبكة.

AWS صممتهما كدة عشان لو حصلت كارثة في مكان (زي حريق أو فيضان)، الـ AZs التانية اللي في نفس المنطقة تفضل شغالة وما متتاثرش.



الـ AZs متوصلين ببعض بشبكة خاصة سريعة جداً (Private Fiber).

دايماً وزع شغلك على أكثر من AZ عشان تضمن إن موقعك ما يقعش أبداً (Resiliency).

3. مراكز البيانات (AWS Data Centers)

ده بيت السيرفرات.

المكان اللي فيه البيانات بتتعالج وتتخزن.

مراكز البيانات متأمّنة بأجهزة إنذار وحراسة ومحدث يقدر يدخلها بسهولة.

كل داتا سنتر فيه خطوط كهرباء وشبكات احتياطية، وبيكون فيه ما بين 50,000 لـ 80,000 سيرفر حقيقي.

4. نقاط التواجد (Points of Presence)

عشان المحتوى يوصل للمستخدم النهائي بسرعة من غير لاج (Lag)، AWS عملت شبكة عالمية فيها أكثر من 410 نقطة تواجد.

• Edge Locations

هو موقع صغير تابع لـ AWS CloudFront موجود قريب من المستخدمين حول العالم.

وظيفته الأساسية إنه يخزن الـ Cached Content زي الصور، الفيديوهات، وملفات الـ CSS و JavaScript.

لما المستخدم يطلب المحتوى، يتبعته من أقرب Edge Location بدل ما يروح للـ Origin Server البعيد، وده يقلل الـ Latency ويخلي الموقع أسرع (وده اللي بتعمله خدمة Amazon CloudFront).

مثال:

لو السيرفر الأساسي في أمريكا والمستخدم في مصر، المحتوى ممكن يوصله من Edge Location قريبة في الشرق الأوسط بدل أمريكا.

• Regional Edge Caches

الـ Regional Edge Cache عبارة عن Cache أكبر موجود بين الـ Edge Locations والـ Origin Server.

وظيفته إنه يحتفظ بالمحتوى لفترة أطول ويقلل عدد الطلبات اللي بتروح للـ Origin.

يعني لو الـ Edge Location مش لاقى المحتوى بدل ما يروح مباشرة للـ Origin Server يروح الأول للـ Regional Edge Cache، وده يقلل الضغط على الـ Origin ويحسن الأداء أكثر.

◆ مميزات بنية AWS التحتية

أمازون صممت النظام ده عشان يحقق 4 صفات أساسية:

- المرونة والتوسع (Elasticity & Scalability): السيستم بيكبر ويصغر لوحده حسب الزحمة اللي على موقعك.
- تحمل الأخطاء (Fault-tolerance): لو قطعة باظت، السيستم بيفضل شغال لن فيه بديل جاهز فوراً (Built-in redundancy).
- الإتاحة العالية (High Availability): الموقع بيفضل شغال بأعلى كفاءة وأقل وقت تعطل ممكن، ومن غير ما تحتاج تتدخل بنفسك.

Section 1 Summary

البنية التحتية العالمية لـ AWS قائمة بشكل أساسي على حاجتين

- المناطق الجغرافية (Regions)
- مناطق التوافر (Availability Zones)

لما تيجي تختار الـ Region لمشروعك، عينك لازم تكون على حاجتين:

- مُتطلبات القوانين والالتزام بخصوص البيانات (Compliance)
- أو إنك تختار أقرب مكان للمستخدمين عشان تقلل اللاج أو زمن الإستجابة (Latency)

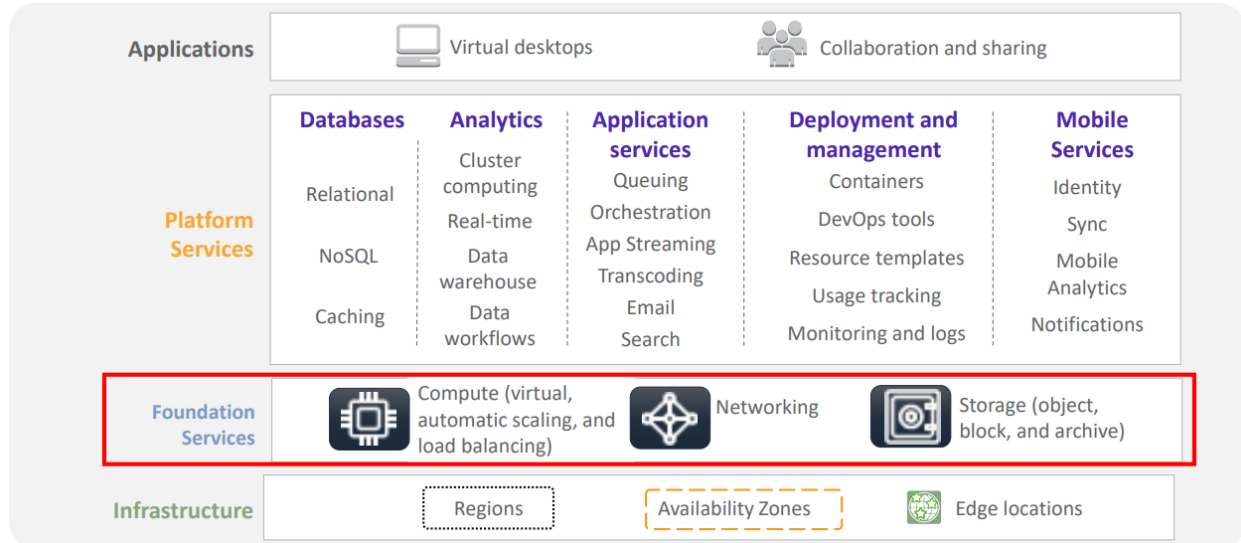
كل Availability Zone معزولة فيزيائياً ومكانياً عن المناطق التانية، ولها مصادر طاقة وشبكات واتصالات احتياطية (Redundant) خاصة بيها، وده اللي بيخلي السيستم يفضل شغال حتى لو حصلت كارثة في مكان واحد.

الـ Edge Locations والـ Regional Edge Caches هما السر في سرعة وصول المحتوى للناس؛ لإنهم بيخزنوا نسخة من البيانات (Caching) في نقطة قريبة جداً من المُستخدم النهائي حول العالم.

2. AWS services and service category overview

◆ نظرة عامة على خدمات AWS وفئاتها (Service Overview)

AWS بتقديم أكثر من 200 خدمة، وعشان يسهلوا علينا الموضوع، قسموهم لطبقات وفئات.



1. AWS Foundational Services

بُناءً على التقسيم المُعتمد في AWS، الخدمات بتتقسم لأربع طبقات أساسية بتعتمد على بعضها:

Infrastructure: ودي اللي شرحناها (Regions, AZs, Edge Locations).

Foundation Services: دي الخدمات الخام اللي مفيش سيستم بيشتغل من غيرها، ويتكون فوق البنية التحتية مباشرةً ، ويتشمل:

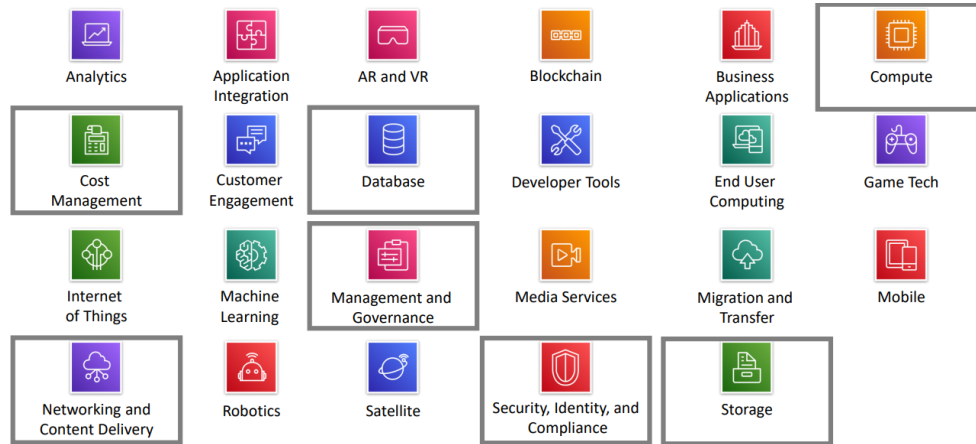
- **Compute:** السيرفرات الافتراضية، وتوزيع الاحمال (Load Balancing)، والتوسع التلقائي (Auto Scalling).
- **Networking:** الوصلات اللي بتربط كل حاجة ببعضها.
- **Storage:** الهاردات والمساحات اللي بنشيل عليها البيانات بمختلف أنواعها.

Platform Services: دي خدمات مُتطورة بتقدم أدوات جاهزة للمطورين، زي:

- **Databases:** سواء كانت Relational أو NoSQL.
- **Analytics:** لمعالجة البيانات الضخمة (Big Data).
- **خدمات التطبيقات والموبايل:** زي إرسال الإشعارات والبحث والايملات.
- **Deployment and Management:** زي أدوات ال DevOps ومراقبة الأداء.

التطبيقات الجاهزة (Applications): دي خدمات نهائية موجهة للإستخدام المباشر، زي أجهزة الديسكتوب الافتراضية وأدوات التعاون والمشاركة.

2. خدمات AWS وتصنيفاتها (Categories of Services)



لما تفتح لوحة تحكم AWS، هنلاقي الخدمات منقسمة لمجموعات (Categories).

أهم المجموعات اللي لازم تكون عارفها عشان الإمتحان وعشان شغلك هي:

الحوسبة (Compute): دي أهم فئة، فيها الـ EC2 والـ Lambda، وهي المسؤولة عن تشغيل الأكواد بتاعتك.

التخزين (Storage): المكان اللي بيخزن فيه، زي خدمة S3 المشهورة.

قواعد البيانات (Database): لو عندك بيانات مُنظمة ومحتاجة ترتيب، زي RDS و DynamoDB.

الشبكات وتوصيل المحتوى (Networking & Content Delivery): المسؤولة عن سرعة وصول موقعك للناس وأمان الربط.

الأمن والهوية والإمتثال (Security, Identity, and Compliance): هي الحماية بتاعت حسابك اللي بتحدد مين يدخل ويعمل ايه.

الإدارة والحوكمة (Management and Governance): الأدوات اللي بتراقب بيها السيستم بتاعك شغال صح ولا لا.

إدارة التكاليف (Cost Management): الأدوات اللي شرحناها في الموديول الثاني عشان تتابع فاتورتك.

بعد ما عرفنا الهيكل العام، هنتمق أكثر في 3 فئات بيمثلوا عمود الأساس لأي مشروع على الكلاود.

1. Storage Service Category

دي الخدمات المسؤولة عن حفظ بياناتك بمختلف أنواعها وأحجامها:

Amazon S3: خدمة تخزين (Object Storage)، بتتميز بقابلية توسع هائلة وأمان عالي جداً، وهي الأنسب لحفظ الصور، الفيديوهات، والملفات اللي بنحتاجها بشكل مُتكرر.

Amazon EBS: ده بمثابة الهارد ديسك بتاع سيرفرات الـ EC2، وهو تخزين من نوع (Block Storage) مُخصص للأداء العالي وسرعة المعالجة.

Amazon S3 Glacier: مُخصص للأرشفة والنسخ الاحتياطي طويل المدى، تكلفته قليلة جداً لكنه بيستخدم للبيانات اللي مش محتاجين نوصل لها فوراً.

Amazon EFS: نظام ملفات شبكي (Network File System) مرن وسهل الإدارة، بتقدر توصله من كذا سيرفر في نفس الوقت، وينفع جداً في ربط السيرفرات ببعضها.

2. Compute Service Category

دي المُحرّكات اللي بتشغل الأكواد والتطبيقات بتاعتك.

Amazon Elastic Compute Cloude: هي السيرفرات الافتراضية اللي تقدر تتحكم في حجمها وإمكانياتها بالكامل.

Amazon EC2 Auto Scaling: الخدمة اللي بتزود أو تنقص عدد السيرفرات أوتوماتيكياً حسب الضغط.

Amazon Elastic Conatiner Service: خدمات مُخصصة لإدارة وتشغيل ال Docker وال Kubernetes بسهولة.

AWS Lambda: الخدمة الأشهر في عالم ال Serverless، بتخليك تشغل كودك من غير ما تشغل بالك بإدارة سيرفرات خالص.

AWS Fargate: بتسمح لك تشغل Containers من غير ما تضطر تدير السيرفرات اللي شايلة Containers دي.

AWS Elastic Beanstalk: أداة للمطورين عشان يرفعوا تطبيقات الويب بتاعتهم وهي بتتكفل بكل تفاصيل النشر والتوسع تلقائياً.

3. Database Service Category

عشان تنظم بياناتك وتسترجعها بسرعة وبأمان:

Amazon RDS: خدمة بتسهل عليك تشغيل وإدارة قواعد البيانات (Relational) المشهورة زي MySQL و PostgreSQL.

Amazon Aurora: قاعدة بيانات مُتوافقة مع MySQL و PostgreSQL لكنها متطورة جداً ومُصممة خصيصاً عشان تستفيد من قوة الكلاود.

Amazon DynamoDB: قاعدة بيانات NoSQL بتديك أداء ثابت وسريع جداً (في أجزاء من الثانية) مهما كان حجم البيانات.

Amazon Redshift: مستودع بيانات ضخمة (Data Warehouse) مُخصص لتحليل كميات مهولة من البيانات بسرعة عالية جداً.

4. Networking & Content Delivery

دي الخدمات المسؤولة عن بناء البنية التحتية اللي بتربط مواردك ببعضها وبالإنترنت.

Amazon VPC: بتسمح لك بإنشاء شبكة معزولة ومنطقية خاصة بيك جوه الكلاود AWS، بتتحكم فيها في كل حاجة زي ال IP addresses وال Subnets.

Elastic Load Balancing: وظيفته توزيع الترافيك اللي جاية لموقعك على كذا سيرفر بشكل آلي عشان مفيش سيرفر يقع من الضغط.

Amazon CloudFront: خدمة (CDN) عالمية بتوصل المحتوى للمستخدمين بسرعة عالية جداً وبأقل زمن استجابة عن طريق تخزينه في ال Edge Locations القريبة منهم.

Amazon Route 53: نظام أسماء النطاقات (DNS) عالي الكفاءة، وظيفته يوجه المُستخدمين لموقعك بطريقة يُعتمد عليها.

AWS Direct Connect: بيعمل وصلة خاصة ومُباشرة بين الداتا سنتر بتاعة شركتك وبين AWS، بعيداً عن الإنترنت العام، لضمان سرعة واستقرار أكبر.

AWS VPN: بيعمل نفق آمن ومُشفّر بين جهازك أو شركتك وبين شبكة AWS.

AWS Transit Gateway: بيشتغل كسنترال يربط ال VPCs المُختلفة وشبكاتك المحلية ببعضها في مكان واحد وبسهولة.

5. Security, Identity, and Compliance

دي الخدمات اللي بتضمن إن نظامك متآمن ومحدث بيدخله غير المصرح ليهم:

AWS IAM: دي أهم خدمة لإدارة الهويات؛ بتحدد مين مسموح له يدخل على أنهي خدمة وبأنهي صلاحيات بالظبط.

AWS Organizations: بتسمح لك تدير كذا حساب AWS مع بعض وتطبق عليهم سياسات موحدة بخصوص الخدمات المسموحة.

Amazon Cognito: مُخصصة للمطورين عشان يضيفوا خاصية تسجيل الدخول (Sign-up/Sign-in) لتطبيقات الموبايل والويب بتاعتهم بسهولة.

AWS Shield: خدمة حماية مُخصصة لصد هجمات الـ DDOS اللي بتهدف لتعطيل المواقع عن طريق إغراقها بطلبات وهمية.

AWS Key Management Service (KMS): بتخليك تنشئ وتدير مفاتيح التشفير اللي بتستخدمها لحماية بياناتك.

AWS Artifact: ده المتجر اللي بتقدر تنزل منه تقارير الإمتثال والشهادات الأمنية الرسمية الخاصة بـ AWS عشان تثبت إن شركتك مُلتزمة بالمعايير العالمية.

6. Management and Governance

الخدمات دي هي اللي بتديك لوحة القيادة والتحكم الكامل في كل الموارد بتاعتك:

AWS Management Console: دي واجهة المُستخدم اللي بنفتحها من المُتصفح عشان نتحكم في حساب AWS ونوصل لكل الخدمات بسهولة.

AWS Config: خدمة بتساعدك تتبع حالة الموارد بتاعتك وتعرف إيه التغييرات اللي حصلت فيها مع الوقت، وده مُهم جداً للإمتثال الأمني.

Amazon CloudWatch: العين اللي بتراقب أداء السيستم؛ بتجمع بيانات (Metrics) عن استهلاك السيرفرات وبتبعت لك تنبيهات لو فيه حاجة مش طبيعية.

AWS Auto Scaling: بتضمن إن عدد الموارد (زي السيرفرات) يزيد أو يقل أوتوماتيكياً عشان يغطي احتياجات السيستم في أي وقت.

AWS Command Line Interface (CLI): أداة موحدة بتخليك تتحكم في خدمات AWS من خلال أوامر نصية (Commands) بدل الواجهة الرسومية، وده بيسهل الأتمتة.

AWS Trusted Advisor: بمثابة مُستشار تقني آلي؛ يفحص حسابك وبيديك نصائح عشان تحسن الأداء، تزود الأمان، وتقلل التكاليف.

AWS Well-Architected Tool: أداة بتساعدك تراجع التصميم بتاع السيستم بتاعك وتقارنه بأفضل الممارسات اللي AWS وضعتها في 6 ركائز أساسية.

AWS CloudTrail: سجل العمليات؛ بيسجل كل حركة أو طلب (API Call) حصل في حسابك، يعني بتقدر تعرف مين عمل إيه وإمتى بالظبط.

7. Cost Management

عشان مفيش حاجة تخرج عن السيطرة، AWS بتديك الأدوات دي عشان تتابع وتخطط لمصاريفك.

AWS Cost and Usage Report: ده التقرير الأكثر تفصيلاً؛ بيديك بيانات دقيقة جداً عن تكاليفك واستخدامك، وتقدر تفتحه ببرامج تحليل البيانات.

AWS Budgets: بتخليك تحط ميزانية مُحددة، وبتبعت لك تنبيهات فورية لو صرفت أكثر منها أو حتى لو السيستم توقع إنك هتعدّيها.

AWS Cost Explorer: أداة سهلة بتديك رسوم بيانية واضحة عشان تفهم وتدير مصاريفك، وتعرف إيه أكثر الخدمات اللي بتسحب من ميزانيتك مع الوقت.

اسئلة مُهممة ممكن تيجي في الإمتحان.

س1: تحت أنهي فئة (Category) بنلاقي خدمة ال IAM؟

الإجابة: بتلاقيها تحت فئة Security, Identity, & Compliance.

ليه؟ لأن ال IAM هي المسؤول الأول عن أمن الحساب وإدارة هويات المُستخدمين وصلاحياتهم.

س2: تحت أنهي فئة بنلاقي خدمة ال Amazon VPC؟

الإجابة: بتلاقيها تحت فئة Networking & Content Delivery.

ليه؟ لأن ال VPC هي الشبكة الخاصة اللي بتبني عليها مشروعك، فهي أساس الربط الشبكي.

س3: ال Subnet (الشبكة الفرعية) اللي بتختارها، هل بتكون موجودة على مستوى ال Region ولا ال Availability Zone؟

الإجابة: ال Subnet موجودة على مستوى ال Availability Zone.

ال Subnet الواحدة لازم تكون جوه AZ واحدة بس، ما ينفعش تمتد بين اثنين AZ.

س4: ال VPC نفسها، هل موجودة على مستوى ال Region ولا ال Availability Zone؟

الإجابة: ال VPC موجودة على مستوى ال Region.

ال VPC هي المظلة الكبيرة اللي بتغطي المنطقة كلها، وجواها بنقسم ال Subnets على مناطق التوافر المُختلفة.

س5: أنهي خدمات من دول بتعتبر "عالمية" (Global) وأنهي "إقليمية" (Regional)؟

(الإختيارات: Amazon EC2, IAM, Lambda, Route 53)

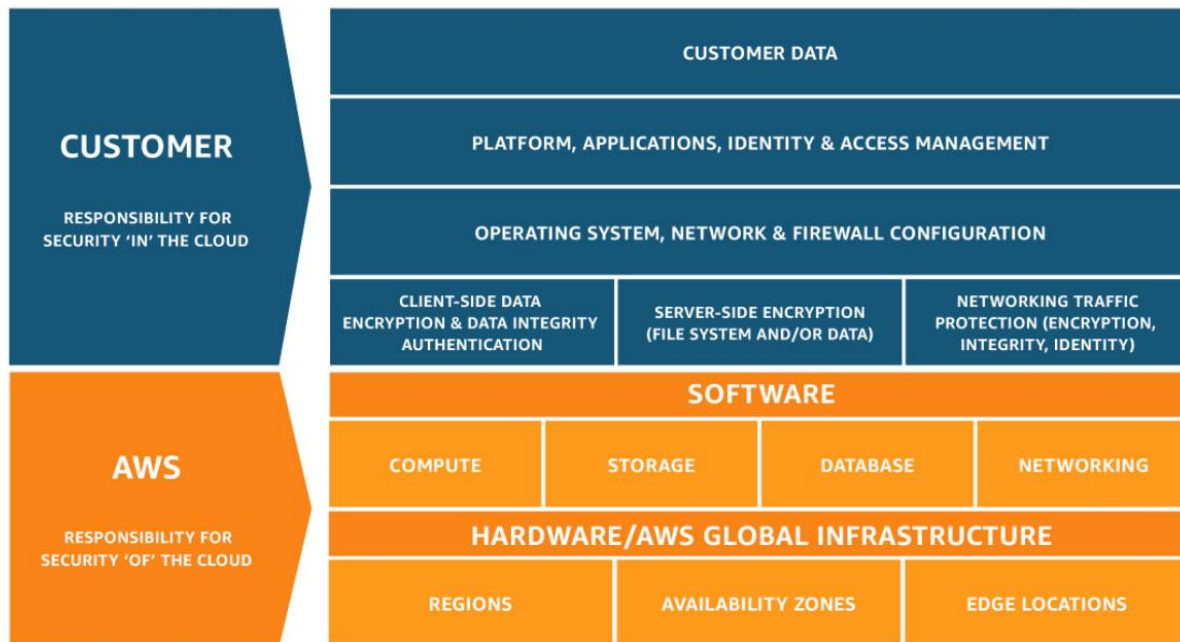
الإجابة:

خدمات عالمية (Global): ال IAM وال Route 53. (لما بتفتحهم مش بتختار Region مُعين، هما شغالين على مستوى العالم كله).

خدمات إقليمية (Regional): ال Amazon EC2 وال Lambda. (لازم تختار Region مُعين عشان تشغلهم فيه).

♦ **Module 4: AWS Cloud Security**

1. AWS shared responsibility model



النموذج ده معمول علشان يحدد مين بيعمل إيه لما تيجي تأمين الشغل على الكلاود.

الأمن هنا مش حكر على شركة AWS لوحدها، ولا على العميل (Customer) لوحده.

- AWS مسؤولة عن حماية وتأمين البنية التحتية الي بتشغل كل الخدمات. (تأمين الكلاود نفسه).
- العميل (أنت) مسؤول عن تأمين وتكوين الحاجات الي بتبنيها جوه البنية التحتية دي. (الأمن جوه الكلاود).

❖ مسؤولية AWS (أمن الكلاود - Security of the cloud)

شركة AWS مسؤولة عن كل المكونات المادية والبرمجية الأساسية (من أول الحديد والخرسانة لحد طبقة السوفت وير الي بتشغل الأجهزة):

1. الأمن المادي لمراكز البيانات (Physical security of data centers):

هما المسؤولين عن حراسة المباني، والتحكم في الدخول ومراقبة مين بيدخل ومين بيخرج بُناءً على الحاجة بس.

2. البنية التحتية للأجهزة والبرامج (Hardware and software infrastructure):

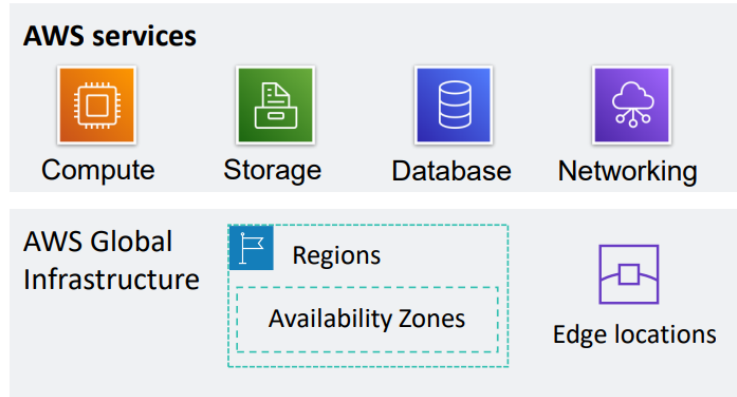
صيانة السيرفرات، والتخلص الآمن من وحدات التخزين القديمة (Storage decommissioning)، وتسجيل الدخول لنظام التشغيل المضيف (Host OS) ومراجعته.

3. البنية التحتية للشبكة (Network infrastructure):

تأمين الكابلات والتوصيلات، وأنظمة كشف التسلل (Intrusion detection) لمنع أي اختراق على مستوى الشبكة العامة بتاعتهم.

4. Virtualization infrastructure

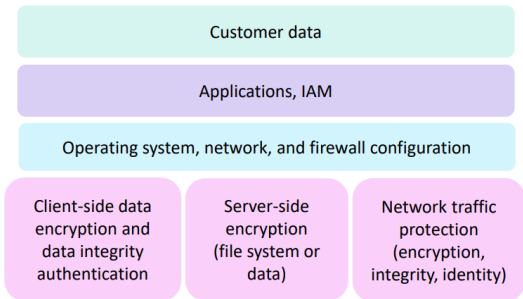
فصل البيئات الافتراضية عن بعضها (Instance isolation) عشان مفيش عميل يقدر يشوف أو يدخل على سيرفر عميل تاني.



الخدمات الأساسية اللي بتديرها AWS في الحقة دي:

- Compute (الحوسبة): السيرفرات والمعالجات.
- Storage (التخزين): الهاردات ومساحات الحفظ.
- Database (قواعد البيانات): الأنظمة اللي بتخزن البيانات المنظمة.
- Networking (الشبكات): التوصيلات والراوترات الافتراضية.

البنية التحتية العالمية لـ AWS اللي بتشمل Regions، Availability Zones، و Edge locations.



❖ مسؤولية العميل (الأمن في الكلاود - Security in the cloud)

كل حاجة هتحتها أو هتعملها جوه الحساب بتاعك هي بتاعتك ومسؤوليتك بالكامل:

1. بيانات العميل (Customer data):

تشفير البيانات وحمايتها، وتحديد مين يشوفها ومين لاء.

2. التطبيقات وإدارة الهوية والوصول (Applications, IAM):

كتابة وتأمين الكود بتاع تطبيقك، إدارة الحسابات، كلمات المرور، وتحديد الصلاحيات (Permissions) لكل مُستخدم.

3. نظام التشغيل، الشبكة، وإعدادات جدار الحماية (OS, Network, & Firewall configuration):

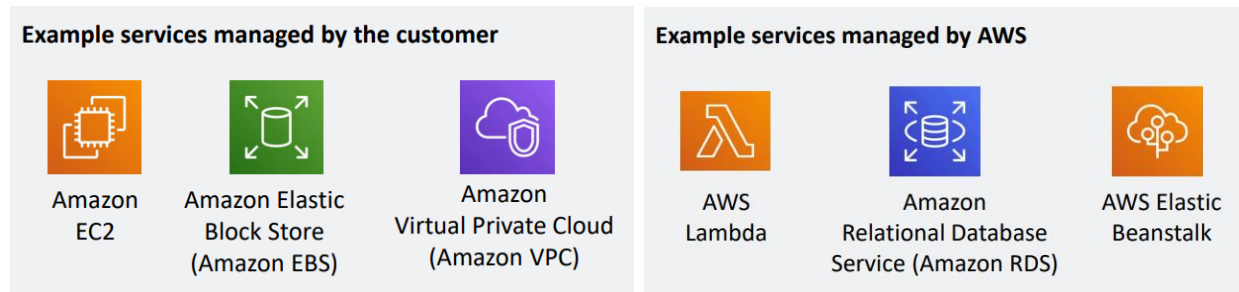
لو حاجز سيرفر، هتبقى أنت المسؤول عن تحديث نظام التشغيل (Patching) بتاعه، وتطبيق جدار الحماية (Firewall) الافتراضي اللي اسمه Security group.

4. التشفير وحماية حركة المرور (Encryption & Traffic protection):

تشفير البيانات من جهة العميل (Client-side) أو جهة السيرفر (Server-side)، وحماية البيانات وهي بتتنقل في الشبكة.

❖ اختلاف المسؤولية حسب نوع الخدمة (Service Characteristics)

المسؤولية دي مش ثابتة، بتزيد وتقل على حسب نوع الخدمة اللي بتشتريها.



1. IaaS - Infrastructure as a Service

الفكرة هنا إن AWS بتديك سيرفر فاضي أو مساحة تخزين وأنت بتعمل كل حاجة.

أمثلة: Amazon EC2, Amazon EBS, Amazon VPC.

المسؤولية: مرونتك عالية جداً، بس مسؤوليتك بتزيد؛ العميل هو المسؤول عن تنزيل نظام التشغيل وتحديثه، وإعدادات جدار الحماية، وإدارة الوصول.

2. PaaS - Platform as a Service

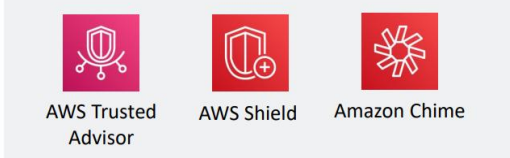
AWS بتشيل عنك هم إدارة البنية التحتية ونظام التشغيل، وتديك منصة جاهزة تحط عليها كودك أو بياناتك بس.

أمثلة: AWS Lambda, Amazon RDS, AWS Elastic Beanstalk.

المسؤولية: AWS هي اللي بتعمل تحديث لنظام التشغيل (OS patching)، وتطبيق جدار الحماية الأساسي، وإصلاح الكوارث (Disaster recovery).

العميل بيركز فقط على إدارة الأكواد والبيانات والصلاحيات.

SaaS examples



3. SaaS - Software as a Service

الفكرة: تطبيق كامل جاهز للاستخدام ومتستضيف مركزياً، بتدخله برابط أو تطبيق.

أمثلة: AWS Trusted Advisor, AWS Shield, Amazon Chime.

المسؤولية: العميل مش بيشتغل مع أي بنية تحتية نهائية، هو مجرد مُستخدم للخدمة (User)، والمسؤولية الأكبر لبرمجة وتأمين وصيانة السوفت وير ده تقع على AWS.

AWS Trusted Advisor: دي عبارة عن أداة أونلاين بتعمل فحص وتحليل كامل وشامل للبيئة بتاعتك (AWS environment) بشكل فوري ومُستمر (Real-time)، بتديك نصائح وتوصيات عشان تشغل خدماتك بُناءً على AWS best practices.

Amazon Chime: دي خدمة اتصالات متكاملة ومؤمنة، بتخليك تعمل اجتماعات (Meet)، ومحادثات (Chat)، ومُكالمات بيزنس مع ناس جوه أو بره شركتك.

Section 1 Summary

AWS: مسؤولة عن أمن الكلاود (حماية البنية التحتية والمباني والشبكات).

العميل: مسؤول عن الأمن في الكلاود (بياناته، كوده، صلاحياته، وتحديثات أنظمة التشغيل التي يختارها بنفسه).

في خدمات ال IaaS العميل مسؤوليته أكبر (تحديثات وإعدادات أمنية)، وفي خدمات ال PaaS وال SaaS المسؤولية تنتقل أكثر لـ AWS والعميل يرتاح من وجع دماغ الإدارة البنيوية.

2. AWS Identity and Access Management (IAM)

دي خدمة مجانية بالكامل جوه حسابك (No-cost feature)، وظيفتها الأساسية هي التحكم في مين يقدر يدخل على حسابك، وإيه الموارد (Resources) التي مسموح له يوصلها، وإزاي يتعامل معاها.

النظام بيحدد بدقة:



AWS Identity and Access
Management
(IAM)

- **Who:** مين يقدر يدخل على المورد.
- **Which & What:** إيه الموارد التي مسموح الدخول ليها، وإيه التي العميل يقدر يعملها في المورد ده.
- **How:** إزاي بيتتم الدخول للموارد دي.

أمثلة للموارد: سيرفر Amazon EC2 أو Amazon S3.

مثال للتحكم: التحكم في مين يقدر يقفل أو يلغي سيرفرات Amazon EC2.

❖ المكونات الأساسية لـ IAM (Essential Components)

1. المُستخدم (IAM User):

شخص أو تطبيق حقيقي محتاج يدخل على الحساب ويثبت هويته (Authenticate).

2. المجموعة (IAM Group):

مجموعة من المُستخدمين ليهم نفس الصلاحيات (زي مجموعة المبرمجين أو المحاسبين)، عشان تظبط صلاحياتهم مرة واحدة بدل ما تلف على واحد واحد.

3. السياسة (IAM Policy):

عبارة عن وثيقة أو مُستند (ملف JSON) مكتوب فيه بدقة إيه الخدمات المسموح بالوصول ليها وإيه الممنوع.

4. الدور (IAM Role):

صلاحيّة مؤقتة بتديها لمُستخدم أو لخدمة ثانية جوه AWS عشان تعمل مُهمة مُعينة، ومن غير ما تديهم اسم مُستخدم أو باسورد دائمين.

❖ إزاي العميل بيدخل على الحساب؟ (Authentication Types)

لما تيجي تعمل مُستخدم (IAM user)، عندك طريقتين عشان تثبت هويتك وتدخل:



AWS CLI

AWS Tools
and SDKs

1. Programmatic access

ده للمطورين والبرامج.

طريقة إثبات الهوية إنه بيسجل دخول وبيأخذ صلاحية باستخدام:

- Access key ID
- Secret access key

الطريقة دي بتوفر إمكانية الدخول لأدوات الـ CLI أو الـ SDK.

AWS Management
Console

2. AWS Management Console access

وده الدخول العادي من المُتصفح.

طريقة إثبات الهوية إنه بيسجل دخول وبيأخذ صلاحية باستخدام:

- رقم الحساب المكون من 12 رقم أو الاسم المُستعار للحساب (digit Account ID or alias-12).
- اسم المُستخدم (IAM user name).
- كلمة السر (IAM password).

الأمان الإضافي (MFA): لو الخاصية دي متفعلة، نظام المُصادقة MFA بيطلب من المُستخدم كود أمان لإثبات الهوية.

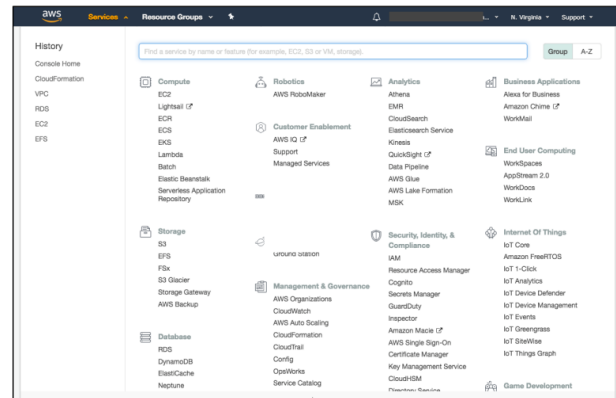
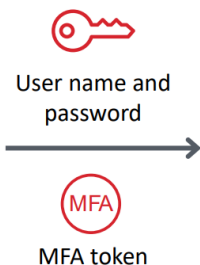
❖ نظام الأمان الإضافي (IAM MFA)

Account:

User Name:

Password:

MFA users, enter your code on the next screen.



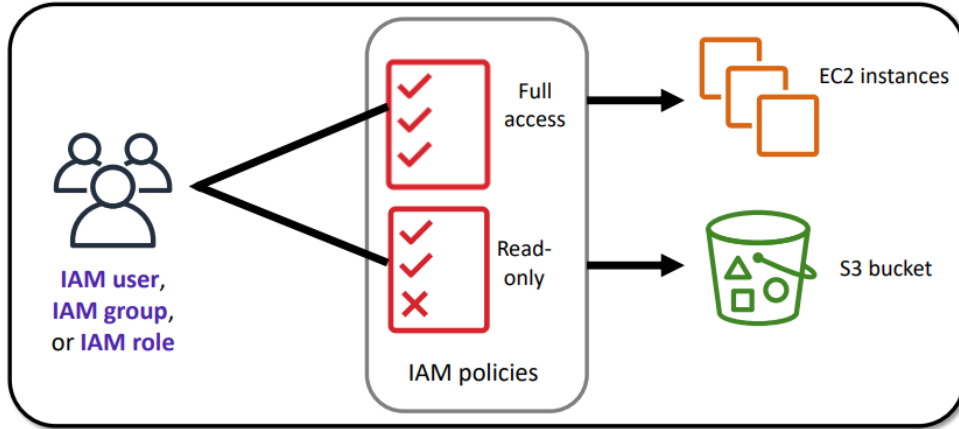
AWS Management Console

الـ MFA (Multi-Factor Authentication) بيوفر مستوى أمان أعلى وحماية زيادة.

بالإضافة لاسم المُستخدم وكلمة السر، نظام الـ MFA بيطلب كود أمان مؤقت وفريد (Unique authentication code) عشان يسمح بالدخول لخدمات AWS.

❖ مفهوم الصلاحيات وتحديدتها

After the user or application is connected to the AWS account, what are they allowed to do?



بعد ما المُستخدم (IAM user) أو التطبيق بيتصل بحساب AWS ويُثبت هويته، بيشفوف إيه اللي مسموح له يعمل بالظبط؟
 تعيين الصلاحيات، الصلاحيات دي بتحدد وبنديهاهم عن طريق إننا بنعمل حاجة اسمها سياسة ال IAM أو ال IAM policy.
 السياسات دي بتطبق وتتعين على ال IAM user، أو ال IAM group، أو ال IAM role.
 الصلاحيات هي اللي بتحدد بدقة إيه هي الموارد (Resources) والعمليات أو الأفعال (Operations) المسموح بيها.
 زي مثلاً إعطاء صلاحية وصول كاملة (Full access) لموارد ال EC2 instances، أو صلاحية قراءة فقط (Read-only) لل S3 bucket.

❖ القواعد الأساسية لحساب الصلاحيات:

كل الصلاحيات بتكون ممنوعة تلقائياً وضمنياً إفتراضياً (Implicitly denied)، يعني لو مفيش سطر بيقولك مسموح، يبقى ممنوع.
 لو فيه أي حاجة ممنوعة بشكل صريح (Explicitly denied)، فالحاجة دي مش مسموح بيها أبداً وبتلغي أي سماح تاني.
 أفضل ممارسة أمنية (Best practice): دائماً اتبع مبدأ إعطاء أقل صلاحيات ممكنة (Principle of least privilege)، يعني متديش المُستخدم غير الصلاحية اللي على قد شغله بالظبط.
 خلي بالك إن إعدادات وتهيئات خدمة ال IAM نطاقها عالمي (Global)، وده معناه إن الإعدادات دي بتطبق على كل مناطق AWS عبر العالم (All AWS Regions).

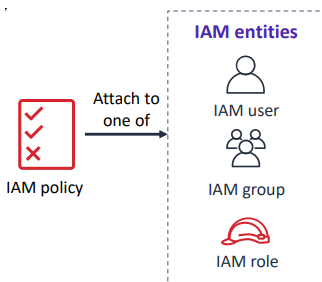
❖ سياسات إدارة الهوية والوصول (IAM Policies)

سياسة ال IAM عبارة عن مستند بيحدد ويوضح الصلاحيات.

المستند ده بيسمح بوجود تحكم دقيق جداً في عمليات الوصول للموارد (Fine-grained access control).

عندنا نوعين من السياسات:

- سياسات قائمة على الهوية (Identity-based)
- سياسات قائمة على الموارد (Resource-based)



♦ السياسات القائمة على الهوية (Identity-based policies)

النوع ده بيتربط مباشرةً بال IAM entity.

ال IAM entity دي ممكن تكون مُستخدم (IAM user)، أو مجموعة (IAM group)، أو IAM role.

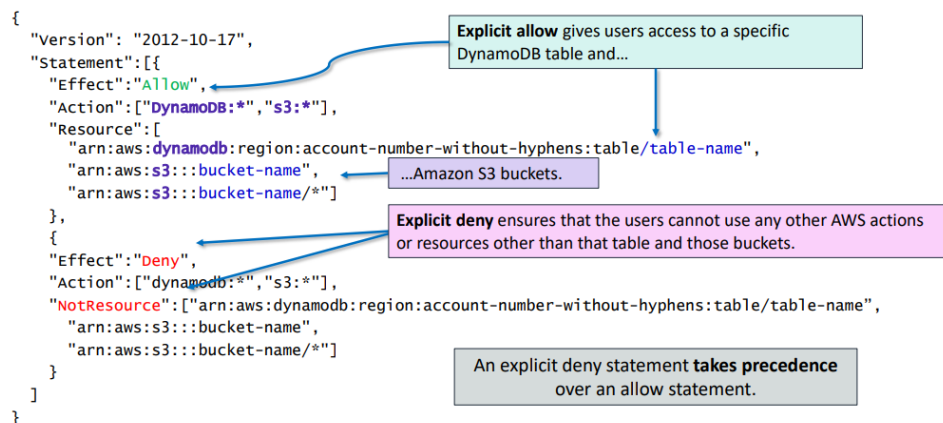
السياسات (Policies) دي بتحدد بالتفصيل:

- الأفعال أو العمليات اللي مسموح لـ entity إنه ينفذها.
- الأفعال أو العمليات اللي مش مسموح لـ entity إنه ينفذها.

المُرونة في الربط:

- السياسة الواحدة ممكن ترتبط بكذا entity في نفس الوقت.
- ال entity الواحد ممكن يرتبط بيه كذا سياسة (Policy) مُختلفة.

مثال:



بُنَاءً على الكود المكتوب بلغة JSON، بتنقسم جمل الصلاحيات لشكلين:

Explicit allow: ده السطر اللي بيكون فيه ال "Effect": "Allow".

وظيفته هنا إنه بيدي المُستخدمين صلاحية دخول مباشرةً لجدول محدد في DynamoDB و Amazon S3 buckets مُحددة.

Explicit deny: ده السطر اللي بيكون فيه ال "Effect": "Deny".

وظيفته هنا إنه بيضمن إن المُستخدمين مش هيقدرُوا يستخدموا أي أفعال أو موارد ثانية تابعة لـ AWS بره الجدول ده وال S3 دي بالذات (باستخدام خاصية NotResource).

مُلاحظة: ال Explicit deny statement دايماً بيكون ليها الأولوية عن ال Explicit Allow.

♦ السياسات القائمة على الموارد (Resource-based policies)

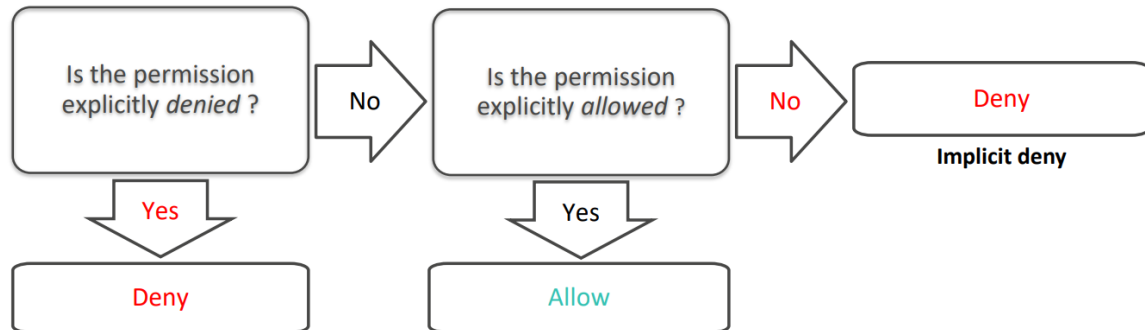
السياسة دي بتحدد مين ليه حق الوصول لـ Resource ده بالذات، وإيه الأفعال والعمليات اللي يقدر يعملها عليه، ومش بتتربط بمُستخدم أو Group أو Role.

السياسات دي بتكون مُدمجة ومكتوبة جوه المورد نفسه بس (inline only)، ومش بتكون سياسات (Policies) مُدارة مُنفصلة.

النوع ده من السياسات مش مدعوم في كل مكان، خدمات مُعينة بس جوه AWS هي اللي بتدعمه وتشغله.

❖ إزاي نظام ال IAM بيحدد الصلاحيات؟ (How IAM determines permissions)

لما بيبتدي أي مُستخدم أو تطبيق يطلب يعمل حركة جوه الحساب، نظام ال IAM بيمشي في خطوات مُحددة بالترتيب ده عشان ياخذ القرار النهائي.



الخطوة الأولى: فحص المنع الصريح (Explicit Deny)

السيستم بيسأل أول سؤال: هل الصلاحية دي ممنوعة بشكل صريح؟ (Is the permission explicitly denied?).

- لو الإجابة نعم (Yes): القرار النهائي بيكون المنع (Deny) والسيستم يرفض الطلب ومبيكملش فحص.
- لو الإجابة لا (No): السيستم بينتقل تلقائياً للخطوة اللي بعدها.

الخطوة الثانية: فحص السماح الصريح (Explicit Allow)

السيستم بيسأل ثاني سؤال: هل الصلاحية دي مسموح بيها بشكل صريح؟ (Is the permission explicitly allowed?).

- لو الإجابة نعم (Yes): القرار النهائي بيكون السماح (Allow) والطلب بيعدي ويتنفذ.
- لو الإجابة لا (No): السيستم يروح للقرار البديل.

لو الإجابة على السؤالين اللي فاتوا كانت "لا" (يعني مفيش سطر بيقول مسموح، وفي نفس الوقت مفيش سطر بيقول ممنوع)، السيستم بياخذ قرار تلقائي بالمنع (Deny).

القاعدة دي اسمها المنع الضمني (Implicit deny)؛ لأن الأصل في AWS هو قفل أي حاجة طالما متوافقش عليها بوضوح.



IAM group: Admins	IAM group: Developers	IAM group: Testers
Carlos Salazar	Li Juan	Zhang Wei
Márcia Oliveira	Mary Major	John Stiles
	Richard Roe	Li Juan

❖ IAM groups

ال IAM Groups عبارة عن تجميعية من المُستخدمين (IAM Users).

بنستخدم المجموعة عشان نممنح نفس الصلاحيات لمجموعة مُستخدمين مع بعض مرة واحدة.

الصلاحيات دي بتُمنح للجروب عن طريق ربط سياسة أو كذا سياسة IAM بالمجموعة نفسها.

المستخدم الواحد يقدر ينتمي لكذا مجموعة عادي جداً.

زي ما واضح في الرسمة إن المستخدم Li Juan موجود في جروب Developers وجروب Testers في نفس الوقت.

مفيش حاجة اسمها مجموعة إفتراضية جوه السيستم.

المجموعات متقدرش تدخل جوه بعضها، يعني متقدرش تحط جروب جوه جروب تاني.



**IAM role
IAM role**

IAM Roles ❖

ال IAM Role هو عبارة عن IAM identity وليها صلاحيات مُحددة.

شبه ال IAM user في إننا بنربط بيه السياسات (Policies) عادي جداً.

بيختلف عن ال IAM user في إيه؟

- مش مربوط ومخصص بشكل فريد لشخص واحد بس.
- مُصمم ومقصود بيه إن يتم انتحاله (Assumable) بواسطة شخص، أو تطبيق برمجى، أو خدمة تابعة لـ AWS.
- صلاحيات مؤقتة: بيوفر بيانات أمان مؤقتة (Temporary security credentials) للإستخدام ومش دائمة.

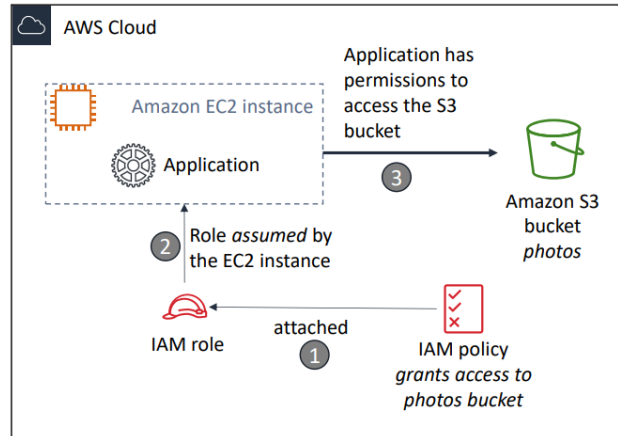
عندنا ٣ أمثلة أساسية لإزاي بنستخدم الأدوار عشان نفوض الصلاحيات للغير:

مُستخدم من نفس الحساب: يتم استخدام ال Role بواسطة IAM User موجود في نفس حساب AWS اللي جواه ال Role دي.

خدمة تابعة لـ AWS: يتم استخدام Role بواسطة خدمة من خدمات AWS زي سيرفرات Amazon EC2 موجودة في نفس الحساب بتاع Role.

مُستخدم من حساب ثاني (Cross-Account): يتم استخدام Role بواسطة IAM User موجود في حساب AWS ثاني ومُختلف تماماً عن الحساب اللي متعرف جواه ال Role.

مثال عملي على استخدام IAM role.



السيناريو أو المُشكلة (Scenario)

- عندك تطبيق برمجى (Application) شغال جوه سيرفر افتراضي (Amazon EC2 instance).
- التطبيق ده محتاج يدخل على S3 bucket عشان ياخذ أو يتعامل مع البيانات اللي جواها.

الحل (Solution)

عشان نحل المُشكلة دي بأمان من غير ما نكتب باسورودات جوه الكود، بنمشي في ٣ خطوات حسب الرسمة التوضيحية:

الخطوة رقم (1): بنعمل IAM policy بتسمح بالوصول لحاوية photos bucket، وبعدين بنربط السياسة دي بال IAM Role.

الخطوة رقم (2): بنسمح لسيرفر ال EC2 إنه ياخذ ال Role دي (Assumed by the EC2 instance).

الخطوة رقم (3): النتيجة النهائية إن التطبيق بياخذ الصلاحيات اللازمة أوتوماتيك ويقدر يدخل على ال S3 bucket بكل أمان.

Section 2 Summary

ال IAM Policy سيتم بنائها وكتابتها باستخدام JSON، وهي التي بتحدد وتعرف الصلاحيات.

ال IAM Policy ممكن تترتبط بأي entity تابع لل IAM أو (IAM entity).

ال Entites دي زي المُستخدمين (IAM users)، المجموعات (IAM groups)، وال IAM roles.

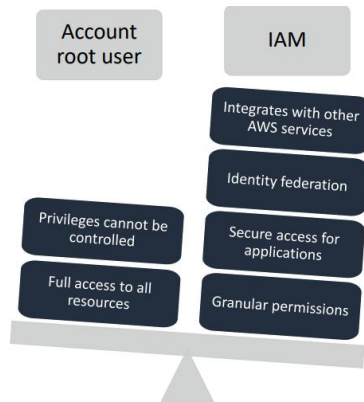
ال IAM User: بيوفر طريقة لشخص، أو تطبيق، أو خدمة عشان يعمل تسجيل دخول وإثبات هوية (Authenticate) لـ AWS.

ال IAM Group: هي طريقة بسيطة وسهلة عشان نربط ونطبق نفس السياسات والصلاحيات على كذا مُستخدم مع بعض في نفس الوقت.

ال IAM Role: بنقدر نربط بيه IAM Policies عادي، ويستخدم أساساً عشان يفوض صلاحيات وصول مؤقتة للمستخدمين أو التطبيقات.

3. Securing a new AWS account

عند التعامل مع حساب AWS، لازم نفرق بين نوعين من الهويات لإدارة الموارد، وكل نوع له خصائص وصلاحيات مُختلفة تماماً.



1. Account Root User

هو الهوية الأساسية التي بتتعمل تلقائياً أول ما بتفتح الحساب.

المستخدم ده عنده سلطة كاملة ومش مُقيدة على كل الموارد والخدمات جوه الحساب.

نظام الحساب مش بيقدّر يقيد أو يتحكم في امتيازات المُستخدم ده.

2. نظام إدارة الهوية والوصول (IAM)

هو النظام المخصص لإدارة المُستخدمين والعمليات الإعتيادية بشكل آمن.

النظام بيسمح بمنح حقوق وصول مُفصلة ومُحددة بدقة (Granular permissions).

ببتكامل مع خدمات AWS الثانية، وبيدعم نظام توحيد الهوية (Identity federation)، وبيوفر وصول آمن للتطبيقات.

❖ أفضل الممارسات الأمنية للـ Root User

بُناءً على المعايير الأمنية، الممارسة الأفضل دائماً هي عدم استخدام الـ Root User للحساب في الشغل اليومي، وبيقصر استخدامه بس على حالات الضرورة القصوى.

وعشان تدخل على الـ Root User، بتحتاج تسجيل الدخول باستخدام عنوان البريد الإلكتروني وكلمة السر اللي استخدمتهم وأنت بتعمل الحساب أول مرة، ولازم تحمي البيانات دي كويس جداً.

فيه مجموعة من المهام الحساسة والإستراتيجية اللي مستحيل أي مُستخدم عادي أو مدير نظام (حتى لو معاه صلاحيات المسؤول الإداري عبر IAM) إنه يعملها، وتكون حصرية للـ Root User بس:

- تعديل معلومات الإتصال الرسمية، أو التحكم في المناطق الجغرافية المسموح بالعمل فيها.
- إدارة كلمة السر: تحديث أو تغيير كلمة المرور الخاصة بالـ Root User نفسه.
- تغيير خطة الدعم الفني: تعديل خطة الدعم الفني الخاصة بشركة (AWS Support plan).
- استعادة الصلاحيات: استعادة وإعادة تعيين صلاحيات مُستخدم IAM لو اتلغت بالخطأ وفقد الوصول الإداري.

❖ خطوات تأمين حساب AWS

عشان تحمي حساب AWS الجديد بتاعك بشكل كويس، فيه ٤ خطوات أساسية لازم تعملهم بالترتيب:

1. التوقف عن استخدام الـ Root User

الـ Root User عنده صلاحيات كاملة ومش مُقيدة لكل الموارد جوه حسابك، وعشان توقف استخدامه وتحمي حسابك، اتبع الآتي: وأنت مسجل دخولك بالـ Root User، أنشئ IAM Rule لنفسك، واحفظ مفاتيح الوصول (Access keys) لو محتاجها. أنشئ IAM group، واديهها صلاحيات مدير نظام كاملة (Full administrator permissions)، وضيف المُستخدم بتاعك للمجموعة دي.

عطل واحذف مفاتيح الوصول (Access keys) الخاصة بالـ Root User لو كانت موجودة.

فعل سياسة كلمة المرور (Password policy) لباقي المُستخدمين.

سجل خروجك، وادخل تاني ببيانات المُستخدم الجديد (IAM user) اللي أنت لسه عامله.

احفظ بيانات الـ Root User في مكان آمن جداً.

2. تفعيل المُصادقة (Multi-Factor Authentication)

لازم تفرض تشغيل خاصية الـ MFA على الـ Root User للحساب، وعلى كل الـ IAM Users التانيين.

تقدر كمان تستخدم الـ MFA عشان تتحكم في الوصول لواجهات برمجة تطبيقات خدمات (AWS Service APIs).

خيارات وأجهزة الحصول على رمز الـ MFA:

- تطبيقات الـ MFA الافتراضية (Virtual MFA): زي تطبيق Google Authenticator وتطبيق Authy.
- أجهزة مفاتيح الأمان U2F: زي جهاز YubiKey.
- خيارات أجهزة الـ MFA (Hardware): زي Key fob أو كروت العرض اللي بتقدمها شركة Gemalto.

3. تشغيل خدمة AWS CloudTrail لمراقبة الحساب

خدمة CloudTrail بتعمل تتبع ومراقبة لنشاط وتحركات المُستخدمين جوه حسابك.

بتسجل كل طلبات الـ API اللي بتتم على الموارد في جميع الخدمات المدعومة جوه الحساب.

سجل الأحداث الأساسي (Basic event history) بيكون شغال تلقائي ومجاني، وبيحتفظ ببيانات أحداث الإدارة لآخر ٩٠ يوم من النشاط.

عشان تدخل على السجل ده ادخل على الـ Console واختار خدمة CloudTrail، واضغط على Event history عشان تشوف وتصفى وتبحث في أحداث آخر ٩٠ يوم.

عشان تحتفظ بالسجلات لأكثر من ٩٠ يوم وتعمل تنبيهات مخصصة، لازم تعمل مسار (Trail) من صفحة الـ Trails في Console الخدمة، اضغط على Create trail.

اكتب اسم للمسار، وطبّقه على كل المناطق الجغرافية (All Regions)، وأنشئ حاوية Amazon S3 جديدة عشان تخزن فيها السجلات.

اضبط قيود الوصول للـ S3 دي (مثلاً: خلي المديرين بس "Admin users" هما اللي ليهم حق يدخلوا عليها).

4. تفعيل تقارير الفواتير والإستهلاك (Billing reports)

تفعيل تقارير الفواتير، زي تقرير تكلفة واستخدام (AWS Cost and Usage Report)، بيوفر لك معلومات دقيقة عن استهلاكك للموارد والتكاليف التقديرية ليها.

شركة AWS بتبعت التقارير دي لـ Amazon S3 أنت اللي بتحددها بنفسك.

التقرير ده بيتحدث مرة واحدة على الأقل كل يوم.

تقرير AWS Cost and Usage Report بيتبع استهلاكك وبيديك التكاليف والرسوم التقديرية المرتبطة بحسابك، وإما بيعرضها لك بالساعة أو باليوم.

Section 3 Summary

لازم تحمي عمليات تسجيل الدخول لكل الحسابات باستخدام (Multi-Factor Authentication)

احذف فوراً أي مفاتيح وصول (Access keys) معمولة لـ Root User عشان تمنع أي اختراق.

اعمل حساب IAM مُستقل لكل شخص شغال معاك، واديه صلاحيات على قد أداء شغلانته بالظبط من غير أي زيادة.

استخدم المجموعات (Groups) عشان تمنح وتدير الصلاحيات لكذا مستخدم مع بعض مرة واحدة بدل ما تعدل على كل مُستخدم لوحده.

اضبط إعدادات الحساب عشان تجبر المُستخدمين يعملوا كلمات مرور قوية ومُعقدة (Strong password policy).

لما تحب تفوض صلاحيات لحد، استخدم الأدوار (Roles) بدل ما تشارك معاه بيانات الأمان والباسوردات بتاعتك.

شغل خدمة AWS CloudTrail عشان تفضل عينك على الحساب وتراقب وتتبع كل الأنشطة والحركات اللي بتحصل جواه أول بأول.

4. Securing accounts

❖ خدمة AWS Organizations



AWS Organizations

هي خدمة بتسمح لك بتجميع وإدارة كذا حساب AWS مع بعض من مكان واحد مركزي.

الميزات الأمنية في الخدمة:

تنظيم الحسابات (OUs):

تقدر تقسم وتجمع حسابات AWS في وحدات تنظيمية اسمها (Organizational Units)، وتربط سياسات وصول مختلفة لكل وحدة لوحدها.

التكامل مع نظام IAM:

الخدمة بتتكامل تماماً مع نظام الـ IAM؛ والصلاحيات الفعلية للمستخدم بتكون هي "نقطة التقاطع" أو الحاجة المشتركة بين المسموح بيه في AWS Organizations والممنوح ليه جوه حساب الـ IAM ده.

سياسات التحكم في الخدمات (Service Control Policies):

بتستخدم السياسات دي عشان تفرض تحكم مركزي على خدمات AWS وعمليات الـ API اللي يقدر أي حساب يدخل عليها.

❖ سياسات التحكم في الخدمات (Service Control Policies - SCPs)

بتوفر تحكم مركزي كامل على الحسابات التي جوه المنظمة.

بتحط حد أقصى للصلاحيات المتاحة جوه الحساب عشان تضمن إن الحسابات ماشية مع معايير التحكم في الوصول.

مهم جداً: الشبه والإختلاف مع سياسات IAM:

هي شبه سياسات ال IAM في طريقة الكتابة وال Syntax، لكن الإختلاف الجوهرى إن ال SCP مُستحيل تمنح صلاحيات؛ هي بس بتحدد الحد الأقصى للصلاحيات المسموحة جوه الحساب.



AWS Key Management
Service (AWS KMS)

❖ خدمة إدارة مفاتيح التشفير (AWS Key Management Service (AWS KMS)

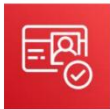
الخدمة دي مُخصصة للتحكم في تشفير البيانات، وميزاتها هي:

بتسمح لك بإنشاء وإدارة مفاتيح التشفير (Encryption keys).

بتخليك تتحكم في عمليات التشفير وفك التشفير عبر خدمات AWS المختلفة أو جوه تطبيقاتك الخاصة.

بتتكامل مع خدمة AWS CloudTrail عشان تسجل وتراقب أي استخدام للمفاتيح دي.

بتستخدم Hardware Security Modules المعتمدة والمُتوافقة مع معايير (FIPS 140-2) لحماية المفاتيح وتأمينها.



Amazon Cognito

❖ خدمة Amazon Cognito

الخدمة دي مُخصصة لإدارة هويات المُستخدمين في التطبيقات.

المُميزات:

بتضيف ميزات إنشاء الحسابات (Sign-up)، تسجيل الدخول (Sign-in)، والتحكم في الوصول لتطبيقات الويب والموبايل بتاعتك.

بتقدر تتوسع تلقائياً عشان تشيل وتخدم ملايين المُستخدمين.

بتدعم تسجيل الدخول عبر شبكات التواصل الاجتماعي (زي Facebook، Google، Amazon)، أو عبر مزودي الهويات للشركات والمؤسسات (زي Microsoft Active Directory) بإستخدام بروتوكول SAML 2.0.



AWS Shield

❖ خدمة الحماية AWS Shield

هي خدمة مُدارة ومُخصصة للحماية من هجمات حجب الخدمة الموزعة (DDoS).

بتحمي وتؤمن التطبيقات الشغالة جوه بيئة AWS.

بتوفر ميزة الرصد والمُراقبة الشغالة دائماً (Always-on detection)، مع مُعالجة وتقليل أثر الهجمات تلقائياً.

المُستويات المتاحة:

- **AWS Shield Standard:** دي النسخة القياسية وبتكون متفعلة تلقائياً لكل العملاء من غير أي تكلفة إضافية.
- **AWS Shield Advanced:** دي نسخة مُتطورة ومدفوعة (اختيارية) بتقدم ميزات حماية أكبر.

بنستخدم الخدمة دي أساساً عشان نقلل من وقت توقف التطبيقات (Downtime) وبطء استجابة الشبكة (Latency) وقت الهجوم.

5. Securing data on AWS

❖ تشفير البيانات أثناء السكون (Encryption of data at rest)

مفهوم التشفير: التشفير يقوم بترميز وتحويل البيانات باستخدام مفتاح سري (Secret key)، وده بيخليها غير قابلة للقراءة تماماً لأي حد بره النظام.

فك التشفير: الأشخاص اللي معاهم المفتاح السري ده بس، هما اللي يقدرُوا يفكُوا التشفير ويقروا البيانات دي تاني.

إدارة المفاتيح: تقدر تستخدم خدمة AWS KMS عشان تدير وتتحكم في المفاتيح السرية بتاعتك دي بالكامل وبأمان.

ما هي البيانات أثناء السكون (Data at rest)؟

هي البيانات المتخزنة فعلياً ومادياً على وسائط التخزين (زي الهارد ديسك "Disks" أو أشرطة التخزين "Tapes").

نظام AWS بيدعم تشفير النوع ده من البيانات بالكامل في أي خدمة مُدمجة ومدعومة من قبل خدمة AWS KMS.

أمثلة على الخدمات المدعومة للتشفير أثناء السكون:

- Amazon S3
- Amazon EBS
- Amazon Elastic File System (Amazon EFS)
- Amazon RDS

❖ تشفير البيانات أثناء الانتقال (Encryption of data in transit)

المقصود بالبيانات أثناء الانتقال (Data in transit) هي البيانات اللي بتتحرك وتتنقل عبر الشبكة من مكان لمكان تاني.

♦ البروتوكولات والخدمات الأساسية للتشفير

بروتوكول TLS (Transport Layer Security): المعروف سابقاً بإسم SSL، وهو بروتوكول قياسي مفتوح ومسؤول عن حماية البيانات أثناء حركتها في الشبكة.

خدمة AWS Certificate Manager: هي الخدمة المسؤولة عن إدارة، ونشر، وتجديد شهادات الرقمية الخاصة بـ TLS أو SSL بشكل تلقائي ومريح.

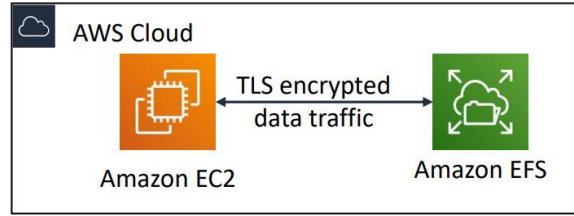
♦ تكنولوجيا الإتصال الآمن (HTTPS)

بروتوكول HTTPS بيقوم بإنشاء نفق مروري آمن ومُشفّر (Secure tunnel) لنقل البيانات.

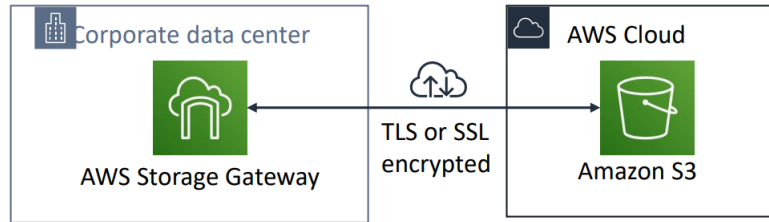
بيعتمد بشكل أساسي على بروتوكولات TLS أو SSL عشان يضمن تبادل البيانات في الإتجاهين (Bidirectional exchange) بين العميل والخادم بكل أمان ومن غير ما حد يقدر يتجسس عليها.

خدمات AWS بتدعم تشفير البيانات وهي بتتنقل بينها وبين بعضها، أو بينها وبين مراكز البيانات الخارجية.

أمثلة توضيحية من الأنظمة:



حركة البيانات الداخلية: التشفير يتم لحركة مرور البيانات بين السيرفرات الافتراضية (Amazon EC2) وأنظمة الملفات المشتركة (Amazon EFS) باستخدام بروتوكول TLS.



حركة البيانات الخارجية والربط: التشفير يحمي البيانات المتنقلة بين مراكز البيانات الخاصة بالشركات (Corporate data center) عبر خدمة (AWS Storage Gateway) وبين Amazon S3 باستخدام شهادات TLS أو SSL.

❖ تأمين S3 buckets and objects

أي Bucket أو ملف (Object) جديد بتعمله على Amazon S3 سيكون خاص ومحمي بشكل افتراضي (Private by default)، ومفيش أي حد يقدر يدخل عليه بره الحساب.

لما طبيعة شغلك تحتاج إنك تشارك الملفات دي مع بره، من الضروري جداً تدير وتتحكم في عملية الوصول للبيانات.

لازم دائماً تطبق مبدأ إعطاء أقل صلاحيات ممكنة وتفكر دائماً في تفعيل تشفير البيانات لزيادة الأمان.

أدوات وآليات التحكم في الوصول لبيانات S3

عندك ه أدوات وآليات أساسية تقدر تستخدمهم عشان تتحكم في مين يشوف أو يعدل على بيانات الـ S3:

1. Amazon S3 Block Public Access

أداة سهلة وبسيطة جداً في الإستخدام.

بتشتغل كحائط صد مركزي بيقفل ويمنع أي محاولة لفتح الحاوية أو الملفات للعامة (Public) بالخطأ، حتى لو فيه صلاحية ثانية بتسمح ب ده.

2. IAM policies

خيار ممتاز جداً لما يكون المستخدم أو التطبيق يقدر يعمل إثبات هوية وتسجيل دخول (Authenticate) باستخدام نظام الـ IAM جوه AWS.

3. Bucket policies

دي سياسات قائمة على الموارد (Resource-based policies) بتكتبها وتربطها في بال Bucket نفسها. بتستخدمها عشان تحدد صلاحيات واسعة أو شروط مُعينة على مستوى الحاوية كلها (زي السماح لحساب AWS تاني بالكامل إنه يدخل على الحاوية).

4. Access Control Lists - ACLs

آلية قديمة للتحكم في الوصول (Legacy mechanism). كانت بتستخدم زمان لتحديد صلاحيات على مستوى ملفات مُعينة جوه الحاوية، وحالياً في أغلب الحالات بينصحوا بالإعتماد على السياسات التانية بدالها.

5. AWS Trusted Advisor

ميزة مجانية بتوفرها لك خدمة Trusted Advisor. الخدمة دي بتعمل فحص وتدقيق أوتوماتيك للحاويات بتاعتك، وبتنبهك فوراً لو فيه أي حاوية مفتوحة للعامة ومكشوفة على الإنترنت بالخطأ عشان تلحق تقفلها.

6. Working to ensure compliance

AWS بتساعد العملاء والشركات على الإلتزام بالقوانين والتشريعات الأمنية والرقابية المُختلفة من خلال توفير معلومات تفصيلية عن عناصر التحكم والسياسات المطبقة في بنية AWS التحتية، وتنقسم لثلاثة أقسام رئيسية:

1. الشهادات والإقرارات (Certifications):

شهادات بيتم تقييمها ومراجعتها بواسطة مُراجعين مُستقلين (طرف ثالث)، مثل شهادات الأيزو (ISO 27001, 27017, 27018).

2. القوانين والتشريعات والخصوصية (Laws & Regulations)

مميزات قانونية وأمنية لدعم الإلتزام بالتشريعات الحكومية، مثل قانون حماية البيانات الأوروبي (GDPR) وقانون التأمين الصحي الأمريكي (HIPAA).

3. المعايير الإسترشادية والنماذج الأمنية (Alignments)

مُتطلبات أمنية خاصة بصناعة أو وظيفة مُعينة، مثل معايير مركز أمن الإنترنت (CIS).

**❖ خدمة AWS Config (أداة التقييم والمراقبة المُستمرة)**

تُستخدم لتقييم ومراجعة وفحص إعدادات (Configurations) موارد AWS الخاصة بك بشكل مُستمر. بتراقب التغييرات وبتقارن تلقائياً بين الإعدادات الفعلية الحالية والإعدادات المثالية المطلوبة (Desired configurations). بتسمح لك بمراجعة أي تعديل حصل ورؤية السجل التاريخي المفصل للموارد. بتبسط وبتهون عمليات التدقيق الأمني والتحليل الرقابي للحساب.



AWS Artifact

❖ خدمة AWS Artifact (مستودع وثائق الإمتثال)

هي المورد والمرجع الأساسي المجاني لكل المعلومات والوثائق القانونية والأمنية المتعلقة بالإمتثال.

بتوفر وصولاً مباشراً لتقارير الأمن والإمتثال الدولية وتتيح قبول الإتفاقيات عبر الإنترنت.

شهادات الأيزو (AWS ISO certifications)، تقارير بطاقات الدفع (PCI)، وتقارير الرقابة الداخلية (SOC).

طريقة الدخول مباشرةً من الـ AWS Console تحت قسم (Security, Identity & Compliance) ثم اختيار Artifact.

Section 6 Summary

برامج الإمتثال: بتوفر للعميل كل المعلومات والبيانات الخاصة بالسياسات والعمليات وعناصر التحكم الأمنية التي بتديرها AWS.

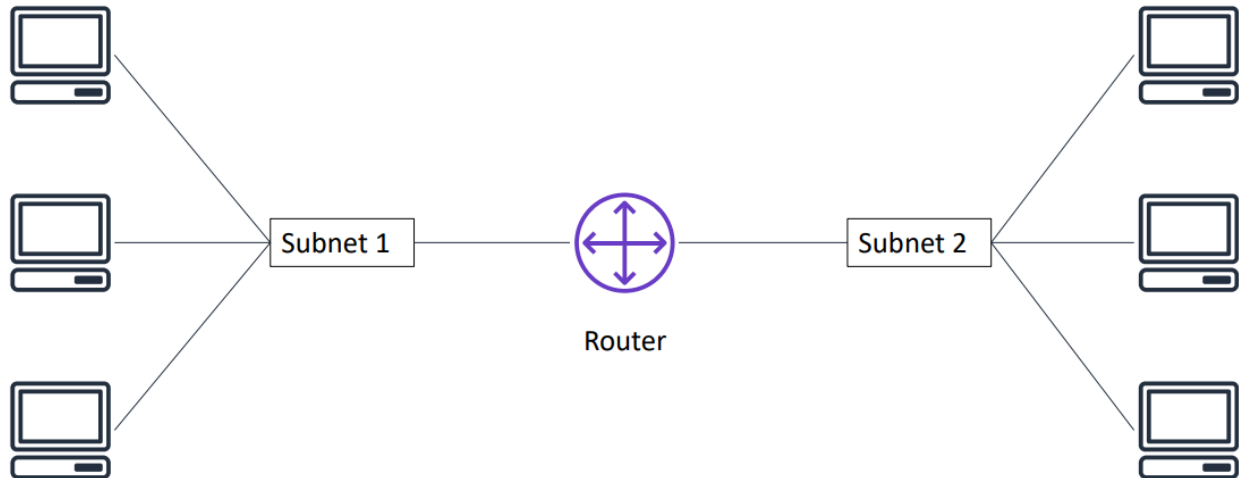
AWS Config: الأداة المُخصصة لتقييم ومراجعة وتدقيق إعدادات ومواصفات موارد AWS.

AWS Artifact: البوابة المركزية المُخصصة للحصول على تقارير الأمن والامتثال المعتمدة.

♦ **Module 5: Networking and Content Delivery**

1. Networking Basics

Networks & Subnets ❖

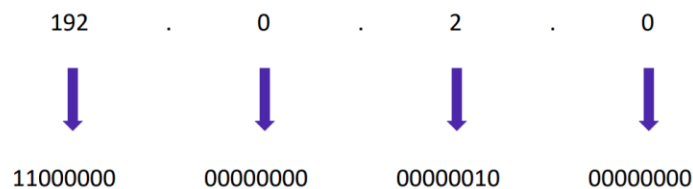


Network: هي النظام الذي يسمح للأجهزة والكمبيوترات إنها تتصل وتتواصل مع بعضها.

Subnet: ال Subnet هي مدى (Range) من عناوين ال IP داخل شبكة أكبر، ويتم تخصيصه لمجموعة معينة من الأجهزة، وكل Subnet لديها بداية ونهاية وعدد مُحدد من عناوين ال IP حسب ال Subnet Mask.

Router: هو الجهاز المسؤول عن ربط الشبكات الفرعية دي ببعضها وتوجيه حركة البيانات بينها.

IP Addresses ❖



جهاز الكمبيوتر يفهم العناوين دي على شكل نظام ثنائي (Binary) مكون من أصفار وواحدات (1s and 0s).

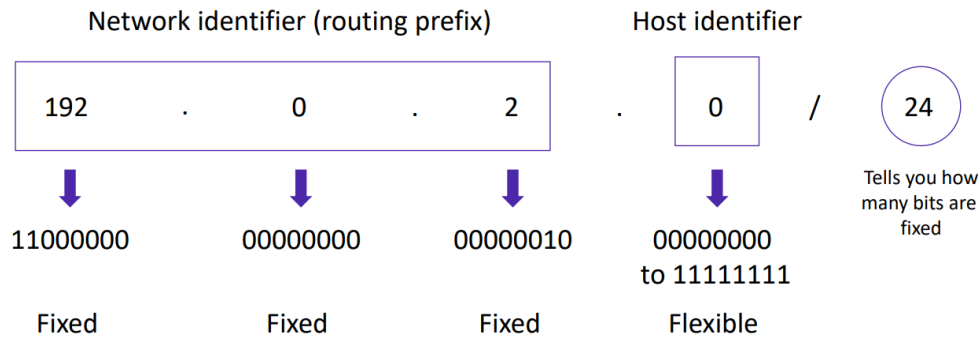
IPv4 (32-bit) address: 192.0.2.0

IPv6 (128-bit) address: 2600:1f18:22ba:8c00:ba86:a05e:a5ba:00FF

IPv4: عنوان بيتكون من 32 بت (32-bit)، ويتم تمثيله بأربع أرقام تفصل بينهم نقطة (مثل: 192.0.2.0).

IPv6: عنوان أكبر بكثير بيتكون من 128 بت (128-bit)، وتعمل عشان يحل مشكلة نفاذ عناوين IPv4.

Classless Inter-Domain Routing (CIDR) ❖



طريقة كتابة عناوين ال IP في الشبكات بتتبع نظام ال CIDR (مثل: 24/192.0.2.0)، وينقسم العنوان هنا لجزأين:

Network identifier / routing prefix: الأرقام الأولى اللي بتكون ثابتة (Fixed) ومش بتتغير جوه النطاق ده.

Host identifier: الجزء المرن (Flexible) اللي أرقامه بتتغير عشان تدي عناوين للأجهزة المختلفة جوه الشبكة (مثلاً من 00000000 إلى 11111111).

الرقم بعد الشرطة (/24): الرقم ده بيعبر عن عدد البتات الثابتة (Fixed bits) في العنوان من ناحية الشمال؛ وكل ما الرقم ده يكبر، عدد العناوين المتاحة للأجهزة بيقل.

Open Systems Interconnection (OSI) model ❖

النموذج ده بيقسم عملية اتصال البيانات عبر الشبكة لـ 7 طبقات (Layers 7)، لكل طبقة وظيفة وبروتوكولات معينة:

1. Layer 1 - Physical Layer

دي الطبقة الأولى، مسؤولة عن نقل البيانات في شكل إشارات (Bits) عبر السلوك أو الواي فاي.

أشهر البروتوكولات/التقنيات: (802.11) Wi-Fi، RJ45، Fiber Optic، Ethernet (Physical).

2. Layer 2 - Data Link Layer

دي الطبقة الثانية، بتنظم البيانات في شكل إطارات (Frames)، وتكتشف الأخطاء، وتتحكم في نقل البيانات داخل نفس الشبكة باستخدام MAC Address.

أشهر البروتوكولات: (802.3) Ethernet، (802.11) Wi-Fi، PPP، (802.1Q) VLAN، STP.

3. Layer 3 - Network Layer

دي الطبقة الثالثة، مسؤولة عن التوجيه (Routing) واختيار أفضل مسار للبيانات بين الشبكات المختلفة باستخدام IP Address.

أشهر البروتوكولات: IP، ICMP، OSPF، RIP، EIGRP، BGP.

4. Layer 4 - Transport Layer

دي الطبقة الرابعة، بتضمن إن البيانات تنتقل من البداية للنهاية بشكل سليم ومرتب، وبتتحكم في تدفق البيانات. أشهر البروتوكولات: TCP، UDP.

5. Layer 5 - Session Layer

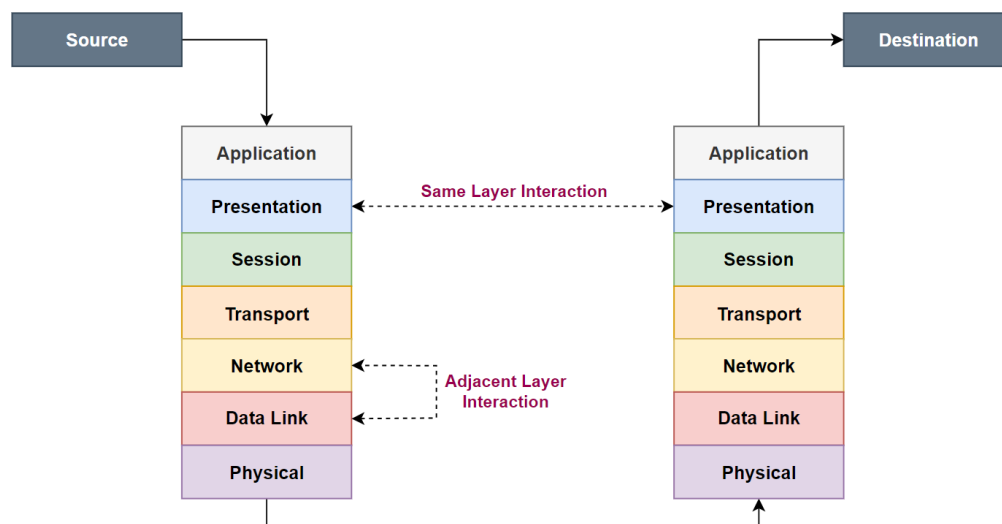
دي الطبقة الخامسة، بتفتح وتدير وتقفّل الجلسات (Sessions) بين الأجهزة عشان البيانات متتداخلش في بعضها. أشهر البروتوكولات: NetBIOS، RPC.

6. Layer 6 - Presentation Layer

دي الطبقة السادسة، بترجم البيانات لصيغة مفهومة، ومسؤولة كمان عن التشفير (Encryption) وفك التشفير (Decryption) وضغط الملفات (Compression). أشهر البروتوكولات/المعايير: SSL/TLS، ASCII، JPEG، GIF، MPEG.

7. Layer 7 - Application Layer

دي الطبقة السابعة، هي اللي بيتعامل معها المستخدم مباشرة، وتتوفر خدمات الشبكة للتطبيقات. أشهر البروتوكولات: HTTP، HTTPS، FTP، DNS، DHCP، SMTP، POP3، IMAP، SSH، Telnet، LDAP.



يبقى الـ OSI Model هو نموذج نظري عملته منظمة ISO علشان ينظم عملية التواصل بين الأجهزة في الشبكة، ويساعدنا نفهم كل مرحلة بتحصل إزاي وإمتى.

2. Amazon VPC

كلمة VPC هي اختصار لـ (Virtual Private Cloud)، وهي الخدمة التي بتخليك تبني شبكتك الافتراضية الخاصة جوه AWS Cloud. الخصائص والمميزات الأساسية:

عزل منطقي كامل: بتسمح لك بحجز مدي معزول تماماً ومنطقياً جوه AWS Cloud عشان تشغل فيها مواردك (زي السيرفرات وقواعد البيانات).



Amazon
VPC

تحكم كامل في الشبكة: الخدمة بتديك صلاحية التحكم الكامل في كل تفاصيل شبكتك الافتراضية، ومنها:

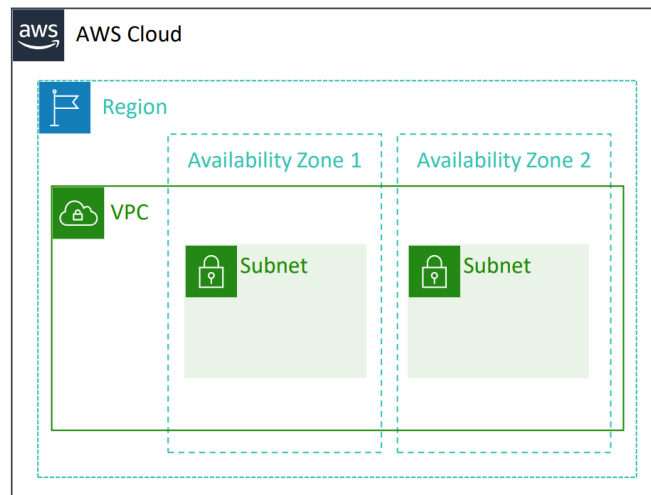
- اختيار نطاق عناوين الـ IP الخاصة بـك (باستخدام نظام الـ CIDR).
- تقسيم الشبكة الكبيرة لشبكات فرعية (Subnets).
- ضبط وإعداد جداول التوجيه (Route Tables) وبوابات الشبكة (Network Gateways).

تخصيص وتعديل مرن: تقدر تفصل وتعديل إعدادات الشبكة بالشكل اللي يناسب تطبيقك أو شركتك.

طبقات أمان متعددة: بتديك القدرة على حماية مواردك بأكثر من خط دفاع وأمن (زي الـ Security Groups والـ NACLs).

الـ VPC بتكون معزولة تماماً عن أي VPC ثانية، وبتكون مخصصة لحساب AWS بتاعك أنت بس.

الـ VPC بتتبع منطقة جغرافية واحدة بس (Single AWS Region) (زي منطقة أيرلندا مثلاً)، لكنها تقدر تتمدد وتغطي كذا مركز بيانات/منطقة إتاحة (Multiple Availability Zones - AZs) جوه المنطقة دي.



الـ Subnets (الشبكات الفرعية)

هي عبارة عن تقسيم لنطاق عناوين الـ IP بتاعة الـ VPC الكبيرة لأجزاء أصغر.

الـ Subnet الواحدة مستحيل تخرج بره مركز البيانات بتاعها؛ يعني بتتبع منطقة إتاحة واحدة بس (Single Availability Zone).

بتنقسم الشبكات الفرعية دي لنوعين حسب اتصالها بالإنترنت:

- شبكة فرعية عامة (Public Subnet): مُتصلة بالإنترنت مباشرةً (زي اللي بنحط فيها سيرفرات الويب).
- شبكة فرعية خاصة (Private Subnet): معزولة تماماً عن الإنترنت الخارجي (زي اللي بنحط فيها قواعد البيانات الحساسة).

توزيع عناوين ال IP جوه ال VPC بيحكمها شوية قواعد ثابتة ومهمة جداً ، وبتنقسم لجزأين:

1. قواعد وقوانين نطاقات ال IP في ال VPC

أول ما بتعمل VPC، لازم تحدد ليها نطاق عناوين IPv4 بإستخدام نظام ال CIDR (ودي بتكون عناوين داخلية private).

مستحيل تغير أو تعدل نطاق العناوين الأساسي لل VPC بعد ما تتركدها خلاص.

الحد الأقصى وال الأدنى للحجم:

x.x.x.x/16 or 65,536 addresses (max)
to
x.x.x.x/28 or 16 addresses (min)

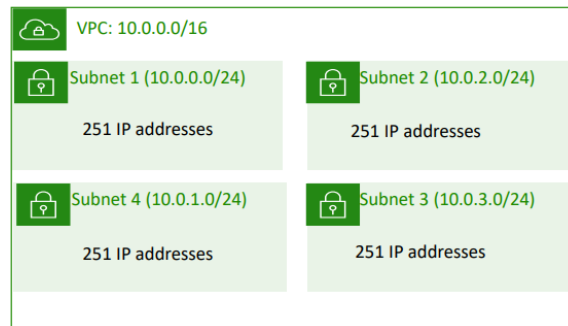
- أكبر حجم مسموح به: (/16)، وده بيديك 65,536 عنوان IP.
- أصغر حجم مسموح به: (/28)، وده بيديك 16 عنوان IP بس.

نظام AWS بيدعم ال IPv6 كمان، بس بحدود وأحجام مختلفة لل CIDR block.

لما تقسم ال VPC ل Subnets، مستحيل نطاقات ال CIDR بتاعة ال Subnets دي تتداخل أو تدخل في قيع بعضها.

2. العناوين المحجوزة في ال Subnet (Reserved IP Addresses)

في أي Subnet بتعملها جوه AWS، شركة AWS بتحجز تلقائياً 5 عناوين IP لإستخداماتها الخاصة، العناوين دي مستحيل أنت كعميل تقدر تديهم لأي سيرفر أو جهاز عندك.



لو عندك Subnet النطاق بتاعها هو 10.0.0.0/24 (يعني حسابياً بتشيل 256 عنوان)، عدد العناوين المتاحة الفعلي للإستخدام هيكون 251 عنوان بس (251 = 5 - 256).

ال 5 عناوين المحجوزين:

10.0.0.0: محجوز كعنوان الشبكة (Network address).

10.0.0.1: محجوز لل VPC Local Router (عشان التواصل الداخلي مع الراوتر الافتراضي).

10.0.0.2: محجوز لخدمة ال DNS (عشان حل وتحويل أسماء النطاقات جوه ال VPC).

10.0.0.3: محجوز بواسطة AWS للإستخدامات المستقبلية (Future use).

10.0.0.255: محجوز كعنوان بث الشبكة (Network broadcast address)، ورغم إن AWS مش بتدعم ال Broadcast التقليدي، بس يفضل محجوز.

❖ أنواع عناوين ال IP العامة وكروت الشبكة الافتراضية

عند التعامل مع السيرفرات (Instances) جوه ال VPC، بنحتاج ساعات نديها عناوين IP عامة (Public IPs) عشان تقدر تتواصل مع الإنترنت الخارجي.

1. أنواع عناوين ال IPv4 Public

عشان السيرفر بتاعك ياخذ عنوان IPv4 عام، عندك طريقتين أساسيتين:

Automatically assigned: بيتم تلقائياً عن طريق تشغيل إعدادات (Auto-assign public IP) على مستوى ال Subnet.

أول ما السيرفر يقوم بياخذ IP، بس العيب هنا إنه عنوان "ديناميكي" يعني بيتغير لو السيرفر اتعمله إيقاف وتشغيل (Stop / Start).

Manually assigned: بيتم يدوياً عن طريق حجز واستخدام عنوان من نوع Elastic IP address.

2. ال Elastic IP Address

هو حل مُشكلة تغير ال IP الديناميكي، وخصائصه كالتالي:

ثابت وعام (Static & Public): عنوان IPv4 عام ومش بيتغير أبداً مهما قفلت السيرفر وفتحته.

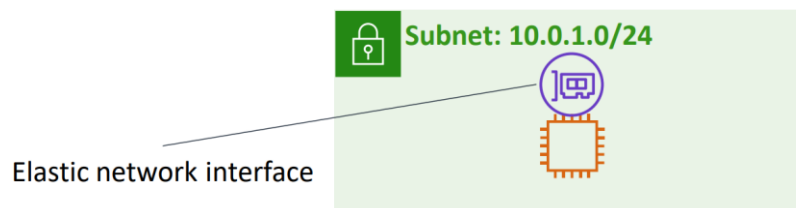
مرتبط بالحساب: ال IP ده بيتحجز ويفضل مربوط بحساب ال AWS بتاعك مش بسيرفر مُعين.

مرونة الربط: تقدر تربطه (Associate) بأي سيرفر أو كارت شبكة (Network interface) جوه أي VPC في حسابك.

إعادة التوجيه (Remapped): تقدر تفكه من سيرفر باظ أو وقع، وتربطه فوراً بسيرفر تاني شغال في أي وقت عشان يرجع الخدمة للناس بسرعة.

التكلفة: ممكن يطبق عليه مصاريف إضافية (AWS بتديهولك مجاناً طول ما هو مربوط بسيرفر شغال، لكن لو حجزته وسبته من غير ما تربطه بسيرفر، بتدفع عليه فلوس عشان ميتسبش محجوز على الفاضي).

❖ كارت الشبكة الافتراضي (Elastic Network Interface - ENI)



ال ENI هو عبارة عن كارت شبكة افتراضي (Virtual network interface) زي الكارت الفيزيائي الي بتركبه في الكمبيوتر في الحقيقة، وتقدر تعمل بيه الآتي:

تقدر تتركبه (Attach) في سيرفر، وتقدر تفكه (Detach) من السيرفر ده وتتركبه في سيرفر تاني خالص عشان تحول وتوجه حركة البيانات (Redirect network traffic) بسرعة لجهاز تاني.

لما بتفك الكارت وتتركبه في سيرفر جديد، كل الخصائص والمواصفات بتاعته بتتنقل معاه (زي ال IP الخاص وال MAC Address وال Security Groups).

أي سيرفر (Instance) بتعمله جوه ال VPC بيكون عنده كارت شبكة افتراضي أساسي وافتراضي (Primary/Default network interface)، والكارت ده بياخذ تلقائياً عنوان Private IPv4 مُستقطع من نطاق العناوين بتاع ال Subnet الي السيرفر شغال فيها.

❖ جداول التوجيه والمسارات (Route Tables and Routes)

عشان البيانات تعرف تمشي جوه ال VPC وتوصل للمكان الصح، بنحتاج زي "عسكري مرور" أو خريطة توجيه، وده بالضبط دور جداول التوجيه (Route Tables).

♦ ما هو جدول التوجيه (Route Table)؟

هو عبارة عن مجموعة من القواعد أو المسارات (Routes) اللي بتضبطها وتهيئها عشان توجيه حركة البيانات الخارجة من الشبكة الفرعية (Subnet) بتاعتك.

كل سطر أو قاعدة جوه الجدول بتحدد حاجتين أساسيتين:

- **الوجهة (Destination):** النطاق أو عنوان ال IP اللي البيانات رايقاله (مثلاً: إنترنت خارجي، أو سيرفر تاني).
- **الهدف (Target):** البوابة أو المعبر اللي البيانات هتتمر منه عشان تروح للوجهة دي (Gateway أو كارت شبكة).

Main (Default) Route Table

Destination	Target
10.0.0.0/16	local

VPC CIDR block

♦ القواعد الثابتة لجدول التوجيه في AWS

بشكل تلقائي وبدون تدخل منك، أي جدول توجيه بتعمله بيكون جواه سطر ثابت اسمه local.

السطر ده هدفه السماح لجميع الموارد والسيرفرات اللي جوه نفس ال VPC إنها تلمح وتكلم بعضها داخلياً.

شكل السطر الافتراضي في الجدول:

- Destination: 10.0.0.0/16 (نطاق ال VPC كله)
- Target: local

كل شبكة فرعية (Subnet) بتعملها لازم ولا بد تكون مُرتبطة بجدول توجيه (Route Table).

ال Subnet الواحدة تقدر ترتبط بجدول توجيه واحد بس كحد أقصى في نفس الوقت (مُستحيل ال Subnet تتبع جدولين)، في المُقابل، الجدول الواحد ينفع يخدم كذا Subnet عادي جداً.

♦ جدول التوجيه الرئيسي/الافتراضي (Main Route Table)

أول ما بتعمل VPC جديدة، شركة AWS بتعملك تلقائياً جدول توجيه أساسي بيسموه ال Main Route Table.

لو أنت عملت Subnet جديدة وما ربطهاش يدوياً بجدول توجيه مُعين، ال Subnet دي بتروح تلقائياً وترتبط بال Main Route Table ده كوضع افتراضي.

Section 2 Summary

ال VPC: هي عبارة عن شبكة معزولة تماماً ومنطقياً (Logically isolated) جوه سحابة AWS لتشغيل مواردك بأمان.

حدود ال VPC الجغرافية: ال VPC تتبع منطقة جغرافية واحدة بس (One Region)، ولازم تحدد لها النطاق العناوين الأساسي بتاعها (CIDR block).

تجزئة ال VPC: ال VPC الكبيرة بتنقسم وتتجزأ داخلياً لقطع أصغر اسمها شبكات فرعية (Subnets).

حدود ال Subnet الجغرافية: ال Subnet الواحدة تتبع منطقة إتاحة واحدة بس (One Availability Zone)، ولازم هي كمان يتحدد لها نطاق عناوين (CIDR block) مُستقطع من ال VPC وبدون تداخل.

جدول التوجيه (Route Tables): جداول التوجيه هي المسؤولة تماماً عن التحكم في حركة البيانات وتوجيهها من وإلى ال Subnet.

كل جدول توجيه جواه مسار محلي مدمج (Built-in local route) عشان يسمح للموارد اللي جوه ال VPC بتلميح وتكليم بعضها تلقائياً.

تقدر تضيف مسارات إضافية يدوياً للجدول حسب رغبتك (زي مسار يوجه البيانات لبوابة الإنترنت).

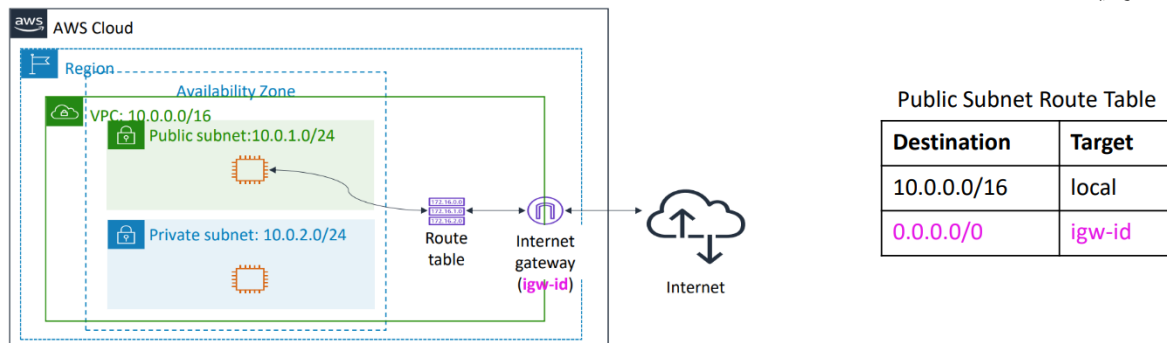
مُستحيل وبأي شكل من الأشكال مسح أو حذف المسار المحلي (Local route) من جدول التوجيه؛ ده خط أحمر ومحمي من السيستم دايماً.

3. VPC Networking

Internet Gateway – IGW ❖

هو مكون برمجي داخل ال VPC، يتميز بأنه مرن، ويتحمل الأعطال تلقائياً (Scalable, redundant, and highly available).

الوظيفة: هو الجسر أو البوابة الأساسية اللي بتسمح بالاتصال المُتبادل (رايح جاي) بين السيرفرات اللي جوه ال VPC وبين شبكة الإنترنت الخارجية.



❖ كيف يتم تحويل ال Subnet إلى Public Subnet؟

بمجرد ما نربط ال Internet Gateway بال VPC، بنروح لجدول التوجيه (Route Table) الخاص بال Subnet ونضيف مسار (Route) جديد بالشكل ده:

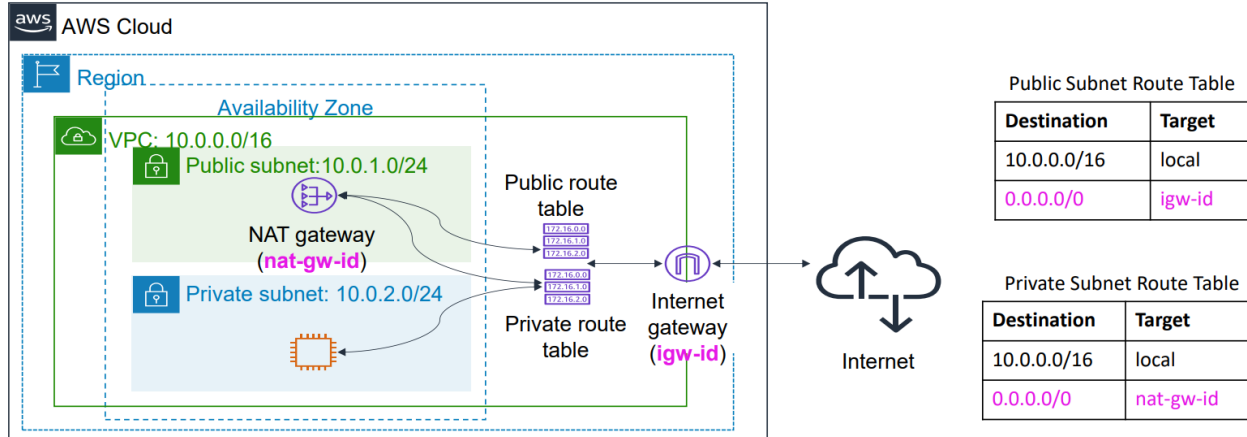
- **Destination:** 0.0.0.0/0 (وهذا التعبير يعني أي مكان في العالم بره ال VPC أو الإنترنت بالكامل)
- **Target:** مُعرف ال Internet Gateway الخاصة بك (مثل: igw-id).

أي Subnet جدول التوجيه الخاص بيها بيحتوي على مسار يوجه ال 0.0.0.0/0 إلى ال igw-id، تبقي ال Public Subnet.

Network address translation (NAT) gateway ❖

السيرفرات الموجودة في الشبكة الفرعية الخاصة (Private Subnet) مثل سيرفرات قواعد البيانات، مینفعش نربطها بالإنترنت مباشرةً عشان متبقاش مكشوفة للهكرز، لكن في نفس الوقت السيرفرات دي محتاجة تطلع إنترنت عشان تنزل تحديثات نظام التشغيل (Patches أو Updates).

الحل هو **NAT Gateway** هي خدمة وظيفتها السماح للسيرفرات اللي جوه ال Private Subnet إنها تطلع وتتصل بالإنترنت الخارجي أو خدمات AWS الأخرى، ولكنها تمنع الإنترنت الخارجي تماماً من بدء أي اتصال مباشر مع هذه السيرفرات (الاتصال بيطلع من جوه لبره بس، ومينفعش يجي من بره لجوه).



♦ أين يتم إنشاؤها؟

لازم ال NAT Gateway يتم إنشاؤها ووضعها جوه Public Subnet عشان تقدر تشوف الإنترنت.

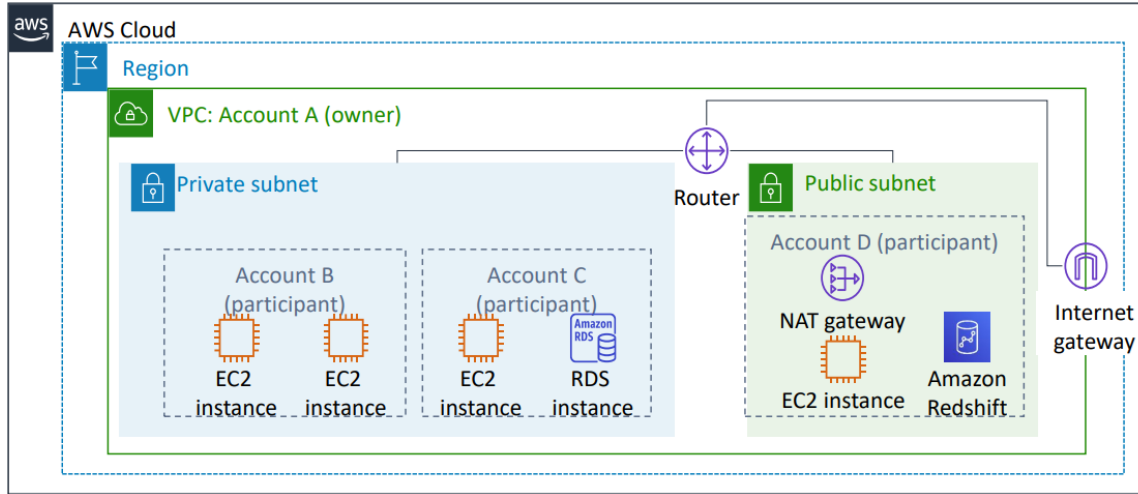
طريقة ضبط جداول التوجيه (مهمة جداً):

- في جدول ال Public Subnet: ال 0.0.0.0/0 يرمي على ال (Internet Gateway) igw-id.
- في جدول ال Private Subnet: ال 0.0.0.0/0 يرمي على ال (NAT Gateway) nat-gw-id.

وال NAT Gateway لما يكون عاوز يوصل ل 0.0.0.0/0 هيرج علي Internet Gateway، لأن ال Nat Gateway موجود في ال Public Subnet اللي بتوصل ل 0.0.0.0/0 عن طريق ال Internet Gateway.

VPC Sharing ❖

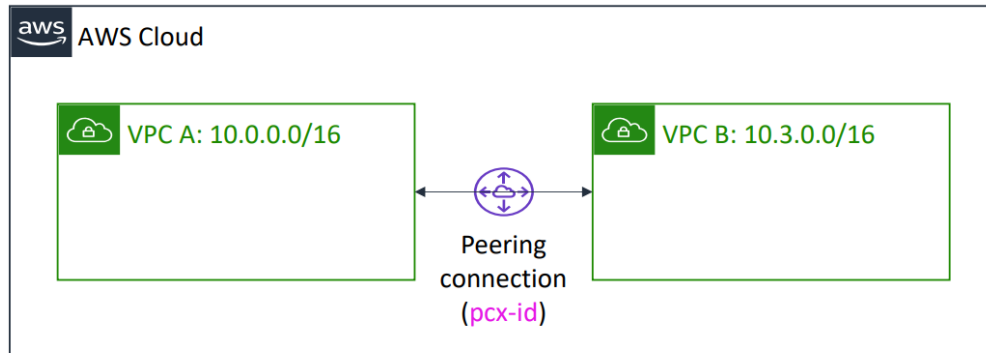
ميزة بتمكن الشركات من مُشاركة شبكات الفرعية (Subnets) مع حسابات AWS ثانية تابعة لنفس المُنظمة (AWS Organization).
بتخلي كذا حساب AWS مُختلفين يقدروا يكرتوا ويشغلوا الموارد بتاعتهم (زي سيرفرات EC2، قواعد بيانات RDS، مستودعات Redshift، والدوال البرمجية Lambda) جوه شبكة VPC واحدة مُشتركة ومُدارة مركزياً.



الحساب المالك (Owner - Account A): هو الحساب الأساسي اللي بيمتلك ال VPC وال Subnets ويربط بيهم بوابة الإنترنت وال NAT.

الحسابات المشاركة (Participants - Accounts B, C, D): دي حسابات ثانية خالص، بس مسموح ليها تدخل تعمل سيرفراتها وشغلها جوه ال Subnets المعزولة والمؤمنة دي مباشرةً.

VPC Peering ❖



Route Table for VPC A

Destination	Target
10.0.0.0/16	local
10.3.0.0/16	pcx-id

Route Table for VPC B

Destination	Target
10.3.0.0/16	local
10.0.0.0/16	pcx-id

هو اتصال شبكي (Peering connection) يربط بين شبكتين VPC مختلفتين تماماً، والهدف منه توجيه حركة البيانات بين الشبكتين دول بشكل خاص وآمن تماماً (Privately) عبر شبكة AWS الداخلية وبدون الحاجة للمرور بالإنترنت العام.

تقدر تعمل ال Peering ده بين شبكتين جوه حسابك أنت، أو بين شبكة في حسابك وشبكة في حساب عميل تاني، أو حتى بين شبكتين في منطقتين جغرافيتين مختلفتين (Cross-Region).

إعداد جدول التوجيه (Route Table):

عشان ال Peering يشتغل، لازم تضيف مسار في جدول كل شبكة يرمي على الثانية باستخدام مُعرف الإتصال (pcx-id)؛ مثلاً:

- جدول ال VPC A يوجه البيانات الرايحة لنطاق VPC B (16/10.3.0.0) إلى ال pcx-id.
- جدول ال VPC B يوجه البيانات الرايحة لنطاق VPC A (16/10.0.0.0) إلى ال pcx-id.

شروط وقيود ال VPC Peering (مُهمة جداً):

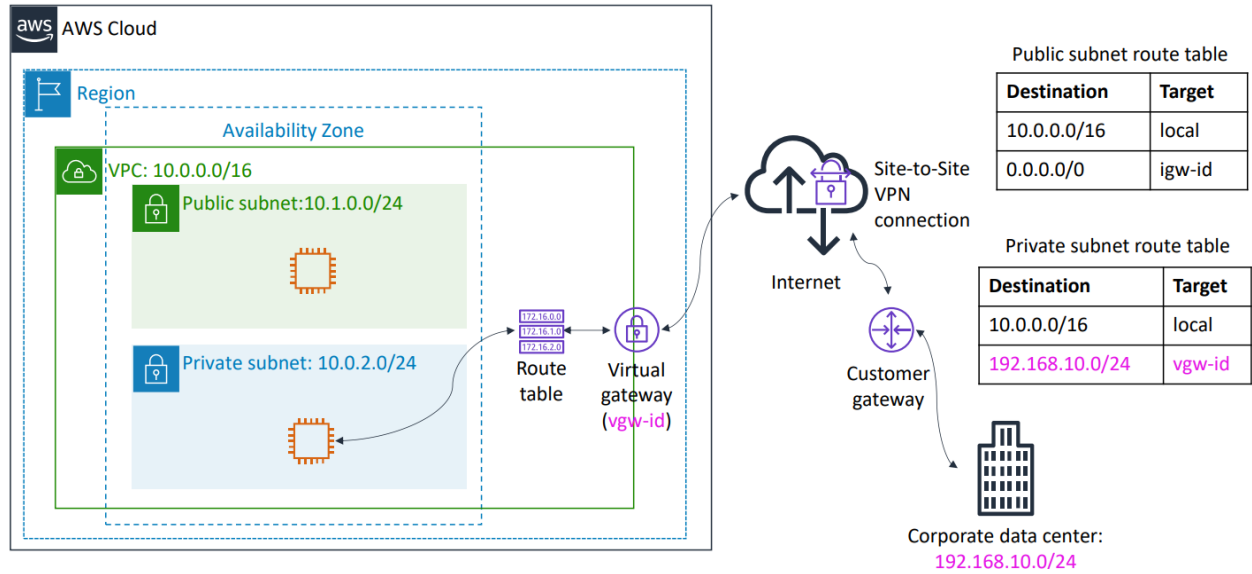
ممنوع تداخل العناوين (No Overlapping IPs): مُستحيل تعمل Peering بين شبكتين ليهم نفس نطاق ال IP أو ال CIDR block.

الإتصال غير متعدٍ (No Transitive Peering): لو شبكة (أ) متصلة بـ (ب)، وشبكة (ب) متصلة بـ (ج)، ده مش معناه إن (أ) تقدر توصل لشبكة (ج)؛ لو عاوز (أ) تتواصل مع (ج) لازم تعمل خط اتصال Peering مُباشر ومخصوص بينهم.

لا يمكنك إنشاء أكثر من خط اتصال Peering واحد بين نفس الشبكتين.

AWS Site-to-Site VPN ❖

هي طريقة لإنشاء اتصال آمن ومُشفّر بين شبكة الشركة وال VPC عبر شبكة الإنترنت العامة باستخدام بروتوكول IPsec الآمن.



المكونات الأساسية للإتصال (مُهمّة جداً):

البوابة الافتراضية الخاصة (Virtual Private Gateway - VGW): دي البوابة الي بنعملها ونربطها من ناحية ال VPC جوه AWS (معرفها بالجدول vgw-id).

بوابة العميل (Customer Gateway - CGW): ده بيمثل جهاز الراوتر أو الفاير وول الحقيقي الي موجود من ناحية شركتك (Corporate data center).

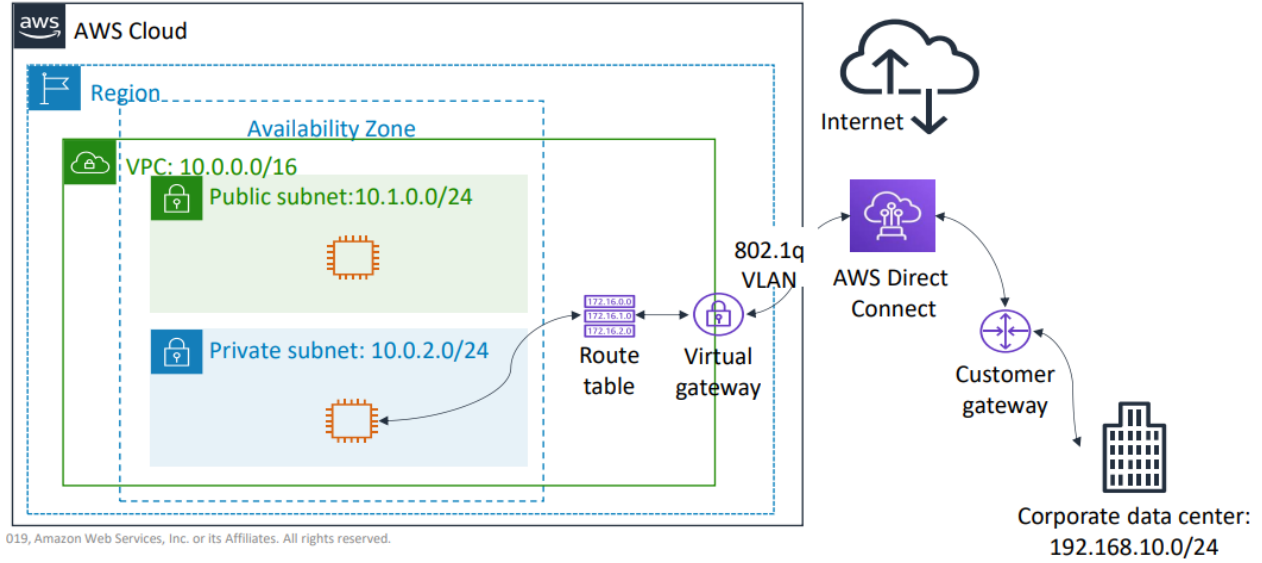
إعداد جدول التوجيه (Route Table):

عشان السيرفرات في ال Private Subnet تقدر توصل للأجهزة الي في الشركة (مثلاً نطاق أجهزتهم 24/192.168.10.0)، بنضيف مسار في ال Private route table:

- Destination: 192.168.10.0/24
- Target: vgw-id

AWS Direct Connect ❖

بتخليك تعمل خط ربط فيزيائي مُخصص ومُباشر (Dedicated physical network connection) بين شبكة شركتك وبين AWS خلال كابلات ألياف ضوئية مُخصصة، تماماً بره شبكة الإنترنت العامة.



المُميزات والفوائد: ❖

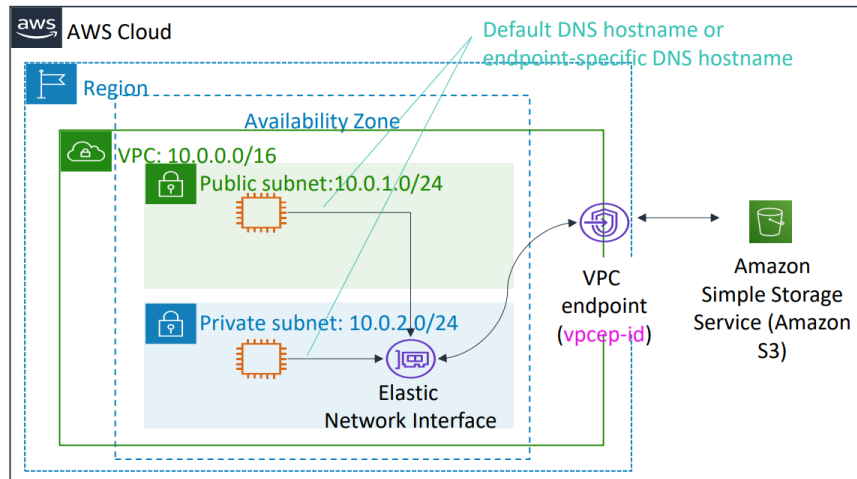
تقليل تكاليف الشبكة: بيوفر في مصاريف نقل البيانات الضخمة (Bandwidth costs).

زيادة سرعة نقل البيانات (Throughput): بيديك سرعات عالية جداً وثابتة.

تجربة شبكة مُستقرة (Consistent experience): لأن البيانات مش بتمشي في نفس مسارات الإنترنت العام، فال Latency (زمن التأخير) بيكون ثابت ومضمون.

طريقة العمل الفنية: بتستخدم معايير شبكات الـ VLAN القياسية وتحديداً بروتوكول 802.1Q للربط الافتراضي عبر البوابة الافتراضية (Virtual Private Gateway).

VPC Endpoints ❖



Public Subnet Route Table

Destination	Target
10.0.0.0/16	local
Amazon S3 ID	vpcep-id

هو جهاز افتراضي (Virtual device) يسمح لك بربط الـ VPC بتاعتك بخدمات AWS الثانية (زي خدمة التخزين Amazon S3 أو قاعدة بيانات DynamoDB) بشكل خاص وسري تماماً.

الإتصال ده مش بيحتاج Internet gateway، ولا جهاز NAT، ولا اتصال VPN، ولا Direct Connect.

البيانات بتفضل جوه شبكة AWS الداخلية اللي بتكون غير مُنفصلة عن الإنترنت العام.

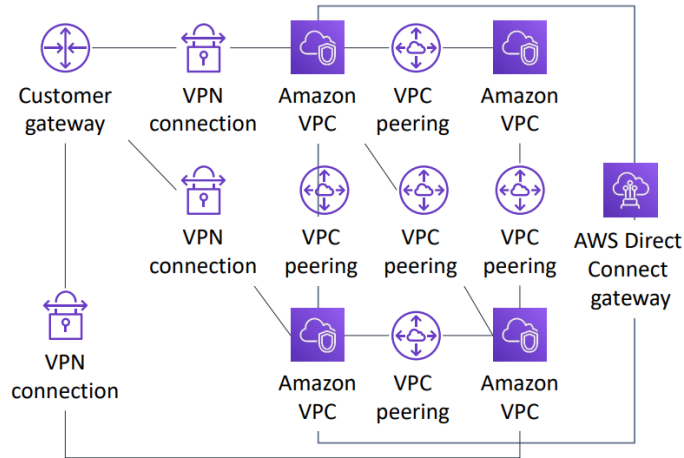
❖ في نوعين من الـ VPC Endpoints:

Interface Endpoints: دي بتشتغل بتكنولوجيا اسمها AWS PrivateLink، وفكرتها إنها بتحجز كارت شبكة افتراضي (ENI) بـ IP داخلي جوه الـ Subnet بتاعتك عشان تكلم الخدمة من خلاله.

Gateway Endpoints: دي مُخصصة لخدمتين اتنين بس في AWS ومفيش غيرهم: (Amazon S3 و Amazon DynamoDB). بتشتغل عن طريق إضافة مسار (Route) تلقائي في جدول التوجيه يرعي على معرف الـ Endpoint (مثل: vpcep-id).

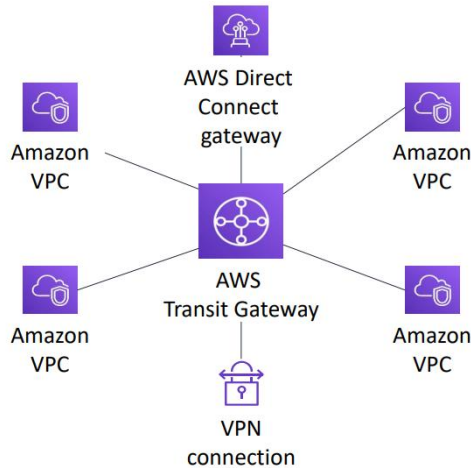
AWS Transit Gateway ❖

From this...



المُشكلة لما شركتك تكبر ويكون عندك شبكات VPC كثيرة جداً، وعاوز تربطهم كلهم ببعض، وتربطهم كمان بال VPN وال Direct Connect بتاع شركتك؛ كنت بتضطر تعمل شبكة مُعقدة جداً من ال VPC Peering والإتصالات المُتداخلة الي الإدارة بتاعتها شبه مُستحيلة بسبب شرط أن ال Peering غير متعدٍ (Non-transitive).

To this...



الحل هو استخدام **AWS Transit Gateway**.

ال Transit Gateway بتشتغل كسنترال مركزي أو موزع رئيسي (Cloud router) في نص الشبكة.

بتعمل خط اتصال واحد بس من كل VPC أو مركز بيانات أرضي وتوصله بالبوابة المركزية دي (AWS Transit Gateway).

وهي بتتولى تلقائياً توجيه البيانات وتوصيل الكل ببعضه من نقطة إدارة واحدة مركزية (Centrally managed).

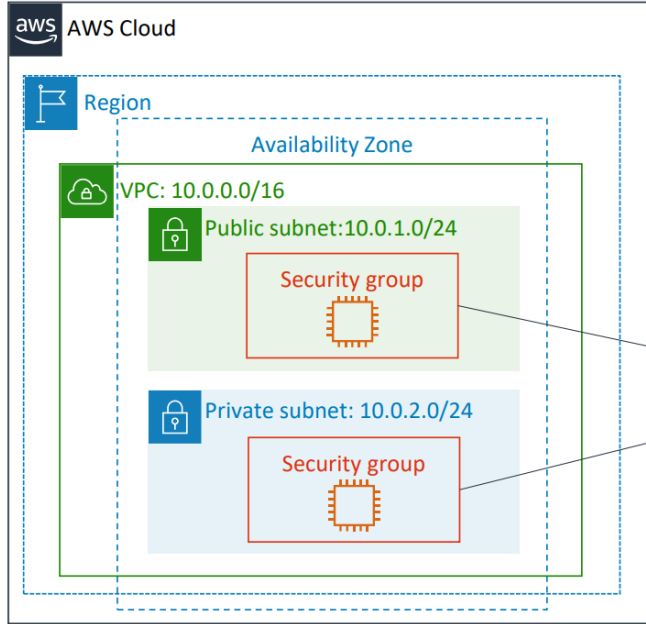
النتيجة: بتبسط تصميم الشبكة جداً، وبتلغي تعقيدات ال VPC Peering تماماً.

4. VPC Security

Security Group ❖

هي عبارة عن فايرول افتراضي (Virtual Firewall) مُخصص للسيرفرات بتاعتك.

بنتحكم بشكل كامل في حركة البيانات الداخلة (Inbound traffic) والخارجة (Outbound traffic) من وإلى السيرفر.



Security groups act at the **instance level**.

تعمل على مستوى السيرفر (Acts at the Instance level)، دي أهم قاعدة؛ ال Security Group بتشتغل وتحمي السيرفر نفسه (كارت الشبكة بتاع السيرفر) مش ال Subnet.

بُناءً على القاعدة اللي فوق، ده معناه إن لو عندك سيرفرين محطوتين جوه نفس ال Subnet، تقدر عادي جداً تدي لكل سيرفر منهم Security Groups مُختلفة تماماً وقواعد حماية مُنفصلة عن الثاني.

ال Security Groups بتحتوي على جداول تسمى القواعد (Rules).

Inbound				
Type	Protocol	Port Range	Source	Description
All traffic	All	All	sg-xxxxxxx	
Outbound				
Type	Protocol	Port Range	Source	Description
All traffic	All	All	sg-xxxxxxx	

القواعد دي بتنقسم لنوعين:

- Inbound Rules: قواعد بتحدد مين مسموح له يدخل للسيرفر.
- Outbound Rules: قواعد بتحدد السيرفر مسموح له يكلم مين بره.

في ال Security Groups، أنت بتضيف قواعد للسماح فقط (Allow rules)، ومينفعش تعمل قاعدة للمنع (Deny rule) بشكل صريح.

أول ما بتعمل ال Security Group الافتراضية، بيكون فيها قواعد (Rules):

- حظر كل الدخول (Deny all inbound traffic): كوضع افتراضي، أي اتصال جاي من بره لجوه السيرفر بيكون ممنوع ومرفوض تماماً، لحد ما أنت تدخل بنفسك وتسمح ببورت مُعين (زي بورت 80 للويب).
- السماح بكل الخروج (Allow all outbound traffic): كوضع افتراضي، السيرفر مسموح له يخرج ويكلم أي حاجة بره في العالم بدون قيود.

ال Security Groups بتتميز بإنها Stateful.

♦ يعني ايه Stateful عملياً؟

يعني لو في اتصال جاي من بره (Inbound) وأنت وافقت عليه وسمحت له يدخل من خلال ال Inbound Rules، ال Security Group ذكية كفاية إنها تفتكر الإتصال ده وتسمح لبيانات الرد بالخروج (Outbound) تلقائياً عشان ترجع للمستخدم، حتى لو كانت ال Outbound Rules عندك قافلة كل حاجة.

والعكس صحيح؛ لو السيرفر خرج يكلم سيرفر بره (Outbound)، الرد هيدخل دائماً تلقائياً.

سؤال، لو عملت سيرفر ويب وفتحت بورت ال HTTP (بورت 80) في ال Inbound عشان الناس تشوف الموقع، وقمت ماسح كل السطور اللي في ال Outbound وعملت Block لكل الخروج. هل المشتركين هيعرفوا يفتحوا الموقع؟

الإجابة: نعم، هيفتح عادي جداً! لأن ال Security Group نوعها Stateful، فبما أن طلب المستخدم مسموح له يدخل، الرد هيخرج له تلقائياً ومش محتاج تفتح له بورت في ال Outbound.

Inbound				
Type	Protocol	Port Range	Source	Description
HTTP	TCP	80	0.0.0.0/0	All web traffic
HTTPS	TCP	443	0.0.0.0/0	All web traffic
SSH	TCP	22	54.24.12.19/32	Office address
Outbound				
Type	Protocol	Port Range	Source	Description
All traffic	All	All	0.0.0.0/0	
All traffic	All	All	::/0	

♦ قواعد الدخول (Inbound Rules):

السطر الأول والثاني (تصفح الموقع): تم فتح بورت HTTP (بورت 80) وبورت HTTPS (بورت 443)، وجعل المصدر (Source) هو 0.0.0.0/0. ده معناه السماح لأي شخص في العالم عبر الإنترنت بالدخول وتصفح الموقع.

السطر الثالث (التحكم عن بُعد): تم فتح بورت SSH (بورت 22) (المستخدم للتحكم في سيرفرات لينكس)، ولكن لاحظ المصدر هنا! المصدر هو IP مُحدد وخاص جداً 54.24.12.19/32 (ومعني 32/ إن ال IP ده فقط).

♦ قواعد الخروج (Outbound Rules):

تم السماح بـ All traffic (كل أنواع البيانات) لكل العناوين في العالم سواء IPv4 (0.0.0.0/0) أو IPv6 (::/0). ده بيضمن إن السيرفر يقدر يرد على المُستخدمين أو يحمل أي ملفات يحتاجها بدون قيود شبكية.

ملاحظات مهمة:

تقدر تحدد قواعد للسماح فقط، ومفیش هنا قاعدة تكتب فيها "امنع ال IP الفلاني"، المنع بيحصل تلقائياً لأي حاجة أنت ما سمحتش بيها برمجياً.

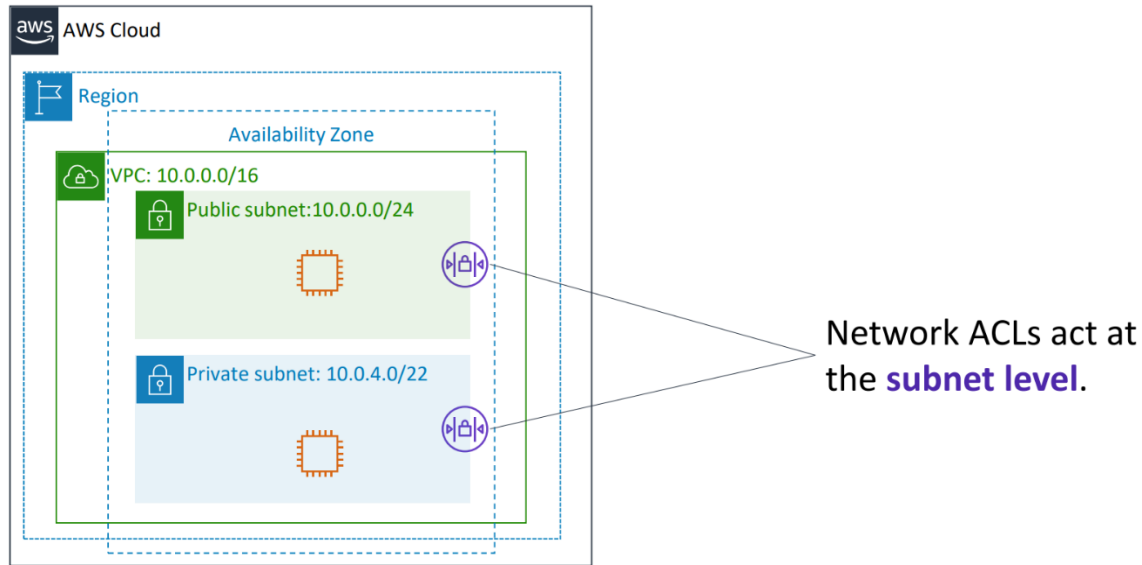
لما البيانات (Packet) بتوصل للسيرفر، ال Security Group بتفحص كل السطور والقواعد الموجودة في الجدول بالكامل قبل ما تأخذ القرار النهائي بالسماح بمرورها أو حظرها، يعني مفیش ترتيب سطور أو أولوية سطر عن سطر، الكل بيتم تقييمه مع بعضه. ممنوع تماماً فتح بورت ال SSH أو ال RDP لل 0.0.0.0/0 (العالم كله)، لازم يتم تحديده ل IP الإدارة فقط لحماية السيرفر من الإختراق.

سؤال، بُناءً على السلايد، لو هاكلر IP بتاعه 1.2.3.4 حاول يعمل اتصال SSH (بورت 22) بالسيرفر ده، إيه اللي هيحصل؟
الإجابة: ال Security Group هتعمل له حظر (Block/Deny) تلقائياً؛ لأنها فحست كل السطور ولقت إن ال SSH مسموح بيه فقط لل IP بتاع المكتب 54.24.12.19/32 ، وبما إنه مش مكتوب له سطر سماح، بيتم رفضه فوراً.

❖ Network Access Control Lists (Network ACLs)

هي طبقة أمان إختيارية تضاف إلى ال VPC الخاص بك.

تعمل بمثابة فايروول (Firewall) كامل للتحكم في حركة البيانات الصادرة والواردة على مستوى Subnet واحدة أو أكثر (Subnets).



تعمل على مستوى ال Subnet Level على عكس Security Groups التي تحمي السيرفر نفسه، ال Network ACL بيتم تطبيقها مباشرةً على حدود ال Subnet.

ده معناه إن أي بيانات (Packet) تحاول تدخل لل Subnet أو تخرج منها، لازم تعدي الأول وتتفحص عن طريق ال Network ACL. لو ال NACL سمحت ليها بالمرور، البيانات بتتوجه بعد كده للسيرفر عشان تصطدم بخط الدفاع الثاني وهو ال Security Group.

Inbound					
Rule #	Type	Protocol	Port Range	Source	Allow/Deny
100	All IPv4 traffic	All	All	0.0.0.0/0	ALLOW
*	All IPv4 traffic	All	All	0.0.0.0/0	DENY
Outbound					
Rule #	Type	Protocol	Port Range	Source	Allow/Deny
100	All IPv4 traffic	All	All	0.0.0.0/0	ALLOW
*	All IPv4 traffic	All	All	0.0.0.0/0	DENY

كل سطر في الجدول يحدد بدقة رقم القاعدة (# Rule)، نوع البيانات (Protocol)، المنفذ (Port Range)، المصدر (Source)، والقرار النهائي بالسماح أو المنع (Allow/Deny).

♦ الترتيب الرقمي للقواعد (Numbered Rules)

القواعد بترتيب بأرقام واضحة (زي قاعدة رقم 100).

السيستم يقرأ الجدول بالترتيب التنازلي تصاعدياً (من أصغر رقم لأكبر رقم)، وأول قاعدة بتطابق البيانات بتطبق فوراً والسيستم يوقف قراءة.

أي جدول NACL لازم ينتهي بقاعدة رقمها نجمة (*)، القاعدة دي بتشتغل أوتوماتيك لعمل منع وحظر (DENY) لأي حركة بيانات لم تتوافق مع القواعد الرقمية الي فوقها.

الوضع الافتراضي لل NACL الأساسية الي بتنزل مع الحساب بتسمح تلقائياً بكل حركة البيانات الداخلة والخارجة ل IPv4.

♦ ال Network ACLs هي Stateless.

ده معناه إنها مش بتسجل ال Sessions؛ لو سمحت لبيانات بالدخول في ال Inbound، لازم تعمل قاعدة مخصصة تسمح برجع الرد في جدول ال Outbound، وإلا الرد هيتحظر.

Inbound					
Rule #	Type	Protocol	Port Range	Source	Allow/Deny
103	SSH	TCP	22	0.0.0.0/0	ALLOW
100	HTTPS	TCP	443	0.0.0.0/0	ALLOW
*	All IPv4 traffic	All	All	0.0.0.0/0	DENY
Outbound					
Rule #	Type	Protocol	Port Range	Source	Allow/Deny
103	SSH	TCP	22	0.0.0.0/0	ALLOW
100	HTTPS	TCP	443	0.0.0.0/0	ALLOW
*	All IPv4 traffic	All	All	0.0.0.0/0	DENY

على عكس ال Security Groups، في ال Custom NACL تقدر تحدد وتكتب قواعد للسماح وللمنع مع بعض في نفس الجدول.

تقييم القواعد جوه الجدول بيمشي بالترتيب الرقمي تصاعدياً، وبتبدأ المعالجة دائماً من أصغر رقم قاعدة موجود.

أول ما البيانات تطابق قاعدة رقمها صغير (زي 100 مثلاً)، بتننفذ فوراً والسيستم بيتجاهل أي قواعد تانية تحتها (زي قاعدة 103).

❖ مقارنة شاملة بين ال Security Groups وال Network ACLs

Attribute	Security Groups	Network ACLs
Scope	Instance level	Subnet level
Supported Rules	Allow rules only	Allow and deny rules
State	Stateful (return traffic is automatically allowed, regardless of rules)	Stateless (return traffic must be explicitly allowed by rules)
Order of Rules	All rules are evaluated before decision to allow traffic	Rules are evaluated in number order before decision to allow traffic

♦ :Scope

ال Security Groups بتشتغل على مستوى السيرفر (Instance level)، بينما ال Network ACLs بتشتغل على مستوى الشبكة الفرعية (Subnet level).

♦ :Supported Rules

ال Security Groups بتدعم قواعد السماح فقط (Allow rules only)، بينما ال Network ACLs بتدعم قواعد السماح والمنع معاً (Allow and deny rules).

♦ :State

ال Security Groups حافظة للحالة (Stateful) فالرد يرجع تلقائياً، بينما ال Network ACLs عديمة الحالة (Stateless) ولازم تفتح الرد يدوياً بقاعدة صريحة (Return traffic must be explicitly allowed by rules).

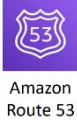
♦ :Order of Rules

في ال Security Groups بيتم فحص وتقييم جميع القواعد معاً قبل اتخاذ القرار، أما في ال Network ACLs بيتم فحص القواعد بالترتيب الرقمي للقواعد قبل اتخاذ القرار.

5. Amazon Route 53

❖ ما هي خدمة Amazon Route 53؟

هي عبارة عن خدمة ويب لنظام أسماء النطاقات (Domain Name System - DNS) بتوفرها AWS، وبتميز بالإتاحة العالية (High Availability) والقابلية للتوسع (Scalability)، وهدفها الأساسي هو توجيه المستخدمين إلى الموارد أو التطبيقات الصحيحة بأسرع وأدق طريقة.



هي خدمة DNS سحابية تتضمن إن الخدمة تفضل شغالة باستمرار حتى لو حصلت أعطال في جزء من البنية التحتية، وبتقدير تستوعب عدد ضخم جداً من طلبات المستخدمين في نفس الوقت بدون ما يتأثر الأداء.

وظيفتها الأساسية إنها تحول أسماء المواقع أو الدومينات التي المستخدم يقدر يقرأها ويفهمها، إلى عناوين IP رقمية التي الأجهزة والشبكات تستخدمها عشان تقدر تتواصل مع بعضها، وده بيتم بشكل تلقائي بمجرد إن المستخدم يكتب اسم الموقع في المتصفح.

الخدمة بتدعم بشكل كامل بروتوكولي IPv4 و IPv6، وبالتالي تقدر تتعامل مع أي نوع من عناوين IP سواء القديمة أو الحديثة، وده بيساعد على التوافق مع مختلف الشبكات والأجهزة.

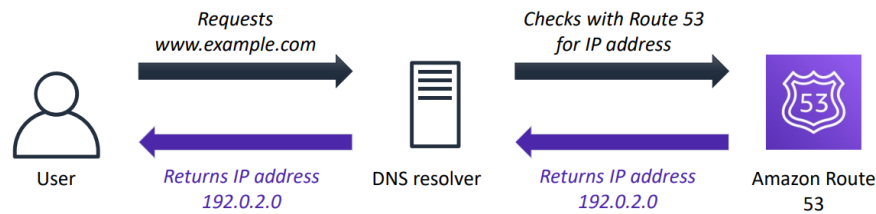
بتقدر توجه طلبات المستخدمين إلى الموارد المستضافة داخل AWS، زي خدمات EC2 أو Load Balancer أو S3، وكمان تقدر توجهها إلى أي سيرفرات أو بنية تحتية موجودة خارج AWS، سواء كانت في مراكز بيانات خاصة بالشركة أو عند أي مزود خدمة ثاني.

بتوفر خاصية Health Checks التي من خلالها تتابع حالة الموارد بشكل مستمر، وتتأكد إنها شغالة وتستجيب للطلبات بشكل طبيعي. ولو مورد معين بقي غير متاح أو حصل فيه عطل، تقدر توقف توجيه المستخدمين ليه بشكل تلقائي، وتوجههم لمورد سليم بدل منه.

بتوفر ميزة Traffic Flow، وهي أداة بتساعدك تتحكم في طريقة توزيع حركة المرور بين الموارد المختلفة باستخدام سياسات توجيه متعددة، وده بيسمح بإدارة حركة المستخدمين بطريقة مرنة ومنظمة حسب احتياجات التطبيق.

بتسمحلك بحجز وتسجيل أسماء الدومين (Domain Registration) مباشرةً من خلال AWS، وكمان تقدر تدير إعدادات الـ DNS الخاصة بالدومين من نفس الخدمة، بدل ما تعتمد على جهة خارجية لإدارة الدومين.

❖ DNS Resolution



هي الرحلة الفنية التي بيمشي فيها طلب المستخدم عشان يتحول من اسم موقع مفهوم للبشر لعنوان IP رقمي تفهمه الأجهزة.

بتنظم التبادل الفني بين ثلاث أطراف رئيسية:

- المستخدم (User)
- خادم الـ DNS المحلي (DNS resolver)
- خدمة Amazon Route 53

خطوة الطلب الأولية (User Request):

المستخدم يفتح المتصفح ويكتب اسم الدومين الذي عاوزه (زي `www.example.com`)، الطلب ده بيروح لجهاز الـ DNS resolver الخاص بشركة الإنترنت.

خطوة الإستعلام من AWS (Check with Route 53):

خادم الـ DNS resolver مش بيكون عارف الـ IP بتاع السيرفر، فيروح يسأل خدمة Amazon Route 53 بشكل مباشر عن عنوان الـ IP المقترن بالاسم ده.

خطوة إرجاع الـ IP Address (Returns IP address):

خدمة Amazon Route 53 بتبص في الجداول اللي عندك وترجع عنوان الـ IP الرقمي الصحيح (زي `192.0.2.0`) للـ DNS resolver، والـ DNS resolver بدوره بيبيعه لجهاز المستخدم النهائي.

ملحوظة مهمة

Amazon Route 53 مش هو اللي بيستضيف الموقع.

دوره الوحيد هو إنه يدل جهاز المستخدم على عنوان الـ IP الصحيح، بعد كده الإتصال بيكون مباشر بين جهاز المستخدم والسيرفر، سواء كان السيرفر ده:

- EC2
- Application Load Balancer
- S3 Static Website
- أو حتى سيرفر موجود خارج AWS.

❖ Amazon Route 53 supported routing

سياسات التوجيه (Routing Policies) عبارة عن خوارزميات وقواعد فنية بتختارها عشان تحدد الـ Route 53 هيرد بـ أي IP لما مُستخدم يسأله عن موقعك.

بتديك تحكم كامل في إدارة حركة البيانات وتوزيع الأحمال، والتعامل مع حالات الطوارئ لو سيرفر وقع.

القواعد:

1. Simple Routing (Round Robin)

بُستخدم لما يكون عندك مورد واحد (Single Resource) أو لما يكون عندك أكثر من Resource لنفس الخدمة ومش محتاج أي شروط مُعينة في التوجيه.

لو فيه أكثر من عنوان IP لنفس الدومين، Route 53 هيقوم بإرجاعهم بالتبادل (Round Robin) بحيث الطلبات تتوزع بينهم بالتساوي، بدون أي أولوية أو تفضيل.

تُستخدم في:

- التطبيقات البسيطة.
- البيئات اللي فيها سيرفر واحد.
- التوزيع البسيط للطلبات بدون شروط.

2. Weighted Routing (Weighted Round Robin)

بإسمحك تحديد وزن (Weight) لكل Resource، بحيث نسبة الطلبات التي تروح لكل Resource تعتمد على الوزن الذي أنت محدده.

كل ما ال Weight يزيد، كل ما نسبة المُستخدمين التي هتتوجه للمورد ده تزيد.
يعني التوزيع مش بيكون بالتساوي، لكنه بيكون حسب ال Weights التي أنت محددها.
تُستخدم في:

- اختبار إصدار جديد من التطبيق (Canary Deployment).
- توزيع الحمل بنسبة مُعينة.
- نقل المُستخدمين تدريجياً لسيرفر جديد.

3. Latency Routing

بتقوم بتوجيه المُستخدم إلى أقل Region من حيث زمن الإستجابة (Lowest Latency).
Route 53 بيقيس أقل زمن للوصول بين المُستخدم وال AWS Region، وبعدها يوجه الطلب للمورد الذي هيقدم أسرع استجابة.
الهدف الأساسي هو تحسين أداء التطبيقات العالمية وتقليل زمن الإستجابة للمُستخدمين.
تُستخدم في:

- التطبيقات المُنتشرة في أكثر من Region.
- المواقع العالمية.
- الخدمات التي بتحتاج استجابة سريعة.

4. Geolocation Routing

بتوجه المُستخدمين بناءً على الموقع الجغرافي للمُستخدم نفسه.
يعني القرار بيتأخذ حسب الدولة أو القارة أو المنطقة التي جاي منها المُستخدم.
كل منطقة جغرافية ممكن يكون ليها Resource مُختلف.
تُستخدم في:

- عرض محتوى مختلف حسب الدولة.
- تطبيق قوانين أو سياسات خاصة بكل دولة.
- توجيه المُستخدمين إلى خدمات محلية.

بيعتمد على Location المُستخدم.

5. Geoproximity Routing

بتوجه المُستخدمين بُناءً على الموقع الجغرافي للموارد (Resources) وليس المُستخدم فقط.
Route 53 يحسب أقرب Resource للمستخدم، ويقدر كمان يوسع أو يقلل نطاق التغطية لكل Resource باستخدام Bias.
بالتالي تقدر تتحكم في توزيع المُستخدمين بين الـ Regions حسب قرب الموارد منهم.
تُستخدم في:

- توزيع الحمل بين الـ Regions.
 - تحسين الأداء.
 - التحكم في نطاق خدمة كل Region.
- يعتمد على Location الموارد.

6. Failover Routing

بتوفر خاصية High Availability.
يكون عندك:

- Primary Resource (الرئيسي)
- Secondary Resource (الاحتياطي)

Route 53 يعمل Health Checks بشكل مُستمر على المورد الأساسي.
لو المورد الأساسي بقي غير مُتاح، يقوم تلقائياً بتحويل كل الطلبات إلى المورد الإحتياطي بدون تدخل يدوي.
تُستخدم في:

- Disaster Recovery
- Business Continuity
- الأنظمة التي لازم تفضل شغالة باستمرار.

7. Multivalue Answer Routing

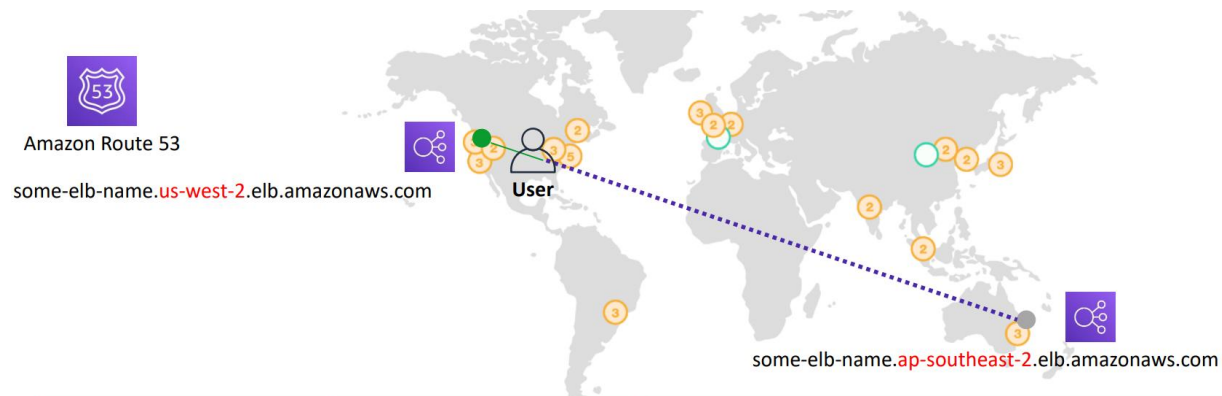
بتقوم بإرجاع أكثر من عنوان IP في نفس استجابة الـ DNS، بحيث يتم اختيار العناوين السليمة فقط.
يمكن ترجع حتى 8 عناوين IP سليمة (Healthy Records) في الرد الواحد.
لو أحد الموارد أصبح غير مُتاح، يتم استبعاده تلقائياً من النتائج طالما الـ Health Check فشل.
الهدف منها هو تحسين التوفر (Availability) وتوزيع الحمل بشكل بسيط بين الموارد السليمة.

Multi-Region Deployment ❖

المقصود بـ Multi-Region Deployment إن التطبيق أو الموقع يكون مُستضاف في أكثر من AWS Region في نفس الوقت، بدل ما يكون موجود في Region واحدة فقط.

وده بيحقق:

- زيادة التوافر (High Availability).
- تحسين الأداء.
- تقليل زمن الإستجابة (Latency).
- استمرارية الخدمة في حالة تعطل إحدى الـ Regions.



Name	Type	Value
example.com	ALIAS	some-elb-name.us-west-2.elb.amazonaws.com
example.com	ALIAS	some-elb-name.ap-southeast-2.elb.amazonaws.com

الصورة بتوضح إن في Domain واحد (**example.com**) وده متسجل على Amazon Route 53. لكن الدومين ده مُرتبط بإثنين Application Load Balancers (ELB) موجودين في منطقتين مُختلفتين.

❖ الـ Region الأولى هي US West (Oregon)

some-elb-name.us-west-2.elb.amazonaws.com

وده الـ Load Balancer الموجود في Region us-west-2.

❖ الـ Region الثانية هي Asia Pacific (Sydney)

some-elb-name.ap-southeast-2.elb.amazonaws.com

وده Load Balancer ثاني موجود في Region ap-southeast-2.

في أسفل الصورة موجود جدول الـ DNS Records، لاحظ إن نفس الدومين اللي هو **example.com** ليه سجلين DNS. يعني نفس الدومين مُرتبط بأكثر من Load Balancer.

◆ معنى ALIAS Record

بدل ما تكتب عنوان IP ثابت، Route 53 يسمحك تربط الدومين مباشرةً بخدمات AWS زي:

- ELB
- CloudFront
- S3 Static Website

ميزة ALIAS:

- مفيش حاجة اسمها IP ثابت.
- AWS بتحدث ال IP تلقائيًا لو اتغير، وأنت مش محتاج تعدل ال DNS يدويًا.

◆ ماذا يحدث عندما يدخل المُستخدم إلى الموقع؟

لما المُستخدم يكتب example.com، الطلب يوصل إلى Amazon Route 53.

بعدها Route 53 يحدد أي Load Balancer هيووجه إليه المُستخدم.

اختيار ال Load Balancer بيعتمد على Routing Policy اللي انت مكوونها.

لو كنت مُستخدم Weighted Routing، هيقسم المُستخدمين بين ال Regions بالنسبة اللي انت محددها.

لو كنت مُستخدم Latency Routing، هيفتار ال Region اللي أقل في زمن الاستجابة بالنسبة للمستخدم.

لو كنت مُستخدم Geolocation Routing، هيفتار ال Region حسب الدولة اللي المستخدم موجود فيها.

لو كنت مُستخدم Failover Routing، هيبعت الطلب لل Primary Region، ولو ال Region دي وقعت هيحول الطلب تلقائيًا لل Secondary Region.

◆ ليه بنستخدم Load Balancer؟

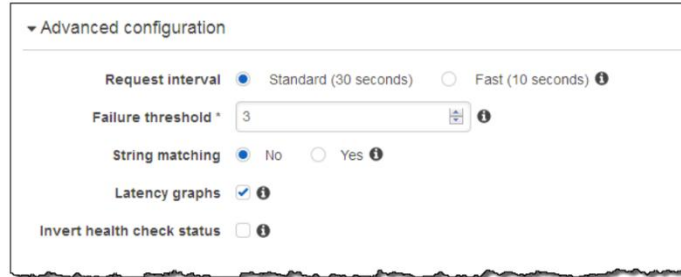
لأن ال Load Balancer بيقوم بعدة وظائف:

- توزيع الطلبات على أكثر من EC2.
- زيادة التوافر.
- التعامل مع زيادة عدد المُستخدمين.
- إزالة السيرفرات غير السليمة من الخدمة.

علشان كده Route 53 بيوجه المُستخدم لل Load Balancer وليس مباشرةً إلى ال EC2.

Amazon Route 53 DNS failover ❖

هي خاصية في Amazon Route 53 تساعد على زيادة توافر (Availability) التطبيقات، بحيث لو المورد الأساسي حصل فيه عطل، يتم تحويل المُستخدمين تلقائياً إلى المورد الإحتياطي (Backup Resource).



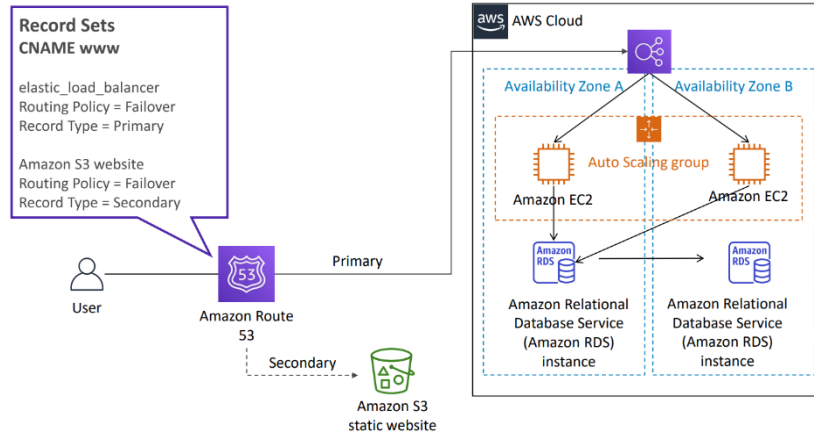
أهم استخداماته ❖

بتسمحك تعمل Primary Resource و Secondary Resource (Backup)، لو المورد الأساسي أصبح غير مُتاح، Route 53 يحول كل الطلبات تلقائياً للمورد الإحتياطي.

بتساعد على إنشاء تطبيقات شغالة في أكثر من AWS Region، بحيث لو Region كاملة حصل فيها عطل، المُستخدمين يتم توجيههم إلى Region ثانية بدون تدخل يدوي.

بيعتمد Route 53 على Health Checks لمراقبة حالة الموارد بشكل مُستمر، لو ال Health Check فشل، يعتبر المورد غير مُتاح، ويبدأ تنفيذ عملية Failover تلقائياً.

DNS failover for a multi-tiered web application ❖



المقصود بـ DNS Failover for a Multi-Tiered Web Application إن السيستم يطبق استراتيجية حماية كاملة لتطبيق متقسم لطبقات؛ بحيث لو المعمارية الأساسية كلها حصل فيها عطل مفاجئ، يتم تحويل المستخدمين أوتوماتيك لصفحة بديلة ثابتة.

وده بيحقق:

- تأمين التطبيقات المُعقدة وضمان استمرارية عملها.
- تجنب ظهور رسائل خطأ شبكية صريحة للمستخدمين عند انهيار النظام.
- عزل الكوارث على مستوى البنية التحتية بالكامل.
- توفير حلول طوارئ مُنخفضة التكلفة باستخدام المواقع الاستاتيكية.

الصورة بتوضح إن البيئة الأساسية (Primary) عبارة عن تطبيق ويب حقيقي شغال ومتكامل جوه الـ AWS Cloud. والبيئة دي بتعتمد على كذا خدمة مع بعض لضمان التوافر العالي وتوزيع الأحمال.

Elastic Load Balancer: وده خط الدفاع الأول اللي بيستقبل الـ Primary traffic من الـ Route 53 عشان يوزع الطلبات على السيرفرات.

Auto Scaling group: بتحتوي على سيرفرات Amazon EC2 وممتدة عبر منطقتين توافر (Availability Zone A & B) عشان لو منطقة وقعت التانية تشيل الشغل.

Amazon RDS instance: وهي قاعدة البيانات الرئيسية اللي مُتصلة بالسيرفرات عشان تخزين واستدعاء بيانات التطبيق. في يسار الصورة موجود مربع الـ Record Sets، لاحظ إن الدومين (www) مربوط بسياسة الـ Failover على وجهتين مُختلفتين تماماً. يعني السيستم مُهيأ ببديل فوري وجاهز لو المعمارية دي كلها مبقتش تستجيب.

♦ معنى الـ Failover لـ تطبيق مُتعدد الطبقات

بدل ما تسبب المُستخدم يواجه صفحة معطلة لو السيرفرات أو قاعدة البيانات وقعت، Route 53 بيحوله للمورد البديل المذكور في الجدول وهو Amazon S3 static website. ميزة الـ S3 البديل:

- موقع استاتيكي ثابت ومُستضاف بشكل مُستقل تماماً عن السيرفرات وقواعد البيانات.
- بيشتغل كـ Secondary record وبتكلفة شبه معدومة وقت الأزمات.

♦ ماذا يحدث عندما يدخل المُستخدم إلى الموقع؟

لما المُستخدم يطلب الرابط (www)، الطلب بيعدي الأول على Amazon Route 53.

بعدها Route 53 بيفحص سلامة Elastic Load Balancer التابع للبيئة الأساسية.

حالة الفحص هي اللي بتحدد مسار البيانات:

لو Healthy، الـ Route 53 بيوجه الطلب كـ Primary للمعارة الأساسية، والشغل بيمشي طبيعي بين الـ EC2 والـ RDS.

لو حصل فيه مُشكلة أو البيئة الأساسية كلها انهارت، فحص السلامة هيفشل.

لو حصل الفشل ده، الـ Route 53 بيلغي التوجيه لـ Primary فوراً، ويحول طلبات المستخدمين تلقائياً لـ Secondary وهو موقع الـ S3 الثابت.

المُتصفح بيفتح للمُستخدم صفحة الطوارئ الثابتة (زي صفحة "نعتذر عن العطل ونعمل على الصيانة الآن") لحد ما البيئة الأساسية ترجع تشتغل وتعدي الفحص، فيرجع التوجيه ليها أوتوماتيك.

8. Amazon CloudFront

Content delivery and network latency ❖

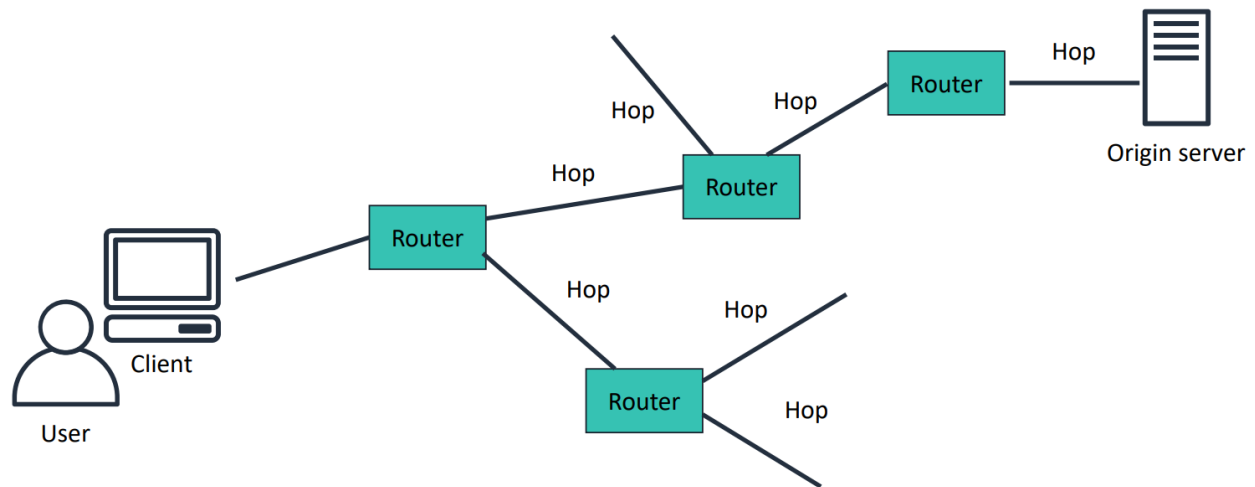
المقصود بـ Content delivery and network latency هو فهم التحديات الأساسية التي تواجه جودة وأداء الشبكات عند نقل البيانات عبر مسافات طويلة، وهي السبب الرئيسي وراء ابتكار خدمات شبكات توصيل المحتوى (CDNs).

وده بیوضح:

أداء الشبكة (Network performance): الصعوبة التي تواجه المواقع في الحفاظ على سرعة ثابتة لما يدخل عليها مُستخدمين من أماكن مُختلفة.

مُشكلة زمن الإستجابة (Network latency): تفاوت الوقت اللي بتاخذه البيانات عشان تسافر بين العميل والسيرفر، وده بيختلف حسب الموقع الجغرافي (different in various geographic locations).

تأثير المسافة: المسافة الطويلة التي يقطعها الطلب بتأثير بشكل مباشر وكبير على استجابة الموقع وسرعته (performance and responsiveness).



الصورة بتوضح المعمارية التقليدية لتنقل البيانات في الشبكة والمكونات المتسببة في حدوث التأخير:

User / Client: المُستخدم الى يفتح مُتصفح أو يعمل بث لفديو (stream a video) ويبدأ يرسل الطلب.

Routers & Network Hops: أجهزة التوجيه التي بتمر عليها البيانات؛ فالطلب بعيد على شبكات كثيرة ومختلفة، وكل نقلة بين راوتر وراوتر تسمى قفزة (Hop).

Origin Server: السيرفر الأصلي والنهائي للموقع، وهو اللي بيخزن النسخ الأصلية والرسمية من الملفات زي صفحات الويب، الصور، وملفات الميديا.

في الجانب الأيمن من الصورة، نلاحظ إن السيرفر الأصلي بعيد جداً عن العميل.

يعني كل ما زادت القفزات الشبكية (Network hops) والمسافة، كل ما قل أداء الموقع وزاد زمن التأخير، وده اللي بيخلي الـ CDN هو الحل الأمثل.

♦ ماذا يحدث عندما يدخل المُستخدم إلى الموقع؟

لما المُستخدم يكتب رابط الموقع أو يشغل ميديا، الطلب بيتحرك فوراً من جهاز العميل (Client) متوجهاً للسيرفر الأصلي. بعدها يبدأ الطلب يمر بسلسلة من الراوترات والشبكات المُختلفة. عدد القفزات والمسافة هما اللي بيحددوا النتيجة: إذا كان المُستخدم قريباً جغرافياً، الطلب بيعدي على قفزات (Hops) قليلة، والموقع بيفتح معاه بسرعة كويسة وبأداء مستقر. إذا كان المستخدم بعيداً جغرافياً، الطلب بياخد وقت طويل ويتنقل بين راوترات كثيرة جداً (More Hops)، والنتيجة إن أداء واستجابة الموقع بتقل جداً. عدد القفزات الشبكية (The number of network hops) والمسافة الجغرافية هما المسبب الأول لبطء استجابة التطبيقات. زمن استجابة الشبكة (Network latency) غير ثابت؛ فهو دائماً أسوأ للمستخدمين البعاد جغرافياً عن السيرفر الأصلي.

❖ Content delivery network (CDN)

المقصود بـ Content delivery network (CDN) إنها شبكة من سيرفرات تخزين الكاش الموزعة عالمياً، هدفها الأساسي تسريع نقل البيانات وحل مشاكل بطء الشبكة للمستخدمين. وده بيحقق:

- زيادة سرعة الأداء (Performance): تحسين أداء التطبيقات وسرعة فتحها بشكل ملحوظ.
- تحسين عملية التوسع (Scaling): مساعدة السيستم إنه يستوعب أعداد مُستخدمين ضخمة في نفس الوقت.
- تقليل زمن الاستجابة (Latency): تقديم المحتوى للمستخدم من أسرع وأقرب نقطة شبكية تابعة له.

♦ مكونات وآلية عمل الـ CDN

بدل ما كل الطلبات تروح للسيرفر الأصلي البعيد، الـ CDN بيقسم الشغل على كذا عنصر:

Globally distributed system: نظام كامل من سيرفرات تخزين الكاش الموزعة في كل مكان في العالم.

Static Content Caching: تخزين نسخ من الملفات الثابتة والأكثر طلباً (زي الـ HTML، CSS، والـ JavaScript، والصور) المُستضافة على السيرفر الأصلي (Origin Server).

Point of Presence (PoP) / Cache Edge: مراكز البيانات المُصغرة القريبة من المُستخدمين، والتي بتسلمهم النسخة المحلية (Local copy) بأعلى سرعة.

Dynamic Content Acceleration: تسريع توصيل المحتوى الديناميكي المُتغير والخاص بكل مُستخدم (Unique) وغير القابل للتخزين (Not cacheable).

♦ ماذا يحدث عندما يدخل المُستخدم إلى الموقع؟

لما المُستخدم يطلب ملف أو صفحة، الطلب يبروح مباشرةً لأقرب Point of Presence (PoP) جغرافياً له.

بعدها الـ CDN يبيشوف نوع المحتوى الي الـ المُستخدم طالبه عشان يحدد طريقة التعامل:

لو المحتوى المطلوب ثابت (Static Content): الـ CDN بيسلم المُستخدم نسخة محلية (Local copy) فوراً من الكاش بتاع الـ PoP، والطلب مش بيحتاج يسافر للسيفرر الأصلي.

لو المحتوى المطلوب ديناميكي ومتغير (Dynamic Content): الـ CDN بيستغل وجوده في مكان قريب من المُستخدم وبيعمل اتصالات آمنة وسريعة (Secure connections)، ويوجه الطلب للسيفرر الأصلي (Origin) عبر شبكة محسنة وسريعة جداً عشان يجيب المحتوى ويرجع بيه.

لو المُستخدم بيبعت بيانات (Form data / Images / Text): الـ PoP بياخد البيانات دي من المُستخدم ويمررها للسيفرر الأصلي بسرعة فائقة مُستفيداً من ميزة الـ Proxy والاتصالات ذات زمن التأخير المنخفض (Low-latency connections).

❖ Amazon CloudFront

المقصود بـ Amazon CloudFront إنها شبكة من سيرفرات تخزين الكاش الموزعة عالمياً (Content Delivery Network) سريعة وآمنة، بتقدم البيانات، الفيديوها، التطبيقات، وAPIs للمُستعرضين حول العالم.

وده بيحقق:



Amazon
CloudFront

سرعة نقل عالية (High transfer speeds): نقل الملفات والبيانات للمُستخدمين بأقصى سرعة ممكنة.

زمن استجابة مُنخفض جداً (Low latency): فتح المواقع وتشغيل الفيديو بدون أي تأخير ملحوظ.

بيئة صديقة للمطورين (Developer-friendly): سهولة الإعداد والربط البرمجي بدون تعقيدات.

توفير مالي كبير: خدمة ذاتية الدفع (Self-service model) بنظام الدفع مُقابل الإستخدام (Pay-as-you-go pricing) بدون عقود طويلة أو رسوم.

بدل ما تعتمد على سيفرر تقليدي مُحدد بمكان واحد، الخدمة بتدير شبكة فروع عملاقة متوزعة في كل القارات:

Global network of edge locations: شبكة عالمية من مراكز البيانات المُصغرة (نقاط التواجد) القريبة جغرافياً من المُستخدمين لتوصيل الملفات بسرعة عالية.

Regional edge caches: طبقة تخزين مؤقت إقليمية أكبر بتساعد في الحفاظ على المحتوى قريب من الـ Edge locations وبتقلل الرجوع للسيفرر الأصلي.

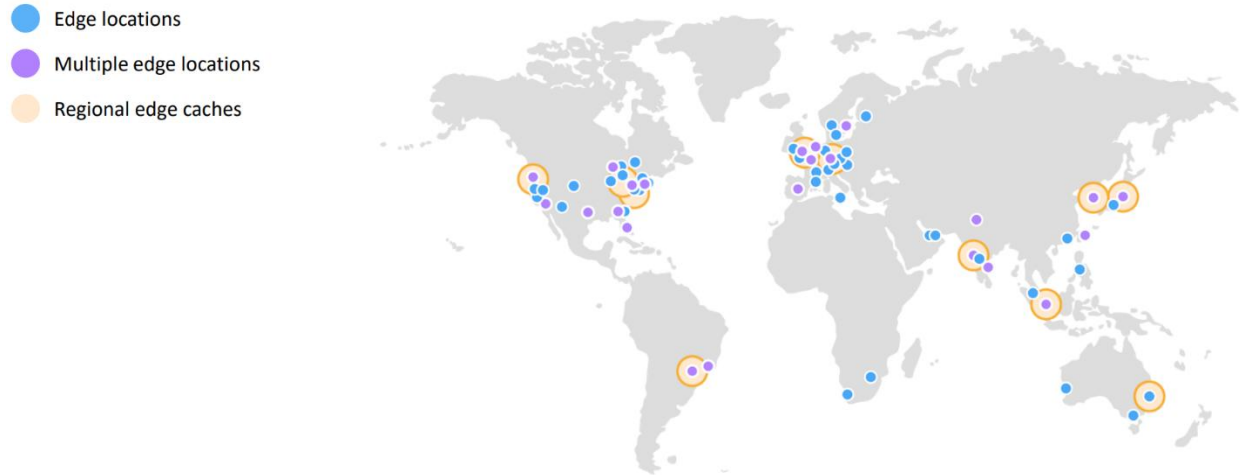
Traditional solution differences: بتختلف عن الحلول التقليدية إنك مش محتاج تتفاوض على عقود أو تدفع مبالغ ضخمة مقدماً عشان تحصل على أداء عالي.

Amazon CloudFront infrastructure ❖

المقصود بـ Edge locations و Regional edge caches هي البنية التحتية والمراكز العالمية التي تستخدمها خدمة CloudFront لتخزين وتوصيل المحتوى بأعلى كفاءة ممكنة.

وده بيحقق:

- توصيل المحتوى بأفضل أداء: توجيه المُستخدم تلقائياً لأقرب مركز بيانات بيقدم أقل زمن تأخير (Lowest latency).
- إدارة ذكية للكاش: النظام في التخزين يفرق بين المحتوى الأكثر طلباً والمحتوى الأقل طلباً.
- تقليل الضغط على السيرفر: الحفاظ على أكبر قدر من الملفات قريبة من المستخدمين لمنع العودة للسيرفر الأصلي.



♦ الفرق بين الـ Edge Locations والـ Regional Edge Caches

بدل ما يتم تخزين كل الملفات في مكان واحد وبنفس المدة، السيستم بيقسمهم لطبقتين:

• Edge Locations

دي السيرفرات القريبة جداً من المُستخدم.

بتخزن المحتوى الأكثر شعبية وطلباً بس (Popular content) عشان توصله للناس بسرعة عالية.

أول ما الملف شعبيته تقل والطلب عليه يقل، الـ Edge location بتمسحه فوراً عشان توفر مساحة لملفات تانية جديدة مطلوبة.

• Regional Edge Caches

دي طبقة تخزين موجودة في النص بين الـ Edge locations وبين السيرفر الأصلي (Origin).

حجم الكاش فيها أكبر بكثير (Larger cache) من الـ Edge locations الفردية.

وظيفتها تخزين الملفات التي شعبيتها قلت واتمست من الـ Edge locations؛ فبدل ما الـ CloudFront يروح للسيرفر الأصلي البعيد عشان يجيبها، بيلاقها متخزنة هنا لفترة أطول وقريبة من المُستخدمين، ولكن سرعة تكون أقل من الـ Edge Locations.

الـ Edge Location أسرع من الـ Regional Edge Cache، والـ Regional أسرع بكثير من الـ Origin Server.

❖ Amazon CloudFront benefits (المميزات)

1. Fast and Global

CloudFront يستخدم شبكة عالمية من Edge Locations و Regional Edge Caches، بحيث المحتوى يوصل للمستخدم من أقرب موقع جغرافي، وده يقلل Latency ويحسن سرعة تحميل الموقع.

2. Security at the Edge

CloudFront بتوفر حماية على مستويين:

- Network-Level Protection: بتحمي الشبكة نفسها من الهجمات.
- Application-Level Protection: بتحمي التطبيق أثناء استقبال الطلبات.

كمان بتوفر:

- AWS Shield Standard للحماية من هجمات DDoS بدون تكلفة إضافية.
- دعم SSL/TLS Certificates من خلال AWS Certificate Manager (ACM) لتأمين الإتصال باستخدام HTTPS بدون تكلفة إضافية.

3. Highly Programmable

CloudFront تقدر تخصصها حسب احتياجات التطبيق.

بتدعم Lambda@Edge، واللي بيسمح بتشغيل كود على الـ Edge Locations قبل وصول الطلب إلى الـ Origin، وبالتالي يتم تنفيذ جزء من منطق التطبيق بالقرب من المستخدم لتحسين سرعة الاستجابة.

كمان بتتكامل مع أدوات DevOps المختلفة، وبتدعم بيئات CI/CD (Continuous Integration / Continuous Delivery) لتسهيل نشر وتحديث التطبيقات بشكل مستمر.

4. Deeply Integrated with AWS

CloudFront متكاملة بشكل كامل مع خدمات AWS والبنية التحتية العالمية الخاصة بيها.

تقدر تربطها بسهولة مع خدمات زي:

- Amazon S3
- Elastic Load Balancing (ELB)
- EC2
- Route 53

وكمان تقدر تدير كل إعداداتها باستخدام:

- AWS Management Console
- AWS CLI
- AWS SDKs
- APIs

يعني كل حاجة تقدر تعملها يدوياً، تقدر تعملها برمجياً كمان.

5. Cost-Effective

CloudFront بتشتغل بنظام Pay as You Go، يعني مفيش اشتراك ثابت أو حد أدنى للدفع، وبتدفع على قد الإستخدام فقط. كمان بتوفر في التكلفة لإنها:

بتخزن المحتوى في الـ Cache، وبالتالي تقلل عدد الطلبات اللي بتوصل للـ Origin Server.

بتقلل الحمل على الـ Origin Server، وبالتالي ممكن محتاجش تزود عدد السيرفرات.

لو مجموعة من المُستخدمين طلبوا نفس الملف في نفس الوقت من نفس الـ Edge Location، CloudFront بتبعث طلب واحد فقط للـ Origin Server، وبعدها توزع نفس النسخة على كل المُستخدمين، وده بيققل الحمل على السيرفر ويوفر في استهلاك الموارد.

لو الـ Origin هو Amazon S3 أو Elastic Load Balancer (ELB)، مفيش رسوم على نقل البيانات بين CloudFront والخدمات دي، وبتدفع فقط تكلفة التخزين أو الخدمة نفسها.

❖ Amazon CloudFront pricing

رسوم استخدام Amazon CloudFront بتتسبب على حسب الإستخدام الفعلي للخدمة، وبتنقسم إلى 4 أجزاء رئيسية.

1. Data Transfer Out

بتدفع مُقابل كمية البيانات اللي CloudFront بيبعتها من Edge Locations إلى المُستخدمين أو إلى الـ Origin (السيرفر الأصلي). الحساب بيكون بوحدة GB.

كل منطقة جغرافية ليها سعر مُختلف، يعني تكلفة نقل البيانات بتختلف حسب المكان اللي البيانات رايحة له.

ملحوظة: لو الـ Origin عبارة عن خدمة AWS زي Amazon S3 أو EC2، فهدفع تكلفة استخدام الخدمة نفسها (زي التخزين أو تشغيل السيرفر)، لإنها رسوم مُنفصلة عن CloudFront.

2. HTTP(S) Requests

بتدفع مُقابل عدد طلبات HTTP و HTTPS اللي المُستخدمين بيبعتها إلى CloudFront.

كل Request بيتسبب، سواء كان لملف أو صفحة أو صورة أو أي محتوى تاني.

يعني كل ما عدد الطلبات يزيد، التكلفة تزيد.

3. Invalidation Requests

لما CloudFront يخزن الملفات في الـ Cache، أحياناً بتحتاج تسمح نسخة قديمة عشان المُستخدمين يشوفوا النسخة الجديدة.

العملية دي اسمها Invalidation.

بتدفع مقابل كل Path بتعمله Invalidation.

AWS بتديك 1000 Path مجاناً كل شهر.

بعد أول 1000 Path، أي Path إضافي بيكون عليه رسوم.

4. Dedicated IP Custom SSL

لو استخدمت Dedicated IP SSL Certificate بدل ال SSL العادي، بيكون عليه رسوم إضافية.

الرسوم 600 دولار شهرياً لكل شهادة SSL.

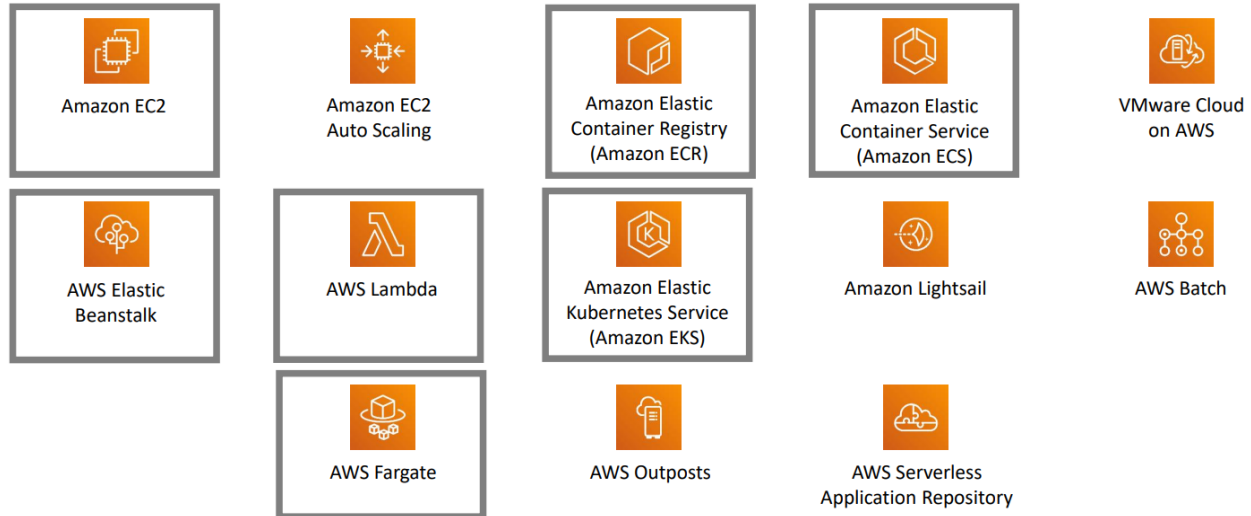
ولو استخدمتها لمدة أقل من شهر، التكلفة بتتسبب حسب عدد الساعات أو الأيام اللي استخدمتها فيها.

ملحوظة: دي خدمة اختيارية، ومش كل التطبيقات بتحتاجها.

◆ **Module 6: Compute**

1. Compute Services Overview

AWS compute services ❖



شركة AWS بتقديم خدمات حوسبة كثيرة ومتنوعة، وده ملخص سريع وبسيط كدة عشان تفهم وظيفة كل خدمة لأول مرة:

◆ **Amazon Elastic Compute Cloud (Amazon EC2)**

دي بتديك أجهزة كمبيوتر وهمية أو افتراضية (Virtual Machines)، وميزة الأجهزة دي إنك تقدر تكبر أو تصغر حجمها ومواصفاتها في أي وقت حسب ضغط الشغل عندك.

◆ **Amazon EC2 Auto Scaling**

دي ميزة بتخلي الموقع بتاعك يفضل شغال وميقعش؛ لإنها بتخلي السيرفرات تزود عددها لوحدها لما يكون في ضغط وزحمة، وتلغي السيرفرات الزيادة لما الضغط يقل.

◆ **Amazon Elastic Container Registry (Amazon ECR)**

ده عبارة عن مخزن رقمي بتشيل وتخزن فيه الـ Images بتاعت الدوكر (Docker Images) اللي بنحتاجها عشان نشغل الحاويات (Containers).

◆ **Amazon Elastic Container Service (Amazon ECS)**

دي خدمة وظيفتها تدير وتنظم الحاويات (Containers) وبتدعم نظام الـ Docker بشكل كامل.

◆ **VMware Cloud on AWS**

دي بتخليك تقدر تعمل Hybrid Cloud، بتربط فيها بين سيرفرات شركتك و AWS Cloud، ومن غير ما تضطر تشتري أجهزة أو هاردوير مخصص.

◆ AWS Elastic Beanstalk :

دي أسهل طريقة تشغل بيها موقع؛ كل اللي عليك إنك ترفع كود الموقع بتاعك، والخدمة دي بتتولى كل حاجة تانية وتجهز السيرفرات لوحدها.

◆ AWS Lambda

خدمة بتقدم Computing من غير ما تشيل هم إدارة السيرفرات خالص (Serverless)، والميزة هنا إنك مش بتدفع اشتراك ثابت، أنت بتدفع بس مُقابل الوقت اللي الكود بتاعك بياخده وهو بيتنفذ بالملي ثانية.

◆ Amazon Elastic Kubernetes Service (Amazon EKS)

دي بتخليك تشغل نظام Kubernetes عشان تدير الحاويات (Containers) بتاعتك على AWS بس بشكل جاهز ومُدار بالكامل.

◆ Amazon Lightsail

خدمة سهلة وبسيطة جداً، معمولة مخصوص للمبتدئين أو اللي عاوز يعمل موقع أو تطبيق صغير بسرعة ومن غير تعقيد وبسعر ثابت خفيف.

◆ AWS Batch

دي أداة مخصصة عشان تشغل المهام الكبيرة اللي بتتعمل على دفعات (Batch Jobs) بأي حجم، زي مثلاً معالجة وتحليل بيانات ضخمة ورا بعضها.

◆ AWS Fargate

دي طريقة بتشغل بيها الحاويات (Containers)، ووظيفتها إنها بتقلل من احتياجك لإدارة السيرفرات أو المجموعات (Clusters). يعني بتخليك تشغل حاوياتك من غير ما تضطر تدير أو تضبط السيرفرات بنفسك.

◆ AWS Outposts

دي بتخليك تقدر تشغل خدمات مُعينة من خدمات AWS جوة مركز البيانات أو السيرفرات المحلية بتاعة شركتك نفسها.

◆ AWS Serverless Application Repository

ده زي مكتبة أو متجر بتقدر من خلاله تكتشف، وتنشر، وتلاقي تطبيقات جاهزة بتشغل بنظام ال Serverless (من غير سيرفرات).

❖ Categorizing compute services

Services	Key Concepts	Characteristics	Ease of Use
Amazon EC2	- Infrastructure as a service (IaaS) - Instance-based - Virtual machines	Provision virtual machines that you can manage as you choose	A familiar concept to many IT professionals.
AWS Lambda	- Serverless computing - Function-based - Low-cost	- Write and deploy code that runs on a schedule or that can be triggered by events - Use when possible (architect for the cloud)	A relatively new concept for many IT staff members, but easy to use after you learn how.
- Amazon ECS - Amazon EKS - AWS Fargate - Amazon ECR	- Container-based computing - Instance-based	Spin up and run jobs more quickly	AWS Fargate reduces administrative overhead, but you can use options that give you more control.
AWS Elastic Beanstalk	- Platform as a service (PaaS) - For web applications	- Focus on your code (building your application) - Can easily tie into other services—databases, Domain Name System (DNS), etc.	Fast and easy to get started.

تقدر تقسم خدمات الحوسبة في AWS لأربع تصنيفات رئيسية، وكل خدمة بتقديم نوع معين من التحكم والإدارة:

1. الأجهزة الافتراضية - IaaS:

الخدمة: Amazon EC2

بتقدم لك أجهزة كمبيوتر افتراضية (Virtual Machines).

بتديك مرونة عالية جداً وبتسبب عليك مسؤولية إدارة السيرفر؛ يعني أنت اللي بتختار نظام التشغيل (OS)، وأنت اللي بتختار حجم وإمكانيات السيرفر.

الفكرة دي مألوفة جداً لمهندسين الـ IT اللي اشتغلوا على سيرفرات محلية، الخدمة دي من أوائل خدمات AWS ولسه من أكثر الخدمات شعبية.

2. الحوسبة بدون سيرفرات (Serverless):

الخدمة: AWS Lambda

منصة Computing مفيهاش أي إدارة سيرفرات من ناحيتك (Zero-administration).

بتخليك تشغل الكود بتاعك من غير ما تجهز أو تدير سيرفرات، وبتدفع بس مُقابل الوقت اللي الكود بتاعك بياخده وهو بيتنفذ.

الفكرة دي تعتبر جديدة على مهندسين الـ IT، لكن شعبيتها بتزيد لأنها بتدعم معمارية الـ Cloud-native، واللي بتديك قدرة توسع هائلة بتكلفة أقل بكثير من إنك تشغل سيرفرات 24 ساعة في اليوم لنفس الشغل.

3. الخدمات القائمة على الحاويات (Container-based):

الخدمات: تشمل Amazon ECS و Amazon EKS و AWS Fargate و Amazon ECR.

بتسمح لك تشغل كذا تطبيق أو كذا عملية (workloads) على نفس نظام التشغيل (OS) الواحد.

الحاويات (Containers) بتفتح وتشغل بسرعة أكبر بكثير من الأجهزة الافتراضية (VMs)، وده بيديها سرعة استجابة عالية، وحلول الحاويات دي شعبيتها بتكبر باستمرار.

4. PaaS

الخدمة: AWS Elastic Beanstalk

بتقدم منصة جاهزة (Platform as a service).

بتسهل عليك نشر وتشغيل تطبيقاتك بسرعة لأنها بتوفر لك كل خدمات التطبيق اللي محتاجها.

هنا AWS هي اللي بتدير نظام التشغيل (OS)، وسيرفر التطبيق، وكل مكونات البنية التحتية، وده بيخليك تركّز بس في كتابة وتطوير كود التطبيق بتاعك.

❖ Choosing the optimal compute service

بتوفر AWS خدمات Compute كتير، لإن كل تطبيق أو نظام ليه احتياجات مختلفة، وبالتالي مفيش خدمة واحدة تنفع لكل الحالات.

اختيار خدمة ال Compute المناسبة بيعتمد على طبيعة التطبيق وطريقة تشغيله.

يعني قبل ما تختار الخدمة، لازم تعرف:

- طبيعة التطبيق.
- حجم الإستخدام.
- مُتطلبات الأداء.
- طريقة إدارة البنية التحتية.

في بعض الحالات، التطبيق بيكون معمول بتقنيات قديمة (Legacy Applications)، وده ممكن يخليك تبدأ بخدمة مُعينة.

لكن مع الوقت تقدر تطور التطبيق وتنقله لتصميم سحابي (Cloud-Native Architecture) للإستفادة من مُميزات AWS.

Best Practices

1. Evaluate the available compute options

قبل ما تختار، لازم تقارن بين خدمات ال Compute المُختلفة، وتشوف أنهي خدمة مُناسبة لتطبيقك.

2. Understand the available compute configuration options

لازم تكون فاهم إعدادات كل خدمة، زي:

- حجم الموارد.
- نظام التشغيل.
- الشبكات.
- التخزين.
- إعدادات الأمان.

عشان تقدر تظبط الخدمة بالشكل المُناسب.

3. Collect compute-related metrics

لازم تتابع أداء الموارد باستخدام Metrics، زي:

- استهلاك المعالج (CPU).
- استخدام الذاكرة.
- عدد الطلبات.
- استهلاك الشبكة.

وده بيساعدك تعرف إذا كانت الموارد مُناسبة ولا محتاجة تعديل.

4. Use the available elasticity of resources

استفد من خاصية Elasticity في AWS، بحيث تزود أو تقلل الموارد حسب الحاجة، بدل ما تفضل مشغل موارد أكثر من المطلوب.

5. Re-evaluate compute needs based on metrics

راجع أداء التطبيق بشكل مُستمر، ولو الـ Metrics وضحت إن الخدمة الحالية مش مُناسبة، غَيّر التصميم أو استخدم خدمة Compute مختلفة.

عوامل اختيار خدمة الـ Compute

قبل ما تختار خدمة Compute، اسأل نفسك:

1. What is your application design ؟ (إيه تصميم التطبيق؟)

هل هو:

- Web Application ؟
- Microservices ؟
- Containers ؟
- Serverless ؟
- تطبيق تقليدي ؟

2. What are your usage patterns ؟ (إيه طبيعة الاستخدام؟)

- هل الاستخدام ثابت ؟
- ولا ييزيد ويقل باستمرار ؟
- هل فيه ضغط في أوقات مُعينة ؟

3. Which configuration settings will you want to manage ؟ (قد إيه عايز تتحكم في إعدادات السيرفر؟)

- هل عايز تدير نظام التشغيل بنفسك ؟
- ولا تسبب AWS تدير كل حاجة ؟

الإجابة على السؤال ده هتساعدك تختار الخدمة المُناسبة.

4. Selecting the wrong compute solution

اختيار خدمة Compute غير مناسبة ممكن يسبب:

- ضعف في الأداء.
- زيادة في التكلفة.
- استهلاك غير ضروري للموارد.
- صعوبة في التوسع.

5. A good starting place

أول خطوة صح هي إنك تكون فاهم خدمات ال Compute اللي AWS بتوفرها، وبعدها تختار الخدمة المناسبة حسب احتياج التطبيق.

2. Amazon EC2

❖ Amazon Elastic Compute Cloud (Amazon EC2)

قبل ظهور الحوسبة السحابية، الشركات كانت بتشغل التطبيقات على Servers داخل الشركة (On-Premises)، وده كان بيحتاج تكلفة كبيرة سواء في شراء الأجهزة أو تشغيلها وصيانتها.

علشان كده AWS قدمت خدمة Amazon EC2، واللي بتوفر سيرفرات افتراضية تقدر تشغل عليها تطبيقاتك من غير ما تشتري أي Hardware.



♦ مشاكل ال On-Premises Servers

تشغيل السيرفرات داخل الشركة بيكون مكلف لأنه بيحتاج:

- شراء أجهزة (Hardware) باستمرار.
- إنشاء وتجهيز Data Center.
- صيانة الأجهزة وتشغيلها.
- فريق مسؤول عن إدارة البنية التحتية.

♦ مشكلة التخطيط للسعة (Capacity Planning)

وفي ال On-Premises لازم تشتري أجهزة تكفي أعلى حمل ممكن يحصل في المستقبل.

وده معناه إنك أوقات كتير بتشتري أجهزة أكبر من احتياجك الفعلي، عشان تكون مُستعد لأي زيادة في عدد المُستخدمين.

وبعد ما يتم تجهيز السيرفرات، ممكن تفضل شغالة لفترات طويلة وهي مُستغلة جزء بسيط من إمكانياتها.

وده بيؤدي إلى:

- هدر في الموارد.
- زيادة في التكلفة.
- استهلاك كهرباء بدون استفادة حقيقية.

Amazon EC2 هي خدمة بتوفر Virtual Machines (Instances) على AWS. بتسمحك تشغل نفس التطبيقات اللي كنت بتشغلها على السيرفرات التقليدية، لكن على الكلاود (السحابة).

♦ مميزات Amazon EC2

1. بتوفر Compute Capacity في السحابة.
2. الموارد قابلة للزيادة أو التقليل بسهولة (Resizable) حسب احتياج التطبيق.
3. بتوفر بيئة آمنة لتشغيل التطبيقات.
4. بتدفع مقابل فترة استخدام ال Instance فقط.
5. استخدامات Amazon EC2

♦ تقدر تستخدم EC2 لتشغيل أنواع مختلفة من السيرفرات، مثل:

1. Application Servers لتشغيل التطبيقات.
2. Web Servers لإستضافة مواقع الويب.
3. Database Servers لتشغيل قواعد البيانات.
4. Game Servers لتشغيل الألعاب.
5. Mail Servers لإدارة البريد الإلكتروني.
6. Media Servers لتشغيل وبث الملفات الصوتية والمرئية.
7. Catalog Servers لإدارة بيانات المنتجات أو الخدمات.
8. File Servers لتخزين ومشاركة الملفات.
9. Computing Servers لتنفيذ العمليات الحسابية ومعالجة البيانات.
10. Proxy Servers لإستقبال وتميرير طلبات المستخدمين.

❖ Amazon EC2 Overview

اسم EC2 اختصار ل Elastic Compute Cloud، وكل كلمة ليها معنى.

♦ Elastic

كلمة Elastic معناها المرونة.



Amazon
EC2

يعني تقدر تزود أو تقلل عدد ال EC2 Instances بسهولة حسب احتياج التطبيق. وكمان تقدر تغير إمكانيات ال Instance نفسها (CPU - RAM) لو احتجت موارد أكبر أو أقل. الهدف هو استخدام الموارد المطلوبة فقط بدون هدر.

◆ Compute

كلمة Compute معناها موارد المُعالجة.

يعني EC2 بتوفر الموارد اللي التطبيقات محتاجها عشان تشتغل، زي:

- CPU (Processing Power) لمُعالجة البيانات.
- RAM (Memory) لتشغيل البرامج والتطبيقات.

◆ Cloud

كلمة Cloud معناها إن ال Servers موجودة على البنية التحتية الخاصة بـ AWS، مش على أجهزة داخل شركتك.

وبالتالي تقدر تنشئ أو توقف أو تعدل السيرفرات من أي مكان.

• التحكم الكامل في نظام التشغيل

Amazon EC2 بيديك Full Administrative Control على ال Instance.

يعني تقدر:

- تختار نظام التشغيل.
- تثبت البرامج اللي محتاجها.
- تعدل إعدادات النظام.
- تدير السيرفر بالكامل.

EC2 بيدعم أنظمة تشغيل مُختلفة، سواء Windows أو Linux.

تقدر تنشئ أي عدد من ال EC2 Instances، وبأي حجم (Instance Type)، وفي أي Availability Zone داخل أي AWS Region، وكل ده خلال دقائق.

في بيئة ال Virtualization فيه نوعين من أنظمة التشغيل:

◆ Host Operating System

هو نظام التشغيل اللي بيكون مُثبت مُباشرةً على الجهاز (Physical Server)، وبيكون مسؤول عن تشغيل ال Virtual Machines.

◆ Guest Operating System

هو نظام التشغيل اللي بيشتغل داخل ال Virtual Machine (EC2 Instance)، وده اللي أنت بتتعامل معاه وتديره.

• Amazon Machine Image (AMI)

ال AMI هي قالب جاهز (Template) بيحتوي على نظام التشغيل والإعدادات الأساسية.

كل EC2 Instance لازم يتم إنشاؤها من AMI، يعني ال AMI هي نقطة البداية لإنشاء أي Instance.

• Security Groups

ال Security Group ببشتغل ك Virtual Firewall لـ EC2 Instance.

من خلاله تقدر تتحكم في:

- الترافيك الي يدخل لـ Instance (Inbound Traffic).
- الترافيك الي يخرج منها (Outbound Traffic).

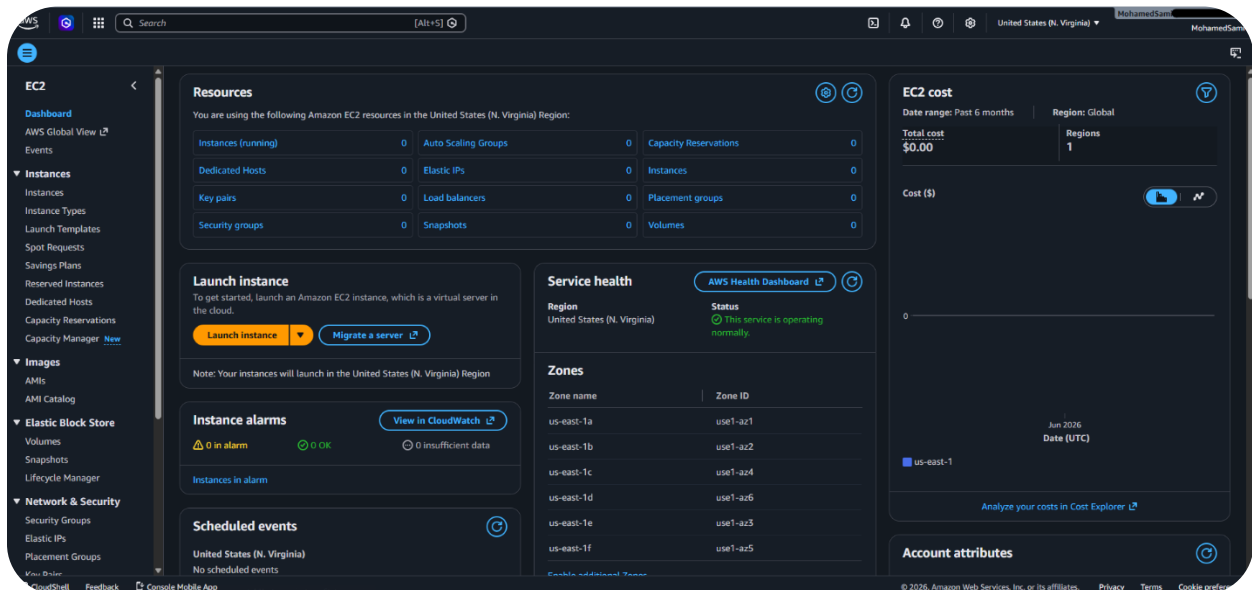
وده بيساعد في تأمين السيرفر.

• التكامل مع خدمات AWS

بما إن EC2 موجودة داخل AWS، فهي تقدر تتكامل بسهولة مع باقي الخدمات، وده بيساعدك تبني حلول متكاملة بإستخدام أكثر من خدمة مع بعض.

❖ Launching an Amazon EC2 Instance

أول مرة تنشئ EC2 Instance، غالبًا هتستخدم Launch Instance Wizard الموجود في AWS Management Console. ال Launch Instance Wizard هو مُعالج بيساعدك تنشئ ال Instance خطوة بخطوة، ويبسهل عليك اختيار الإعدادات المطلوبة. ولو استخدمت الإعدادات الافتراضية (Default Settings)، تقدر تنشئ EC2 Instance بسرعة وبعدد قليل من الخطوات. لكن في أغلب الحالات، الإعدادات الافتراضية مش هتكون مُناسبة، لإن كل تطبيق ليه مُتطلبات مُختلفة. علشان كده غالبًا هتحتاج تعدل الإعدادات قبل إنشاء ال Instance.



◆ تخصيص الإعدادات

أثناء إنشاء ال Instance، هنتختار مجموعة من الإعدادات المهمة، زي:

1. نظام التشغيل (AMI).
2. نوع ال Instance (Instance Type).
3. الشبكة (VPC / Subnet).
4. مساحة التخزين (Storage).
5. قواعد الحماية (Security Group).
6. مُفتاح الإتصال (Key Pair).

كل اختيار من دول بيأثر على طريقة تشغيل ال Instance وأدائها.

في الشرح اللي بعد كده هنوضح كل إعداد من الإعدادات دي بالتفصيل، عشان تعرف:

- وظيفة كل إعداد.
- إمتى تستخدمه.
- إزاي تختار الإعداد المناسب حسب احتياج التطبيق.

1. Select an AMI

أول خطوة عند إنشاء EC2 Instance هي اختيار Amazon Machine Image (AMI).

ال AMI عبارة عن قالب (Template) بيحتوي على كل المعلومات اللي AWS محتاجها عشان تنشئ ال Instance.

يعني مينفعش تنشئ EC2 Instance من غير AMI.

◆ مكونات ال AMI

• Root Volume Template

ال AMI بتحتوي على قالب لل Root Volume، والي بيضم:

- نظام التشغيل (Operating System).
- البرامج المثبتة.
- المكتبات (Libraries).
- الإعدادات الأساسية.

عند إنشاء EC2 جديدة، AWS بتنسخ القالب ده ونستخدمه كنقطة بداية لل Instance.

يبقى ال Root Volume هو اللي بيحتوي على نظام التشغيل وكل الملفات الأساسية.

• Launch Permissions

بتحدد مين يقدر يستخدم ال AMI.

يعني تقدر تخلي ال AMI:

- خاصة بحسابك فقط.
- أو تشاركها مع حسابات AWS ثانية.
- أو تكون متاحة للجميع (Public).

• Block Device Mapping

بتحدد وحدات التخزين (Volumes) اللي هتتربط بال Instance وقت إنشائها.

يعني بتحدد نوع وعدد وأحجام ال Storage اللي هتكون متوصلة بال EC2.

♦ أنواع ال AMIs

1. Quick Start AMIs

دي AMIs جاهزة من AWS.

بتحتوي على أنظمة تشغيل مُختلفة، سواء Linux أو Windows، وتعتبر أسهل اختيار لمُعظم الحالات.

2. My AMIs

دي AMIs أنت اللي أنشأتها بنفسك.

بتستخدمها لو عايز تنشئ Instances جديدة بنفس الإعدادات اللي حضرتها قبل كده.

3. AWS Marketplace AMIs

دي AMIs موجودة في AWS Marketplace.

بتحتوي على أنظمة تشغيل أو برامج أو حلول جاهزة من شركات مختلفة، وتساعدك تبدأ بسرعة.

4. Community AMIs

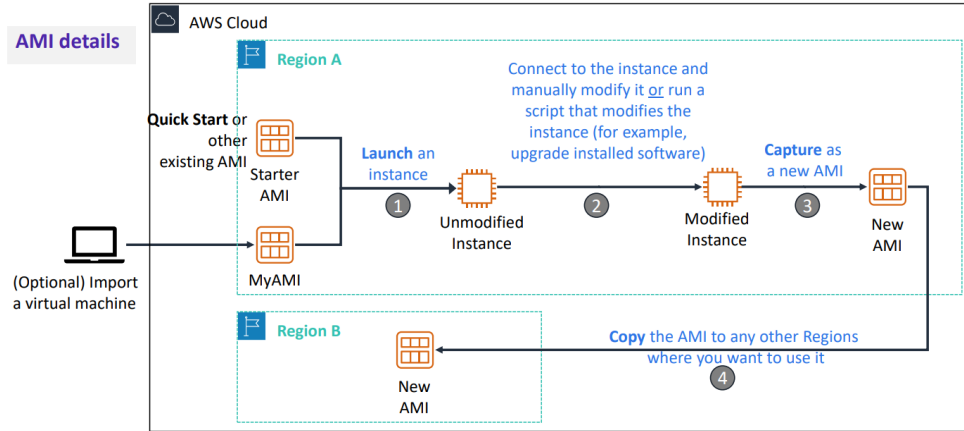
دي AMIs بيعملها مستخدمون من المجتمع.

AWS مش بتراجعها أو تضمناها، لذلك لازم تستخدمها بحذر، ومُفضلش استخدامها في بيئات الإنتاج (Production).

Creating a New AMI ♦

تقدر تنشئ AMI جديدة من EC2 Instance موجودة بالفعل.

الهدف من ده إنك تحتفظ بنسخة جاهزة من السيرفر بكل إعداداته، بحيث تقدر تنشئ منها Instances جديدة في أي وقت.



طرق إنشاء AMI:

تقدر تبدأ بطريقتين:

- استيراد Virtual Machine من خارج AWS، وتحويلها إلى EC2 Instance.
- أو استخدام AMI جاهزة من AWS وإنشاء EC2 Instance منها.

في الحالتين، هيكون عندك EC2 Instance تقدر تعدل عليها.

إنشاء Golden Instance

بعد إنشاء ال EC2 Instance، بتبدأ تضيف كل الإعدادات اللي محتاجها، زي:

- نظام التشغيل.
- البرامج.
- التحديثات.
- الإعدادات الخاصة بالتطبيق.

بعد ما تخلص كل التعديلات، بيبقى عندك Golden Instance.

ال Golden Instance هي EC2 Instance مُجهزة بالكامل وجاهزة للإستخدام كنموذج لإنشاء سيرفرات جديدة.

إنشاء AMI من ال Golden Instance

بعد تجهيز ال Golden Instance، تقدر تنشئ منها AMI جديدة.

أثناء إنشاء ال AMI، AWS بتعمل الخطوات دي:

1. توقف ال EC2 Instance مؤقتًا.
2. تنشئ Snapshot لل Root Volume.
3. تسجل ال Snapshot كـ AMI جديدة.

بعد كده تقدر تستخدم ال AMI دي لإنشاء أي عدد من ال EC2 Instances.

استخدام ال AMI

بعد تسجيل ال AMI، تقدر تستخدمها لإنشاء EC2 Instances جديدة داخل نفس ال AWS Region. ولو محتاج تستخدمها في Region ثانية، تقدر تعمل Copy لل AMI إلى أي Region أخرى.

2. Select an Instance Type

بعد ما تختار ال AMI، لازم تختار Instance Type.

ال Instance Type هو اللي بيحدد إمكانيات ال EC2 Instance، زي:

- CPU (المُعالج).
- Memory (RAM).
- Storage (التخزين).
- Network Performance (أداء الشبكة).

يعني هو اللي بيحدد قوة وإمكانيات السيرفر.

لماذا يوجد أكثر من Instance Type؟

مش كل التطبيقات بتحتاج نفس الموارد.

علشان كده AWS بتوفر أنواع مُختلفة من ال Instance Types، بحيث تختار النوع المُناسب حسب احتياج التطبيق، بدل ما تستخدم موارد أكثر أو أقل من المطلوب.

Instance Sizes

كل Instance Type بيكون فيه أكثر من Size.

ال Size بيحدد حجم الموارد داخل نفس النوع، وبالتالي تقدر تزود أو تقلل الإمكانيات حسب حجم ال Workload.

أنواع Instance Types

1. General Purpose

بتوفر توازن بين:

- CPU
- Memory
- Networking

وتعتبر مُناسبة لمُعظم التطبيقات العامة.

2. Compute Optimized

بتوفر قدرة مُعالجة CPU أعلى.

بتستخدم مع التطبيقات الي بتعتمد بشكل كبير على المُعالج.

3. Memory Optimized

بتوفر RAM أكبر.

بُستخدم مع التطبيقات التي محتاجة ذاكرة كبيرة.

4. Storage Optimized

بتوفر أداء عالي في عمليات القراءة والكتابة على التخزين.

بتناسب التطبيقات التي تتعامل مع كميات كبيرة من البيانات.

5. Accelerated Computing

بتحتوي على وحدات مُعالجة إضافية مثل GPU أو Accelerators.

بتستخدم مع التطبيقات التي محتاجة مُعالجة مُتقدمة.

EC2 Instance Type Naming and Sizes

اسم أي EC2 Instance Type يتكون من 3 أجزاء، وكل جزء بيوضح معلومة عن ال Instance.

مثال: **t3.large**

♦ **T : Family**

أول جزء هو Family.

وده بيحدد نوع ال Instance والفئة التي بتنتمي ليها.

في المثال T، يعني ال Instance من عائلة T، وهي من فئة General Purpose.

♦ **3 : Generation**

الرقم بعد ال Family هو Generation.

في المثال 3، يعني دي الجيل الثالث من عائلة T.

كل ما رقم ال Generation يكون أحدث، كل ما الأداء يكون أفضل، وكفاءة استخدام الموارد أعلى.

♦ **Large : Size**

آخر جزء هو Size.

وده اللي بيحدد حجم الموارد الموجودة في ال Instance، زي:

- عدد ال vCPU.
- حجم ال RAM.
- أداء الشبكة.

كل ما ال Size يكبر، الموارد تزيد.

:Instance Sizes

Instance Name	vCPU	Memory (GB)	Storage
t3.nano	2	0.5	EBS-Only
t3.micro	2	1	EBS-Only
t3.small	2	2	EBS-Only
t3.medium	2	4	EBS-Only
t3.large	2	8	EBS-Only
t3.xlarge	4	16	EBS-Only
t3.2xlarge	8	32	EBS-Only

داخل نفس ال Family، فيه أحجام مُختلفة.

ملاحظات مهمة:

كل ما تكبر ال Size:

- عدد ال vCPU يزد.
- حجم ال RAM يزد.
- Network Bandwidth تزد.
- كل مستوى أكبر من الي قبله في الموارد.

لما تلاقي في الجدول مكتوب EBS-Only، ده معناه إن ال EC2 Instance نفسها مفيهاش هارد داخلي لتخزين البيانات. يعني لو شغلت ال Instance، لازم توصلها بوحدة تخزين من خدمة Amazon EBS، وهي الي هيتخزن عليها:

- نظام التشغيل (Operating System).
- الملفات.
- البرامج.
- البيانات.

بمعنى تاني:

- EBS-Only = التخزين هيكون على Amazon EBS.
- مش EBS-Only = ممكن يكون فيه تخزين محلي (Instance Store) موجود مع ال Instance نفسها.

كل ما ال Size يكبر، غالباً أداء الشبكة كمان يزد.

ولو التطبيق بيعتمد على نقل بيانات بكميات كبيرة، لازم تختار Size أكبر عشان تحصل على Network Bandwidth أعلى.

Select Instance Type Based on Use Case

ال EC2 Instance Types مش كلها زي بعض، لكنها بتختلف في:

- CPU Type → نوع المعالج.
- CPU/Core Count → عدد أنوية أو وحدات المعالجة (vCPU).
- Memory Amount → حجم الذاكرة (RAM).
- Storage Type → نوع التخزين.
- Storage Amount → حجم التخزين.
- Network Performance → أداء وسرعة الشبكة.

علشان كده لازم تختار ال Instance Type المناسبة حسب احتياجات التطبيق.

Instance type details

	 General Purpose	 Compute Optimized	 Memory Optimized	 Accelerated Computing	 Storage Optimized
Instance Types	a1, m4, m5, t2, t3	c4, c5	r4, r5, x1, z1	f1, g3, g4, p2, p3	d2, h1, i3
Use Case	Broad	High performance	In-memory databases	Machine learning	Distributed file systems

1. T3 Instance (General Purpose)

ال T3 من فئة General Purpose، يعني بتوفر توازن بين:

- المُعالج (CPU).
- الذاكرة (RAM).
- الشبكة (Network).

كمان بتدعم Burstable Performance، وده معناه إن ال Instance بيكون لديها مستوى أساسي (Baseline) من أداء ال CPU، ولو التطبيق احتاج قدرة مُعالجة أعلى لفترة قصيرة، تقدر تستخدم قوة إضافية مؤقتًا، وبعدها ترجع للمستوى الطبيعي.

يعني هي مُناسبة للتطبيقات اللي استهلاكها للمُعالج مش بيكون عالي طول الوقت.

2. C5 Instance (Compute Optimized)

ال C5 من فئة Compute Optimized.

يعني تم تصميمها للتطبيقات اللي بتحتاج قوة مُعالجة كبيرة.

بتركز على توفير:

- عدد أكبر من ال CPU.
- أداء أعلى للمعالج.
- أفضل كفاءة في تشغيل العمليات الحسابية.

3. R5 Instance (Memory Optimized)

ال R5 من فئة Memory Optimized.

يعني تم تصميمها للتطبيقات التي تحتاج RAM كبيرة.

بتركز على توفير:

- حجم ذاكرة أكبر.
- أداء أفضل للتطبيقات التي تعتمد على الذاكرة بشكل أساسي.

أهم الحاجات التي بتيجي في الإمتحان

T3

- General Purpose
- Burstable CPU
- مناسب للأحمال العادية أو المتغيرة.

C5

- Compute Optimized
- High CPU
- مناسب للتطبيقات التي تعتمد على المعالجة.

R5

- Memory Optimized
- High RAM
- مناسب للتطبيقات التي تعتمد على الذاكرة.

Networking Features

عند اختيار EC2 Instance Type، متبصش بس على:

- CPU
- RAM
- Storage

لازم كمان تبص على Network Performance، لأن سرعة الشبكة بتختلف من Instance Type للتانية.

كل Instance Type ليها سرعة شبكة مُختلفة، وبتتقاس بوحدة (Gbps (Gigabits per second.

كل ما ال Instance تكون أقوى أو أكبر، غالبًا هتلاقي سرعة الشبكة أعلى.

لازم تختار Instance Type توفر سرعة الشبكة المناسبة للتطبيق.

يعني ال Network Bandwidth بتختلف حسب ال Instance Type.

بشكل افتراضي، AWS تحاول توزيع الـ EC2 Instances على أجهزة فعلية مُختلفة، وده علشان لو جهاز حصل فيه مُشكلة، متتأثرش كل الـ Instances مع بعض.

لكن لو عندك مجموعة EC2 Instances بتتواصل مع بعض باستمرار، تقدر تستخدم Placement Group. الـ Placement Group بتخلي الـ Instances تكون قريبة من بعض داخل نفس Availability Zone، وده يؤدي إلى:

- تقليل Latency (زمن التأخير).
 - زيادة Network Throughput (سرعة نقل البيانات بين الـ Instances).
- يعني الـ Placement Group هدفه تحسين الإتصال بين مجموعة EC2 Instances.

بعض الـ Instance Types بتدعم ميزة اسمها Enhanced Networking. الميزة دي بتحسن أداء الشبكة عن طريق:

- زيادة سرعة نقل البيانات.
- تقليل Latency.
- تقليل Network Jitter (اختلاف أو تذبذب زمن وصول البيانات).
- زيادة عدد الـ Packets Per Second (PPS)، يعني عدد حزم البيانات اللي تقدر تنتقل في الثانية.

أنواع Enhanced Networking

1. Elastic Network Adapter (ENA)

هو أحدث نوع من تقنيات Enhanced Networking. بيدعم سرعات تصل إلى 100 Gbps. مدعوم في معظم الـ Instance Types الحديثة.

2. Intel 82599 Virtual Function

نوع أقدم من تقنيات Enhanced Networking. بيدعم سرعات تصل إلى 10 Gbps.

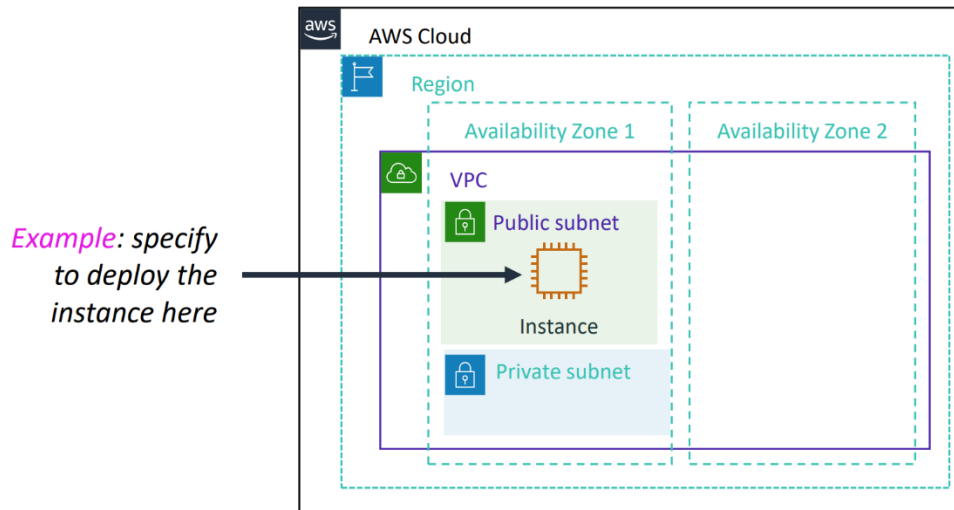
3. Specify network settings

بعد ما تختار:

- AMI
- Instance Type

تيجي خطوة تحديد إعدادات الشبكة (Network Settings)، يعني هتحدد الـ EC2 هتشتغل فين وإزاي هتتصل بالشبكة. قبل ما تبدأ إنشاء الـ EC2، لازم تتأكد إنك واقف على الـ AWS Region الصح. وده لإن الـ EC2 هتتنشئ داخل الـ Region اللي أنت مختارها.

بعد كده بتحدد ال VPC اللي هتشتغل فيها ال EC2، وال Subnet اللي هتكون موجودة فيها. وده بيحدد مكان ال Instance داخل الشبكة.



أثناء إنشاء ال EC2، AWS ممكن تديها Public IP Address أو لأ، حسب نوع ال VPC وال Subnet.

♦ لو استخدمت Default VPC

لو أنشأت ال EC2 داخل Default VPC، ف AWS بتديها Public IP Address تلقائياً (Default). يعني هتقدر توصل لل EC2 من الإنترنت إذا كانت باقي الإعدادات (زي ال Security Group) تسمح بذلك.

♦ لو استخدمت Nondefault VPC

لو أنشأت ال EC2 داخل VPC أنت اللي عاملها (Nondefault VPC)، ف AWS مش هتديها Public IP بشكل تلقائي. في الحالة دي، لازم تحدد بنفسك إذا كنت عايز ال Instance تاخد Public IP ولا لأ.

تقدر تتحكم في إعطاء ال Public IP بطريقتين:

- تغيير إعدادات ال Subnet نفسها.
- أو أثناء إنشاء ال EC2، تختار Enable أو Disable Public IP، وده هيكون له الأولوية على إعدادات ال Subnet.

4. Attach IAM Role (Optional)

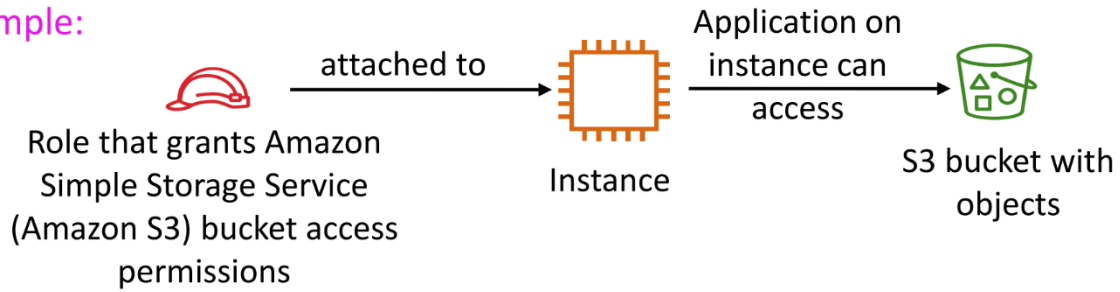
لو التطبيق اللي شغال على ال EC2 Instance محتاج يتعامل مع خدمات AWS تانية (زي S3 أو DynamoDB أو SQS)، فهو محتاج صلاحيات عشان يقدر يستخدم الخدمات دي.

وأفضل طريقة لعمل ده هي IAM Role.

ال IAM Role عبارة عن مجموعة من Permissions بتتربط بال EC2 Instance، وبتمسح للتطبيق اللي شغال عليها إنه ينفذ API Calls لخدمات AWS المسموح بيها.

يعني بدل ما التطبيق يستخدم اسم مُستخدم وكلمة سر، بيستخدم ال IAM Role للحصول على الصلاحيات بشكل آمن.

Example:



لماذا نستخدم IAM Role؟

AWS بتنصح بعدم تخزين AWS Credentials (زي Access Key و Secret Key) داخل الـ EC2 Instance.

لأن لو حد وصل للسيرفر، ممكن يسرق بيانات الدخول ويستخدم حساب AWS.

بدل كده، اربط IAM Role بالـ EC2، و AWS هتوفر الصلاحيات المطلوبة بشكل آمن.

الـ Instance Profile هو عبارة عن Container ببيحتوي على الـ IAM Role.

لما تنشئ IAM Role خاصة بـ EC2 من خلال AWS Console، AWS بتنشئ Instance Profile تلقائياً بنفس اسم الـ Role.

وعند إنشاء الـ EC2، اللي بتختاره في الحقيقة هو الـ Instance Profile اللي مرتبط بالـ IAM Role.

تقدر تربط الـ IAM Role أثناء إنشاء الـ EC2 Instance، أو بعد ما الـ EC2 تكون شغالة بالفعل.

يعني مش لازم تحدد وقت الإنشاء فقط.

الـ IAM Role بيحدد:

- مين يقدر يستخدم الـ Role (زي EC2).
- إيه الصلاحيات المسموح بيها.
- إيه الـ API Actions اللي التطبيق يقدر ينفذها على خدمات AWS.

لما تنشئ IAM Role خاصة بـ EC2، بتحدد حاجتين أساسيتين:

1. من يقدر يستخدم الـ Role؟ (Assume Role)

بتحدد مين المسموح له يستخدم الـ Role، سواء خدمة Amazon EC2، أو أي خدمة AWS ثانية حسب احتياجك.

2. ما هي الصلاحيات؟ (Permissions)

بتحدد:

- API Actions: العمليات اللي التطبيق يقدر ينفذها على خدمات AWS.
- Resources: الموارد اللي التطبيق يقدر يوصل لها.

يعني بتحدد إيه اللي التطبيق يقدر يعمل، وعلى أي موارد يقدر يعمل.

لو عدلت صلاحيات الـ IAM Role، فالتعديل بيتطبق تلقائياً على كل الـ EC2 Instances اللي مرتبطة بنفس الـ Role، يعني مش محتاج تدخل تعدل كل Instance واحدة واحدة.

5. User data script (optional)

أثناء إنشاء EC2 Instance، تقدر تخصيص User Data Script.

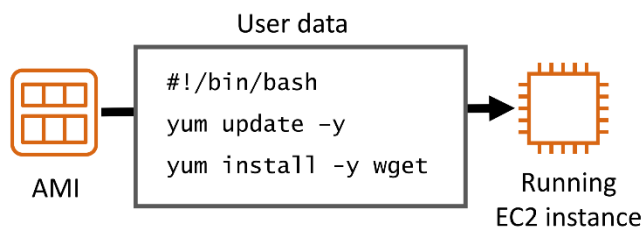
ال User Data Script هو عبارة عن Script بيتنفذ تلقائياً أول ما ال EC2 تشتغل، ويستخدم لتنفيذ إعدادات أو تثبيت برامج بشكل أوتوماتيكي.

لماذا نستخدم User Data؟

بدل ما تدخل على ال EC2 بعد إنشائها وتبدأ تثبت البرامج أو تعمل الإعدادات يدوياً، تقدر تكتب كل الأوامر في ال User Data Script، وهي تنفذ تلقائياً أثناء تشغيل ال Instance لأول مرة.

وده يساعد في:

- تحديث نظام التشغيل.
- تثبيت البرامج المطلوبة.
- تنفيذ الإعدادات الأساسية.
- تشغيل أوامر تلقائياً بعد إنشاء ال Instance.



أنوع ال Script

نوع ال Script يختلف حسب نظام التشغيل.

Linux

لو ال EC2 بتشتغل بنظام Linux، بيكون ال Script مكتوب بلغة Bash.

Windows

لو ال EC2 بتشتغل بنظام Windows، بيكون ال Script مكتوب باستخدام:

- Command Prompt (Batch)
- PowerShell أو

صلاحيات تنفيذ ال Script

عند تشغيل ال EC2، ال User Data Script بيتنفذ بصلاحيات Root في Linux، يعني بيكون عنده أعلى مستوى من الصلاحيات.

متى يتم تشغيل ال User Data؟

بشكل افتراضي، ال User Data Script بيتنفذ مرة واحدة فقط، وهي أول مرة يتم فيها تشغيل ال EC2 Instance.

ولو عايز ال Script يشتغل في كل مرة ال Instance تعمل Boot، لازم تستخدم إعدادات خاصة (MIME Multipart)، لكن ده مش شائع.

6. Storage Options

أثناء إنشاء EC2 Instance، لازم تحدد إعدادات التخزين اللي هتستخدمها ال Instance.

التخزين هو المكان اللي هيتحفظ عليه:

- نظام التشغيل.
- البرامج.
- الملفات.
- البيانات.

Root Volume

كل EC2 Instance بيكون فيها Root Volume.

وده هو القرص الأساسي اللي بيتثبت عليه نظام التشغيل (Operating System).

تقدر تحدد:

- حجم ال Root Volume.
- نوع التخزين المستخدم.

Additional Storage Volumes

لو التطبيق محتاج مساحة أكبر، تقدر تضيف Storage Volumes إضافية أثناء إنشاء ال EC2.

وبالتالي ممكن يكون عندك:

- Root Volume
- Volume أو أكثر إضافيين لتخزين البيانات.

Storage Configuration

لكل Volume تقدر تحدد:

- Size: حجم القرص.
- Volume Type: نوع وحدة التخزين.
- Delete on Termination: هل يتم حذف ال Volume عند حذف ال EC2 أم يتم الاحتفاظ به.
- Encryption: هل يتم تشفير البيانات المخزنة على ال Volume أم لا.

Delete on Termination

تقدر تحدد إذا كان ال Volume:

- يتم حذفه تلقائيًا عند حذف ال EC2 Instance.
- أو يفضل موجود حتى بعد حذف ال Instance، بحيث تقدر تستخدم البيانات مرة ثانية.

Encryption

تقدر تفعل Encryption لتشفير البيانات المخزنة على ال Volume. وده بيحافظ على سرية البيانات ويزود مستوى الأمان.

Amazon EC2 Storage Options

AWS بتوفر أكثر من نوع تخزين تقدر تستخدمه مع EC2 Instance، وكل نوع مناسب لإستخدام مُعين.

1. Amazon Elastic Block Store (Amazon EBS)

Amazon EBS هو خدمة Block Storage مُصممة للعمل مع EC2 Instances.

يُستخدم كوحدة التخزين الأساسية أو كقرص إضافي، ويتميز إنه:

- سهل الإستخدام.
- أداء عالي.
- بياناته دائمة (Persistent)، يعني البيانات بتفضل موجودة حتى لو أوقفت ال EC2.
- تقدر تغير حجم ال Volume أو نوعه بدون التأثير على تشغيل التطبيق.

2. Amazon EC2 Instance Store

ال Instance Store هو Block Storage موجود على الجهاز الفعلي اللي مُستضيف ال EC2.

بيتميز إنه:

- سريع جدًا.
- مناسب للبيانات المؤقتة.

لكن أهم نقطة فيه:

لو ال EC2 Instance اتوقفت (Stop) أو حصل لها عطل أو اتعملها Terminate، كل البيانات الموجودة على Instance Store بتضيع.

عشان كده يُسمى Ephemeral Storage أو Temporary Storage.

3. Amazon Elastic File System (Amazon EFS)

Amazon EFS هو خدمة File Storage.

بيوفر نظام ملفات (File System) تقدر توصله بأكثر من EC2 Instance في نفس الوقت.

كمان بيتميز إنه:

- بيتوسع تلقائياً حسب حجم البيانات.
- مفيش حاجة اسمها تحدد حجم التخزين من البداية.
- AWS بتزود أو تقلل المساحة تلقائياً.

4. Amazon Simple Storage Service (Amazon S3)

Amazon S3 هو خدمة Object Storage.

يُستخدم لتخزين أي كمية من البيانات بطريقة آمنة وقابلة للتوسع.

يتميز بـ:

- قابلية توسع عالية جدًا.
- توافر عالي.
- أمان مرتفع.
- مُناسب لتخزين الملفات والنسخ الاحتياطية والأرشفة والبيانات الضخمة.

أهم الحاجات التي بتيجي في الإمتحان

Amazon EBS

- Block Storage
- Persistent
- يستخدم مع EC2.
- يمكن تغيير الحجم أو النوع.

Instance Store

- Block Storage
- Temporary (Ephemeral)
- البيانات تضيع عند Stop أو Terminate لل Instance.

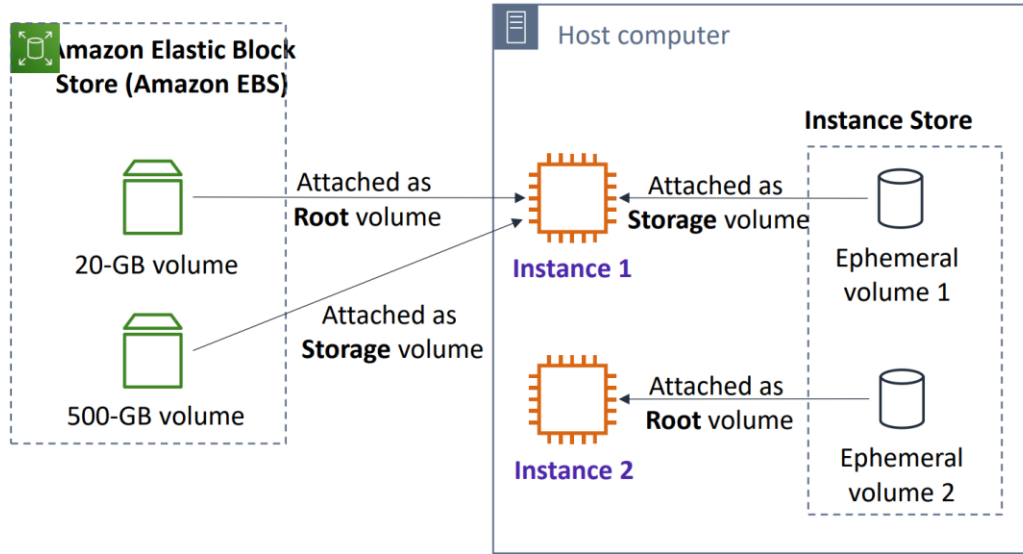
Amazon EFS

- File Storage
- يمكن توصيله بأكثر من EC2.
- يتوسع تلقائيًا.

Amazon S3

- Object Storage
- قابلية توسع عالية.
- يُستخدم لتخزين الملفات والبيانات.

مثالين لطريقة تخزين البيانات في EC2 Instances.



المثال الأول (Instance 1)

في المثال ده فيه Root Volume موجود على Amazon EBS.

وفيه كمان:

- Volume إضافي من Amazon EBS.
- Volume من Instance Store.

لو ال EC2 اتعملها Stop ثم Start

- بيانات ال Root Volume (EBS) هتفضل موجودة.
- بيانات ال EBS Volume الإضافي هتفضل موجودة.
- أما البيانات الموجودة على Instance Store هتضيع نهائياً.

وده لأن Instance Store تخزين مؤقت (Temporary).

المثال الثاني (Instance 2)

في المثال ده ال Root Volume نفسه موجود على Instance Store.

وده معناه إن نظام التشغيل نفسه موجود على تخزين مؤقت.

لو ال Instance اتعملها Terminate أو حصل عطل أدى لإيقافها نهائياً:

- نظام التشغيل هيتضيع.
- كل البيانات هتضيع.
- مش هتقدر تشغل نفس ال Instance مرة ثانية.

علشان كده AWS بتنصح بعدم استخدام Instance Store لتخزين البيانات المهمة أو الدائمة.

7. Add Tags

ال Tag هو عبارة عن Label (تصنيف أو علامة) بتضيفها لأي مورد (Resource) في AWS، علشان يسهل التعرف عليه وإدارته.

كل Tag بيتكون من:

- **Key**: اسم ال Tag.
- **Value**: قيمة ال Tag (اختيارية).

Key (128 characters maximum)	Value (256 characters maximum)
Name	WebServer1
Add another tag (Up to 50 tags maximum)	

ال Tags بتستخدم لتنظيم وتصنيف موارد AWS.

وبالتالي تقدر:

- تميز الموارد عن بعضها.
- تبحث عن الموارد بسهولة.
- تعمل فلتر (Filter) للموارد.
- تسهل إدارة عدد كبير من الموارد.

ال Tags تعتبر Metadata، يعني معلومات إضافية بتتوصف بيها ال Resource، لكنها مش بتأثر على تشغيلها.

ال Tag Keys وال Tag Values حساسة لحالة الأحرف (Case Sensitive).

يعني:

- Name
- name

دول يعتبروا مُفتاحين مُختلفين.

AWS بتعرض ال Tag Key اللي اسمه Name (بحرف N كبير) في قائمة ال EC2 Instances بشكل افتراضي.

أما لو كتبت name بحرف صغير، هيشغل عادي، لكن مش ه يظهر في عمود Name.

Best Practice

من أفضل الممارسات إنك تستخدم Tagging Strategy موحدة.

يعني تستخدم نفس أسماء ال Tags على كل الموارد، لأن ده بيسهل إدارتها والبحث عنها.

8. Security Group Settings

ال Security Group هو عبارة عن Virtual Firewall (جدار حماية افتراضي) يحمي ال EC2 Instance، ويبتحكم في حركة البيانات التي داخلة أو خارجة منها.

أثناء إنشاء ال EC2 Instance، لازم تختار:

- إنشاء Security Group جديدة.
 - أو استخدام Security Group موجودة بالفعل داخل نفس ال VPC.
- ولو مختارتش واحدة، هيتم استخدام Default Security Group.

Security Group Rules

ال Security Group بيحتوي على مجموعة من Rules.

ال Rules هي التي بتحدد:

- مين يقدر يدخل على ال EC2.
- وإيه الترافيك اللي يقدر يخرج منها.

Inbound Rules

ال Inbound Rules بتتحكم في الترافيك الداخلى إلى ال EC2.

يعني بتحدد:

- مين يقدر يوصل لل Instance.
- وعلى أي Port.
- وبأي Protocol.

Outbound Rules

ال Outbound Rules بتتحكم في الترافيك الخارج من ال EC2.

يعني بتحدد ال EC2 تقدر تبعت بيانات لمين.

Destination و Source

عند إنشاء Rule، تقدر تحدد مصدر أو وجهة الإتصال.

ممکن يكون:

- IP Address
- IP Range
- Security Group أخرى.
- VPC Endpoint
- أو Anywhere، وده معناه السماح لأي مصدر.

القواعد بتطبيق إمتى؟

أي تعديل عمله على Security Group بيتطبق فوراً على كل ال EC2 Instances المرتبطة بيها.
يعني مش محتاج تعمل Restart لل Instance.

ال EC2 Instance ممكن تكون مرتبطة بأكثر من Security Group.
وفي الحالة دي AWS بتجمع كل ال Rules الموجودة في كل ال Security Groups وتطبقها مع بعض.

القواعد الافتراضية (Default Rules)

بشكل افتراضي Outbound Traffic مسموح بالكامل.
أما ال Inbound Traffic، فلازم تضيف له Rules حسب احتياجك.
ولو حذفت كل ال Outbound Rules، فال EC2 مش هتقدر تبعث أي بيانات للخارج.

مثال على Security Group Rule

Type ⓘ	Protocol ⓘ	Port Range ⓘ	Source ⓘ
SSH	TCP	22	My IP 72.21.198.67/32

في المثال الموجود، تم إنشاء Inbound Rule تسمح بالاتصال باستخدام:

- Type :SSH
- Protocol :TCP
- Port :22
- Source :My IP (72.21.198.68)

وده معناه إن ال EC2 هتسمح باتصالات SSH على المنفذ 22، لكن فقط من عنوان ال IP الحالي بتاعك.
لما تختار My IP، AWS بتحدد تلقائياً عنوان ال Public IP اللي أنت متصل بيه وقت إنشاء ال Rule.
وبالتالي، أنت فقط من هتقدر تعمل اتصال بال EC2 من خلال ال SSH، طالما عنوان ال IP بتاعك متغيرش.
ال (SSH (Secure Shell هو بروتوكول بيستخدم للاتصال وإدارة سيرفرات Linux EC2 عن بُعد بشكل آمن.
وبيستخدم بشكل افتراضي:

- Protocol :TCP
- Port :22

بالإضافة إلى Security Groups، فيه خدمة تانية اسمها Network ACL (Access Control List).
ال Network ACL هي كمان عبارة عن Firewall، لكنها بتحمي ال Subnet بالكامل داخل ال VPC، مش ال EC2 Instance نفسها.

9. Identify or create the key pair

بعد ما تخلص كل إعدادات ال EC2 Instance، وتضغط Launch، AWS هتطلب منك تختار:

- Key Pair موجودة.
- أو تنشئ Key Pair جديدة.
- أو تكمل بدون Key Pair (وده غير مستحب لو محتاج تدخل على ال Instance).

ما هي ال Key Pair؟

ال Key Pair عبارة عن مُفتاحين يُستخدموا في Public-Key Cryptography لتأمين عملية تسجيل الدخول إلى ال EC2. وتتكون من:

- Public Key
- Private Key

كيف تعمل؟

- **Public Key**: AWS بتحطه تلقائياً على ال EC2 Instance
 - **Private Key**: بتقوم بتحميله والاحتفاظ بيه عندك، وتستخدمه للدخول على ال Instance.
- يعني الإتصال بيتتم بإستخدام Private Key بدل استخدام كلمة مرور.

حفظ ال Private Key

لو أنشأت Key Pair جديدة، لازم تحمل ملف ال Private Key وتحفظه في مكان آمن. وده لإن AWS مش هتسمحلك تنزله مرة ثانية بعد إنشاء ال Key Pair.

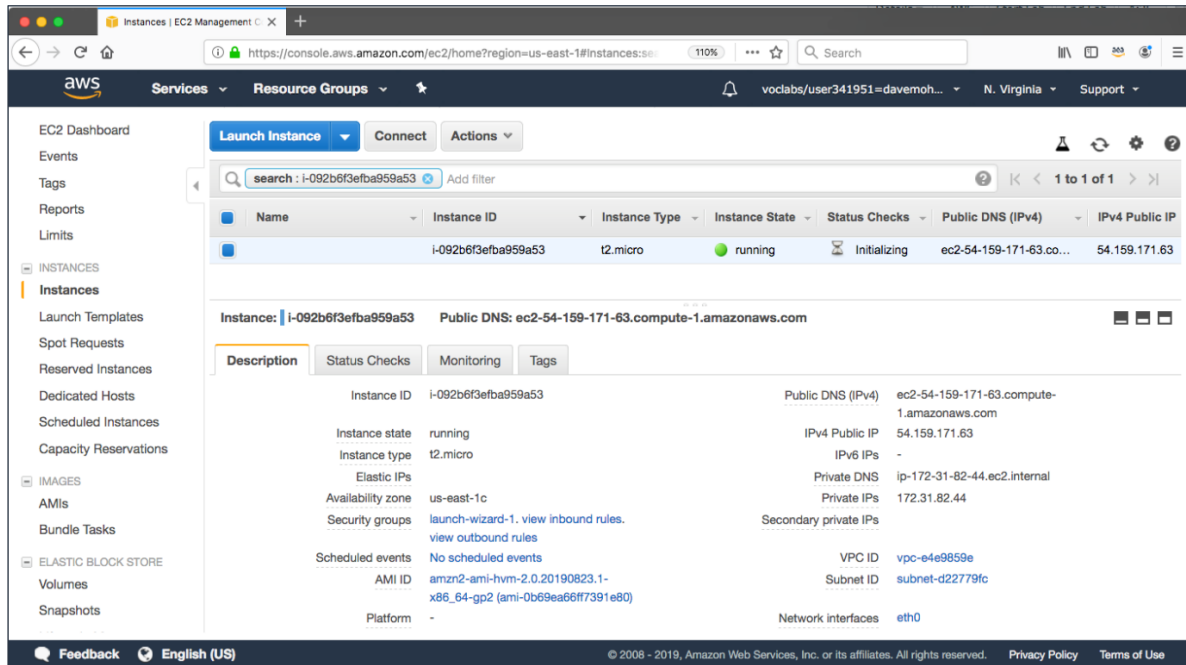
الإتصال بـ Windows EC2

لو ال EC2 بتستخدم Windows بتستخدم ال Private Key للحصول على Administrator Password، وبعدها تدخل على السيرفر بإستخدام (RDP) Remote Desktop Protocol.

الإتصال بـ Linux EC2

لو ال EC2 بتستخدم Linux، AWS بتحط ال Public Key داخل ال Instance أثناء تشغيلها. ولما تعمل إتصال بإستخدام SSH، لازم تقدم ال Private Key لإثبات هويتك.

Amazon EC2 console view of a running EC2 instance ❖



بعد ما تعمل Launch الـ EC2 Instance وتختار View Instances، هتظهرلك صفحة فيها كل المعلومات الخاصة بالـ Instance. في الصفحة دي هتلاقي أغلب الإعدادات اللي اخترتها أثناء إنشاء الـ Instance، بالإضافة إلى معلومات تعريفية عنها.

أهم المعلومات اللي هتظهر

• **Public IP Address**

عنوان الـ Public IP الخاص بالـ EC2، وبستخدامه لو هتتصل بالـ Instance من خلال الإنترنت.

• **DNS Address**

عنوان DNS الخاص بالـ EC2، وده اسم بدل ما تستخدم عنوان الـ IP.

• **Instance Type**

نوع الـ EC2 Instance اللي اخترته، زي:

- t3.micro
- t3.large
- c5.large

• **Instance ID**

رقم تعريف (ID) فريد لكل EC2 Instance.

كل Instance بيكون ليها Instance ID مختلف عن أي Instance تانية.

• AMI ID

هو رقم تعريف ال Amazon Machine Image (AMI) التي تم استخدامها لإنشاء ال EC2.

• VPC ID

رقم تعريف ال VPC التي موجودة فيها ال EC2 Instance.

• Subnet ID

رقم تعريف ال Subnet التي انتشرت فيها ال EC2 داخل ال VPC.

• Hyperlinks

مُعظم المعلومات دي بيكون عليها Hyperlinks.

يعني تقدر تضغط عليها علشان تفتح المورد نفسه، زي:

- ال VPC.
- ال Subnet.
- ال AMI.
- أو أي Resource مُرتبط بال EC2.

❖ Launch an EC2 Instance with the AWS CLI

بدل ما تنشئ EC2 Instance من خلال AWS Management Console، تقدر تنشئها باستخدام ال Interface (AWS CLI) أو باستخدام AWS SDKs داخل البرامج والتطبيقات.

وده بيسمح بإنشاء وإدارة ال EC2 بشكل آلي (Automation) ومن خلال الأوامر البرمجية.



AWS Command Line
Interface (AWS CLI)

AWS CLI

ال AWS CLI هي أداة سطر أوامر (Command Line Tool) بتسمح لك بتنفيذ أوامر AWS من خلال ال Terminal أو Command Prompt بدون الحاجة لإستخدام ال Console.

أمر إنشاء EC2

لإنشاء EC2 باستخدام ال AWS CLI بنستخدم الأمر:

`aws ec2 run-instances`

بعدها بنحدد مجموعة من Parameters التي بتوصف ال Instance التي عايزين ننشئها.

Example command:

```
aws ec2 run-instances \
--image-id ami-1a2b3c4d \
--count 1 \
--instance-type c3.large \
--key-name MyKeyPair \
--security-groups MySecurityGroup \
--region us-east-1
```

أهم ال Parameters

• image-id

بيحدد AMI ID اللي هيتم إنشاء ال EC2 منها.

• count

بيحدد عدد ال EC2 Instances اللي هيتم إنشاؤها.

• instance-type

بيحدد نوع ال EC2 Instance، زي:

- t3.micro
- t3.large
- c5.large

• key-name

بيحدد اسم ال Key Pair اللي هتستخدمها للدخول على ال EC2، وللازم تكون ال Key Pair موجودة بالفعل.

• security-groups

بيحدد ال Security Group اللي هتتريبط بال EC2، وللازم تكون موجودة بالفعل.

• region

بيحدد ال AWS Region اللي هيتم إنشاء ال EC2 فيها، وللازم تكون ال AMI موجودة في نفس ال Region.

شروط نجاح الأمر

علشان الأمر ينجح، لازم:

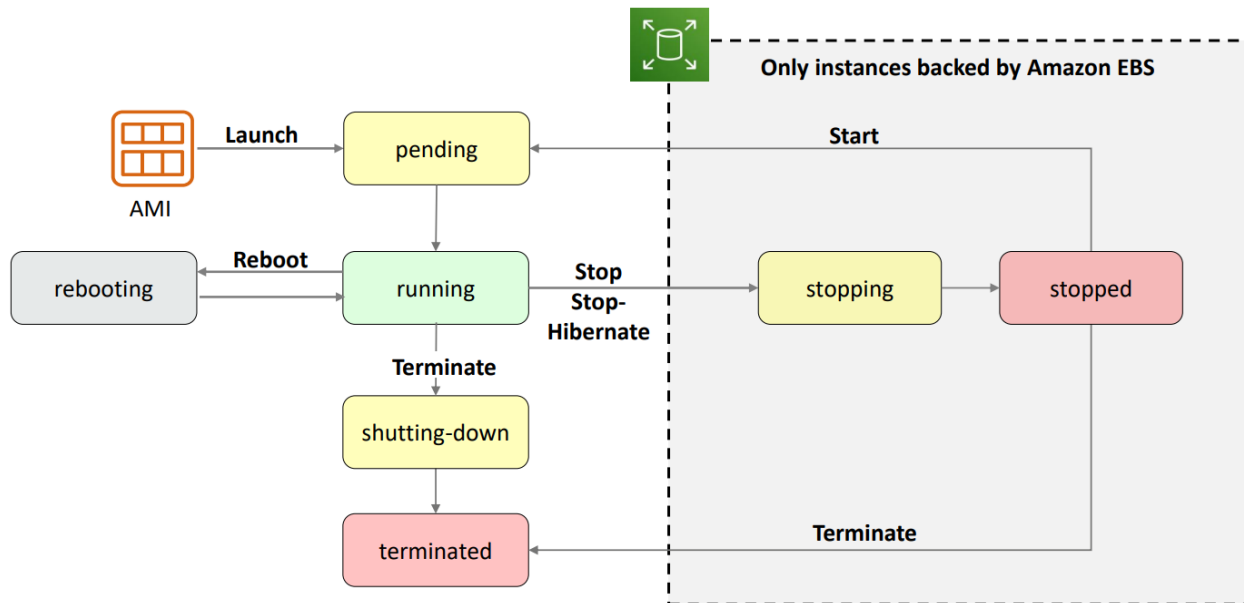
- يكون الأمر مكتوب بشكل صحيح.
- ال AMI وال Key Pair وال Security Group موجودين بالفعل.
- يكون عندك صلاحيات (IAM Permissions) تسمح بإنشاء ال EC2.
- يكون عندك موارد وسعة متاحة في حساب ال AWS.

نتيجة تنفيذ الأمر

لو الأمر نجح، ال AWS بترجع معلومات عن ال Instance الجديدة، أهمها:

- Instance ID.
- وبعض البيانات الأخرى اللي ممكن تستخدمها في أوامر أو عمليات لاحقة.

Amazon EC2 instance lifecycle ❖



ال Instance Lifecycle هي المراحل (States) التي يمر بها ال EC2 Instance من وقت إنشائها لحد ما يتم حذفها.

1. Pending

دي أول حالة بيدخلها ال EC2 بعد ما تعمل Launch أو بعد ما تعمل Start ل Instance كانت متوقفة.

في المرحلة دي AWS بتقوم بـ:

- إنشاء ال Instance.
- تشغيل نظام التشغيل.
- تجهيزها للعمل.

بعد ما تخلص التجهيز، تنتقل لحالة Running.

2. Running

دي الحالة الطبيعية لل EC2.

في الحالة دي:

- ال Instance شغالة بالكامل.
- تقدر تتصل بيها.
- التطبيقات بتكون شغالة.
- يتم احتساب تكلفة التشغيل.

3. Rebooting

دي حالة إعادة تشغيل (Restart) للـ EC2.

AWS بتنصح تعمل Reboot من خلال:

- AWS Console
- AWS CLI
- AWS SDK

أثناء الـ Reboot:

- الـ Instance تفضل على نفس الجهاز (Host).
- تحتفظ بنفس Public IP (إذا كان متغيرًا ديناميكيًا فلا يتغير مع الـ Reboot).
- تحتفظ بنفس Public DNS.
- بيانات الـ Instance Store لا تضيع.

4. Stopping

دي مرحلة انتقالية قبل ما الـ EC2 تقف بالكامل.

الحالة دي موجودة فقط للـ EC2 اللي بتستخدم Amazon EBS كـ Root Volume.

5. Stopped

في الحالة دي:

- الـ Instance متوقفة.
- نظام التشغيل مش شغال.
- مش تقدر تتصل بيها.

لكن:

- البيانات الموجودة على Amazon EBS تفضل موجودة.
 - تقدر تعمل لها Start في أي وقت.
- كمان تكلفة التشغيل بتتوقف، لكن هتفضل تدفع تكلفة تخزين الـ EBS.

6. Shutting Down

دي مرحلة انتقالية بين: Running → Terminated

يعني الـ EC2 بيتم إنهاؤها.

7. Terminated

دي آخر مرحلة في الدورة.

في الحالة دي:

- ال Instance اتحذفت نهائياً.
- مش هتقدر تعمل لها Start تاني.
- مش هتقدر تتصل بيها.
- مش هتقدر تسترجعها.

لو عملت Stop لل EC2:

Running



Stopping



Stopped

ولما تعمل Start:

Stopped



Pending



Running

قد يتم تشغيل ال Instance على Host مُختلف عن اللي كانت عليه قبل الإيقاف.

معناها	الحالة
جاري إنشاء وتشغيل ال EC2	Pending
ال EC2 شغالة وجاهزة للاستخدام	Running
إعادة تشغيل بدون فقدان البيانات	Rebooting
مرحلة قبل التوقف	Stopping
ال EC2 متوقفة ويمكن تشغيلها مرة أخرى	Stopped
مرحلة قبل الحذف النهائي	Shutting Down
حذف نهائي لل EC2	Terminated

❖ Consider using an Elastic IP address

ال Public IP Address هو عنوان IPv4 يسمح لل EC2 Instance إنها تكون مُتاحة عبر الإنترنت.
ولما ال EC2 تأخذ Public IP، AWS بتديها كمان Public DNS Name تقدر تستخدمه للوصول إليها.
لو فعلت خيار Assign Public IP أثناء إنشاء ال EC2، AWS هتخصص عنوان Public IP من مجموعة عناوينها العامة.
لكن لازم تعرف إن ال Public IP ده مش ثابت، ومش مُرتبط بحسابك.

ماذا يحدث عند Stop أو Terminate؟

لو عملت:

- Stop لل EC2، ال Public IP بيتشال، ولما تعمل Start هتأخذ Public IP جديد.
- Terminate، ال Public IP بيرجع لمجموعة عناوين AWS، ومش هتقدر تستخدمه تاني.

Elastic IP Address (EIP)

ال Elastic IP هو Public IPv4 Address ثابت (Static) بتخصصه AWS لحسابك.
بعد ما تعمل Allocate لل Elastic IP، تقدر تربطه بأي EC2 Instance داخل نفس ال Region.
الميزة الأساسية إنه بيفضل ثابت حتى لو عملت:

- Stop.
- Start.

طالما لسه مُرتبط بال Instance أو موجود في حسابك.

لإستخدام ال Elastic IP:

1. تعمل Allocate لل Elastic IP داخل ال Region.
2. تعمل Associate للعنوان مع ال EC2 Instance.

بشكل افتراضي، كل حساب AWS يقدر يمتلك 5 Elastic IP Addresses لكل Region.
ولو احتجت عدد أكبر، تقدر تطلب زيادة الحد من AWS.

❖ **EC2 instance metadata**

ال Instance Metadata هي عبارة عن معلومات خاصة بال EC2 Instance نفسها، ويتكون مُتاحة من داخل ال Instance فقط. يعني تقدر تعرف تفاصيل عن ال Instance وهي شغالة، بدون الحاجة للدخول على AWS Management Console.

كيفية الوصول إلى Instance Metadata

أثناء اتصالك بال EC2، تقدر تعرض ال Metadata باستخدام الرابط:

• `http://169.254.169.254/latest/meta-data`

أو من خلال ال Terminal باستخدام الأمر:

• `curl http://169.254.169.254/latest/meta-data`

عنوان ال Metadata دائماً هو: **169.254.169.254**

أهم المعلومات التي يمكن الحصول عليها

من خلال ال Instance Metadata تقدر تعرف معلومات زي:

- .Public IP Address
- .Private IP Address
- .Public Hostname
- .Instance ID
- .Security Groups
- .AWS Region
- .Availability Zone

وذي تقريباً نفس المعلومات الي بتشوفها في ال AWS Management Console.

User Data

لو كنت أضفت User Data Script أثناء إنشاء ال EC2، تقدر تعرضه من خلال:

• `http://169.254.169.254/latest/user-data`

وده بيسمح للتطبيقات أو السكريبتات إنها تقرأ ال User Data بعد تشغيل ال Instance.

استخدامات Instance Metadata

ال Instance Metadata بـُستخدم في:

- معرفة معلومات ال EC2 أثناء تشغيلها.
- إعداد البرامج أو نظام التشغيل تلقائياً.
- تشغيل Scripts تعتمد على بيانات ال Instance.

Link-Local Address

عنوان 169.254.169.254 هو Link-Local Address.

يعني يُمكن الوصول إليه من داخل الـ EC2 Instance فقط، لا يمكن الوصول إليه من الإنترنت أو من أي جهاز خارج الـ Instance.

❖ Amazon CloudWatch for monitoring

Amazon CloudWatch هي خدمة مُراقبة (Monitoring Service) في AWS.

بتستخدم لمُراقبة موارد AWS، ومنها EC2 Instances.

بتقوم بجمع بيانات التشغيل (Metrics) الخاصة بالـ EC2، وتحولها إلى رسوم بيانية وإحصائيات تقدر تتابع من خلالها أداء الـ Instance.

بيعمل إيه الـ CloudWatch؟

الـ CloudWatch يقوم بـ:

- جمع بيانات الأداء (Metrics).
 - عرض البيانات بشكل قريب من الوقت الحقيقي (Near Real-Time).
 - الإحتفاظ بالإحصائيات لمدة 15 شهر، بحيث تقدر ترجع للبيانات القديمة وتحلل أداء الـ Instance.
- بشكل افتراضي، أي EC2 Instance يكون عليها Basic Monitoring، وفيه يتم إرسال البيانات إلى CloudWatch كل 5 دقائق.

Detailed Monitoring

لو محتاج بيانات بتتحدث بشكل أسرع، تقدر تفعل Detailed Monitoring. وفي الحالة دي يتم إرسال البيانات إلى CloudWatch كل دقيقة واحدة (1 Minute)، لكن دي خاصية إختيارية ولازم تقوم بتفعيلها.

Metrics

الـ Metrics هي بيانات الأداء اللي الـ CloudWatch بيجمعها عن الـ EC2، زي:

- استخدام المُعالج (CPU Utilization).
 - الترافيك على الشبكة.
 - عمليات القراءة والكتابة على الأقراص.
- وبيتم عرضها في شكل رسوم بيانية داخل AWS Console.

RAM Metrics

بشكل افتراضي، الـ CloudWatch لا يجمع بيانات استخدام الـ RAM للـ EC2. ولو محتاج تراقب استهلاك الذاكرة، لازم تقوم بإعدادات إضافية (مثل تثبيت CloudWatch Agent).

3. Amazon EC2 Cost Optimization

Amazon EC2 Pricing Models ❖

AWS بتوفر أكثر من Pricing Model لتشغيل EC2 Instances، بحيث تختار النموذج المناسب حسب طبيعة استخدامك وتوفر في التكلفة.

1. On-Demand Instances

ال On-Demand هو النموذج الافتراضي لتشغيل EC2.

يتميز بأنه:

- مفيش أي عقد أو التزام طويل.
- مفيش دفع مُقدم.
- بتدفع فقط مقابل مدة تشغيل ال EC2.
- مناسب للإستخدامات قصيرة المدة أو الأحمال غير المُتوقعة.

كمان هو النموذج المؤهل للإستفادة من AWS Free Tier.

2. Dedicated Hosts

ال Dedicated Host هو سيرفر فعلي (Physical Server) مُخصص بالكامل لحسابك.

بيستخدم لو عندك تراخيص برامج (Licenses) خاصة، وعازب تستخدمها على نفس السيرفر.

3. Dedicated Instances

ال Dedicated Instance هي EC2 Instance بتشغل على جهاز مُخصص لعميل واحد فقط، ومش بتشارك نفس ال Host مع عملاء تانيين.

لكن على عكس Dedicated Host، أنت مش بتتحكم في السيرفر الفعلي نفسه.

4. Reserved Instances (RI)

ال Reserved Instance بتخليك تحجز سعة تشغيل لمدة سنة (Year 1) أو 3 سنوات (Years 3).

وفي المُقابل بتحصل على خصم كبير مُقارنةً ب On-Demand.

مناسب لو عارف إن التطبيق هيشغل باستمرار لفترة طويلة.

5. Scheduled Reserved Instances

دي نوع من ال Reserved Instances.

بتحجز ال EC2 بحيث تشتغل في مواعيد مُحددة بشكل مُتكرر، مثل:

- يوميًا.
- أسبوعيًا.
- شهريًا.

لكن بتدفع تكلفة الفترة المحجوزة حتى لو استخدمتها أو لا.

6. Spot Instances

ال Spot Instance هي عبارة عن EC2 Instance بتستخدم السعة (Capacity) اللي AWS مش محتاجها حاليًا.

يعني أحيانًا بيكون عند AWS سيرفرات فاضية أو جزء من إمكانياتها غير مُستخدم، فبدل ما تفضل بدون استخدام، AWS بتوفرها للعملاء بسعر أقل بكثير من السعر العادي.

مُميزاتُها:

- تعتبر أرخص نوع من أنواع EC2، ويمكن التوفير يوصل إلى 90% مقارنة بـ On-Demand.
- سعرها بيتغير باستمرار حسب العرض والطلب على الموارد.
- مناسبة لتقليل تكلفة تشغيل التطبيقات.

العيب الأساسي:

بما إن AWS بتبيع السعة الفاضية، فلو احتاجت السعة دي لأي عميل ثاني، تقدر توقف أو تنهي (Interrupt) ال Spot Instance في أي وقت.

عشان كده مش ينفع تعتمد عليها للتطبيقات اللي لازم تفضل شغالة بدون انقطاع.

تُستخدم مع التطبيقات أو المهام اللي تقدر تكمل شغلها حتى لو ال Instance اتوقفت، أو تقدر تعيد تشغيلها مرة ثانية بدون مُشكلة.

Per-Second Billing

بعض أنواع ال EC2 بتتْحاسب بالثانية بدل الساعة.

وده مُتاح مع:

- On-Demand.
- Reserved Instances.
- Spot Instances.

لكن فقط عند إستخدام:

- Amazon Linux.
- Ubuntu.

Amazon EC2 pricing models: Benefits ❖

كل Pricing Model في Amazon EC2 له مميزات، واختيار النوع المناسب يعتمد على طبيعة استخدامك.



On-Demand Instances	Spot Instances	Reserved Instances	Dedicated Hosts
Low cost and flexibility	Large scale, dynamic workload	Predictability ensures compute capacity is available when needed	- Save money on licensing costs - Help meet compliance and regulatory requirements

1. On-Demand Instances

ال On-Demand بيوفر أكبر قدر من المرونة.

يتميز بأنه:

- مفيش عقد أو التزام طويل.
- مفيش دفع مُقدم.
- بتدفع حسب مُدة الإستخدام فقط.
- مُناسب لما يكون استخدامك غير ثابت أو غير مُتوقع.

2. Spot Instances

ال Spot Instances بتوفر أقل تكلفة لأنها بتستخدم السعة غير المُستغلة عند AWS.

بتناسب التطبيقات اللي تقدر تتحمل إن ال Instance تتوقف في أي وقت.

3. Reserved Instances

ال Reserved Instances مُناسبة لو عندك تطبيق هيشغل لفترة طويلة وبشكل مُستمر.

بتحجز السعة لمدة سنة أو 3 سنوات، وتحصل على خصم مُقارنةً بـ On-Demand.

4. Dedicated Hosts

ال Dedicated Host مُناسب لو عندك:

- تراخيص برامج (Software Licenses) خاصة.
- مُتطلبات أمنية أو قانونية (Regulatory Requirements أو Compliance) بتفرض إن السيرفر يكون مُخصص ليك بالكامل.

Amazon EC2 pricing models: Use cases ❖

كل Pricing Model في Amazon EC2 مناسب لحالات استخدام معينة، واختيار النوع الصحيح يعتمد على طبيعة التطبيق.

1. On-Demand Instances

تُستخدم لما يكون:

- التطبيق هيشغل لفترة قصيرة.
- الاستخدام غير ثابت أو غير متوقع.
- أثناء التطوير (Development) أو الاختبار (Testing).
- الأحمال بتزيد وتقل بشكل مفاجئ (Spiky Workloads).

2. Spot Instances

تُستخدم لما يكون التطبيق يقدر يتحمل توقف ال Instance.

لو AWS احتاجت السعة، هتبعث إشعار قبل الإيقاف بحوالي دقيقتين (2-Minute Warning).

بعدها ممكن ال Instance:

- يتم Terminate (الافتراضي).
- أو Stop إذا قمت بإعداد ذلك.
- أو Hibernate إذا كانت مدعومة ومفعلة.

مناسبة للتطبيقات التي تتحمل الإنقطاع أو تحفظ بياناتها باستمرار في تخزين دائم مثل Amazon S3.

3. Reserved Instances

تُستخدم لما يكون عندك تطبيق:

- هيشغل لفترة طويلة.
- استخدامه ثابت ويمكن توقعه.
- محتاج توفير في التكلفة.

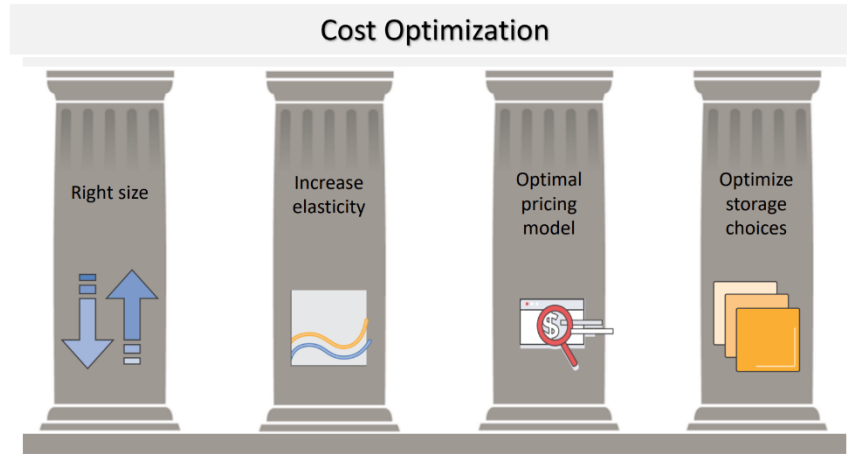
4. Dedicated Hosts

تُستخدم لما يكون عندك:

- تراخيص برامج موجودة بالفعل (BYOL - Bring Your Own License).
- متطلبات أمنية أو قانونية (Compliance أو Regulatory Requirements).

❖ The four pillars of cost optimization

علشان تقلل تكلفة تشغيل الموارد على AWS، فيه 4 مبادئ أساسية (Four Pillars) لازم تعتمد عليها.



1. Right-Size

المقصود بـ Right-Size إنك تختار حجم ونوع الـ EC2 Instance المناسب لإحتياجات التطبيق. يعني متستخدمش Instance بإمكانيات أكبر من المطلوب، ولو لقيت إن الـ Instance أكبر من احتياج التطبيق، ممكن تصغرها أو توقفها لو مش محتاجها، بشرط إن الأداء يفضل مُناسب.

2. Increase Elasticity

المقصود بـ Elasticity إن عدد الـ EC2 يزيد أو يقل تلقائياً حسب حجم الضغط على التطبيق. وده بيققل عدد الـ Instances اللي بتفضل شغالة بدون استخدام، وبالتالي يقلل التكلفة. غالباً بيتم تنفيذ ده بإستخدام Auto Scaling.

3. Optimal Pricing Model

اختار Pricing Model المناسب لطبيعة استخدامك.

يعني:

- On-Demand للإستخدام المؤقت.
 - Reserved للإستخدام المُستمر.
 - Spot لتقليل التكلفة إذا كان التطبيق يتحمل الإنقطاع.
- اختيار النموذج الصحيح يساعد على تقليل التكلفة بشكل كبير.

4. Optimize Storage Choices

اختار نوع التخزين المناسب للتطبيق، ومتستخدمش تخزين أغلى من احتياجك. كمان حاول تقلل المساحات التخزينية غير المُستخدمة، واستخدم أنواع التخزين الأقل تكلفة طالما بتحقق الأداء المطلوب.

Pillar 1: Right size ❖

المقصود بـ Right-Size هو أنك تختار الـ EC2 Instance بالحجم والإمكانيات المناسبة لتشغيل التطبيق، بحيث تحقق الأداء المطلوب بأقل تكلفة ممكنة.

بما إن AWS بتوفر عدد كبير من أنواع وأحجام الـ EC2 Instances، فاختيار النوع المناسب يساعد على تقليل المصاريف بدون التأثير على أداء التطبيق.

كيف تقوم بعملية Right-Sizing؟

1. اختر أقل Instance يحقق الأداء المطلوب

متستخدمش Instance أكبر من احتياج التطبيق، لأن ده هيزود التكلفة بدون فائدة.

2. راقب استهلاك الموارد

راجع استخدام الموارد باستمرار، مثل:

- CPU
- RAM
- Storage
- Network

لو لقيت إن استهلاك الموارد مُنخفض، ممكن تستخدم Instance أصغر وتوفر في التكلفة.

3. اختبر أكثر من Instance Type

قبل ما تعتمد نوع معين، جرب تشغيل التطبيق على أكثر من Instance Type وحجم مختلف، وشوف أي واحد بيديك أفضل توازن بين الأداء والتكلفة.

وده غالباً بيتم باستخدام Load Testing لمعرفة أداء التطبيق تحت الضغط.

4. استخدم Amazon CloudWatch

استخدم Amazon CloudWatch لمراقبة أداء الـ EC2 وجمع الـ Metrics مثل:

- CPU Utilization
- Network Usage
- Disk Usage

وَبُنَاءً على البيانات دي تقدر تعرف إذا كان الـ Instance مُناسب ولا يحتاج تصغير أو تكبير.

Pillar 2: Increase elasticity ❖

المقصود بـ Increase Elasticity هو إن الموارد في AWS تزيد أو تقل تلقائياً حسب احتياج التطبيق، بحيث متدفعش مقابل موارد شغالة وهي مش مُستخدمة.

وده يعتبر من أهم مُميزات الـ Cloud Computing.

كيف تطبق Elasticity؟

1. إيقاف الـ Instances غير المستخدمة

لو عندك Instances خاصة بـ:

- Development
- Testing
- Non-Production

ومش محتاجها بعد ساعات العمل، تقدر توقفها (Stop) أو تعملها Hibernate بدل ما تفضل شغالة وتكلفك فلوس. وده ممكن يوفر نسبة كبيرة من التكلفة.

2. استخدام Auto Scaling

بالنسبة لتطبيقات الإنتاج (Production)، يُفضل استخدام Auto Scaling.

الـ Auto Scaling ييزود عدد الـ EC2 Instances لما الضغط يزد، ويقلل العدد لما الضغط يقل.

وده معناه إنك مش محتاج تشغل عدد كبير من الـ Instances طول الوقت لمُجرد الإستعداد لأوقات الضغط.

3. استخدام On-Demand و Spot عند الحاجة

AWS بتنصح إن جزء من الـ EC2 Instances يكون من نوع:

- On-Demand
- Spot

علشان تستفيد من المرونة وتقلل التكلفة حسب طبيعة الأحمال.

❖ Pillar 3: Optimal pricing model

المقصود بـ Optimal Pricing Model هو اختيار نموذج التسعير (Pricing Model) التي يناسب طبيعة تشغيل التطبيق، علشان تحقق أفضل أداء بأقل تكلفة.

AWS بتوفر أكثر من طريقة لتشغيل EC2، وكل واحدة مناسبة لحالة استخدام مختلفة، واختيار النوع المناسب يساعد على تقليل المصاريف.

كيف تطبق Optimal Pricing Model؟

1. اختر نموذج التسعير المناسب

بدل ما تستخدم On-Demand لكل الـ EC2 Instances، اختار النوع المناسب لكل حمل عمل، مثل:

- On-Demand للإستخدام المؤقت أو غير المتوقع.
- Reserved Instances للتشغيل المستمر لفترة طويلة.
- Spot Instances للتطبيقات التي تتحمل الإنقطاع.
- Dedicated Hosts إذا كنت تحتاج تراخيص خاصة أو متطلبات امتثال (Compliance).

2. امزج أكثر من Pricing Model

مش لازم تستخدم نوع واحد فقط.

ممكن تشغيل جزء من الـ Instances بـ Reserved Instances لأنها شغالة دائماً، وجزء آخر بـ Spot Instances لتقليل التكلفة، أو تستخدم On-Demand للأحمال المفاجئة.

وده يساعد على تحقيق أفضل توازن بين التكلفة والمرونة.

3. راجع تصميم التطبيق

قبل ما تشغيل التطبيق على EC2، اسأل نفسك:

هل التطبيق فعلاً محتاج EC2؟

في بعض الحالات، ممكن تستخدم خدمات ثانية مثل AWS Lambda بدل EC2، وده يقلل التكلفة لإنك هتدفع فقط وقت تنفيذ الكود، بدل تشغيل سيرفر طوال الوقت.

Pillar 4: Optimize storage choices ❖

المقصود بـ Optimize Storage Choices هو اختيار نوع وحجم التخزين المناسب، وعدم استخدام مساحة أو نوع تخزين أعلى من احتياج التطبيق.

كيف تطبق Optimize Storage؟

1. استخدم حجم التخزين المناسب

راجع أحجام وحدات التخزين (EBS Volumes) باستمرار.

لو لقيت إن حجم الـ Volume أكبر من احتياج التطبيق، قلل حجمه لتوفير التكلفة.

2. اختر نوع الـ EBS المناسب

Amazon EBS يوفر أكثر من Volume Type، وكل نوع له أداء وسعر مختلف.

اختر أقل نوع تكلفة يحقق الأداء المطلوب، بدل استخدام نوع أعلى بدون حاجة.

3. احذف الـ Snapshots غير المُستخدمة

الـ EBS Snapshots يُستخدم لعمل نسخ احتياطية.

لو فيه Snapshots قديمة أو مش محتاجها، احذفها لأنها بتستهلك مساحة وبتزود التكلفة.

4. استخدم خدمة التخزين المناسبة

مش كل البيانات لازم تتخزن على Amazon EBS.

حسب طبيعة البيانات، ممكن تستخدم خدمات تخزين ثانية مثل:

- Amazon S3 لتخزين البيانات العامة.
- Amazon S3 Glacier للبيانات القديمة أو قليلة الاستخدام، لأنه أقل تكلفة.

5. استخدم Lifecycle Policies

اعمل Lifecycle Policies لنقل البيانات تلقائياً من التخزين الأعلى إلى التخزين الأرخص لما تقل الحاجة لإستخدامها.

وده يساعد على تقليل تكلفة التخزين بدون تدخل يدوي.

❖ Measure, monitor, and improve

تقليل التكاليف في AWS مش بيتم مرة واحدة، لكن لازم تراجع استخدامك باستمرار، تراقب الموارد، وتحسنها كل فترة علشان تفضل بتدفع أقل تكلفة مُمكنة.

1. Measure (قياس الاستخدام والتكلفة)

أول خطوة هي إنك تعرف مواردك بتستهلك كام وبتكلفك كام. وده بيساعدك تعرف الأماكن اللي ممكن تقلل فيها المصاريف.

2. Monitor (مراقبة الموارد)

راقب استخدام الموارد باستمرار علشان تكتشف:

- موارد مش مُستخدمة.
- موارد أكبر من احتياجها.
- زيادة غير طبيعية في التكلفة.

3. Improve (تحسين التكلفة)

بعد ما تجمع البيانات، عدّل الموارد أو طريقة تشغيلها لتقليل التكلفة مع الحفاظ على الأداء.

4. استخدم Tags

ال Tags بتساعدك تنظم مواردك وتعرف كل Resource تابع لمين أو بيستخدم في إيه.

تقدر تستخدمها لتقسيم التكاليف حسب:

- المشروع (Project).
- التطبيق (Application).
- صاحب المورد (Owner).
- مركز التكلفة (Cost Center).

وبعد تفعيل Cost Allocation Tags، تقدر تشوف تكلفة كل مجموعة موارد بشكل مُنفصل.

5. استخدم AWS Cost Explorer

AWS Cost Explorer أداة مجانية بتعرض استهلاكك وتكلفة خدمات AWS في شكل تقارير ورسوم بيانية.

بتساعدك:

- متابعة المصروفات.
- معرفة الخدمات اللي بتكلفك أكثر.
- تحليل اتجاهات الإنفاق مع الوقت.

6. استخدم AWS Trusted Advisor

AWS Trusted Advisor يحلل بيئة AWS بتاعتك ويقدم توصيات لتحسينها. من ضمنها اقتراحات لتقليل التكلفة، بالإضافة لتحسين الأداء والأمان.

7. اجعل Cost Optimization مسؤولية مستمرة

من الأفضل يكون فيه شخص أو فريق مسؤول عن مراجعة التكاليف وتحسينها باستمرار، لإن استخدام الموارد بيتغير مع الوقت.

4. Container services

❖ Container Basics

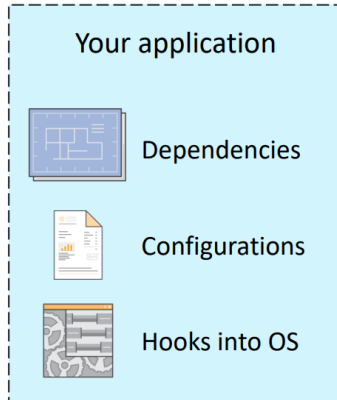
ال Container هو أسلوب لتشغيل التطبيقات بحيث يتم وضع التطبيق مع كل الملفات والمكتبات والإعدادات التي يحتاجها داخل حاوية (Container) مستقلة، بحيث يشتغل بنفس الطريقة على أي بيئة تشغيل.

إزاي بيشتغل ال Container؟

بدل ما كل تطبيق يكون له نظام تشغيل (Operating System) خاص به، كل ال Containers بتشارك نفس نظام التشغيل، لكن كل Container بيشتغل في بيئة معزولة (Isolated) عن باقي ال Containers.

وده بيخلي تشغيلها أسرع واستهلاكها للموارد أقل.

Your Container



مكونات ال Container

ال Container بيحتوي على كل ما يحتاجه التطبيق للتشغيل، مثل:

- كود التطبيق (Application Code).
- المكتبات (Libraries).
- أدوات النظام (System Tools).
- ملفات الإعدادات (Configurations).
- بيئة التشغيل (Runtime).

لذلك التطبيق بيشتغل بنفس الشكل في أي مكان بدون الحاجة لإعادة ضبط الإعدادات.

مميزات Containers

1. حجم أصغر

ال Containers أصغر بكثير من ال Virtual Machines لأنها لا تحتوي على نظام تشغيل كامل، وده بيقلل استهلاك مساحة التخزين.

2. سرعة التشغيل

تشغيل ال Container يتم في وقت قصير جداً، غالباً خلال أجزاء من الثانية.

وده بيساعد على تشغيل التطبيقات بسرعة.

3. ثبات بيئة التشغيل (Environmental Consistency)

بما إن كل احتياجات التطبيق موجودة داخل الـ Container، فالتطبيق يشتغل بنفس الطريقة في أي بيئة بدون اختلافات.

4. سهولة النقل (Portability)

تقدر تنقل الـ Container من جهاز أو بيئة إلى أخرى بدون الحاجة لتعديل التطبيق.

5. كفاءة استخدام الموارد

بما إن الـ Containers بتشارك نفس نظام التشغيل، فهي تستهلك موارد أقل من الـ Virtual Machines، وبالتالي تسمح بتشغيل عدد أكبر من التطبيقات على نفس الخادم.

6. سرعة وموثوقية نشر التطبيقات

الـ Containers بتساعد على نشر التطبيقات بسرعة وبشكل ثابت، لأن نفس الـ Container هيشتغل بنفس الطريقة في كل البيئات.

الفرق بين Virtual Machine و Container

Container	Virtual Machine
لا يحتوي على نظام تشغيل كامل	يحتوي على نظام تشغيل كامل
يشارك نظام التشغيل مع باقي الـ Containers	لكل VM نظام تشغيل مُستقل
حجمه أصغر	حجمه أكبر
أسرع في التشغيل	أبطأ في التشغيل
يستهلك موارد أقل	يستهلك موارد أكثر

❖ What is Docker?

الـ Docker هو منصة (Platform) بتُستخدم لإنشاء وتشغيل وإدارة الـ Containers. هو المسؤول عن تجميع التطبيق مع كل الملفات والإعدادات والإعتماديات (Dependencies) داخل Container جاهز للتشغيل.



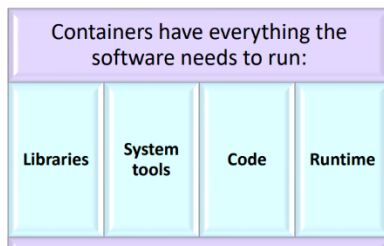
Container

إزاي بيشتغل الـ Docker؟

يتم تثبيت Docker على السيرفر الذي سيشغل الـ Containers.

بعد تثبيته، بيوفر أوامر بسيطة تمكّنك من:

- إنشاء (Build) الـ Containers.
- تشغيل (Start) الـ Containers.
- إيقاف (Stop) الـ Containers.
- إدارة الـ Containers.



مميزات ال Docker**1. توحيد بيئة التشغيل (Standardize Environments)**

بيخلي التطبيق يشتغل بنفس الطريقة في جميع البيئات، سواء أثناء التطوير أو الإختبار أو الإنتاج.

2. تقليل مشاكل التوافق (Reduce Conflicts)

بيقل مشاكل اختلاف إصدارات البرامج أو المكتبات (Libraries) بين الأجهزة المختلفة، لإن كل ال Dependencies موجودة داخل ال Container.

3. تشغيل التطبيقات بإستخدام Containers

بيسهل تشغيل وإدارة التطبيقات داخل Containers بدل تشغيلها مباشرةً على نظام التشغيل.

4. دعم Microservices

يساعد على تشغيل تطبيقات Microservices، بحيث كل خدمة (Service) تشتغل داخل Container مُستقل.

5. سهولة النقل (Portability)

تقدر تنقل ال Container وتشغله على أي جهاز أو بيئة فيها Docker بدون الحاجة لإعادة إعداد التطبيق.

6. سهولة التوسع (Scalability)

بيسهل نشر عدد أكبر من ال Containers أو تقليل عددها حسب احتياج التطبيق.

امتي نتستخدم Docker؟

يكون Docker مُناسب عندما تحتاج إلى:

- توحيد بيئات التشغيل.
- تقليل مشاكل توافق الإصدارات.
- تشغيل التطبيقات داخل Containers.
- تشغيل تطبيقات تعتمد على Microservices.
- نقل التطبيقات بسهولة بين البيئات المختلفة.

❖ Containers versus virtual machines

رغم إن Containers و Virtual Machines (VMs) الإثنين يُستخدموا لتشغيل التطبيقات في بيئة معزولة، لكن طريقة عملهم مُختلفة.

أولاً: Virtual Machine (VM)

في ال Virtual Machine:

- كل VM يكون له نظام تشغيل (Operating System) خاص بيه.
- ال VM بيشتغل فوق Hypervisor.
- كل تطبيق بيشتغل داخل VM مُنفصل.
- بسبب وجود نظام تشغيل لكل VM، بيكون استهلاك الموارد أكبر وحجم ال VM أكبر.

ثانياً: Container

في ال Container:

- كل Container بيحتوي على التطبيق وال Dependencies فقط.
 - جميع ال Containers بتشارك نفس نظام التشغيل (Linux OS).
 - تشغيل ال Containers بيكون من خلال Docker Engine.
 - كل Container بيكون معزول عن باقي ال Containers، لكنهم جميعاً بيستخدموا نفس ال Kernel.
- وده بيخلي ال Containers أخف وأسرع من ال Virtual Machines.

دور Docker

ال Docker Engine هو المسؤول عن:

- إنشاء ال Containers.
- تشغيلها.
- إيقافها.
- إدارة دورة حياتها (Container Lifecycle).

ليه ال Containers أكثر كفاءة؟

لإنها لا تحتاج إلى نظام تشغيل مُستقل لكل تطبيق.

لذلك:

- تستهلك موارد أقل.
 - حجمها أصغر.
 - تعمل بسرعة أكبر.
 - يمكن تشغيل عدد كبير منها على نفس ال EC2 Instance.
- في المُقابل، كل Virtual Machine تحتاج إلى نظام تشغيل كامل، وبالتالي تستهلك موارد أكثر.

قابلية النقل (Portability)

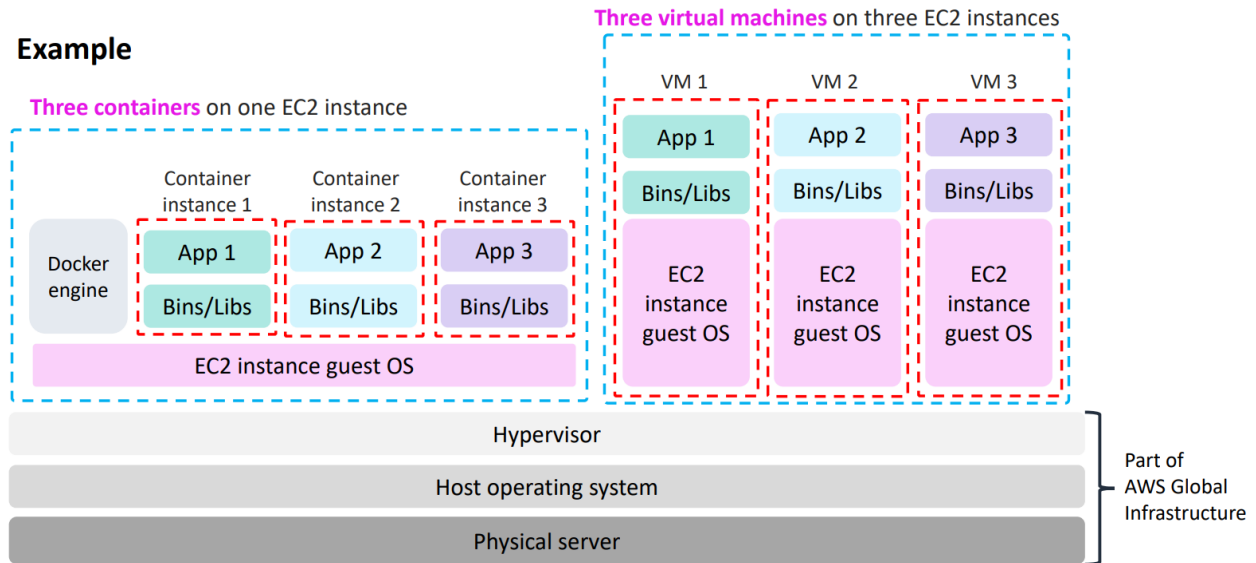
ال Containers يمكن تشغيلها على أي جهاز يوجد عليه:

- .Linux
- .Docker

سواء كان:

- .Laptop
- .EC2 Instance
- .Virtual Machine
- .Bare Metal Server

وده ببسهل نقل التطبيقات بين البيئات المختلفة.

Example**نقطة مهمة جدًا للامتحان**

احفظ الفرق ده لأنه بيتكرر كثير:

ال Virtual Machine فيها كل VM لها Operating System مُستقل وتعمل فوق Hypervisor.

إنما ال Container لا يحتوي على نظام تشغيل مُستقل، بل يشارك نظام التشغيل مع باقي ال Containers ويعمل من خلال Docker Engine.

نتيجة ذلك:

- Containers أسرع، أخف، وتستهلك موارد أقل.
- Virtual Machines أثقل، تستهلك موارد أكثر، لكن كل VM معزولة بالكامل بنظام تشغيل مُستقل.

Amazon Elastic Container Service (Amazon ECS) ❖

ال Amazon ECS هي خدمة مُدارة (Managed Service) من AWS مُخصصة لإدارة وتشغيل Docker Containers بسهولة. بدل ما تنشئ EC2 Instances بنفسك، وتثبت Docker عليها، وتدير ال Containers يدوياً، AWS بتوفر لك Amazon ECS علشان تتولى عملية إدارة ال Containers بشكل أسهل.



Amazon Elastic
Container Service

ليه نستخدم Amazon ECS؟

لو هتدير ال Containers بنفسك، هتحتاج:

- إنشاء EC2 Instances.
- تثبيت Docker على كل Instance.
- تشغيل ال Containers.
- مراقبة ال Containers.
- إدارة ال Cluster.

لكن مع Amazon ECS، AWS بتسهل كل عمليات الإدارة دي.

مميزات Amazon ECS

1. إدارة ال Containers

بيسهل تشغيل وإدارة عدد كبير من Docker Containers بدون الحاجة لإدارتها يدوياً.

2. قابلية التوسع (Highly Scalable)

يقدر يشغل آلاف أو عشرات الآلاف من ال Containers خلال ثوانٍ، وبالتالي يناسب التطبيقات الكبيرة.

3. مراقبة ال Containers

يساعدك على متابعة حالة ال Containers والتأكد إنها شغالة بشكل صحيح.

4. إدارة ال Cluster

بيقوم بإدارة ال Cluster اللي بيشغل ال Containers، ويتابع حالته بدل ما تعمل ده بنفسك.

ال Cluster هو مجموعة من ال EC2 Instances بتشغل مع بعض لتشغيل ال Containers.

5. جدولة تشغيل ال Containers (Scheduling)

بيحدد على أي EC2 Instance هيتم تشغيل كل Container، وده بيتم باستخدام Scheduler مدمج أو Scheduler خارجي.

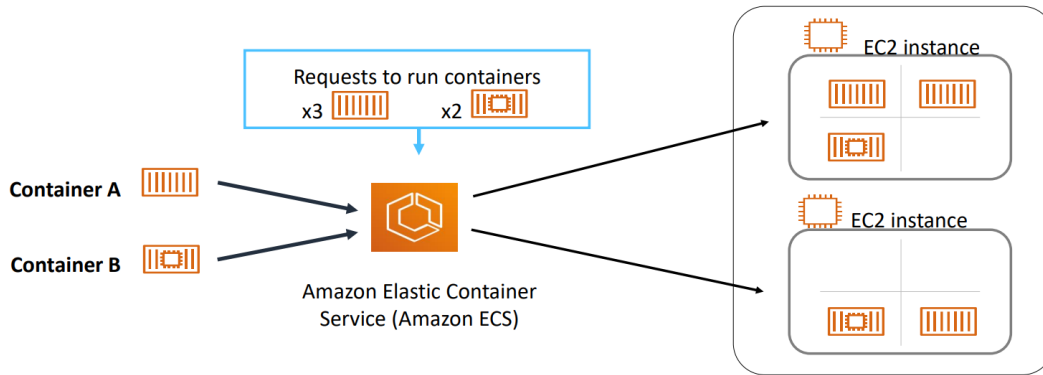
6. يدعم نماذج التسعير المختلفة

يمكن تشغيل الـ ECS باستخدام:

- On-Demand Instances
- Reserved Instances
- Spot Instances

وده يساعد على تقليل التكلفة حسب احتياج التطبيق.

Amazon ECS orchestrates containers ❖



علشان تشغيل تطبيقك على Amazon ECS، لازم الأول تعمل Task Definition.

1. Task Definition

الـ Task Definition هو ملف إعدادات (Configuration File) بيحتوي على كل المعلومات المطلوبة لتشغيل التطبيق.

تقدر تعتبره Blueprint أو قالب للتطبيق.

- الـ Task Definition بيحدد:
- عدد الـ Containers (من 1 إلى 10 Containers).
- صورة الـ Container (Docker Image).
- الـ Ports اللي هتفتح.
- وحدات التخزين (Volumes).
- إعدادات التشغيل الخاصة بالتطبيق.

2. Task

الـ Task هو التطبيق الفعلي للـ Task Definition.

بمعنى إن:

- Task Definition = خطة أو قالب.
- Task = تشغيل فعلي للقالب.

ويمكن تشغيل أكثر من Task من نفس الـ Task Definition.

3. Cluster

كل ال Tasks بتشتغل داخل ال Cluster.

وال Cluster عبارة عن مجموعة من EC2 Instances بتستضيف وتشغل ال Containers.

ولو اخترت EC2 Launch Type، كل EC2 Instance بيكون عليه Amazon ECS Container Agent المسؤول عن التواصل مع خدمة ECS.

4. ECS Task Scheduler

ال Task Scheduler هو المسؤول عن تحديد المكان المناسب لتشغيل كل Task داخل ال Cluster.

وبيختار ال EC2 Instance المناسبة بُناءً على الموارد المُتاحة.

5. Scheduling Strategies

Amazon ECS بيوفر أكثر من طريقة لتوزيع ال Tasks داخل ال Cluster.

وبيختار مكان تشغيل ال Tasks حسب:

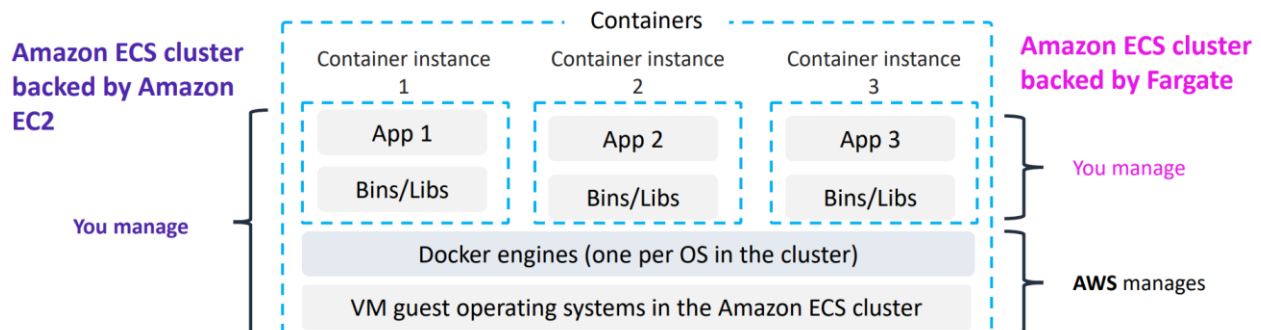
- الموارد المُتاحة (CPU و RAM).
- مُتطلبات التوافر (Availability).

وده بيساعد على توزيع الحمل بشكل مُناسب.

❖ Amazon ECS cluster options

عند إنشاء Amazon ECS Cluster، بيكون عندك 3 اختيارات:

1. Networking Only (AWS Fargate)
2. EC2 Linux + Networking
3. EC2 Windows + Networking



1. Networking Only (AWS Fargate)

في الاختيار ده، AWS هي اللي بتدير البنية التحتية بالكامل.

يعني أنت كل اللي عليك:

- ترفع ال Containers.
- تحدد احتياجاتها من ال CPU وال Memory.
- تحدد إعدادات الشبكة (Networking).
- تحدد صلاحيات ال IAM.
- تشغل التطبيق.

أما إنشاء السيرفرات، إدارتها، وصيانتها، وتوسيعها (Scaling)، فكل ده بيكون مسؤولية AWS.

مميزات Fargate

- مش محتاج تنشئ EC2 Instances.
- مش محتاج تدير أو تصين السيرفرات.
- مش محتاج تختار نوع ال Instance.
- AWS بتتولى عملية ال Scaling.
- يخليك تركز على التطبيق نفسه بدل إدارة البنية التحتية.

2. EC2 Linux + Networking

في الاختيار ده، ال Cluster بيتكون من EC2 Instances بنظام Linux.

أنت المسؤول عن:

- اختيار نوع ال EC2 Instance.
- إنشاء ال Instances.
- إدارتها.
- تحديثها.
- زيادة أو تقليل عددها.

لكن Amazon ECS بيدير تشغيل ال Containers عليها.

3. EC2 Windows + Networking

نفس فكرة Linux، لكن ال EC2 Instances بتكون Windows Server بدل Linux.

لو اخترت EC2 Linux أو EC2 Windows، هتحتاج تحدد:

- حجم ال Instance.
- عدد ال Instances.
- استخدام On-Demand أو Spot Instances.
- باقي إعدادات ال EC2 المُعتادة.
- نوع ال EC2 Instance.

يعني أنت بتدير البنية التحتية بنفسك.

حتى لو أنت اللي بتدير ال EC2 Instances، فال Amazon ECS بيقوم بـ:

- مُتابعة موارد ال Cluster (CPU و Memory وغيرها).
- اختيار أفضل EC2 Instance لتشغيل كل Container حسب الموارد المُتاحة.

الفرق بين EC2 و Fargate

EC2 Launch Type	AWS Fargate
أنت اللي بتدير ال EC2 Instances	AWS تدير السيرفرات بالكامل
بتختار نوع وحجم ال Instance	لا تحتاج لإختيار أي Server
أنت مسؤول عن ال Scaling	AWS هي اللي بتعمل ال Scaling
تحكم أكبر في البنية التحتية	إدارة أقل وأسهل
مُناسب لو محتاج تحكم كامل	مُناسب لو عايز تركز على التطبيق فقط

❖ What is Kubernetes?

ال Kubernetes هو برنامج مفتوح المصدر (Open Source) مُخصص لإدارة وتنظيم وتشغيل ال Containers على نطاق كبير (Container Orchestration).



Amazon Elastic
Kubernetes Service

يعني بدل ما تدير ال Containers يدويًا، Kubernetes بيتولى تشغيلها، توزيعها، وإدارتها بشكل تلقائي.

مُميزات Kubernetes

1. يُدير ال Containers

بيقوم بتشغيل وإدارة التطبيقات اللي شغالة داخل Containers، ويسهل التحكم فيها حتى لو عددها كبير جدًا.

2. بيشتغل مع أكثر من Container Technology

مش بيشتغل مع Docker بس، لكنه بيدعم تقنيات مُختلفة لتشغيل ال Containers.

3. قابل للتوسع (Scalable)

مُصمم لتشغيل وإدارة عدد كبير من ال Containers بسهولة، مع إمكانية زيادة أو تقليل عددها حسب الحاجة.

4. يشتغل في أي بيئة

يمكن تشغيل Kubernetes داخل:

- On-Premises Data Center
- AWS Cloud
- أي Cloud Provider آخر.

وده معناه إنك تقدر تنقل تطبيقاتك بين البيئات المختلفة بدون تغيير طريقة إدارتها.

Nodes

ال Kubernetes يشتغل ال Containers على مجموعة من أجهزة الحوسبة اسمها Nodes.

ال Node هو السيرفر أو الجهاز اللي بيستضيف ال Containers.

Pods

في Kubernetes، ال Containers مبتشتغلش بشكل مُنفرد، لكنها بتتجمع داخل وحدة اسمها Pod.

ال Pod هو أصغر وحدة تشغيل في Kubernetes، وممكن يحتوي على Container واحد أو أكثر.

DNS و IP Address

كل Pod بيحصل على:

- عنوان IP خاص به.
- اسم DNS خاص به.

وده بيساعد ال Pods والخدمات إنها تتواصل مع بعض بسهولة.

أهم ميزة في Kubernetes

أكبر ميزة هي إنك تستخدم نفس الأدوات ونفس طريقة الإدارة سواء التطبيق شغال:

- على جهازك.
- داخل الشركة (On-Premises).
- على Cloud مثل AWS.

يعني مش هتحتاج تغيير طريقة التشغيل أو الإدارة عند نقل التطبيق من مكان لمكان.

Amazon Elastic Kubernetes Service (Amazon EKS) ❖

ال Amazon EKS هي خدمة مُدارة (Managed Service) من AWS مُخصصة لتشغيل وإدارة Kubernetes Clusters. بدل ما تُنشئ EC2 Instances، وتُثبت Kubernetes، وتدير ال Cluster بنفسك، AWS بتقوم بكل المهام دي نيابةً عنك.



ليه نستخدم Amazon EKS؟

لو هتدير Kubernetes بنفسك، هتحتاج:

- إنشاء EC2 Instances.
- تثبيت Kubernetes.
- إدارة ال Control Plane.
- متابعة حالة ال Cluster.
- عمل تحديثات وصيانة.

مع Amazon EKS، AWS بتدير كل المهام دي، وأنت بتركز على تشغيل تطبيقاتك.

مميزات Amazon EKS

1. خدمة مُدارة بالكامل

AWS بتتولى تثبيت وتشغيل وإدارة Kubernetes، وبالتالي مش هتحتاج تدير ال Kubernetes Cluster بنفسك.

2. إدارة ال Control Plane

AWS بتقوم بإدارة ال Kubernetes Control Plane بالكامل، بما يشمل:

- تشغيله.
- صيانه.
- تحديثه.
- الحفاظ على توافره (High Availability).

3. التوسع والتوافر

Amazon EKS بيدير تلقائياً توافر ال Cluster وقابليته للتوسع، ويضمن استمرار تشغيله بشكل مُستقر.

4. اكتشاف الأعطال تلقائياً

لو أي جزء من ال Control Plane حصل فيه عطل، AWS بتكتشف المشكلة وتستبدله تلقائياً بدون تدخل منك.

5. التكامل مع خدمات AWS

Amazon EKS بيتكامل مع خدمات AWS المختلفة مثل:

- Application Load Balancer (ALB) لتوزيع الحمل (Load Balancing).
- IAM لإدارة الصلاحيات.
- Amazon VPC لتوصيل ال Pods بالشبكة.

6. مُتوافق مع Kubernetes القياسي

أي تطبيق معمول باستخدام Kubernetes Standard يقدر يشتغل على Amazon EKS بدون الحاجة لتعديلات.

ليه فيه ECS و EKS؟

الخدمتين بيستخدموا لتشغيل وإدارة ال Containers، لكن كل واحدة موجهة لفئة مُختلفة من المُستخدمين.

Amazon ECS

- خدمة خاصة بـ AWS.
- أسهل في الإستخدام.
- مُناسبة لو شغال بالكامل داخل AWS.

Amazon EKS

- مبنية على Kubernetes.
- مُناسبة لو بتستخدم Kubernetes بالفعل أو عايز تنقل تطبيقاتك بين أكثر من بيئة.
- بتعتمد على المعايير القياسية لـ Kubernetes.

الفرق بين ECS و EKS

أولاً: Amazon ECS

تخيل إنك عندك Docker Containers وعايز تشغلهم على AWS.
 AWS بتقولك: "سيب إدارة ال Containers عليا، ومش محتاج تعرف أي حاجة عن Kubernetes."
 يعني ECS هو الحل الخاص بـ AWS لإدارة ال Containers.

أنت بتعامل مع:

- Task
- Task Definition
- Cluster

وAWS بتديرهم ليك.

يبقى ECS هو AWS Container Orchestrator.

ثانياً: Amazon EKS

أما لو أنت شغال بـ Kubernetes، أو شركتك أصلاً بتستخدم Kubernetes، فمش هينفع تحول كل حاجة لـ ECS.
 AWS بتقولك: "هات ال Kubernetes بتاعك وأنا هديرهولك."
 يعني EKS مش نظام جديد، هو Kubernetes نفسه لكن AWS هي اللي بتديره.

❖ Amazon Elastic Container Registry (Amazon ECR)

ال Amazon ECR هي خدمة مُدارة (Managed Service) من AWS مُخصصة لتخزين وإدارة Docker Container Images. بمعنى إن بدل ما تخزن ال Docker Images على Docker Hub أو أي Registry خارجي، تقدر تخزنها داخل AWS باستخدام Amazon ECR.



ليه نستخدم Amazon ECR؟

لإن ال Docker Image لازم تكون موجودة في مكان مُعين قبل ما يتم تشغيلها. ال Amazon ECR بيكون هو المكان اللي بنرفع عليه ال Images، وبعد كده خدمات زي ECS أو EKS تسحبها وتشغلها.

مُميزات Amazon ECR

1. تخزين Docker Images

بيوفر مكان آمن لتخزين وإدارة جميع ال Docker Images الخاصة بتطبيقاتك.

2. التكامل مع Amazon ECS

بيتكامل مُباشرةً مع Amazon ECS.

كل اللي عليك إنك تحدد اسم ال ECR Repository داخل Task Definition، وECS هيقيم تلقائياً بسحب ال Image وتشغلها.

3. يدعم أدوات Docker

بيدعم أوامر Docker CLI، وبالتالي تقدر تستخدم نفس أوامر Docker اللي متعود عليها بدون تغيير.

4. نقل آمن للبيانات

يتم نقل ال Images بين جهازك وAmazon ECR باستخدام HTTPS لضمان أمان الإتصال.

5. تشفير البيانات

جميع ال Images المُخزنة في Amazon ECR بيتم تشفيرها تلقائياً أثناء التخزين (Encryption at Rest) باستخدام Amazon S3 Server-Side Encryption.

6. بيشتغل مع Amazon EKS أيضاً

مش مُقتصر على ECS فقط، بل يمكن استخدام نفس ال Docker Images مع Amazon EKS أيضاً.

العلاقة بين ECR و ECS

1. بتعمل Docker Image.
2. وترفعها على Amazon ECR.
3. وداخل Task Definition نكتب اسم ال Image الموجودة في ECR.
4. وال Amazon ECS يسحب ال Image تلقائياً ويشغل ال Container.

العلاقة بين ECR و EKS

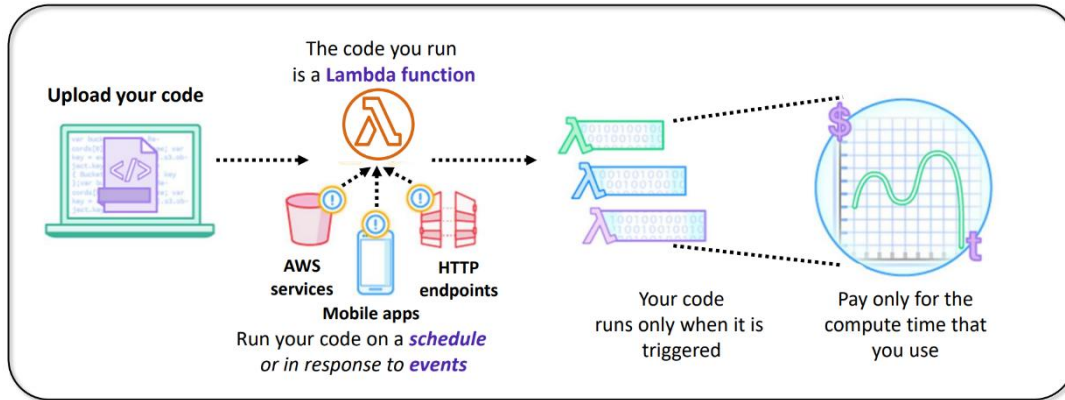
نفس الفكرة.

بدل ما ECS هو اللي يسحب ال Image، ال (EKS) Kubernetes هو اللي بيسحب ال Image من ECR ويشغل ال Containers.

5. Introduction to AWS Lambda

❖ AWS Lambda: Run code without servers

ال AWS Lambda هي خدمة Serverless Compute من AWS بتشغل الكود بتاعك بدون ما تحتاج تُنشئ أو تدير أي Servers. يعني بدل ما تنشئ EC2 أو تدير Containers، كل اللي عليك هو ترفع الكود، و AWS تتولى تشغيله وإدارة البنية التحتية بالكامل.

ما معنى Serverless؟

كلمة Serverless لا تعني إنه مفيش Servers.

السيرفرات موجودة فعلاً، لكن AWS هي اللي بتديرها بالكامل، وأنت مش بتعامل معاها نهائياً.

يعني أنت بتركز على كتابة الكود فقط، و AWS بتتولى:

- تشغيل السيرفرات.
- إدارتها.
- صيانتها.
- التوسع (Scaling).

إزاي بيشتغل AWS Lambda؟

في Lambda، بُنشئ حاجة اسمها Lambda Function.
 ال Lambda Function هي المورد (Resource) اللي بيحتوي على الكود اللي أنت كتبتة.
 بعد كده بتحدد إمتى الكود يشتغل.

متى يتم تشغيل الكود؟

ال Lambda Function بتشتغل بطريقتين:

1. Scheduled

يعني تشتغل في وقت أو موعد مُحدد بشكل دوري.

2. Event-Driven

يعني تشتغل عند حدوث Event مُعين.
 مثلاً لو حصل حدث مُعين داخل AWS، ال Lambda Function تشتغل تلقائياً.

الدفع في AWS Lambda

ميزة Lambda إنك بتدفع فقط مُقابل وقت تنفيذ الكود.

يعني:

- الكود شغال، بيتم احتساب التكلفة.
- الكود متوقف، مش تدفع أي تكلفة.

وده مُختلف عن EC2، لأن EC2 بتدفع مُقابل تشغيل ال Instance حتى لو التطبيق مش بيعمل حاجة.

نقطة مُهمّة جدّاً للإمتحان

احفظ المُقارنة دي لإنها من أشهر الأسئلة:

EC2: أنت اللي بتدير السيرفر، حتى لو التطبيق مش شغال.

ECS / EKS: أنت تدير ال Containers (أو Kubernetes)، و AWS تساعدك في الإدارة.

Lambda: لا تدير أي Servers أو Containers، فقط ترفع الكود، و AWS تشغله تلقائياً عند الحاجة، وتدفع فقط وقت التنفيذ.



Benefits of Lambda ❖

1. يدعم لغات برمجة متعددة

AWS Lambda يدعم عدد كبير من لغات البرمجة، مثل:

- Python
- Java
- Node.js
- #C
- Go
- Ruby
- PowerShell

وكمان تقدر تستخدم أي مكتبات (Libraries) سواء كانت مُدمجة أو خارجية.

2. إدارة كاملة للبنية التحتية (Fully Managed)

AWS هي التي بتتولى:

- تشغيل السيرفرات.
 - إدارة البنية التحتية.
 - الصيانة.
 - تحديثات الأمان (Security Patches).
- وأنت بتكتب الكود فقط.

3. Monitoring و Logging

Lambda بيتكامل مع Amazon CloudWatch، وبالتالي تقدر:

- تتابع تنفيذ الـ Functions.
- تشوف الـ Logs.
- تراقب الأداء.

4. Fault Tolerance و High Availability

Lambda بيشغل الكود على أكثر من Availability Zone داخل الـ Region.

وده بيوفر:

- High Availability (استمرار الخدمة).
- Fault Tolerance (تحمل الأعطال).

ولو حصل عطل في Server أو حتى Data Center، الخدمة تفضل شغالة.

5. Workflow باستخدام AWS Step Functions

لو عندك أكثر من Lambda Function وعائزهم يشتغلوا بترتيب مُعين، تقدر تستخدم AWS Step Functions.

Step Functions بتساعدك تعمل Workflow بين ال Lambda Functions.

وتقدر تحدد إنهم يشتغلوا:

- بالتسلسل (Sequential).
- بالتوازي (Parallel).
- حسب شروط مُعينة (Branching).
- مع مُعالجة الأخطاء (Error Handling).

6. الدفع حسب الإستخدام

في Lambda بتدفع فقط مقابل:

- عدد مرات تشغيل ال Function (Requests).
- مُدة تنفيذ الكود (Execution Time).

ويتم حساب وقت التشغيل بدقة كل 100 Milliseconds.

وده بيخلي Lambda مُناسب للتطبيقات اللي عدد تشغيلها قليل أو مُتغير.

7. التوسع التلقائي (Automatic Scaling)

Lambda بيعمل Scaling تلقائي.

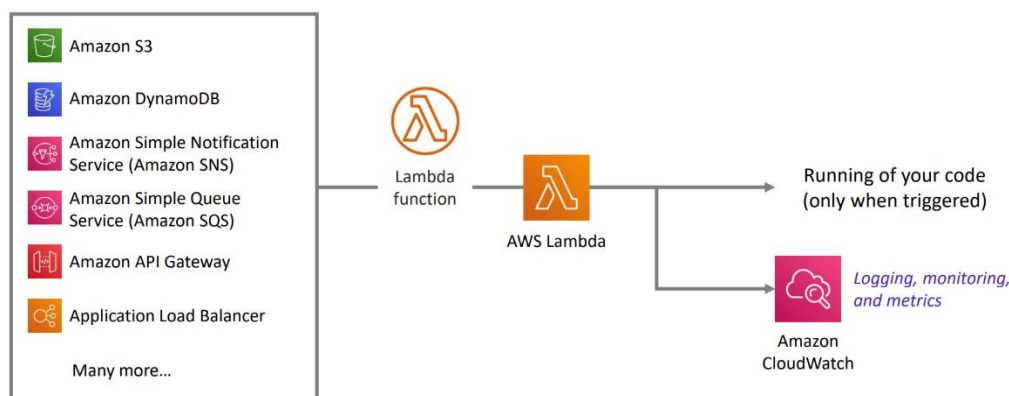
يعني سواء عندك عدد قليل من الطلبات أو آلاف الطلبات في الثانية، AWS بتزود أو تقلل الموارد تلقائياً بدون أي تدخل منك.

❖ AWS Lambda event sources

ال Event Source هو أي خدمة في AWS أو حتى ال Application بتاعك، لما يحصل فيها حدث (Event) تقوم تشغل (Trigger) ال Lambda Function.

يعني بدل ما تشغل ال Lambda بنفسك، الخدمة هي اللي بتشغلها تلقائياً عند حدوث حدث مُعين.

Event sources



الخدمات التي تقدر تشغيل Lambda**1. Amazon S3**

لما يحصل حدث على ال Bucket، زي:

- رفع ملف (Upload)
- حذف ملف (Delete)

ال S3 يقدر يشغل ال Lambda تلقائياً.

2. Amazon SNS (Simple Notification Service)

لما ال SNS يبعث Notification، يقدر يشغل ال Lambda عشان تنفذ كود مُعين.

3. Amazon CloudWatch Events / EventBridge

تقدر تشغيل ال Lambda في وقت مُعين (Schedule) أو عند حدوث Event مُعين داخل AWS.

4. Amazon SQS

ال SQS مش بيبعت Event لل Lambda بنفسه.

لكن ال Lambda بتعمل Polling لل Queue، وكل ما تلاقي رسالة جديدة تشغيل ال Function وتعالج الرسالة.

5. Amazon DynamoDB

لو حصل تغيير في البيانات (Insert - Update - Delete)، تقدر ال Lambda تقرأ التغييرات وتشغل نفسها.

6. Amazon API Gateway

أي HTTP Request يوصل لل API، ال API Gateway يشغل ال Lambda ويرجع النتيجة للمستخدم، وده من أشهر استخدامات Lambda.

تشغيل Lambda يدوياً (Direct Invocation)

مش لازم يكون فيه Event.

تقدر تشغيل ال Lambda بنفسك بإستخدام:

- AWS Management Console
- AWS CLI
- AWS SDK
- Lambda API

وده بيستخدم غالباً أثناء الإختبار أو التطوير.

Monitoring

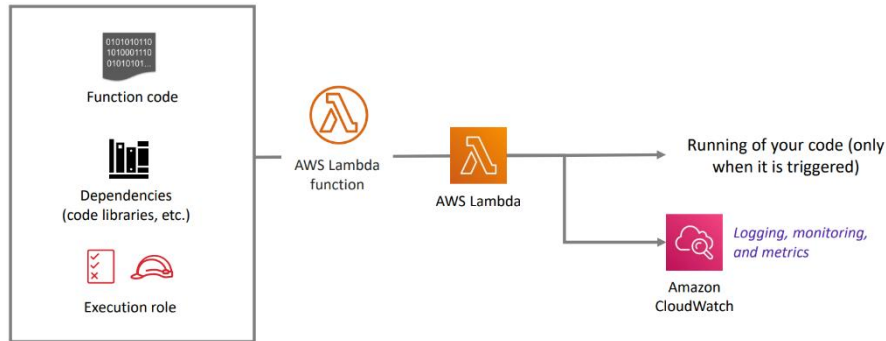
ال Lambda مُتكاملة تلقائياً مع Amazon CloudWatch.

وده معناه إنها:

- بتسجل كل مرة ال Function تشتغل فيها.
- بتسجل أي Errors أو Failures.
- بتخزن ال Logs داخل CloudWatch Logs.
- تقدر تتابع الأداء وتحل المشاكل بسهولة.

❖ AWS Lambda function configuration

بعد ما عرفنا إن Lambda Function هي الكود اللي أنت بتكتبه، وإن AWS هي اللي بتشغله عند حدوث Event، دلوقتي نشوف إزاي بنعمل Configuration لل Function.



لما تيجي تعمل Lambda Function من خلال AWS Management Console، أول حاجة بتحدد:

- **Function Name**: اسم ال Function.
- **Runtime**: لغة البرمجة اللي هتستخدمها (Python, Node.js, Java, C...).
- **Execution Role (IAM Role)**: الصلاحيات اللي ال Lambda هتستخدمها عشان تتعامل مع خدمات AWS المختلفة.

مهم: متحطش Access Keys داخل الكود، استخدم دائماً IAM Role.

بعد ما تضغط Create Function، بتبدأ تضبط باقي الإعدادات.

1. Add Trigger

بتحدد إيه اللي هيشغل ال Lambda.

مممكن يكون:

- Amazon S3
- Amazon SQS
- Amazon SNS
- Amazon API Gateway
- Amazon EventBridge
- وغيرها من الخدمات.

2. Add Function Code

بعدها بتحط الكود بتاعك.

وده ممكن تكتبه مباشرة من الـ AWS Console، أو ترفعه كملف.

3. Memory

بتحدد حجم الـ Memory اللي الـ Lambda هتستخدمها.

من 128 MB لحد 10,240 MB (10 GB).

كل ما تزود الـ Memory غالباً الأداء هيكون أسرع، لكن التكلفة هتزيد.

4. إعدادات اختيارية

تقدر كمان تضبط:

- Environment Variables: مُتغيرات يستخدمها الكود.
- Description: وصف للـ Function.
- Timeout: أقصى مدة تسمح للـ Function إنها تشتغل قبل ما AWS توقفها.
- VPC: لو الـ Lambda محتاجة تتصل بموارد داخل VPC.
- Tags: لتنظيم الموارد.

Lambda Deployment Package

كل الحاجات دي (الكود + المكتبات + الإعدادات المطلوبة) بتتجمع في حاجة اسمها Deployment Package.

وده عبارة عن ملف ZIP بيحتوي على:

- Function Code
- Dependencies (المكتبات).
- أي ملفات إضافية يحتاجها الكود.

لو بتستخدم AWS Management Console، AWS، الـ Deployment Package تلقائياً.

أما لو بتستخدم AWS CLI أو Lambda API، فإنت اللي لازم تعمل ملف الـ ZIP بنفسك وترفعه.

Schedule-based Lambda function example: Start and stop EC2 instances ❖

الـ AWS Lambda مش لازم تشتغل لما يحصل Event بس، لأ ممكن كمان تشتغل في وقت مُعين (Schedule).

في المثال ده، الهدف هو تقليل تكلفة تشغيل EC2.

لو عندك Instances خاصة بالـ Development أو Testing، مش محتاجة تفضل شغالة 24 ساعة.

فتقدر تعمل الآتي:

- بالليل تعمل Stop للـ EC2.
- الصبح تعمل Start للـ EC2.

وده كله يتم أوتوماتيك.

خطوات التنفيذ

1. بتعمل CloudWatch Event (EventBridge) يشتغل كل يوم الساعة 10 مساءً.
2. الـ Event يشتغل Lambda Function.
3. الـ Lambda تستخدم IAM Role عنده صلاحية إنه يعمل Stop للـ EC2 Instances.
4. الـ EC2 تدخل حالة Stopped.
5. تعمل CloudWatch Event ثاني يشتغل الساعة 5 صباحًا.
6. الـ Event يشتغل الـ Lambda مرة ثانية.
7. الـ Lambda تستخدم الـ IAM Role عشان تعمل Start للـ EC2 Instances.
8. الـ EC2 ترجع لحالة Running.

أهم حاجة تعرفها

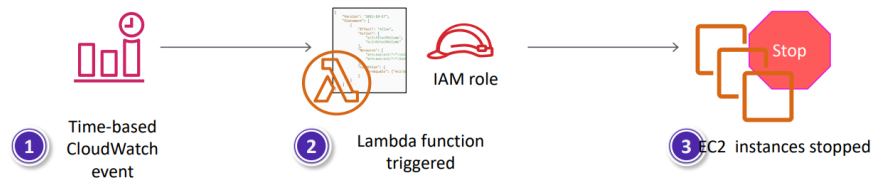
CloudWatch Events / EventBridge مسئول عن تشغيل الـ Lambda في الوقت المُحدد.

الـ Lambda هي اللي تنفذ أوامر Start و Stop.

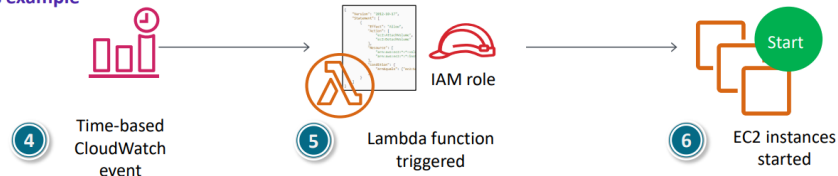
لازم الـ IAM Role يكون عنده صلاحيات تشغيل وإيقاف الـ EC2.

الفكرة الأساسية هي توفير التكلفة عن طريق إيقاف الـ EC2 وقت عدم استخدامها.

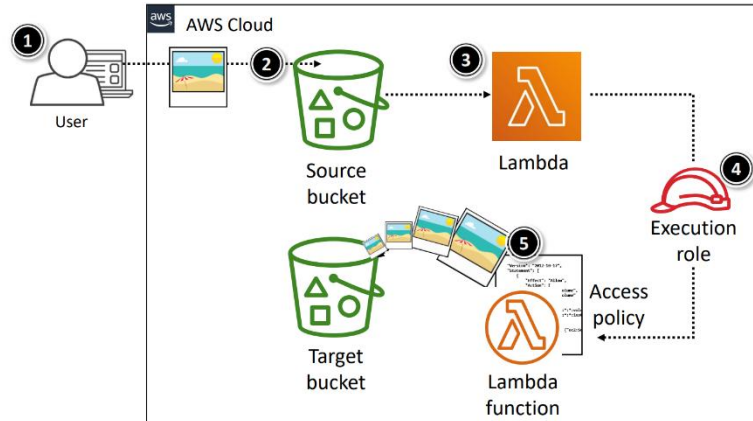
Stop instances example



Start instances example



Event-based Lambda function example: Create thumbnail images ❖



المثال ده بيوضح إزاي Lambda بتشتغل لما يحصل Event.

في الحالة دي، المطلوب إنه كل ما حد يرفع صورة على Amazon S3، يتم إنشاء نسخة صغيرة منها (Thumbnail) تلقائياً. يعني العملية كلها بتحصل بدون أي تدخل منك.

خطوات التنفيذ

المستخدم يرفع صورة (jpg أو png) على Source S3 Bucket.

ال Amazon S3 يكتشف إن فيه Object Created Event (تم رفع ملف جديد).

ال Amazon S3 يشغل (Trigger) ال Lambda Function ويبيعته بيانات عن الملف الي اترفع.

ال Lambda تشتغل بإستخدام Execution IAM Role الي بيديها صلاحية الوصول إلى S3.

ال Lambda تعرف من بيانات ال Event:

- اسم ال Source Bucket.
- اسم الملف (Object Key).

بعد كده:

- تقرأ الصورة من ال Source Bucket.
- تنشئ Thumbnail بإستخدام مكتبات معالجة الصور.
- تحفظ ال Thumbnail في Target S3 Bucket.

أهم حاجة تعرفها

هنا ال S3 هو الي بيشتغل ال Lambda عند رفع ملف جديد، يعني ال Trigger هنا هو Object Created Event في S3.

ال Event بيحتوي على معلومات مهمة زي:

- اسم ال Bucket.
- اسم الملف (Object Key).

ال Lambda لازم يكون عندها IAM Role يسمح لها تقرأ من ال Source Bucket وتكتب في ال Target Bucket.

❖ AWS Lambda quotas

الـ AWS Lambda فيها مجموعة من Quotas (Limits) لازم تعرفها، لإنها بتحدد أقصى الموارد اللي تقدر الـ Lambda تستخدمها. لو وصلت للحدود دي، ممكن الـ Lambda متشتغلش أو يحصل Error.

أهم الـ Quotas

- **Maximum Memory**: أقصى Memory تقدر تخصصها للـ Lambda هي 10,240 MB (10 GB)
- **Concurrent Executions**: تقدر تشغل 1000 Lambda Function في نفس الوقت داخل الـ Region (بشكل افتراضي).
- **Maximum Execution Time (Timeout)**: أقصى مدة لتشغيل الـ Lambda هي 15 دقيقة، وتقدر تحدد الـ Timeout من 1 ثانية لحد 15 دقيقة.
- **Deployment Package Size**: أقصى حجم لملف الـ (ZIP) Deployment Package هو 250 MB.

Lambda Layers

لو عندك مكتبات (Libraries) أو Dependencies كبيرة، بدل ما تضيفها داخل كل Deployment Package، تقدر تستخدم Lambda Layers.

الـ Layer عبارة عن ZIP File بيحتوي على:

- Libraries
- Custom Runtime
- Dependencies

وده بيساعدك:

- تقلل حجم الـ Deployment Package
- تُعيد استخدام نفس المكتبات بين أكثر من Lambda Function
- تشارك الكود بين عدة Functions

Container Image

لو التطبيق أكبر من إنك ترفعه كـ ZIP، AWS بتسمح إنك ترفع الـ Lambda على شكل Container Image. وفي الحالة دي، الحجم ممكن يوصل إلى 10 GB.

وده مناسب للتطبيقات الكبيرة زي:

- Machine Learning
- Data Processing

Soft Limit و Hard Limit

الـ Quotas نوعين:

- **Soft Limit**: تقدر تطلب من AWS تزوده عن طريق Support Ticket لو عندك سبب مقنع.
- **Hard Limit**: ده حد ثابت، ومينفعش تزوده.

6. Introduction to AWS Elastic Beanstalk

AWS Elastic Beanstalk ❖

الـ AWS Elastic Beanstalk هي خدمة من AWS بتتبع Platform as a Service (PaaS). يعني AWS توفرلك البيئة كاملة، وأنت كل اللي عليك إنك ترفع كود التطبيق.



AWS Elastic
Beanstalk

الـ AWS Elastic Beanstalk بتعمل إيه؟

بعد ما ترفع الكود، Elastic Beanstalk تقوم تلقائياً بـ:

- إنشاء الـ EC2 Instances.
- إعداد Load Balancer.
- إعداد Auto Scaling.
- مراقبة حالة التطبيق (Monitoring).
- نشر التطبيق (Deployment).

يعني بدل ما تعمل كل ده بنفسك، الخدمة بتعمله أوتوماتيك.

هل مازال عندي تحكم؟

أيوه، رغم إن Elastic Beanstalk بتدير البنية التحتية، إلا إنك لسه تقدر تتحكم في:

- نوع الـ EC2 Instance.
 - قاعدة البيانات.
 - إعدادات الـ Auto Scaling.
 - تحديث التطبيق.
 - الوصول إلى Log Files.
 - تفعيل HTTPS على الـ Load Balancer.
- وكمان تقدر تدخل على الموارد اللي أنشأتها AWS في أي وقت.

التكلفة

مفیش رسوم إضافية على Elastic Beanstalk نفسها.

أنت بتدفع فقط مقابل الموارد اللي هي استخدمتها لتشغيل التطبيق، مثل:

- EC2
- Elastic Load Balancer
- Amazon S3
- RDS (لو استخدمته).

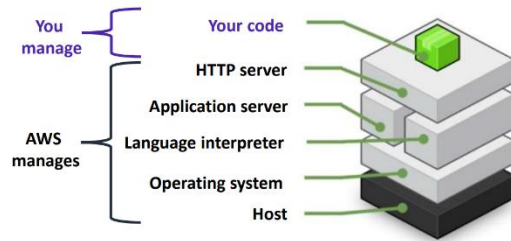
يعني Elastic Beanstalk بتكون مجانية، لكن الموارد اللي بتُنشئها هي اللي عليها تكلفة.

AWS Elastic Beanstalk deployments ❖

الـ AWS Elastic Beanstalk بتسهل عملية نشر (Deployment) التطبيق.

تقدر ترفع الكود بأكثر من طريقة، مثل:

- .AWS Management Console
- .AWS CLI
- .Visual Studio
- .Eclipse



بعد ما ترفع الكود، الـ Elastic Beanstalk بتتولى باقي خطوات النشر تلقائياً.

المنصات (Platforms) اللي بيدعمها

Elastic Beanstalk بيدعم لغات وإطارات عمل كثير، منها:

- Docker
- Go
- Java
- .NET
- Node.js
- PHP
- Python
- Ruby

بيستخدم إيه لتشغيل التطبيق؟

حسب لغة البرمجة اللي بتستخدمها، Elastic Beanstalk بيختار السيرفر المناسب تلقائياً.

- Java : Apache Tomcat
- PHP / Python : Apache HTTP Server
- Node.js : Apache HTTP Server أو NGINX
- Ruby : Passenger أو Puma
- .NET : Microsoft IIS
- Docker / Go / Java SE : بيستخدم البيئة المناسبة تلقائياً.

❖ Benefits of Elastic Beanstalk

الـ AWS Elastic Beanstalk بتوفر مميزات كتير تخلي نشر وإدارة التطبيقات أسهل.

1. سهولة وسرعة النشر

تقدر ترفع التطبيق بإستخدام:

- .AWS Management Console
- .Git Repository
- .Visual Studio
- .Eclipse

وبعدها Elastic Beanstalk تتولى عملية الـ Deployment تلقائياً.

2. إدارة البنية التحتية تلقائياً

Elastic Beanstalk بتدير تلقائياً:

- Provisioning للموارد.
- Load Balancing
- Auto Scaling
- Monitoring لصحة التطبيق.

وده يخليك تركز على كتابة الكود بدل إدارة السيرفرات.

3. زيادة إنتاجية المطور

بدل ما تضيق وقت في إعداد:

- .Servers
- .Databases
- .Load Balancers
- .Firewalls
- .Networks

تقدر تركز على تطوير التطبيق فقط.

4. تحديثات تلقائية

AWS بتحدث البيئة اللي شغال عليها التطبيق تلقائياً، مثل:

- .Security Patches
- .System Updates

بدون ما تحتاج تعملها بنفسك.

5. Auto Scaling

لو عدد المُستخدمين زاد، Elastic Beanstalk تزود الموارد تلقائياً.

ولو الحمل قل، تقلل الموارد.

وده يساعد على:

- تحسين الأداء.
- تقليل التكلفة.

وتقدر تستخدم CPU Utilization كشرط لتشغيل الـ Auto Scaling.

6. تحكم كامل في الموارد

رغم إن Elastic Beanstalk بتدير البنية التحتية، إلا إنك ما زال تقدر تختار وتعديل:

- نوع الـ EC2 Instance.
- إعدادات الـ Auto Scaling.
- باقي موارد AWS المُستخدمة.

ولو حبيت تدير الموارد بنفسك بعد كده، تقدر تعمل ده بدون مُشكلة.

❖ الفرق بين AWS Lambda و AWS Elastic Beanstalk

AWS Lambda

في Lambda أنت بتكتب Function صغيرة، وAWS هي اللي:

- تشغيلها.
- تعمل Scaling.
- تدير ال Servers.
- تعمل Monitoring.

أنت ملكش دعوة بأي Server.

الكود بيشتغل فقط لما يحصل:

- S3 Upload
- API Request
- SQS Message
- Schedule
- أي Event

ولما يخلص التنفيذ.... يقف، يعني مغيث سيرفر شغال طول الوقت.

تستخدم Lambda إمتى؟

لما يكون عندك:

- Image Processing
- File Upload
- Notifications
- Backend APIs صغيرة.
- Automation
- Scheduled Tasks

يعني أي Task قصيرة وسريعة.

مميزات Lambda

- Serverless
- Auto Scaling
- تدفع وقت التشغيل فقط.
- مغيث إدارة Servers.
- Event Driven

عيوب Lambda

- أقصى مدة تشغيل 15 دقيقة.
- مش مناسبة للتطبيقات الكبيرة.
- فيها Limits لل Memory وال Package Size.

AWS Elastic Beanstalk

Elastic Beanstalk مُختلفة تماماً، هنا أنت عندك Application كامل.

مثلاً:

- موقع Web.
- REST API كبيرة.
- نظام ERP.
- E-commerce.

أنت ترفع الكود فقط، و Elastic Beanstalk تقوم تلقائياً بإنشاء:

- EC2
- Load Balancer
- Auto Scaling
- Security Groups
- Monitoring

لكن السيرفرات موجودة بالفعل.

تستخدم Elastic Beanstalk إمتى؟

لما يكون عندك Application يفضل شغال 24 ساعة.

مثلاً:

- Django
- Laravel
- Spring Boot
- ASP.NET
- Node.js Application

مميزات Elastic Beanstalk

- سهل في ال Deployment.
- يعمل Auto Scaling.
- يعمل Load Balancing.
- AWS تحدث ال Platform.
- عندك تحكم في EC2.

عيوب Elastic Beanstalk

- مازال فيه Servers.
- بتدفع ثمن EC2 حتى لو محدش بيستخدم التطبيق.
- إدارة أكثر من Lambda.

♦ **Module 7: Storage**

1. Amazon Elastic Block Store (Amazon EBS)

Amazon Elastic Block Store
(Amazon EBS)



Storage ❖

ال Amazon EBS هو خدمة Block Storage تُستخدم مع Amazon EC2.

تقدر تعتبره الهارد ديسك اللي بتركبه على ال EC2 علشان تخزن عليه نظام التشغيل والملفات والبيانات.

Persistent Storage

ال Persistent Storage معناه إن البيانات بتفضل موجودة حتى لو ال EC2 اعمله Stop أو Restart.

يعني لو عندك Server عليه Windows أو Linux، وبعدها عملت Stop ثم Start...

هتلاقي:

- نظام التشغيل موجود.
- الملفات موجودة.
- التطبيقات موجودة.

وده عكس ال Instance Store اللي البيانات عليه بتضيع.

Replication داخل ال Availability Zone

كل EBS Volume بيتعمله Replication تلقائي داخل نفس ال Availability Zone (AZ).

وده معناه لو حصل عطل في الهارد اللي عليه البيانات، AWS تقدر تسترجعها من النسخة الثانية.

ملحوظة: ال Replication بيكون داخل نفس ال AZ فقط، وليس بين أكثر من AZ.

Performance

تم تصميم ال EBS بحيث يوفر:

- أداء ثابت (Consistent Performance).
- زمن استجابة قليل (Low Latency).

وده يخليه مناسب لتشغيل:

- أنظمة التشغيل.
- قواعد البيانات.
- التطبيقات.

Pricing

بتدفع فقط على حجم ال Storage اللي عملته Provision.

Scalability

من مميزات الـ EBS إنك تقدر:

- تزود حجم الـ Volume.
- تغير نوع الـ Volume.

وكل ده بسهولة بدون الحاجة لإنشاء Server جديد.

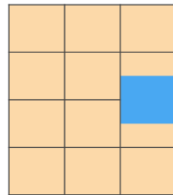
❖ AWS Storage Options: Block Storage VS Object Storage

قبل ما نتكلم عن خدمات التخزين في AWS، لازم تعرف إن فيه طريقتين لتخزين البيانات:

- Block Storage
- Object Storage

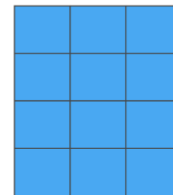
والفرق بينهم مهم جداً لأنه بيأثر على:

- السرعة (Performance).
- زمن الإستجابة (Latency).
- استهلاك الـ Bandwidth.
- التكلفة.



Block storage

Change one block (piece of the file)
that contains the character



Object storage

Entire file must be updated

أولاً: Block Storage

في الـ Block Storage، الملف بيتقسم إلى أجزاء صغيرة اسمها Blocks.

كل Block بيتخزن بشكل مُستقل.

لو عدلت جزء صغير من الملف ... AWS هتعدل الـ Block اللي اتغير فقط، مش الملف كله.

وده بيخلي الـ Block Storage سريع جداً.

ثانياً: Object Storage

في الـ Object Storage، الملف بيتخزن كـ Object واحد كامل.

يعني AWS بتعتبر الملف كله وحدة واحدة.

لو غيرت حتى حرف واحد لازم تعيد رفع الملف بالكامل، مش بس الجزء اللي اتغير.

مثال يوضح الفرق

تخيل عندك ملف حجمه 1 GB، وغيّرت حرف واحد فقط.

لو الملف موجود على (EBS (Block Storage، AWS هتعدل ال Block اللي فيه الحرف فقط.

يعني ممكن تعدل 4 KB بدل ما تعدل 1 GB.

لكن لو الملف موجود على (S3 (Object Storage، ساعتها AWS لازم تعيد رفع ملف ال 1 GB كله.

علشان كده ال Object Storage أبداً في التعديلات.

ليه بنستخدم ال Block Storage؟

لأنه مناسب للحاجات اللي بيحصل فيها قراءة وكتابة باستمرار، زي:

- Operating System
- Databases
- Applications
- Virtual Machines

ليه بنستخدم ال Object Storage؟

لأنه مناسب لتخزين الملفات الكبيرة اللي مش بيتعدل عليها باستمرار، زي:

- الصور
- الفيديوهات
- ملفات ال Backup
- ملفات ال Logs
- ملفات المواقع (Static Files)

نقطة مهمة

بسبب إن ال Block Storage بيعدل جزء صغير فقط من الملف، فهو:

- أسرع
- أقل استهلاكاً لل Network
- لكنه غالباً أغلى

أما Object Storage:

- أرخص
- مُمتاز لتخزين كميات ضخمة من الملفات
- لكنه مش مناسب لو الملف بيتعدل باستمرار

Amazon EBS ❖

ال Amazon EBS (Elastic Block Store) هو خدمة Block Storage في AWS، وتُستخدم مع EC2 لتخزين البيانات بشكل دائم (Persistent Storage).

تقدر تنشئ EBS Volume بشكل مُنفصل، وبعدها توصله (Attach) بأي EC2 Instance.

EBS Volume

ال EBS Volume عبارة عن قرص تخزين (Hard Disk) مُستقل عن ال EC2.

يعني تقدر:

- تنشئ Volume جديد.
 - توصله بـ EC2.
 - تفصله من EC2 وتوصله بـ EC2 أخرى (في نفس ال Availability Zone).
- وده بيديك مرونة كبيرة في إدارة التخزين.

Block-Level Storage

ال EBS بيستخدم Block Storage.

يعني البيانات بتتقسم إلى Blocks، ولو عدلت جزء صغير من ملف، بيتعدل ال Block اللي اتغير فقط. وده بيخلي EBS مناسب للتطبيقات اللي محتاجة سرعة في القراءة والكتابة.

Durable Storage

AWS بتعمل Replication تلقائي لكل EBS Volume داخل نفس ال Availability Zone.

وده معناه إن لو حصل عطل في أحد الأقراص، البيانات تفضل موجودة.

علشان كده بنقول إن EBS خدمة Durable (موثوقة وتحافظ على البيانات).

Detachable Storage

من أهم مميزات EBS إنه Detachable.

يعني تقدر تفصل ال Volume من EC2 وتوصله بـ EC2 ثانية بدون ما تفقد البيانات.

وده شبه إنك تفصل هارد من جهاز وتركبه في جهاز تاني.

Low Latency

بما إن ال EBS متوصل مُباشرةً بال EC2، فهو بيوفر Low Latency وسرعة عالية في الوصول للبيانات.

علشان كده يستخدم مع:

- قواعد البيانات.
- أنظمة التشغيل.
- التطبيقات اللي محتاجة أداء عالي.

العلاقة بين EBS و AMI

لما تعمل (Amazon Machine Image) AMI من EC2، فبيانات ال EBS Volumes بتكون جزء من ال Backup. بعد كده بيتم تخزين ال AMI في Amazon S3، وتقدر تستخدمها بعد كده لإنشاء EC2 جديدة بنفس النظام والإعدادات.

EBS Snapshot

ال Snapshot هو نسخة احتياطية (Backup) من ال EBS Volume.

Baseline Snapshot

أول Snapshot بتأخذها اسمها Baseline Snapshot.

ودي بتحتوي على كل البيانات الموجودة على ال Volume.

Incremental Snapshot

أي Snapshot بعد أول واحدة بيكون Incremental.

يعني بيحفظ التغييرات فقط مُنذ آخر Snapshot، وليس كل البيانات مرة أخرى.

وده يقلل:

- مساحة التخزين.
- وقت إنشاء ال Backup.
- التكلفة.

استخدامات Amazon EBS

يستخدم في:

- Boot Volume لتثبيت نظام التشغيل على EC2.
- Data Storage لتخزين الملفات والبيانات.
- Database Hosts لتشغيل قواعد البيانات.
- Enterprise Applications التي تحتاج أداء عالي واستقرار.

Amazon EBS volume types ❖

لما تيجي تعمل EBS Volume، AWS بتديك أكثر من نوع (Volume Type)، لإن كل تطبيق احتياجاته مختلفة.

يعني مش منطقي إن Website بسيط يستخدم نفس نوع التخزين اللي تستخدمه Database ضخمة عليها ملايين العمليات في الثانية.

علشان كده AWS وفرت أنواع مختلفة بحيث تختار النوع المناسب حسب الأداء المطلوب والتكلفة.

	Solid State Drives (SSD)		Hard Disk Drives (HDD)	
	General Purpose	Provisioned IOPS	Throughput-Optimized	Cold
Maximum Volume Size	16 TiB	16 TiB	16 TiB	16 TiB
Maximum IOPS/Volume	16,000	64,000	500	250
Maximum Throughput/Volume	250 MiB/s	1,000 MiB/s	500 MiB/s	250 MiB/s

1. General Purpose SSD (gp2 / gp3)

ده أكثر نوع مُستخدم في AWS، لأنه يحقق توازن بين الأداء والسعر.

مناسب لمُعظم التطبيقات مثل:

- Web Servers
- Application Servers
- Development & Testing
- Boot Volume

لو معندكش مُتطلبات أداء عالية جداً، فده غالباً أفضل اختيار.

2. Provisioned IOPS SSD (io1 / io2)

النوع ده معمول للتطبيقات اللي محتاجة أعلى أداء ممكن.

AWS بتوفرلك عدد ثابت من عمليات القراءة والكتابة في الثانية (IOPS)، وبالتالي الأداء بيكون ثابت حتى مع الضغط العالي.

بيستخدم في:

- Oracle Database
- SQL Server
- SAP
- التطبيقات البنكية
- Financial Applications

لكن لأنه بيوفر أعلى أداء، فهو أغلى من باقي الأنواع.

3. HDD Volumes

دي أقراص Hard Disk عادية.

الأداء فيها أقل من SSD، لكن سعرها أرخص.

تستخدم في:

- File Storage
- Backups
- Logs
- Big Data
- Archives

مش مناسبة لقواعد البيانات أو التطبيقات التي محتاجة سرعة.

Boot Volume

ال Boot Volume هو ال Volume اللي عليه نظام التشغيل.

AWS بتسمح إن ال Boot Volume يكون من نوع SSD فقط.

يعني:

- General Purpose SSD
- Provisioned IOPS SSD

HDD لا يُمكن استخدامه كـ Boot Volume.

لماذا يوجد أكثر من نوع؟

الهدف هو تقليل التكلفة.

لو تطبيقك بسيط واستخدمت Provisioned IOPS، هتدفع فلوس زيادة بدون أي استفادة.

ولو عندك Database ضخمة واستخدمت HDD، الأداء هيبكون ضعيف جداً.

علشان كده لازم تختار النوع المناسب حسب ال Workload.

Amazon EBS volume type use cases ❖

Solid State Drives (SSD)		Hard Disk Drives (HDD)	
General Purpose	Provisioned IOPS	Throughput-Optimized	Cold
- This type is recommended for most workloads - System boot volumes - Virtual desktops - Low-latency interactive applications - Development and test environments	- Critical business applications that require sustained IOPS performance, or more than 16,000 IOPS or 250 MiB/second of throughput per volume - Large database workloads	- Streaming workloads that require consistent, fast throughput at a low price - Big data - Data warehouses - Log processing - It cannot be a boot volume	- Throughput-oriented storage for large volumes of data that is infrequently accessed - Scenarios where the lowest storage cost is important - It cannot be a boot volume

بعد ما عرفنا أنواع الـ EBS Volumes، دلوقتي هنوضح كل نوع بيستخدم في إيه، لإن اختيار النوع المناسب بيساعدك تحصل على أفضل أداء بأقل تكلفة.

1. General Purpose SSD

ده النوع المناسب لمُعظم الإستخدامات، لأنه بيقدم توازن بين الأداء والتكلفة.

يستخدم في:

- مُعظم التطبيقات (Most Workloads).
- Boot Volume لتشغيل نظام التشغيل.
- Virtual Desktops.
- التطبيقات التي تحتاج Low Latency.
- بيئات التطوير والإختبار (Development & Test).

ملحوظة: لو السؤال في الإمتحان قال أفضل اختيار لمُعظم التطبيقات، فالإجابة غالباً هتكون General Purpose SSD.

2. Provisioned IOPS SSD

النوع ده مُخصص للتطبيقات اللي محتاجة أعلى أداء ممكن وثبات في عدد عمليات القراءة والكتابة (IOPS).

يُستخدم في:

- التطبيقات الحرجة (Critical Business Applications).
- التطبيقات التي تحتاج أكثر من 16,000 IOPS.
- التطبيقات التي تحتاج Throughput قيمته 250 MiB/s أو أكثر.
- قواعد البيانات الكبيرة (Large Database Workloads).

لو التطبيق بيعتمد على Database ضخمة أو بيعمل آلاف عمليات القراءة والكتابة في الثانية، يبقى ده أفضل اختيار.

3. Throughput-Optimized HDD (st1)

ده نوع HDD لكنه معمول للتطبيقات الي محتاجة Throughput عالي وسعر مُنخفض، مش محتاجة IOPS عالية. يُستخدم في:

- Streaming Workloads
- Big Data
- Data Warehouses
- Log Processing

لا يُمكن استخدامه ك Boot Volume

ركز إن النوع ده مُناسب لنقل كميات كبيرة من البيانات (Sequential Read/Write)، لكنه مش مُناسب لقواعد البيانات.

4. Cold HDD (sc1)

ده أرخص نوع في EBS، ومُناسب للبيانات الي نادراً ما يتم الوصول إليها. يُستخدم في:

- تخزين كميات كبيرة من البيانات قليلة الإستخدام.
- الحالات التي يكون فيها تقليل تكلفة التخزين هو الأولوية.
- Archive Storage

لا يُمكن استخدامه ك Boot Volume

ملحوظة: لو السؤال قال "أقل تكلفة للتخزين"، فالإجابة هي Cold HDD.

Amazon EBS features ❖

لا يقتصر دور Amazon EBS على كونه مجرد مساحة تخزين للـ EC2، لكنه يوفر مجموعة من المميزات التي تساعد على حماية البيانات، زيادة المرونة، وتسهيل إدارة التخزين

Point-in-Time Snapshot

من أهم مميزات EBS هي إمكانية إنشاء Snapshot.

الـ Snapshot هي نسخة احتياطية (Backup) من الـ EBS Volume في لحظة مُعينة (Point-in-Time).

يعني AWS بتأخذ صورة (نسخة - Copy) لحالة الـ Volume في وقت مُعين، وتقدر ترجع إليها في أي وقت إذا احتجتها.

Create a New Volume from Snapshot

بعد إنشاء الـ Snapshot، تقدر تنشئ EBS Volume جديد منها في أي وقت.

وده بيفيد في حالات مثل:

- استعادة البيانات بعد حدوث مُشكلة.
- إنشاء نسخة جديدة من نفس الـ Volume.
- نقل البيانات إلى EC2 أخرى.

يعني الـ Snapshot مش مجرد Backup، لكنها تقدر تكون بداية لإنشاء Volume جديد بالكامل.

Share and Copy Snapshots

AWS بتسمح لك تعمل حاجتين مُهمين:

- Share Snapshot مع AWS Account آخر.
- Copy Snapshot إلى Region أخرى.

الميزة دي بتستخدم في Disaster Recovery (DR).

يعني لو حصلت مُشكلة في الـ Region الأساسية، تقدر تستعيد بياناتك من الـ Snapshot الموجودة في Region ثانية.

مثال:

الـ Snapshot موجودة في Virginia.

تعمل لها Copy إلى Tokyo.

لو Region Virginia حصل فيها عطل، تقدر تستخدم نسخة Tokyo.

Encryption

يدعم Amazon EBS تشفير البيانات (Encryption) بدون أي تكلفة إضافية.

وده معناه إن البيانات تكون محمية أثناء انتقالها بين:

- EC2 Instance
- EBS Volume

داخل مراكز بيانات AWS.

وده بيزود مستوى الأمان ويحمي البيانات من أي وصول غير مصرح به.

ملحوظة: التشفير بدون رسوم إضافية، ودي نقطة ممكن تيجي في الإمتحان.

Resize Volume

مع زيادة حجم بيانات الشركة، ممكن تحتاج مساحة أكبر.

بدل ما تنشئ Volume جديد، تقدر تزود حجم ال Volume الحالي.

مثلاً: من 50 GB إلى 500 GB أو حتى 16 TB.

Change Volume Type

مش بس تقدر تكبر الحجم، لكن كمان تقدر تغير نوع ال Volume.

مثلاً: من HDD إلى SSD لو احتجت أداء أعلى، أو العكس لو هدفك تقليل التكلفة.

وده بيديك مرونة كبيرة حسب احتياجات التطبيق.

Dynamic Resize

الميزة المهمة إن كل التعديلات دي تقدر تعملها وال EC2 شغال.

يعني تقدر تكبر مساحة ال Volume أو تغير نوعه، بدون الحاجة لإيقاف ال Instance.

وده بيقلل ال Downtime ويحافظ على استمرار الخدمة.

مثال

شركة بدأت بـ EC2 عليه EBS Volume حجمه 50 GB من نوع General Purpose SSD.

بعد فترة زادت البيانات وأصبحت تحتاج مساحة وأداء أعلى.

بدل إنشاء Volume جديد، قامت بـ:

- زيادة الحجم إلى 500 GB.
- تغيير النوع إلى Provisioned IOPS SSD.

وتمت العملية بدون إيقاف ال EC2.

Amazon EBS: Volumes, IOPS, and pricing ❖

عند حساب تكلفة Amazon EBS، لا تدفع مقابل الـ EC2 نفسه، ولكن بتدفع مقابل الـ Storage والأداء (Performance) التي تستخدمها.

بشكل عام، تكلفة الـ EBS تعتمد على عاملين رئيسيين:

- Volume Size
- IOPS (Performance)

أولاً: Volume Pricing

أي EBS Volume تقوم بإنشائه يفضل موجود بشكل مستقل عن الـ EC2 Instance. يعني لو قمت بإيقاف أو حذف الـ EC2 (حسب طريقة الحذف)، فإن الـ EBS هيفضل موجود ويحتفظ بالبيانات. ولهذا يتم مُحاسبتك على حجم الـ Volume الذي قمت بعمل Provision له (GB/Month) وليس على كمية البيانات الفعلية الموجودة داخله.

مثال

لو أنشأت Volume حجمه 100 GB، وحتى لو استخدمت 20 GB.

هتدفع تكلفة الـ 100 GB لأن هو ذا الحجم اللي حجزته (Provisioned Storage).

ثانياً: تكلفة الـ IOPS

الـ IOPS (Input/Output Operations Per Second) هو عدد عمليات القراءة والكتابة التي يستطيع الـ Volume تنفيذها في الثانية.

لكن طريقة احتساب الـ IOPS بتختلف حسب نوع الـ Volume.

1. General Purpose SSD (gp2 / gp3)

في هذا النوع، تكلفة الـ IOPS تكون مُضمنة داخل سعر الـ Volume.

يعني مش هتدفع مبلغ إضافي مقابل الـ IOPS، ولكن هتدفع فقط مقابل حجم الـ Volume.

2. Magnetic

في هذا النوع، يتم احتساب تكلفة إضافية حسب عدد عمليات القراءة والكتابة (Requests) التي تحصل على الـ Volume.

كلما زاد عدد الـ Requests، زادت التكلفة.

3. Provisioned IOPS SSD (io1 / io2)

في هذا النوع، يتم احتساب تكلفة حجم الـ Volume، وتكلفة عدد الـ IOPS التي هحجزتها (Provisioned IOPS).

ولذلك يعتبر هذا النوع هو الأعلى في الأداء، والأعلى في التكلفة أيضاً.

ملاحظة: كلمة Provisioned معناها الموارد اللي هحجزتها، وليس الموارد التي استخدمتها فعلياً.

هل تسعير EBS بسيط؟

الإجابة هي لا، تسعير Amazon EBS يُعتبر من أكثر خدمات AWS تعقيداً، لأن التكلفة بتختلف حسب:

- نوع ال Volume.
- حجم ال Volume.
- ال IOPS.
- مدة استخدام ال Volume.

لكن بشكل عام يمكنك تذكر القاعدة التالية كلما زاد حجم ال Volume أو زاد الأداء (IOPS)، زادت التكلفة.

❖ Amazon EBS: Snapshots and data transfer

عند استخدام Amazon EBS، هناك تكلفتان إضافيتان يجب الإنتباه لهما، وهما:

- Snapshots
- Data Transfer

أولاً: Snapshots

لما تعمل Snapshot، AWS بتأخذ نسخة احتياطية من ال EBS Volume وتحفظها في Amazon S3.

وده بيساعدك لو حصل:

- حذف للبيانات.
- تلف في ال Volume.
- أو عايز ترجع لحالة قديمة من البيانات.

لكن لازم تعرف إن ال Snapshot ليها تكلفة إضافية.

بتتاسب على حجم البيانات المخزنة داخل ال Snapshot كل شهر (GB/Month)، يعني كل ما حجم البيانات يزيد، التكلفة هتزيد.

ثانياً: Data Transfer

ممکن يبقى عندك Snapshot في Region، وتحتاج تنقلها لـ Region ثانية.

وده بيستخدم غالباً في Disaster Recovery (DR) أو لو عايز يبقى عندك Backup في مكان ثاني.

لكن هنا AWS بتحاسبك على نقل البيانات بين ال Regions.

وبعد ما ال Snapshot توصل لـ Region الجديدة، هتبدأ تتحاسب كمان على تخزينها هناك.

2. Amazon Simple Storage Service (Amazon S3)

Storage ❖

بعد ما خلصنا Amazon EBS، هنبدأ في أشهر خدمة Storage في AWS وهي Amazon S3.

الفرق الأساسي بينهم إن:

Amazon Simple Storage
Service (Amazon S3)



- EBS = Block Storage (زي الهارد اللي بتركبه في الكمبيوتر).
- S3 = Object Storage (بيخزن الملفات كـ Objects).

يعني إيه Amazon S3؟

Amazon S3 (Simple Storage Service) هي خدمة تخزين ملفات (Object Storage) على AWS.

أي ملف ترفعه على S3 بيتخزن على شكل Object داخل مكان اسمه Bucket. فال Bucket تقدر تعتبره زي Folder كبير أو Container بيجمع فيه كل الملفات.

أول خاصية: Persistent Storage

ال S3 يعتبر Persistent Storage.

يعني البيانات بتفضل موجودة حتى لو:

- الجهاز اتقفّل.
 - السيرفر وقف.
 - ال EC2 Instance اتقفّلت أو حتى اتمسحت.
- البيانات مش بتضيع لإنها متخزنة بشكل مُستقل على ال S3.

مثال للتوضيح:

عندك موقع عليه صور.

لو الصور متخزنة على ال EC2، والسيرفر وقع...

الصور هتضيع.

لكن لو الصور على ال S3..

أي Server جديد يقدر يجيبها عادي.

وده سبب إن أغلب المواقع الكبيرة بتحط الصور والملفات على S3.

ثاني خاصية: كل ملف له URL

أي Object على S3 سيكون له URL خاص به.

وده معناه إنك تقدر توصله من أي مكان على الإنترنت (لو عندك صلاحية).

مثلاً: <https://mybucket.s3.amazonaws.com/logo.png>

أي حد معاه ال URL والصلاحيات المناسبة يقدر يفتح الملف.

ثالث خاصية: Object Storage

دي أهم نقطة ولازم تحفظها.

في ال Amazon S3 الملفات بتتخزن ك Objects.

يعني لو عندك ملف حجمه 1 GB، وعدلت حرف واحد فيه، مينفعش AWS تعدل الجزء ده بس، لازم تنزل الملف، وتعدل عليه، وترفع الملف كله مرة ثانية.

وده الفرق الكبير بينه وبين EBS.

رابع خاصية: Buckets

كل الملفات في S3 لازم تكون موجودة داخل Bucket.

يعني مينفعش ترفع Object مباشرة على S3.

لازم الأول تعمل Bucket.

بعدها تبدأ ترفع الملفات جواها.

مثال للتوضيح:

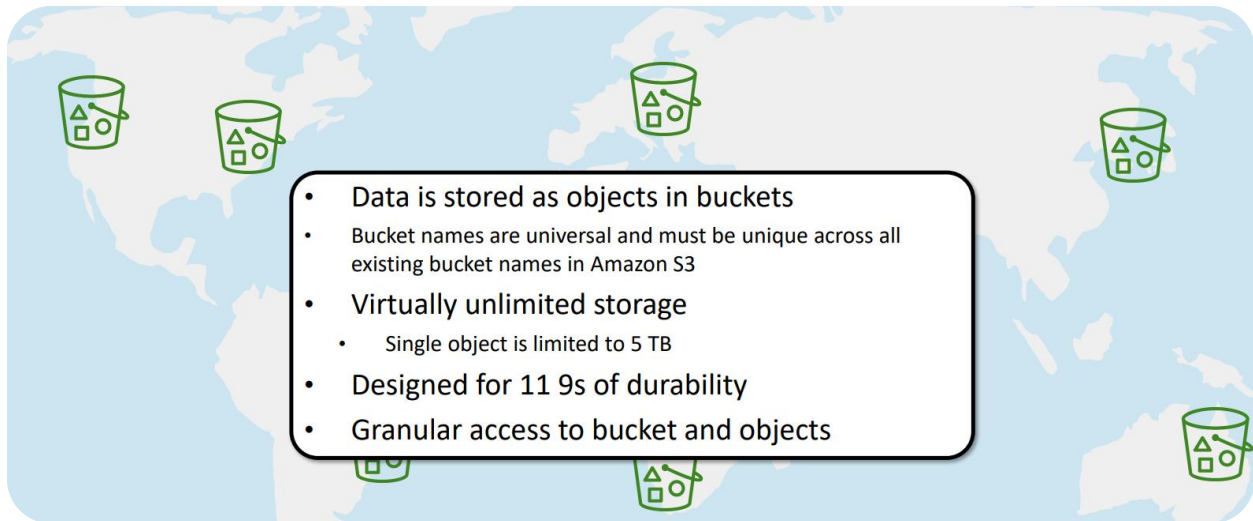
عملت Bucket اسمها company-backup

وجواها رفعت:

- database.sql
- users.csv
- images.zip

يبقى الثلاث ملفات دول اسمهم Objects، والمكان اللي بيحتويهم اسمه Bucket.

Amazon S3 overview ❖



بعد ما عرفنا إن Amazon S3 هو خدمة Object Storage، تعالَ نتعرف على أهم المُميزات اللي بتخليه من أكثر خدمات AWS استخداماً.

أولاً: Managed Storage

S3 خدمة Managed، يعني AWS هي اللي بتدير كل حاجة.

مش محتاج تشيل هم:

- الهاردات.
- السيرفرات.
- الصيانة.
- ال Replication.

كل اللي عليك إنك ترفع الملفات، وAWS هي اللي بتتكفل بالباقي.

ثانياً: البيانات بتتخزن كـ Objects داخل Buckets

أي ملف بترفعه على S3 بيتحول إلى Object، وال Objects دي لازم تكون موجودة داخل Bucket.

يعني:

- Bucket = مكان التخزين.
- Object = الملف نفسه.

ثالثاً: مساحة تخزين شبه غير محدودة

من أكبر مُميزات S3 إنك تقدر تخزن عدد ضخم جداً من الملفات.

AWS بتقول إن S3 يقدر يخزن Trillions of Objects / يعني عملياً مش هتقلق من إن المساحة تخلص.

رابعاً: أقصى حجم للملف

ال Object الواحد ممكن يوصل حجمه إلى 5 TB.

وده يخليك تقدر تخزن ملفات كبيرة جداً زي:

- Database Backups
- Videos
- VM Images
- ملفات Archive

خامساً: 11 9s of Durability

دي من أشهر الحاجات في S3.

AWS بتضمن 99.999999999% Durability.

وده معناها إن احتمالية فقدان البيانات قليلة جداً.

لإن AWS بتعمل نسخ من البيانات على أكثر من جهاز وأكثر من Facility داخل نفس ال Region.

ملحوظة

Durability معناها إن البيانات متضيعش، أما **Availability** معناها إن الخدمة تفضل شغالة وتقدر توصل للبيانات.

سادساً: الوصول للبيانات

تقدر توصل لل Objects بأكثر من طريقة:

- HTTP
- HTTPS
- VPC Endpoint (لو عايز الوصول يكون من داخل ال VPC فقط)

سابعاً: التحكم في الصلاحيات

S3 بيديك تحكم كامل في مين يقدر يشوف الملفات أو يحملها أو يعدلها.

وده بيتم عن طريق:

- IAM Policies
- Bucket Policies
- ACLs

وبشكل افتراضي ال Bucket بتكون Private.

ثامناً: Event Notifications

S3 يقدر يبعث Notification أو يشغل Service ثانية أول ما يحصل Event.

مثلاً:

- رفع ملف جديد.
- حذف ملف.
- تعديل Object.

ومن أشهر الاستخدامات، لما ترفع صورة على S3، يشغل AWS Lambda تعمل Thumbnail تلقائياً.

تاسعاً: Storage Class Analysis

لو عندك ملفات قديمة مبعثش بتتفتح كثير...

ال S3 يقدر يحلل استخدام الملفات، ويقترح ينقلها إلى Storage Class أرخص.

وده يساعد في تقليل التكلفة.

مثال للتوضيح:

شركة عندها:

- صور المنتجات.
- فيديوهات.
- ملفات PDF.
- Backups.

كل الملفات دي بتتفع على S3 داخل Bucket.

لو المستخدم طلب صورة هيجيبها مباشرةً من ال S3، ولو الصورة قديمة ومحدثش بيفتحها، ال S3 يقترح نقلها إلى Storage Class أرخص لتوفير الفلوس.

Amazon S3 storage classes ❖

مش كل الملفات عندك بيكون استخدامهما بنفس الشكل.

يعني مثلاً:

- صور الموقع بتفتح كل دقيقة.
- ال Backup بيتفتح مرة كل شهر.
- ملفات قديمة ممكن متفتحش غير بعد سنة.

علشان كده AWS عملت أكثر من Storage Class، وكل واحدة مناسبة لإستخدام مُعين وتكلفتها مُختلفة.

أولاً: Amazon S3 Standard

دي هي ال Default Storage Class.

بتستخدمها للبيانات اللي بتفتح بإستمرار (Frequently Accessed Data).

مُميزاتُها:

- سرعة عالية جداً
- Low Latency
- 11s 9 Durability
- High Throughput
- High Availability

استخداماتها

- مواقع الويب
- تطبيقات الموبايل
- الصور والفيديوهات
- Content Distribution
- Big Data

مثال للتوضيح:

عندك موقع E-commerce.

كل العملاء بيفتحوا صور المنتجات بإستمرار.

يبقى أفضل اختيار هو Amazon S3 Standard.

ثانياً: Amazon S3 Intelligent-Tiering

دي تُعتبر من أذكى الـ Storage Classes.

بدل ما أنت تقرر الملف يتحط فين، AWS هي اللي تقرر.

هي بتراقب استخدام كل Object.

بيشتغل إزاي؟

لو الملف بيتفتح باستمرار هيفضل في Frequent Access Tier.

لو محدش فتح الملف لمدة 30 يوم، ساعتها AWS هتنقله تلقائياً إلى Infrequent Access Tier.

ولو حد فتحه مرة ثانية، AWS ترجعه تلقائياً للـ Frequent Tier.

المميزات:

- بيقلل التكلفة أوتوماتيك.
- مفيش أي تأثير على الأداء.
- مفيش Retrieval Fees.
- مفيش رسوم لنقل الملفات بين الـ Tiers، لكن في Monitoring Fee بسيطة لكل Object.

إمتى أستخدامه؟

لما تكون مش عارف استخدام الملفات هيبقى عامل إزاي.

ثالثاً: Amazon S3 Standard-IA

IA معناها Infrequent Access، يعني الملفات مش بتتفتح كثير، لكن لما تحتاجها لازم تفتح بسرعة.

المميزات:

- نفس سرعة Standard تقريباً.
- أقل في تكلفة التخزين.
- لكن فيه Retrieval Fee لما تجيب الملف.

الاستخدامات:

- Backups
- Disaster Recovery
- ملفات أرشيف بتحتاجها من وقت للثاني

ملحوظة: الـ Standard-IA أرخص في التخزين لكن لما تعمل Download للملف هتدفع Retrieval Fee.

رابعاً: Amazon S3 One Zone-IA

دي شبه ال Standard-IA، لكن الفرق الأساسي إن البيانات بتتخزن في Availability Zone واحدة فقط مش في ثلاثة AZs. علشان كده سعرها أقل.

إمتى أستخدمها؟

لو البيانات مش بتستخدمها كثير، وسهل تعويضها لو حصلت مشكلة.

ملحوظة:

- لو ال Availability مهمة استخدم Standard-IA.
- لو أهم حاجة عندك السعر استخدم One Zone-IA.

خامساً: Amazon S3 Glacier

دي معموله للأرشفة (Archive)، يعني بيانات قديمة جداً ومش هتستخدمها إلا نادراً.

المميزات:

- تكلفة منخفضة جداً.
- مناسب للأرشفة.
- البيانات آمنة جداً.

لكن سرعة الوصول مش فورية، يعني ممكن تستغرق دقائق أو ساعات حسب نوع ال Retrieval.

سادساً: Amazon S3 Glacier Deep Archive

دي أرخص Storage Class في S3 كلها.

مُخصصة للبيانات اللي ممكن محتاجهاش غير مرة أو مرتين في السنة.

استخداماتها:

- أرشفة طويلة المدى.
- Backup.
- Disaster Recovery.
- حفظ البيانات المطلوبة قانونياً.

سرعة الإسترجاع أبطأ من Glacier، استرجاع البيانات ممكن يوصل إلى 12 ساعة.

Amazon S3 bucket URLs (two styles) ❖

بعد ما عرفنا إن Amazon S3 بيخزن الملفات داخل Buckets، تعال نفهم إزاي نوصل لل Bucket أو للملفات اللي جواها عن طريق ال URL.

أولاً: ال Bucket

ال Bucket هو المكان اللي بتحط فيه ملفاتك (Objects).

لكن فيه شرط مهم جداً إن اسم ال Bucket لازم يكون Unique على مستوى AWS كلها.

يعني مينفعش يكون فيه Buckets بنفس الإسم حتى لو في حسابين مختلفين.

ثانياً: ال Region

كل Bucket بتتعمل في Region معينة.

مثلاً:

- us-east-1
- eu-west-1
- ap-northeast-1 (Tokyo)

وده بيحدد مكان تخزين البيانات.

ثالثاً: ال Objects

بعد ما تعمل Bucket، بتبدأ ترفع الملفات، وكل ملف بيتحول إلى Object.

كل ملف من دول اسمه Object.

رابعاً: ال URL

كل Bucket وكل Object ليهم URL تقدر تستخدمه للوصول ليهم (لو عندك صلاحية).

AWS بتدعم طريقتين لكتابة URL.

❖ الطريقة الأولى: Virtual Hosted-Style URL

https:// bucket-name.s3-ap-northeast-1.amazonaws.com

Bucket name Region code

بيكون اسم ال Bucket في أول ال URL.

<https://mybucket.s3.amazonaws.com/logo.png>

هنا:

- mybucket = اسم ال Bucket.
- logo.png = اسم ال Object.

ودي الطريقة الأكثر استخداماً حالياً.

♦ الطريقة الثانية: Path-Style URL

<https://s3.ap-northeast-1.amazonaws.com/bucket-name>

Region code Bucket name

في الطريقة دي اسم ال Bucket بيكون بعد اسم الخدمة.

<https://s3.amazonaws.com/mybucket/logo.png>

برضو:

- mybucket = Bucket
- logo.png = Object

ملحوظة

AWS حالياً بتشجع استخدام Virtual Hosted-Style URL لأنه هو الأسلوب الحديث.

خامساً: Metadata

كل Object مش بيتخزن لوحده معاه كمان معلومات اسمها Metadata.

وادي بتكون بيانات بتوصف الملف، زي:

- اسم الملف.
- الحجم.
- نوع الملف.
- تاريخ الرفع.
- Permissions

يعني ال Object بيتكون من:

- Data (الملف نفسه).
- Metadata (معلومات عن الملف).

التحكم في ال Bucket

كل Bucket ليها صلاحيات مُستقلة.

تقدر تحدد:

- مين يرفع ملفات.
- مين يحمل ملفات.
- مين يحذف ملفات.
- مين يشوف محتويات ال Bucket.

وكمان تقدر تراجع Access Logs علشان تعرف مين دخل على ال Bucket وإمتى.

❖ Data is redundantly stored in the Region

من أهم مميزات Amazon S3 إنه يحافظ على بياناتك حتى لو حصلت أعطال في بعض مراكز البيانات.

أولاً: ال Bucket بتعمل داخل Region

لما تعمل Bucket لازم تختار AWS Region.

مثلاً:

- us-east-1
- eu-west-1
- ap-northeast-1

وده المكان اللي بياناتك هتتخزن فيه.

ثانياً: هل البيانات بتتخزن على سيرفر واحد؟

لأ ودي نقطة مهمة جداً.

AWS مبيتحطش الملف على جهاز واحد أو حتى Data Center واحد، لكن بتعمل Replication تلقائي داخل نفس ال Region.

يعني نفس الملف بيتخزن على أكثر من مكان.

ثالثاً: ليه AWS بتعمل كده؟

علشان لو حصلت مشكلة في:

- .Server
- .Hard Disk
- .Rack
- أو حتى Data Center كامل.

البيانات تفضل موجودة في النسخ الثانية.

وده السبب إن S3 بيوفر Durability عالية (99.99999999%).

رابعاً: لو حصل عطل في Data Center؟

AWS مصممة S3 بحيث يقدر يتحمل فقدان البيانات في مركزين بيانات (Facilities) في نفس الوقت، والبيانات تفضل محفوظة.

يعني حتى لو حصلت مشكلة كبيرة في أكثر من Facility داخل ال Region...

البيانات مش هتضيع.

ملحوظة مهمة

ال Replication اللي AWS بتعمله هنا بيكون داخل نفس ال Region فقط.

لو عايز نسخة من البيانات في Region ثانية (مثلاً من Frankfurt إلى London).

لازم تستخدم S3 Cross-Region Replication (CRR).

❖ Designed for seamless scaling

من أكبر مميزات Amazon S3 إنه معمول علشان يكبر معاك أوتوماتيك.

يعني مهما حجم بياناتك زاد، مش هتحتاج تقف تزود Storage أو تغير إعدادات، لإن AWS هي اللي بتعمل ده كله في الخلفية.

في Amazon EBS كُنا بنحدد حجم ال Volume.

مثلاً:

- 100 GB
- 500 GB
- 1 TB

لكن في Amazon S3 مفيش الكلام ده، أنت تعمل Bucket وارفع ملفاتك، وAWS هتزود المساحة تلقائياً كل ما بياناتك تكبر.

و مش بس التخزين هو اللي بيكبر كمان عدد الأشخاص اللي بيستخدموا ال Bucket.

يعني سواء:

- 10 مُستخدمين.
- 1000 مُستخدم.
- مليون مُستخدم.

كلهم بيطلبوا الملفات في نفس الوقت، ال S3 يقدر يتعامل معاهم بدون ما تحتاج تعمل أي Scaling بنفسك.

بتدفع على اللي بتستخدمه بس ودي من أحسن مميزات S3.

AWS مش هتجزلك مساحة وتحاسبك عليها.

هي بتحاسبك على:

- حجم البيانات اللي خزنتها.
- وعدد ال Requests.
- وأي خدمات إضافية استخدمتها.

يعني لو عندك 20 GB هتدفع على 20 GB فقط، ولما البيانات تزيد، الفاتورة تزيد.

الفرق بين EBS و S3 هنا مُهم جداً.

EBS: لازم تحدد حجم ال Volume قبل ما تستخدمه.

S3: مفيش Provisioning، وال Storage بيكبر تلقائياً.

Access the data anywhere ❖

من أهم مميزات Amazon S3 إنك تقدر توصل لبياناتك من أي مكان وبأكثر من طريقة.
يعني سواء كنت شغال من الـ AWS Console، أو من جهازك، أو حتى من برنامج كتبتة بنفسك، تقدر تتعامل مع الملفات بسهولة.



AWS Management Console



AWS Command Line Interface



SDK

أولاً: الوصول من AWS Management Console

دي أسهل طريقة.

بتدخل على AWS Console، وبعدها تفتح خدمة Amazon S3.

هتلاقي كل الـ Buckets بتاعتك، وتقدر:

- ترفع ملفات
- تحمل ملفات
- تحذف ملفات
- تنشئ Buckets جديدة

ودي الطريقة اللي أغلب الناس بتبدأ بيها.

ثانياً: باستخدام AWS CLI

لو بتحب تشتغل من الـ Terminal أو Command Prompt، تقدر تستخدم AWS CLI.

وده بيسمحلك تنفذ أوامر زي:

- رفع ملف
- تحميل ملف
- عرض الملفات
- حذف الملفات

كل ده من خلال أوامر بدون ما تفتح الـ Console.

ثالثاً: باستخدام AWS SDK

لو بتبني Application تقدر تستخدم AWS SDK.

يعني من داخل الكود بتاعك (C - JavaScript - Java - Python ... إلخ)، تقدر:

- ترفع ملفات
- تحمل ملفات
- تعمل Buckets
- تحذف Objects

وده اللي بيستخدمه معظم المطورين.

رابعاً: الوصول عن طريق URL

كل Object موجود على S3 له URL، لو عندك الصلاحيات المناسبة، تقدر تفتح الملف مباشرةً من خلال ال URL.

خامساً: REST API

Amazon S3 بيوفر REST API.

وده معناه إن أي برنامج يقدر يتعامل مع S3 بإستخدام طلبات:

- GET
- PUT
- POST
- DELETE

عن طريق بروتوكول HTTP أو HTTPS.

وده السبب إن S3 سهل جداً يتكامل مع أي Application.

ليه اسم ال Bucket لازم يكون Unique؟

بما إن ال Bucket ليها URL خاص بيها فلازم يكون اسمها Unique عالمياً.

مثلاً: `https://my-company-bucket.s3.amazonaws.com`

لو فيه شخص تاني عمل Bucket بنفس الاسم هيحصل تعارض، علشان كده AWS بتمنع وجود Buckets بنفس الاسم على مستوى العالم.

يعني إيه DNS-Compliant؟

يعني اسم ال Bucket لازم يكون صالح إنه يبقى جزء من اسم موقع (Domain Name).

مثال صحيح: `my-company-backup`

مثال غير صحيح: `!My Bucket`

لأن فيه مسافات وحروف غير مسموح بيها.

Object Key

ال Object Key هو اسم الملف داخل ال Bucket.

مثلاً: `photos/profile.jpg`

هنا:

- `/photos` يعتبر Folder.
- `profile.jpg` هو اسم ال Object.

ويُفضل يكون اسم الملف بيستخدم حروف آمنة لل URLs علشان ميحصلش مشاكل أثناء الوصول ليه.

❖ Common use cases

بعد ما عرفنا إن Amazon S3 بيوفر مساحة تخزين كبيرة جداً، وسهل الوصول للبيانات من أي مكان، يبقى السؤال هو نستخدمه في إيه؟

ال S3 فيه استخدامات كتير جداً، ودي أشهرها.

أولاً: تخزين بيانات التطبيقات (Application Data)

من أشهر استخدامات S3 إنه يكون مكان مُشترك لتخزين ملفات التطبيق.
بدل ما كل Server يحتفظ بنسخة من الملفات، كل السيرفرات بتستخدم نفس ال Bucket.
وده ببسهل إدارة الملفات.

مثال للتوضيح:

عندك موقع شغال على خمسة EC2 Instances.
لو مُستخدم رفع صورة شخصية، الصورة هتتخفظ في S3، بعد كده أي EC2 Instance تقدر تجيب الصورة من نفس ال Bucket.
يعني كل السيرفرات بتشوف نفس الملفات.
أمثلة للملفات

- صور المُستخدمين.
- فيديوهات.
- ملفات PDF.
- Server Logs

ثانياً: تخزين الملفات في S3 بدلاً من إرسالها من ال EC2

بدل ما ال EC2 هو اللي بيعت الصور أو الفيديوهات للمستخدم، تقدر تخلي المُستخدم يحملها مُباشرةً من S3.
وده بيققل الحمل على السيرفر.

ثالثاً: Static Website Hosting

من أشهر استخدامات S3.
لو عندك موقع Static مفيهوش Backend، تقدر تستضيفه بالكامل على S3.

يعني إيه Static Website؟

هو موقع مُكون من:

- HTML
- CSS
- JavaScript
- Images

ومفيهوش Database أو كود Backend.

ملحوظة

ال S3 يقدر يستضيف Static Websites فقط.
لكن لو عندك (Python - Java - Node.js - PHP) Backend يبقى هتحتاج خدمة ثانية زي EC2 أو Elastic Beanstalk.

رابعاً: Backup

بسبب إن S3 بيوفر Durability عالية، فهو مكان مُمتاز لحفظ النسخ الاحتياطية.

خامساً: Disaster Recovery

لو عايز تحمي بياناتك أكثر، تقدر تستخدم Cross-Region Replication (CRR).
وده بيخلي AWS تنسخ البيانات تلقائياً من Region إلى Region ثانية.

❖ Amazon S3 common scenarios

دي أشهر السيناريوهات اللي بيستخدم فيها Amazon S3 في الشركات والتطبيقات.

أولاً: Backup and Storage

من أشهر استخدامات S3 إنه يكون مكان لحفظ البيانات والنسخ الاحتياطية.
بسبب إن S3 بيوفر Durability عالية جداً، فهو مُناسب جداً لحفظ الملفات المهمة.

ثانياً: Application Hosting

S3 ممكن يُستخدم مع التطبيقات علشان يخزن الملفات اللي التطبيق بيحتاجها.
زي:

- صور المُستخدمين.
- ملفات PDF.
- الفيديوهات.
- ملفات الـ Logs.

كل سيرفرات التطبيق تقدر توصل لنفس الملفات من خلال الـ S3 Bucket.

ثالثاً: Software Delivery

ال S3 بيستخدم كمان لتوزيع البرامج والملفات على المُستخدمين.
يعني ترفع ملفات البرنامج على S3، والمُستخدمين يحملوها مباشرةً.

رابعاً: Media Hosting

لو عندك موقع أو تطبيق يرفع عليه المُستخدمون:

- صور.
- فيديوهات.
- ملفات صوت.

ف S3 يعتبر من أفضل الحلول.

لأنه يقدر يخزن عدد ضخم جداً من الملفات، ويتعامل مع ملايين المُستخدمين في نفس الوقت.

❖ Amazon S3 pricing

من مُميزات Amazon S3 إنك بتدفع على قد استخدامك فقط (Pay As You Go).
يعني مفيش اشتراك ثابت أو مساحة لازم تحجزها مُقدماً، لكن الفاتورة بتتسبب بُناءً على استخدامك الفعلي.

أولاً: بتدفع على مساحة التخزين (Storage)

أول حاجة بتتاسب عليها هي حجم البيانات المُخزنة.
كل ما تخزن بيانات أكثر، التكلفة تزيد.

ثانياً: بتدفع على نقل البيانات خارج الـ Region (Data Transfer OUT)

لو البيانات خرجت من الـ Region إلى مكان ثاني، هيكون فيه تكلفة.

ثالثاً: بتدفع على الـ Requests

كل عملية بتعملها على S3 تعتبر Request، وبعض العمليات بيكون عليها تكلفة.
زي:

- PUT → رفع ملف.
- COPY → نسخ ملف.
- POST → إرسال بيانات.
- LIST → عرض الملفات داخل الـ Bucket.
- GET → تحميل أو قراءة ملف.

رابعاً: إيه الحاجات اللي ببلاش؟

فيه عمليات AWS مش بتحاسبك عليها.

1. نقل البيانات إلى S3 (Transfer IN)

لو رفعت ملفات من جهازك إلى S3، مش هتدفع أي تكلفة على عملية النقل.

2. نقل البيانات من S3 إلى CloudFront في نفس الـ Region

لو بتستخدم Amazon CloudFront مع S3 في نفس الـ Region، AWS مش بتحاسبك على النقل بينهما. وده بيخلي استخدام CloudFront مع S3 أوفر وأسرع.

3. نقل البيانات من S3 إلى EC2 في نفس الـ Region

لو عندك:

- EC2
- S3

والإثنين موجودين في نفس الـ Region، فالـ Data Transfer بينهم بيكون بدون تكلفة.

❖ Amazon S3: Storage pricing

لما تيجي تحسب تكلفة استخدام Amazon S3، متبصش على حجم التخزين بس. في الحقيقة، فيه 4 عوامل رئيسية هي اللي بتحدد الفاتورة.

أولاً: نوع الـ Storage Class

أول حاجة بتأثر على السعر هي الـ Storage Class اللي اخترتها. كل Storage Class ليها سعر مختلف حسب الاستخدام. مثلاً:

- S3 Standard أغلى شوية لأنه مُخصص للبيانات اللي بتستخدم باستمرار.
- S3 Standard-IA أرخص لأنه مُخصص للبيانات اللي نادراً ما يتم الوصول إليها.

ثانياً: حجم البيانات المُخزنة (Amount of Storage)

التكلفة بتزيد كل ما حجم البيانات يزد.

يعني AWS بتحاسبك على إجمالي حجم الملفات الموجودة داخل الـ Buckets.

ثالثاً: عدد ونوع ال Requests

كل عملية بتعملها على S3 تعتبر Request، وبعض العمليات ليها تكلفة.

أشهر ال Requests هي:

- GET → قراءة أو تحميل ملف.
- PUT → رفع ملف جديد.
- COPY → عمل نسخة من ملف داخل S3.

رابعاً: نقل البيانات (Data Transfer)

آخر حاجة بتأثر على السعر هي نقل البيانات.

القاعدة المهمة جداً هي:

- Data Transfer IN → مجاني.
- Data Transfer OUT → عليه تكلفة.

ملحوظة مهمة

في الإمتحان لو شفت Transfer IN اعرف إنها Free.

أما Transfer OUT فهو غالباً Paid (إلا في حالات معينة زي النقل إلى EC2 أو CloudFront في نفس ال Region).

3. Amazon Elastic File System (Amazon EFS)

Storage ❖

Amazon Elastic File
System (Amazon EFS)



بعد ما عرفنا:

- Block Storage :Amazon EBS
- Object Storage :Amazon S3

دلوقتي هنتكلم عن النوع الثالث من التخزين في AWS وهو File Storage، واللي بتقدمه خدمة Amazon EFS. يعني لو EBS شبه الهارد، وS3 شبه Google Drive، ف EFS شبه Shared Folder أو Network Drive تقدر كذا جهاز يستخدمه في نفس الوقت.

إيه هو Amazon EFS؟

ال Amazon Elastic File System (EFS) هو خدمة بتوفر File Storage تقدر تستخدمها مع:

- Amazon EC2
- AWS Services
- وحتى Servers موجودة On-Premises

وبيشتغل بنفس فكرة ال File System العادي اللي أنت متعود عليه (Files و Folders).

ليه نستخدم EFS؟

لإن بعض التطبيقات بتحتاج أكثر من Server يشتغلوا على نفس الملفات، ودي حاجة EBS ميقدرش يعملها بسهولة. مثال للتوضيح:

موقع كبير شغال عليه عشرة EC2 Instances.

كلهم محتاجين يوصلوا للـ:

- صور.
- ملفات.
- تقارير.

بدل ما تنسخ الملفات على كل سيرفر، تحطها في EFS، وكل السيرفرات تستخدم نفس الملفات.

بيعمل Scaling تلقائي

دي من أهم مميزات EFS، إنك مش محتاج تحدد حجم التخزين.

كل ما تضيف ملفات المساحة بتكبر تلقائياً، ولو حذفت ملفات المساحة تقل تلقائياً.

مفيش Downtime

لما حجم الملفات يزداد، مش محتاج توقف التطبيق، ولا تعمل Upgrade، ولا تعمل Migration.

كل ده بيحصل في الخلفية بدون ما المستخدم يحس بأي حاجة.

Amazon EFS features ❖

بعد ما عرفنا إن Amazon EFS هو File Storage تقدر كذا EC2 Instance تستخدمه في نفس الوقت، خرينا نتعرف على أهم مميزاتة.

أولاً: Fully Managed Service

EFS خدمة Fully Managed، يعني AWS هي اللي بتتكفل بإدارة كل حاجة زي:

- إنشاء ال File System
- الصيانة
- التحديثات
- التوافر (Availability)

وأنت كل اللي عليك إنك تستخدمه.

ثانياً: مناسب لتطبيقات كتير

EFS مش معمول لإستخدام واحد، لكنه مناسب لسيناريوهات كتير، منها:

- Big Data & Analytics: تخزين البيانات اللي بيتم تحليلها.
- Media Processing: معالجة الصور والفيديوهات.
- Content Management: أنظمة إدارة المحتوى (CMS).
- Web Serving: تخزين ملفات المواقع.

ثالثاً: بيتعامل مع الملفات بالطريقة العادية

ال EFS بيستخدم File System Interface.

يعني بالنسبة لنظام التشغيل، هتجس إنه هارد عادي.

تقدر تعمل:

- Create File
- Read File
- Write File
- Delete File

بإستخدام نفس أوامر الملفات اللي متعود عليها في Linux أو Windows.

رابعاً: Automatic Scaling

دي من أقوى مميزات EFS.

مش محتاج تحدد حجم التخزين من البداية.

كل ما البيانات تزيد ال EFS يكبر تلقائياً، ولو البيانات قلت المساحة تقل تلقائياً.

خامساً: بیدعم File Locking و Strong Consistency

ودى من المميزات المهمة.

• Strong Consistency

يعني أي تعديل يحصل على ملف، هيطهر فوراً لكل الأجهزة اللي بتستخدم ال File System.

• File Locking

لو برنامج فاتح ملف وبيعدل عليه، ال EFS يقدر يقفل الملف مؤقتاً (Lock) علشان يمنع جهاز تاني يعدل عليه في نفس اللحظة. وده بيمنع تضارب البيانات (Data Corruption).

سادساً: آلاف ال EC2 Instances تقدر تستخدمه في نفس الوقت

على عكس ال EBS اللي بيتوصل بـ EC2 واحدة.

ال EFS ممكن آلاف ال EC2 Instances تستخدم نفس ال File System في نفس الوقت.

وده مناسب جداً للتطبيقات الكبيرة.

سابعاً: أداء ثابت (Consistent Performance)

حتى لو فيه مئات أو آلاف ال EC2 Instances متصلة بنفس ال EFS، AWS بتحافظ على أداء ثابت لكل Instance.

يعني مش معنى إن عدد المستخدمين زاد إن الأداء هيقع.

ثامناً: High Availability و High Durability

EFS مُصمم علشان يكون:

- Highly Available: الخدمة تفضل شغالة حتى لو حصلت مشكلة في جزء من البنية التحتية.
- Highly Durable: يحافظ على البيانات ويقلل احتمالية فقدانها.

تاسعاً: الدفع حسب الاستخدام

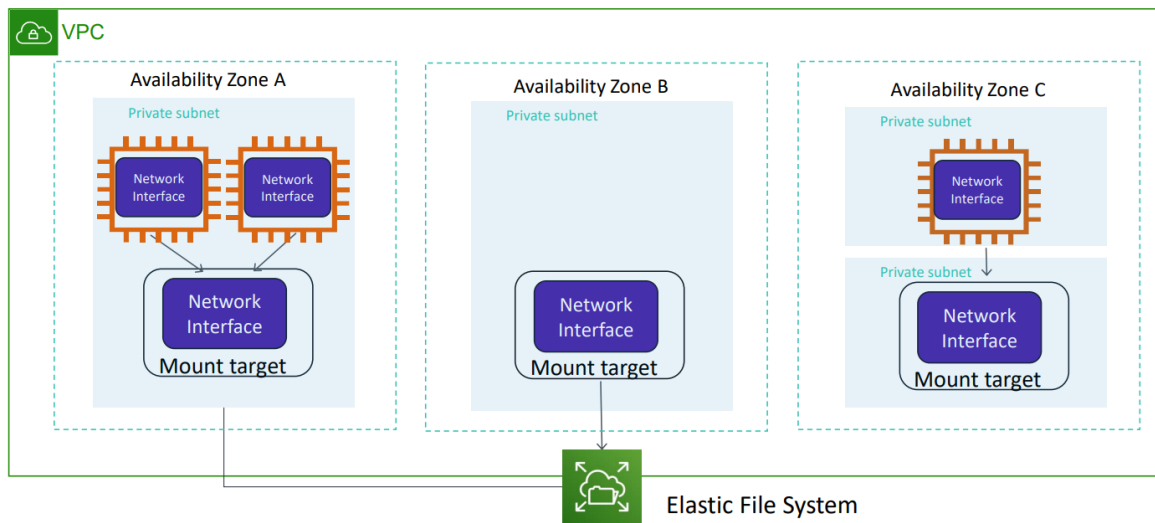
مفيش:

- رسوم إنشاء.
- رسوم إعداد.
- حد أدنى للدفع.

أنت بتدفع فقط مقابل المساحة اللي استخدمتها.

Amazon EFS architecture ❖

بعد ما عرفنا يعني إيه Amazon EFS، خليةنا نفهم هو بيشتغل إزاي في ال Architecture.



الفكرة الأساسية إن EFS بيكون عبارة عن File System مُشترك (Shared File System)، وأي EC2 Instance تقدر توصله وتقرأ أو تكتب فيه، بشرط إنها تكون عندها صلاحية الوصول.

أولاً: بتعمل File System

أول خطوة إنك تنشئ Amazon EFS File System.

وده هيك يكون المكان اللي هتتخزن فيه كل الملفات.

ثانياً: بتعمل Mount لل EFS على EC2

بعد ما تنشئ ال File System بتعمل له Mount على ال EC2.

كلمة Mount معناها تخلي ال File System يظهر داخل نظام التشغيل كإنه هارد عادي.

ثالثاً: القراءة والكتابة

بعد ال Mount ال EC2 تقدر تعمل:

- Read
- Write
- Edit
- Delete

لأي ملف موجود داخل ال EFS، وكنها بتتعامل مع هارد موجود داخل السيرفر.

رابعاً: يستخدم NFS

Amazon EFS يعتمد على بروتوكول اسمه NFS (Network File System)

ويقدم:

- NFSv4.0
- NFSv4.1

وده البروتوكول اللي بيسمح إن أجهزة مُختلفة تشارك نفس ال File System عبر الشبكة.

خامساً: أكثر من EC2 يقدر يستخدم نفس ال EFS

دي أهم ميزة في EFS.

ممکن يكون عندك:

- EC2-1
- EC2-2
- EC2-3

وكلهم متوصلين بنفس ال File System، أي تعديل يحصل من واحد الباقي بيشفه فوراً.

سادساً: بيشتغل عبر أكثر من Availability Zone

ال EFS مش بيكون موجود في AZ واحدة فقط.

هو بيكون مُتاح لكل ال EC2 Instances الموجودة داخل نفس ال Region، حتى لو كانوا في Availability Zones مُختلفة.

سابعاً: Mount Targets

علشان ال EC2 تقدر تتصل بال EFS، AWS بتنشئ حاجة اسمها Mount Target.

وده عبارة عن نقطة إتصال بين ال EFS وال EC2 داخل كل Availability Zone.

علشان:

- يقلل ال Latency.
- يحسن الأداء.
- يخلي الإتصال أسرع.

AWS بتنصح إن كل EC2 تتصل بال Mount Target الموجود في نفس ال Availability Zone.

ثامناً: Mount Target واحد لكل AZ

لو ال AZ فيها أكثر من Subnet، مش لازم تعمل Mount Target في كل Subnet.

بيكون فيه Mount Target واحد فقط لكل Availability Zone، وكل ال Subnets داخل ال AZ تقدر تستخدمه.

❖ Amazon EFS implementation

علشان تستخدم Amazon EFS لأول مرة، AWS بتقسم العملية لـ 5 خطوات بسيطة. الفكرة كلها إنك تنشئ الـ File System، وتوصله بالـ EC2، وبعدها تبدأ تستخدمه.

أولاً: إنشاء EC2 Instance

في البداية لازم يكون عندك EC2 Instance هيسخدم الـ EFS.

وأثناء إنشاء الـ EC2 هتحتاج تعمل:

- اختيار الـ VPC.
- اختيار الـ Subnet.

إنشاء أو استخدام Key Pair علشان تقدر تعمل SSH على السيرفر.

ثانياً: إنشاء Amazon EFS File System

بعد تجهيز الـ EC2، تنشئ Amazon EFS File System.

وده هيكون المكان اللي هتتخزن فيه الملفات المشتركة.

في المرحلة دي AWS بتجهز الـ File System لكن لسه الـ EC2 مش مُتصل بيه.

ثالثاً: إنشاء Mount Targets

بعد إنشاء الـ EFS، لازم تعمل Mount Targets داخل الـ Subnets المناسبة.

الـ Mount Target هو نقطة الإتصال بين الـ EC2 والـ EFS.

الأفضل إن كل الـ EC2 يتصل بالـ Mount Target الموجود في نفس الـ Availability Zone علشان أفضل أداء وأقل Latency.

رابعاً: عمل Mount للـ EFS على الـ EC2

بعد إنشاء الـ Mount Targets، تدخل على الـ EC2 وتعمل Mount للـ EFS.

بعدها هيتظهر كإنه Folder أو Hard Disk عادي داخل نظام التشغيل.

خامساً: تنظيف الموارد (Clean Up)

بعد ما تخلص التجربة أو المشروع، يُفضل تمسح الموارد اللي مش محتاجها علشان متدفعش تكلفة بدون سبب. زي:

- حذف الـ EC2 Instance.
- حذف الـ EFS File System.
- حذف الـ Mount Targets لو مش هتستخدمها.

❖ ملاحظات مهمة جداً

1. إيه هو ال Mount Target؟

ال Mount Target هو عبارة عن Network Endpoint بيخلي ال EC2 يقدر يتصل بال Amazon EFS. يعني ال EC2 مش بيتصل بال EFS مباشرةً، لكنه بيتصل بال Mount Target، وال Mount Target هو اللي بيوصله بال EFS.

2. هل ممكن EC2 يتصل ب Mount Target في Availability Zone ثانية؟

أيوه، ينفع.

يعني لو ال EC2 موجود في AZ-A، وال Mount Target موجود في AZ-B، فال EC2 يقدر يعمل Mount لل EFS عادي. لكن ده مش أفضل اختيار.

لإن الإتصال هيكون بين Availability Zones مختلفة، وده ممكن يسبب:

- Latency أعلى.
- Cross-AZ Traffic.
- وفي بعض السيناريوهات ممكن يكون فيه تكلفة لنقل البيانات بين ال AZs.

3. أفضل ممارسة من AWS هي:

اعمل Mount Target في كل Availability Zone فيها EC2، وخلي كل EC2 يتصل بال Mount Target الموجود في نفس ال AZ. وده بيدي:

- أفضل Performance.
- أقل Latency.
- أفضل Availability.

4. كام Mount Target أقدر أعمل؟

تقدر تعمل Mount Target واحد فقط لكل Availability Zone.

5. لو ال AZ فيها أكثر من Subnet؟

لو عندك Availability Zone فيها أكثر من Subnet، برضه بتعمل Mount Target واحد فقط داخل واحدة من ال Subnets. وباقي ال EC2 الموجودة في نفس ال AZ تقدر تستخدمه.

6. هل Amazon EFS بيشتغل بين Regions؟

لا، ال Amazon EFS خدمة Regional.

يعني ال File System بيتعمل داخل Region واحدة فقط.

❖ Amazon EFS resources

لما تيجي تتعامل مع Amazon EFS، لازم تعرف إن فيه Resource أساسي، وفيه Resources ثانية بتساعده يشتغل.

ال Resource الأساسي هو File System.

أما باقي الحاجات زي Mount Targets و Tags فهي تعتبر موارد فرعية (Subresources).

أولاً: File System (ال Resource الأساسي)

ال File System هو المكان اللي هتتخزن فيه الملفات.

ولما تعمل EFS جديد، AWS بتديله مجموعة من الخصائص (Properties) علشان تقدر تميزه وتديره.

من أهمها:

◆ File System ID

ده رقم أو ID فريد بيميز كل File System، زي كده fs-123456789.

AWS بتستخدمه علشان تعرف أي File System أنت بتتعامل معاه.

◆ Creation Token

ده Token بيتولد وقت إنشاء ال File System.

وظيفته يمنع إنشاء نفس ال File System مرتين بالخطأ لو العملية اتكررت.

◆ Creation Time

هو وقت وتاريخ إنشاء ال File System.

◆ File System Size

بيوضح حجم البيانات الموجودة داخل ال EFS.

الميزة هنا إن الحجم ييزيد ويقل تلقائياً حسب كمية الملفات.

يعني مش محتاج تحدد حجم ثابت زي EBS.

◆ Number of Mount Targets

بيوضح كام Mount Target معمول لل File System.

◆ File System State

بيوضح حالة ال File System.

مثلاً: Deleting - Available – Creating

ثانياً: Mount Targets

ال Mount Target هو الوسيط بين ال EC2 وال EFS، بدوره ال EC2 مش هيعرف يوصل لل File System.

كل Mount Target بيكون ليه بيانات خاصة بيه.

أهم خصائص ال Mount Target:

◆ Mount Target ID

رقم يميز كل Mount Target.

◆ Subnet ID

بيوضح ال Subnet اللي موجود فيها ال Mount Target.

◆ File System ID

بيوضح ال File System اللي ال Mount Target تابع ليه.

◆ IP Address

كل Mount Target بيكون ليه Private IP Address.

وال EC2 بيستخدم ال IP ده علشان يعمل Mount.

◆ DNS Name

بدل ما تستخدم ال IP، AWS بتوفر DNS Name.

وده الأفضل لأنه أسهل ولو ال IP اتغير مش هتحتاج تعدل أي حاجة.

ثالثاً: Tags

ال Tags عبارة عن Metadata.

يعني معلومات إضافية بتضيفها علشان تنظم ال Resources.

كل Tag عبارة عن Key و Value.

ودي بتساعدك جداً لما يبقى عندك عدد كبير من ال Resources.

ملحوظة

ال Tags مش بتأثر على طريقة شغل ال EFS، لكنها بتسهل الإدارة، البحث، وتنظيم الموارد، وكمان ممكن تستخدمها في Cost Allocation لمعرفة تكلفة كل مشروع.

العلاقة بين الـ Resources

افتكر إن الـ File System هو الـ Resource الأساسي.

والـ Mount Target و Tags مينفعش يتعملوا لوحدهم.

لازم يكونوا مرتبطين بـ File System، يعني لو حذف الـ File System، الـ Mount Targets و الـ Tags المرتبطة بيه هتختفي معاه.

4. Amazon S3 Glacier

❖ Storage

الـ Amazon S3 Glacier هو أحد الـ Storage Classes الموجودة في Amazon S3، لكنه معمول مخصوص لتخزين البيانات اللي مش بنستخدمها بشكل متكرر (Cold Data).



يعني بدل ما تخزن البيانات على الـ S3 Standard وتدفع تكلفة أعلى، تقدر تنقلها إلى Glacier وتوفر في التكلفة.

Amazon S3 Glacier

أولاً: يعني إيه الـ Cold Storage؟

الـ Cold Storage هو تخزين للبيانات اللي نادر جداً إنك تفتحها، لكن لازم تحتفظ بيها.

يعني البيانات موجودة وآمنة، لكن مش بتستخدمها كل يوم.

أمثلة:

- نسخ احتياطية (Backups) قديمة.
- ملفات مشاريع انتهت.
- سجلات (Logs) قديمة.
- مستندات قانونية.
- بيانات لازم تحتفظ بيها لسنين بسبب قوانين أو لوائح.

ثانياً: إمتى أستخدم Amazon S3 Glacier؟

بتستخدمه لما يكون عندك بيانات:

- مش بتفتح إلا نادراً.
- محتاج تحتفظ بيها لفترة طويلة.
- عايز تقلل تكلفة التخزين.

ثالثاً: مميزات Amazon S3 Glacier

تكلفة التخزين مُنخفضة جداً مقارنة بـ S3 Standard.

مُناسب للأرشفة (Archiving).

مُناسب للـ Compliance والإحتفاظ بالبيانات لفترات طويلة.

نفس مستوى التحمل (Durability) العالي الخاص بـ Amazon S3 (11 9s).

ملاحظة

لأن البيانات متخزنة للأرشفة، فالوصول إليها مش ببيكون فوري زي S3 Standard، وممكن تحتاج تستنى شوية لحد ما يتم استرجاعها حسب نوع ال Retrieval اللي هتختاره.

Amazon S3 Glacier review ❖

عرفنا قبل كده إن Amazon S3 Glacier مُخصص لتخزين البيانات اللي بنرجعلها نادراً جداً (Cold Data)، وببتميز إنه أرخص من S3 Standard، لكن في المُقابل مش هتقدر توصل للبيانات فوراً.

ليه Amazon S3 Glacier رخيص؟

لأن AWS بتفترض إنك مش هتحتاج البيانات دي إلا كل فترة كبيرة. فبدل ما تكون جاهزة للوصول في أي لحظة زي S3 Standard، بتكون متخزنة بطريقة مُخصصة للأرشفة، وبالتالي التكلفة بتكون أقل.

استرجاع البيانات (Retrieval)

أهم حاجة لازم تعرفها إن البيانات مش بتفتح فوراً. لازم تعمل Restore الأول، وبعدها AWS تجهز البيانات علشان تقدر تحملها. وده ممكن ياخد دقائق أو ساعات، حسب نوع ال Retrieval اللي اخترته.

أهم المصطلحات في Glacier**1. Archive**

ال Archive هو أي ملف بتخزنه داخل Glacier.

يعني ممكن يكون:

- صورة
- فيديو
- PDF
- Backup
- أي ملف

كل Archive ببيكون ليه:

- Archive ID
- Description (اختياري)

.2 Vault

ال Vault هو المكان الذي يتم تخزين فيه ال Archives.

وده يشبه جداً فكرة ال Bucket في Amazon S3.

كل Vault يتم تحدد له:

- اسم
- Region

.3 Vault Access Policy

هي Policy بتحدد:

- مين يقدر يدخل ال Vault؟
- ومين مينفعش؟
- وإيه العمليات التي يقدر يعملها؟

زي:

- Read
- Upload
- Delete

.4 Vault Lock

دي ميزة إضافية.

بتخليك تمنع أي حد من تعديل أو حذف البيانات الموجودة داخل ال Vault.

ودي مهمة جداً في:

- Compliance
- الجهات الحكومية
- البنوك
- المستشفيات

لأن القانون أحياناً بيمنع حذف البيانات قبل عدد معين من السنين.

أنواع استرجاع البيانات (Retrieval Options)

AWS بتوفر 3 طرق لإسترجاع البيانات.

كل واحدة لديها سرعة وسعر مُختلف.

1. Expedited Retrieval

الأسرع، البيانات ستكون جاهزة من 1 إلى 5 دقائق تقريباً.

لكن دي أغلى طريقة.

تستخدمها لو حصل Emergency ومحتاج الملف فوراً.

2. Standard Retrieval

الإختيار المتوسط، البيانات ستكون جاهزة في خلال من 3 إلى 5 ساعات.

وده الأكثر استخداماً.

تستخدمها لما تكون مستعجل شوية، لكن مش محتاج البيانات خلال دقائق.

3. Bulk Retrieval

أرخص اختيار، لكن أبطأ واحد.

الإسترجاع بياخد من 5 إلى 12 ساعة.

تستخدمه لو عندك كمية بيانات ضخمة، ومش مُستعجل عليها.

مُميزات Amazon S3 Glacier

- تكلفة تخزين مُنخفضة جداً.
- مُناسب للأرشفة طويلة المدى.
- 11 9s Durability بيوفر
- يدعم تشفير البيانات أثناء النقل (SSL/TLS) وأثناء التخزين.
- بيدعم Vault Lock لتطبيق سياسات ال Compliance.

Amazon S3 Glacier use cases ❖

بما إن Amazon S3 Glacier مُخصص للأرشفة (Archiving)، فهو يُستخدم في أي سيناريو عندك فيه بيانات ضخمة لازم تحتفظ بيها لفترة طويلة، لكن مش بتحتاجها بشكل مُستمر.

أولاً: أرشفة ملفات الميديا (Media Asset Archiving)

شركات الإعلام والإنتاج تحتفظ بكميات ضخمة من:

- الفيديوهات
- الأفلام
- نشرات الأخبار
- المقاطع القديمة

الملفات دي ممكن توصل ل Petabytes، لكن نادراً ما بيتم استخدامها.

بدل ما تفضل على S3 Standard بتكلفة عالية، بتتخزن على Amazon S3 Glacier.

ولو احتاجوا أي فيديو بعد فترة، بيعملوا Restore ويرجعوه ل Amazon S3 علشان يقدرُوا يستخدموه أو يوزعوه.

ثانياً: أرشفة البيانات الطبية (Healthcare Information Archiving)

المُستشفيات مُطالبة قانونياً إنها تحتفظ ببيانات المرضى لسنين طويلة.

زي:

- ملفات المرضى
- الأشعة الطبية (PACS)
- السجلات الطبية الإلكترونية (EHR)
- بيانات التأمين الصحي

بما إن البيانات دي كبيرة جداً ونادراً ما بيتم الرجوع ليها، ف Amazon S3 Glacier يعتبر حل مُمتاز لإنه:

- آمن
- مُنخفض التكلفة
- مُناسب للتخزين طويل المدى

ثالثاً: أرشفة بيانات ال Compliance والقوانين (Regulatory & Compliance Archiving)

بعض الشركات، خصوصاً:

- البنوك
- شركات التأمين
- المؤسسات الصحية

لازم تحتفظ ببيانات معينة لسنوات علشان تلتزم بالقوانين، هنا بيستخدموا Amazon S3 Glacier مع Vault Lock.

وده بيمنع أي شخص من تعديل أو حذف البيانات قبل انتهاء المدة القانونية.

رابعاً: أرشفة البيانات العلمية (Scientific Data Archiving)

مراكز الأبحاث والجامعات تنتج كميات ضخمة جداً من البيانات.

زي:

- نتائج التجارب
- بيانات الأقمار الصناعية
- أبحاث الذكاء الاصطناعي
- بيانات المعامل

بعد انتهاء الأبحاث، البيانات دي لازم تتحفظ، لكن مش بيتم استخدامها يومياً.

علشان كده بيستخدموا Amazon S3 Glacier لأنه بيوفر:

- تكلفة قليلة
- تخزين طويل المدى
- سعة كبيرة جداً

خامساً: الحفظ الرقمي (Digital Preservation)

المكتبات والهيئات الحكومية بيكون عندها وثائق رقمية مهمة جداً.

زي:

- الكتب القديمة
- الوثائق التاريخية
- المخطوطات
- السجلات الحكومية

لازم البيانات دي تفضل سليمة لسنين طويلة.

ميزة Amazon S3 Glacier إنه بيعمل فحص دوري لسلامة البيانات (Data Integrity Checks)، ولو اكتشف أي مشكلة بيصلحها تلقائياً (Self-Healing).

وده بيساعد في الحفاظ على البيانات بدون تدخل يدوي.

ملحوظة

كل السيناريوهات دي فيها حاجة مُشتركة:

- البيانات كبيرة جداً
- نادراً ما بيتم الوصول ليها
- لازم تتخزن لفترة طويلة
- التكلفة لازم تكون قليلة

وده بالظبط اللي معمول عشانه Amazon S3 Glacier.

❖ Using Amazon S3 Glacier

بعد ما عرفنا إيه هو Amazon S3 Glacier واستخدماته، يبقى السؤال إزاي أستخدمه؟ فيه أكثر من طريقة للتعامل مع Glacier، لكن مش كل العمليات ينفع تتعمل من ال Console.

أولاً: استخدام AWS Management Console

تقدر تستخدم AWS Management Console، لكن بإمكانات محدودة. من خلال ال Console تقدر:

- إنشاء Vault جديد.
- حذف Vault.
- إنشاء وإدارة Vault Policies.

لكن أغلب العمليات التانية مش مُتاحة من ال Console.

ثانياً: استخدام AWS CLI أو SDK أو REST API

لو عايز تعمل كل العمليات الخاصة بـ Glacier، هتستخدم:

- AWS CLI
- AWS SDK (زي Java أو .NET أو Python)
- Amazon S3 Glacier REST API

ودي بتديك تحكم كامل في الخدمة.

تقدر من خلالها:

- رفع Archives
- استرجاع البيانات (Restore)
- حذف Archives
- إدارة ال Vaults
- تنفيذ أي عملية مُقدمة

ثالثاً: استخدام Lifecycle Policies (الأهم)

وده يعتبر أفضل وأشهر استخدام لـ Amazon S3 Glacier.

بدل ما تنقل الملفات بنفسك، بتخلي Amazon S3 يعملها أوتوماتيك.

❖ Lifecycle policies

ال Lifecycle Policy هي ميزة في Amazon S3 بتخليك تدير دورة حياة الملفات بشكل أوتوماتيك.

يعني بدل ما كل شوية تدخل تنقل الملفات من Storage Class للتانية أو تمسحها بنفسك، AWS بتعمل ده تلقائياً حسب القواعد (Rules) اللي أنت بتحددها.

ليه نستخدم Lifecycle Policies؟

مع مرور الوقت، أهمية البيانات بتقل.

مثلاً:

- ملف اترفع النهارده، بيتفتح كثير.
- بعد شهر، نادراً حد يفتحه.
- بعد سنة، مُمكن محدش يحتاجه خالص.

يبقى ليه أفضل أدفع تكلفة S3 Standard طول السنة؟

الأفضل إن الملف ينتقل تدريجياً ل Storage Classes أرخص، وده بالظبط اللي بتعمله Lifecycle Policies.

تقدر تطبقها على إيه؟

ال Lifecycle Policy ممكن تتطبق على:

- Object مُعين.
- مجموعة Objects (عن طريق Prefix أو Tag).
- Bucket بالكامل.

يعني تقدر تحدد إن كل الملفات الموجودة داخل Bucket مُعين تمشي عليها نفس القواعد.

مثال عملي

نفترض إن عندك تطبيق لرفع الفيديوهات، والمستخدم بيرفع فيديو، والتطبيق بيعمل Thumbnail (صورة مصغرة) للفيديو.

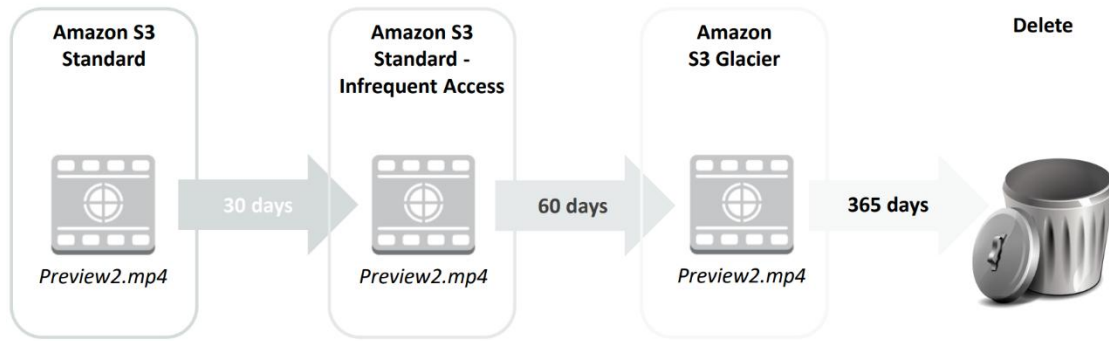
في أول فترة، المستخدم غالباً هيشوف الصورة دي كثير، علشان كده هتتخزن في Amazon S3 Standard.

بعد 30 يوم الإحصائيات بتقول إن أغلب المستخدمين مبغوش بيفتحوا الصور دي.

فال Lifecycle Policy هتنقلها تلقائياً إلى Amazon S3 Standard-IA، علشان تقلل التكلفة.

وبعد 30 يوم كمان بقت نادر جداً إنها تتفتح، فال Lifecycle Policy هتنقلها إلى Amazon S3 Glacier، لإنها بقت مُجرد أرشيف.

بعد سنة... الصورة مبقاش ليها أي قيمة فال Lifecycle Policy هتحدفها تلقائياً.



كل ده بيحصل أوتوماتيك بدون أي تدخل منك.

فوائد Lifecycle Policies

- تقلل تكلفة التخزين.
- تنقل البيانات تلقائياً بين Storage Classes.
- تحذف البيانات القديمة تلقائياً.
- مش محتاج تتابع الملفات بنفسك.
- مناسبة جداً للـ Backups والأرشفة.

ملحوظة مهمة

الـ Lifecycle Policies بتشتغل بناءً على عمر الملف (Age)، يعني أنت بتحدد بعد كام يوم يحصل النقل أو الحذف، وـ AWS هيطبق القاعدة تلقائياً.

❖ Storage comparison

الـ Amazon S3 و الـ Amazon S3 Glacier الإثنين يعتبروا Object Storage، والإثنين تقدر تخزن فيهم كمية ضخمة جداً من البيانات، لكن كل واحد معمول لغرض مختلف.

المقارنة	Amazon S3	Amazon S3 Glacier
Data Volume	غير محدود (Unlimited)	غير محدود (Unlimited)
Average Latency	Milliseconds (في لحظات تقريباً)	Minutes أو Hours (من دقائق إلى ساعات)
Maximum Item Size	5 TB	40 TB
Storage Cost (لكل GB شهرياً)	أعلى	أقل
العمليات اللي بتحاسب عليها	PUT, COPY, POST, LIST, GET	Retrieval و Upload
تكلفة استرجاع البيانات	رسوم لكل Request فقط	رسوم لكل Request + لكل GB يتم استرجاعه

أولاً: الفرق الأساسي

- Amazon S3

معمول للبيانات التي تستخدمها باستمرار.

يعني أول ما تطلب الملف، ب يفتح معاك فوراً (Low Latency).

- Amazon S3 Glacier

معمول للأرشفة (Archiving).

الملفات موجودة، لكن لو احتجتها لازم تعمل Restore الأول، ويمكن تستنى من دقائق لساعات.

ثانياً: أقصى حجم للملف

- Amazon S3

أقصى حجم لل Object الواحد هو 5 TB.

- Amazon S3 Glacier

يقدر يخزن Archive حجمه لحد 50 TB.

وده مناسب جداً لل Backups الضخمة أو البيانات العلمية.

ثالثاً: تكلفة التخزين

- Amazon S3

بما إنه بيوفر وصول سريع جداً للبيانات، ف تكلفة التخزين أعلى.

- Amazon S3 Glacier

تكلفته أقل بكثير، لكن السرعة أقل.

يعني أنت بتبدل سرعة أقل = تكلفة أقل.

رابعاً: تكلفة الـ Requests

• Amazon S3

بتدفع على عمليات زي:

- PUT
- GET
- COPY
- POST
- LIST

وادي العمليات التي بتستخدمها بشكل يومي.

• Amazon S3 Glacier

الرسوم الأساسية بتكون على:

- Upload
- Retrieval (استرجاع البيانات)

ولإن استرجاع البيانات من Glacier مش بيحصل كثير، فتكلفته أعلى من GET في S3.

خامساً: تكلفة استرجاع البيانات

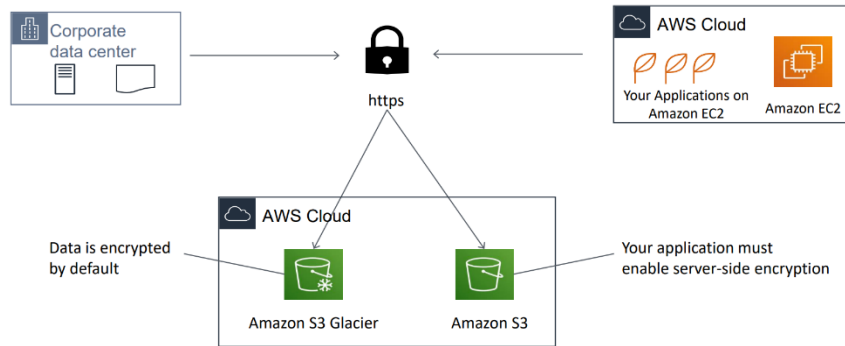
دي نقطة مهمة جداً.

في Amazon S3 لما تعمل GET علشان تحمل ملف، بتدفع رسوم على الـ Request فقط.

أما في Amazon S3 Glacier، ف Retrieval أعلى لأنك بتدفع:

- رسوم على Request.
 - وكمان على حجم البيانات (GB) اللي رجعتها.
- علشان كده Glacier مناسب للأرشفة، مش للإستخدام اليومي.

Server-side encryption ❖



لحد دلوقتي كنا بنتكلم عن تخزين البيانات، لكن فيه سؤال مهم: إزاي AWS بتحمي البيانات وهي متخزنة؟
الإجابة هي **Server-side Encryption (SSE)**.

يعني إيه Server-side Encryption؟

هي عملية تشفير البيانات بعد ما توصل لـ AWS وقبل ما تتخزن على الـ Disk.
ولما تيجي تطلب البيانات مرة ثانية، AWS بتفك التشفير تلقائياً وترجعلك الملف.
يعني أنت مش محتاج تعمل أي حاجة أثناء القراءة.

الـ Server-side Encryption بتحمي إيه؟

بتحمي البيانات وهي At Rest (وهي متخزنة على الـ Storage).

أما أثناء نقل البيانات (In Transit)، فبيتم استخدام:

- HTTPS
- TLS/SSL

يعني عندنا نوعين من الحماية:

- HTTPS / TLS : **In Transit**
- Server-side Encryption : **At Rest**

الفرق بين Amazon S3 و Amazon S3 Glacier

- Amazon S3 Glacier

أي بيانات بتتخزن في Glacier بتكون مُشفرة تلقائياً (Encrypted by Default).
يعني أنت مش محتاج تعمل أي إعدادات إضافية.

- Amazon S3

في S3 لازم أنت اللي تفعل Server-side Encryption (أو تضبط الـ Bucket بحيث يطبقها تلقائياً).
وبعدها AWS هتقوم بتشفير البيانات أثناء التخزين.

أنواع Amazon S3 في Server-side Encryption

AWS بتوفر 3 طرق مُختلفة.

1. SSE-S3 (Amazon S3 Managed Keys)

وده أبسط نوع، AWS هي اللي:

- بتعمل مفاتيح التشفير.
- تخزينها.
- تديرها.
- تغيرها (Rotate) بشكل دوري.

كل اللي عليك إنك تفعل SSE-S3.

AWS بتستخدم AES-256، وده من أقوى وأشهر خوارزميات التشفير.

تستخدمه لو مش محتاج تتحكم في مفاتيح التشفير بنفسك، وده يعتبر الاختيار الافتراضي لمُعظم الإستخدامات.

2. SSE-C (Customer-Provided Keys)

في النوع ده أنت اللي بتجيب مُفتاح التشفير، ولما ترفع الملف بتبعث المُفتاح مع ال Request.

AWS بتستخدم المُفتاح لتشفير الملف، وبعد كده محتفظش بالمُفتاح.

ولما تيجي تحمل الملف لازم تبعث نفس المُفتاح.

تستخدمه لو شركتك عندها نظام إدارة مفاتيح خاص بيها، ومش عايزة AWS تحتفظ بالمفاتيح.

3. SSE-KMS (AWS Key Management Service)

وده أكثر الأنواع استخداماً في الشركات.

بدل ما AWS تستخدم مفاتيح عادية بتستخدم خدمة اسمها AWS KMS.

اللي هي خدمة مُتخصصة لإدارة مفاتيح التشفير.

مُميزات SSE-KMS

تقدر:

- تنشئ مفاتيح بنفسك.
- تتحكم مين يستخدم المُفتاح.
- تعمل صلاحيات للمفاتيح.
- تراجع Logs لمعرفة مين استخدم المُفتاح.
- تعمل Audit بسهولة.

داخل KMS بيكون عندك (CMK) Customer Master Key، وده المفتاح الرئيسي اللي بيستخدم لتشفير بيانات S3.

تستخدمه لو عندك متطلبات أمنية عالية أو Compliance، وده أشهر اختيار في الشركات الكبيرة.

Security with Amazon S3 Glacier ❖

بما إن Amazon S3 Glacier يستخدم لتخزين بيانات مهمة وحساسة لفترات طويلة، فـ AWS موفرة أكثر من وسيلة لحماية البيانات.

أولاً: التحكم في الوصول بإستخدام IAM

بشكل افتراضي (By Default) أنت فقط اللي تقدر توصل للبيانات الموجودة في Amazon S3 Glacier. يعني مفيش أي حد يقدر يشوف أو يحمل البيانات إلا لو أنت سمحتله.

إزاي تسمح لحد يوصل للبيانات؟

عن طريق (IAM (Identity and Access Management.

بتعمل IAM Policy وتحدد فيها:

- مين يقدر يدخل
- مين ميقدرش
- وياه العمليات المسموح بيها

زي:

- Upload Archive
- Retrieve Archive
- Delete Archive
- List Vaults

يعني أنت المُتحكم بالكامل في صلاحيات المُستخدمين.

ثانياً: تشفير البيانات (Encryption)

الـ Amazon S3 Glacier بيشفّر كل البيانات تلقائياً.

وبيستخدم خوارزمية AES-256، ودي من أقوى خوارزميات التشفير المُستخدمة حالياً. وده معناه إن البيانات وهي متخزنة (At Rest) بتكون محمية حتى لو حد قدر يوصل للأقراص نفسها.

ثالثاً: إدارة مفاتيح التشفير

في Amazon S3 Glacier، AWS هي اللي بتدير مفاتيح التشفير نيابة عنك.

يعني:

- إنشاء المفاتيح.
- تخزينها.
- إدارتها.
- استخدامها.

كل ده بيتتم تلقائياً بدون أي تدخل منك.

وده بيخلي استخدام Glacier أسهل، لإنك مش محتاج تقلق بخصوص إدارة مفاتيح التشفير.

♦ **Module 8: Databases**

1. Amazon Relational Database Service (Amazon RDS)

Amazon Relational Database Service ❖

في الشاتر اللي فات كُنا بنتكلم عن Storage Services.

يعني خدمات لتخزين:

- ملفات
- صور
- فيديوهات
- Backups

زي:

- Amazon S3
- Amazon EBS
- Amazon EFS
- Amazon Glacier

لكن دلوقتي هنبدا نتكلم عن ال Databases.

يعني بدل ما أأخزن ملفات، هأخزن بيانات مُنظمة يقدر التطبيق يتعامل معاها بسهولة.

يعني إيه Database؟

ال Database عبارة عن مكان لتخزين البيانات بشكل مُنظم.

مثال:

عندك موقع Ecommerce، والموقع فيه:

- Users
- Products
- Orders
- Payments

كل البيانات دي لازم تتخزن في Database.

مثلاً:

ID	Name	Email
1	Mohamed	mo@gmail.com
2	Ali	ali@gmail.com

طيب يعني إيه Relational Database؟

ال Relational Database بتخزن البيانات في شكل Tables.

وال Tables دي بيكون بينها علاقات (Relations).

مثال:

عندك جدول Users، وعندك جدول Orders.

كل Order لازم يكون تابع لمستخدم مُعين.



Amazon Relational Database Service
(Amazon RDS)

Amazon RDS

Amazon RDS اختصار لـ **Amazon Relational Database Service**.

ودي خدمة من AWS بتوفرلك قاعدة بيانات جاهزة للإستخدام.

يعني بدل ما تقعد تثبت MySQL أو PostgreSQL بنفسك، AWS تعمل كل ده، وأنت تستخدم قاعدة البيانات مباشرة.

طب RDS بيحل مشاكل إيه؟

زمان لو كنت هتشغل Database بنفسك...

كنت هتحتاج:

- Server
- Install Database
- Configure Database
- تعمل Backup
- تعمل Updates
- تراقب الأداء
- تصلح الأعطال

كل ده شغل عليك لكن مع Amazon RDS، AWS هي اللي بتعمل كل ده، وأنت بتركز على شغلك بس.

❖ Unmanaged VS Managed Services

قبل ما نفهم Amazon RDS، لازم نفهم الفرق بين نوعين من الخدمات في AWS:

- Unmanaged Services
- Managed Services

وده من أهم المفاهيم في AWS كلها.

أولاً: Unmanaged Service

في ال Unmanaged Service، AWS بتوفرك البنية الأساسية (Infrastructure) فقط.

أما إدارة الخدمة نفسها، فهي مسؤوليتك بالكامل.

يعني AWS بتقولك "اتفضل السيرفر... والباقي عليك."

مثال للتوضيح:

لو شغلت EC2 Instance وحطيت عليه Web Server.

يبقى AWS وفرتلك ال EC2 فقط.

لكن مين المسؤول عن:

- تثبيت ال Web Server؟
- تحديثه؟
- مراقبة الأداء؟
- إصلاح الأعطال؟
- زيادة عدد السيرفرات لو الضغط زاد؟

الإجابة: أنت.

ولو الضغط زاد فجأة؟

نفترض إن موقعك كان عليه 100 مُستخدم وفجأة بقوا 100,000 مُستخدم.

هل EC2 هيزود سيرفرات لوحده؟

لا، لازم أنت تعمل:

- Auto Scaling
- Load Balancer
- Health Checks

ولو معملتش كده، الموقع ممكن يقع.

مميزات ال Unmanaged

الميزة الوحيدة تقريباً هي:

- إن عندك تحكم كامل.
- يعني أنت اللي بتحدد كل حاجة بالتفصيل.
- وده مناسب لو محتاج إعدادات خاصة جداً.

ثانياً: Managed Service

ال Managed Service مختلفة تماماً.

AWS بتدير الخدمة بدل منك، يعني أنت بتعمل الإعدادات الأساسية فقط، وبعدين AWS تتولى الباقي.

مثال للتوضيح:

لو استخدمت Amazon S3.

كل اللي بتعمله تنشئ Bucket، وتحدد Permissions، وبس.

بعدها AWS هي اللي تعمل:

- Scaling
- High Availability
- Fault Tolerance
- Storage Management

كل ده تلقائي.

♦ أشهر الأمثلة

Unmanaged

- Amazon EC2
- EC2 على Database
- EC2 على Web Server

Managed

- Amazon S3
- Amazon RDS
- Amazon DynamoDB
- Amazon EFS

❖ Challenges of relational databases

قبل ما AWS تعمل خدمة زي Amazon RDS، الشركات كانت بتشغل قواعد البيانات بنفسها على Servers. وده كان بيخليها تواجه مشاكل كتير، لإن إدارة ال Database مش مجرد إنك تثبت MySQL وخلاص. تعالى نشوف أهم المشاكل دي.

أولاً: Server Maintenance & Energy Footprint

لما يكون عندك Server في الشركة (On-Premises)، أنت المسؤول عن:

- صيانة السيرفر.
- تغيير الهارد لو باظ.
- تغيير أي قطعة Hardware.
- الكهرباء والتبريد.
- تشغيل الداتا سنتر.

كل ده بيكلف فلوس ووقت.

أما مع AWS، كل ده بيبقى مسؤولية AWS.

ثانياً: Software Installation & Patches

بعد ما تحيب السيرفر، لازم تثبت قاعدة البيانات. زي:

- MySQL
- PostgreSQL
- Oracle
- SQL Server

وكل شوية ينزل تحديث (Patch).

وأنت لازم تثبته ولو منزلتوش، ممكن يبقى فيه ثغرات أمنية أو مشاكل في الأداء.

ثالثاً: Database Backups & High Availability

ال Database دي فيها بيانات مهمة جداً، لازم تعملها Backup باستمرار.

ولو السيرفر وقع، لازم يكون عندك نسخة تانية شغالة علشان الخدمة متقفش، وده اسمه High Availability.

لو معملتش الحاجات دي، ممكن تخسر بيانات العملاء.

رابعاً: Limits on Scalability

تخيل إن قاعدة البيانات كانت بتخدم 100 مُستخدم، وفجأة بقوا 100 ألف مُستخدم. هل قاعدة البيانات هتكبر لوحدها؟ لا.

لازم أنت:

- تزود CPU
 - تزود RAM
 - تكبر ال Storage
 - أو تنقل لسيرفر أقوى
- وده محتاج تخطيط ووقت.

خامساً: Data Security

البيانات لازم تكون مُؤمنة.

يعني لازم تهتم بـ:

- تشفير البيانات (Encryption)
- صلاحيات المُستخدمين
- التحكم في الوصول
- حماية قاعدة البيانات من الهجمات

كل ده مسؤوليتك لو بتدير ال Database بنفسك.

سادساً: Operating System Installation & Patches

مش قاعدة البيانات بس اللي محتاجة تحديثات، حتى نظام التشغيل نفسه (Linux أو Windows Server) لازم:

- تثبته.
- تحدّثه.
- تعمل Security Patches.
- تصلح أي مشاكل فيه.

وده عبء إضافي عليك.

ليه كل ده مُشكلة؟

لإن بدل ما فريق التطوير يركز على تطوير التطبيق...

هيقضي وقت كبير في:

- صيانة السيرفرات
- تحديث الأنظمة
- مراقبة قواعد البيانات
- عمل Backups
- حل الأعطال

وده بيقلل الإنتاجية.

إيه الحل؟

علشان كده AWS عملت Amazon RDS.

بدل ما تعمل كل الحاجات دي بنفسك، AWS هي اللي تتولى:

- Server Maintenance
- Software Installation
- Patching
- Automatic Backups
- High Availability
- Monitoring
- Scalability

وأنت تركز على استخدام قاعدة البيانات فقط.

ملحوظة مهمة

دي تعتبر من أشهر مميزات Amazon RDS في الامتحان.

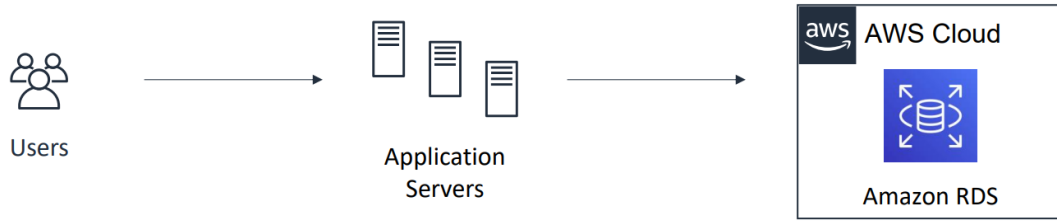
لو السؤال قال: Which service reduces the operational overhead of managing relational databases?

الإجابة هتكون: **Amazon RDS**.

لأنه بيثيل عنك كل المهام الإدارية دي.

Amazon RDS ❖

بعد ما عرفنا المشاكل اللي بتواجه أي حد بيشغل Relational Database بنفسه، AWS قدمت الحل، وهو Amazon RDS.



يعني إيه Amazon RDS؟

Amazon RDS (Relational Database Service) هي خدمة Managed Database Service من AWS.

يعني AWS هي اللي:

- تنشئ قاعدة البيانات
- تشغيلها
- تديرها
- تعملها Scaling
- وتتابع صيانتها

وأنت تركز على استخدام قاعدة البيانات فقط.

إيه اللي AWS بتعمله في Amazon RDS؟

AWS بتتولى المهام اللي كانت بتضيع وقت كبير، زي:

- إنشاء قاعدة البيانات
- تشغيلها
- تحديثها (Patching)
- عمل Automatic Backups
- مراقبة الأداء
- التوسع (Scaling)
- الصيانة
- توفير High Availability (لو فعلتها)

يعني بدل ما تقضي وقتك في إدارة ال Database، هتركز على تطوير التطبيق.

Cost-efficient and Resizable Capacity**أولاً: Cost-efficient**

يعني AWS بتخليك تدفع على حسب الموارد اللي محتاجها.
مش لازم تشتري Server كبير من البداية.

ثانياً: Resizable Capacity

لو قاعدة البيانات كبرت، تقدر بسهولة تزود:

- CPU
- RAM
- Storage

من غير ما تبدأ من الصفر.

Automating Administrative Tasks

دي من أهم مميزات RDS.

AWS بتعمل تلقائياً (Automatically):

- Backups
- Software Updates
- Maintenance
- Monitoring
- Failure Detection

يعني بتوفر عليك شغل إداري كتير.

أنت بتركز على إيه؟

بدل ما تنشغل بإدارة السيرفر، أنت بتركز على:

- تصميم ال Database
- كتابة ال Queries
- تحسين أداء التطبيق
- تطوير ال Application

وده الهدف الأساسي من Amazon RDS.

إليه اللي التطبيق بيستفيد؟

Amazon RDS بيوفر للتطبيق:

أداء عالي (Performance): قاعدة البيانات تشتغل بكفاءة وسرعة

High Availability: لو حصل عطل، AWS تقدر توفر استمرارية للخدمة (حسب الإعدادات اللي فعلتها).

Security: AWS بتوفر أدوات لحماية قاعدة البيانات: زي:

- Encryption
- IAM Integration
- Network Security

Compatibility: RDS بيدعم أشهر أنواع قواعد البيانات، وبالتالي تقدر تستخدم نفس الأدوات وال Applications اللي متعود عليها.

❖ From on-premises databases to Amazon RDS

الجزء ده مهم لأنه بيوضح إيه المسؤوليات اللي بتقل كل ما تنتقل من On-Premises لـ EC2 ثم لـ RDS.

يعني هنقارن بين 3 حالات:

- On-Premises Database
- Database على EC2
- Database على Amazon RDS / Aurora

أولاً: On-Premises Database

دي الحالة التقليدية، يعني الشركة عندها Data Center خاص بيها، كل حاجة مسؤوليتها هي.

أنت المسؤول عن:

1. Power (الكهرباء)
2. HVAC (التبريد)، Heating, Ventilation, and Air Conditioning (يعني أنظمة التبريد والتهوية الخاصة بالداتا سنتر)
3. شراء السيرفرات وتركيبها

وده المقصود بـ Rack and Stack Servers

يعني:

- تشتري السيرفر
- تركيبه في الـ Rack
- توصله بالكهرباء
- توصله بالشبكة
- 4. Server Maintenance: أي عطل في الهاردوير أنت اللي تصلحه.
- 5. تثبيت نظام التشغيل
- 6. تحديث نظام التشغيل

7. تثبيت قاعدة البيانات، زي:

- MySQL
- PostgreSQL
- Oracle

8. تحديث قاعدة البيانات

9. عمل Backup

10. High Availability

11. Scaling

12. تحسين أداء التطبيق

يعني باختصار: أنت مسؤول عن كل حاجة من أول الكهرباء لحد الـ SQL Query.

ثانياً: Database على Amazon EC2

دلوقتي نقلت قاعدة البيانات على EC2، إيه اللي اتغير؟

AWS بقت مسؤولة عن:

- السيرفرات
- الكهرباء
- التبريد
- الشبكات
- الهاردوير

يعني مش هتشيل هم الـ Data Center.

لكن لسه أنت مسؤول عن:

- تثبيت الـ OS
- تحديث الـ OS
- تثبيت الـ Database
- تحديث الـ Database
- Backups
- High Availability
- Scaling
- تحسين التطبيق

يعني AWS شالت منك مسؤولية الهاردوير فقط، لكن إدارة قاعدة البيانات لسه عليك.

ثالثاً: Database على Amazon RDS أو Aurora

هنا بقي الفرق الحقيقي، AWS أصبحت مسؤولة عن أغلب المهام.

AWS تدير:

- Server
- Hardware
- Network
- OS Installation
- OS Patches
- Database Installation
- Database Patches
- Automatic Backups
- High Availability
- Scaling

وأنت مسؤول عن حاجة واحدة تقريباً Application Optimization.

يعني:

- تصميم ال Database
- كتابة SQL
- تحسين ال Queries
- تطوير التطبيق

ليه الشركات بتحب RDS؟

لإن بدل ما فريق كامل يقضي يومه في:

- تثبيت Updates
- عمل Backups
- إصلاح الأعطال
- مراقبة السيرفرات

الفريق يركز على:

- تحسين التطبيق
- تحسين الأداء
- تطوير Features جديدة

❖ Managed services responsibilities

هنشرح دلوقي مين مسؤول عن إيه لما تستخدم Amazon RDS؟
ودي من أكثر الحاجات اللي بيجي عليها أسئلة في الإمتحان.

أولاً: أنت مسؤول عن إيه؟

لما تستخدم Amazon RDS، في حاجة أساسية هتفضل مسؤول عنها وهي Application Optimization.
يعني تحسين التطبيق وقاعدة البيانات نفسها.
وده يشمل:

- تصميم ال Database
- إنشاء الجداول (Tables)
- كتابة SQL Queries
- تحسين أداء ال Queries
- إنشاء Indexes
- ربط التطبيق بقاعدة البيانات

بمعنى إنك مسؤول عن البيانات والتطبيق، مش عن تشغيل قاعدة البيانات.

ثانياً: AWS مسؤولة عن إيه؟

1. Operating System Installation & Patches

AWS هي اللي:

- تثبت نظام التشغيل
- تحدّثه
- تنزل Security Patches
- تصلح أي مشاكل فيه

أنت مش بتتعامل مع ال OS أصلاً.

2. Database Software Installation & Patches

بدل ما تثبت MySQL أو PostgreSQL بنفسك، AWS هي اللي:

- تثبت قاعدة البيانات
- تحدّثها
- تنزل آخر Patches
- تصلح الثغرات الأمنية

3. Automatic Backups

AWS بتعمل نسخ احتياطية تلقائياً.

ولو احتجت ترجع البيانات، تقدر تعمل Restore بسهولة.

يعني مش محتاج تعمل Backup يدوي كل يوم.

4. High Availability

لو فعلت خاصية Multi-AZ، وحصل عطل في السيرفر الأساسي، AWS هتنقل التشغيل تلقائياً للسيرفر الإحتياطي (Standby).
وده بيخلي الخدمة تفضل شغالة بدون تدخل منك.

5. Scaling

لو التطبيق كبر تقدر AWS تساعدك في تكبير الموارد بسهولة، زي:

- زيادة ال CPU
- زيادة ال RAM
- زيادة مساحة التخزين

من غير تعقيد كبير.

6. Power & Rack and Stack Servers

AWS مسؤولة عن:

- الكهرباء
- التبريد
- الشبكات
- تركيب السيرفرات
- تشغيل ال Data Centers

ودي حاجات أنت مش بتشوفها أصلاً.

7. Server Maintenance

أي مُشكلة في الهاردوير، AWS هي اللي تصلحها.

- لو هارد باظ
- لو Server وقف.
- لو محتاج تغيير مكونات

كل ده مسؤولية AWS.

ليه ده مُهم؟

لإن كل الحاجات اللي AWS شالتها منك كانت بتاخذ وقت كبير جداً.

بدل ما تقضي يومك في:

- تحديث السيرفرات
- عمل Backups
- إصلاح الأعطال
- مراقبة ال Database

هتتركز على:

- تطوير التطبيق
- تحسين الأداء
- إضافة Features جديدة

يعني AWS شالت عنك المهام التشغيلية، فبقى عندك وقت تركز على شغلك الأساسي.

ملحوظة مهمة

ناس كتير بتفتكر إن RDS بيدير قاعدة البيانات بالكامل.

وده مش دقيق.

AWS بتدير تشغيل وصيانة قاعدة البيانات (Infrastructure & Operations).

أما:

- إنشاء الجداول
- إدخال البيانات
- حذف البيانات
- كتابة ال SQL
- تحسين ال Queries

كل ده مسؤوليتك أنت.

❖ Amazon RDS DB instances

بعد ما عرفنا يعني إيه Amazon RDS، دلوقتي هنتكلم عن أهم مكون فيه وهو DB Instance.

وده يعتبر قلب خدمة Amazon RDS.

أولاً: يعني إيه DB Instance؟

ال DB Instance هو قاعدة البيانات نفسها اللي بتشتغل عليها داخل Amazon RDS.

تقدر تعتبره كإنه Server مُخصص لتشغيل قاعدة البيانات، لكن AWS هي اللي بتديره.

يعني لما تعمل Create ل RDS، اللي بيتعمل فعلياً هو DB Instance.

ثانياً: Isolated Database Environment

يعني كل DB Instance سيكون مُنفصل عن أي DB Instance ثاني.
كل Instance معزول عن الثاني، لو حصل مشكلة في واحد مش هتأثر على الثاني.

ثالثاً: ممكن يحتوي على أكثر من Database

يعني ال DB Instance الواحد، ممكن يكون جواه أكثر من Database.

رابعاً: الوصول لل DB Instance

AWS بتقول: تقدر تتصل بال DB Instance بنفس الأدوات اللي كنت بتستخدمها مع أي Database عادية.
يعني لو بتستخدم:

- MySQL Workbench
 - pgAdmin
 - SQL Server Management Studio (SSMS)
- هتفضل تستخدمهم عادي، مش محتاج تتعلم برنامج جديد.

خامساً: Database Instance Class

ودي نقطة مُهمة جداً.
كل DB Instance ليه مواصفات، اسمها DB Instance Class، ودي بتحدد قوة السيرفر.
زي:

- عدد ال CPUs
- حجم ال RAM
- الأداء

سادساً: Storage Type

تقدر تختار نوع التخزين، زي:

- SSD
- Provisioned IOPS SSD
- HDD

كل نوع له:

- سرعة مُختلفة
- سعر مُختلف
- أداء مُختلف

سابعاً: الأداء والسعر

AWS بتديك حرية الاختيار.

يعني لو مشروع صغير مش لازم تدفع في أقوى Instance.

ولو عندك تطبيق عليه ملايين المستخدمين تقدر تختار مواصفات أعلى.

يعني كل ما تزود الموارد الأداء يزيد، لكن التكلفة كمان تزيد.

ثامناً: Database Engine

قبل ما تنشئ ال DB Instance، AWS هتسألك: عايز تستخدم أنهي Database Engine؟

يعني نوع قاعدة البيانات.

حالياً Amazon RDS بيدعم:

- MySQL
- PostgreSQL
- MariaDB
- Oracle
- Microsoft SQL Server
- IBM DB2

يعني لو شركتك شغالة بـ MySQL، هتختار MySQL.

ولو شغالة بـ SQL Server، هتختار SQL Server.

وهكذا.

ملحوظة مهمة

ناس كتير بتتلخبط بين:

- Database Engine
- DB Instance

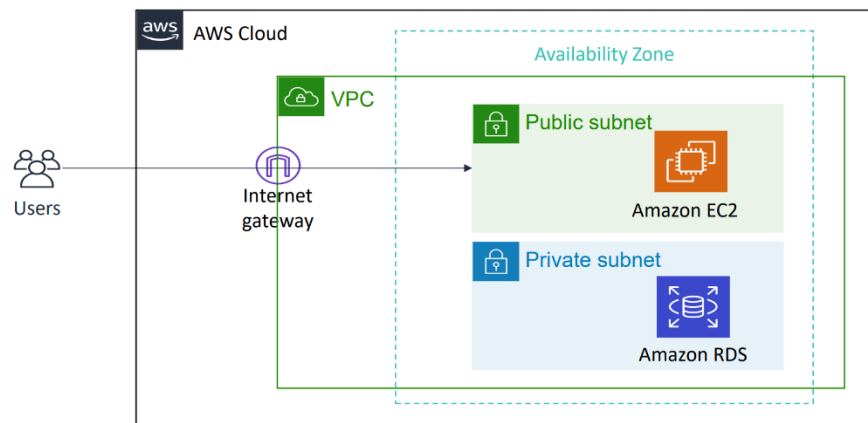
الفرق بينهم:

Database Engine = نوع قاعدة البيانات (MySQL - PostgreSQL - Oracle - ...).

DB Instance = البيئة أو السيرفر اللي بيشتغل قاعدة البيانات.

يعني ال Engine بيشتغل داخل ال DB Instance.

Amazon RDS in a virtual private cloud (VPC) ❖



لحد دلوقتي عرفنا إن Amazon RDS بيشغل قاعدة البيانات.

السؤال بقي: قاعدة البيانات دي بتكون فين؟

الإجابة: بتكون جوه (VPC (Virtual Private Cloud.

يعني ال RDS بيتحط داخل شبكة خاصة أنت اللي بتتحكم فيها.

ليه RDS بيتحط داخل VPC؟

علشان تأمين قاعدة البيانات.

قاعدة البيانات غالبًا فيها:

- بيانات العملاء
- كلمات المرور (بعد تشفيرها)
- الطلبات
- المدفوعات

مينفعش تكون متوصلة بالإنترنت مباشرة، علشان كده AWS بتحطها داخل VPC.

ليه اللي ال VPC بيديهولك؟

أنت المتحكم في الشبكة بالكامل.

تقدر تحدد:

- ال IP Address Range
- ال Subnets
- ال Route Tables
- ال ACLs
- ال Security Groups

يعني أنت اللي بتحدد مين يدخل ومين ميقدرش يدخل.

ملاحظة مهمة جداً

في أغلب الشركات الـ RDS بيتحت داخل Private Subnet مش Public.
 علشان مش أي حد من الإنترنت يقدر يوصل لقاعدة البيانات.
 يعني المستخدم بيدخل على الـ Web Server، والـ Web Server هو اللي يكلم الـ Database.
 يبقى المستخدم عمره ما يوصل للـ Database مباشرةً، وده يُعتبر Best Practice.

هل ينفع أخلي RDS في Public Subnet؟

ينفع لكن نادر جداً وبيكون في حالات خاصة.
 أما في أغلب التطبيقات الحقيقية بيكون داخل Private Subnet.

إيه علاقة الـ Subnet بالـ Availability Zone؟

دي نقطة مهمة جداً.
 كل Subnet موجودة داخل Availability Zone واحدة فقط، يعني مينفعش Subnet واحدة تكون موجودة في أكثر من AZ.

أثناء إنشاء الـ RDS بتختار مثلاً Private Subnet A، يبقى أنت بشكل غير مباشر اخترت Availability Zone A
 لأن كل Subnet مرتبطة بـ AZ واحدة.

ليه ده مهم؟

لأنه بيزود الأمان بشكل كبير.
 بدل ما تفتح قاعدة البيانات للعالم كله، بتخلي الوصول ليها من السيرفرات المسموح لها فقط.
 وده من أهم مبادئ تصميم أي Application على AWS.

❖ High availability with Multi-AZ deployment (1 of 2)

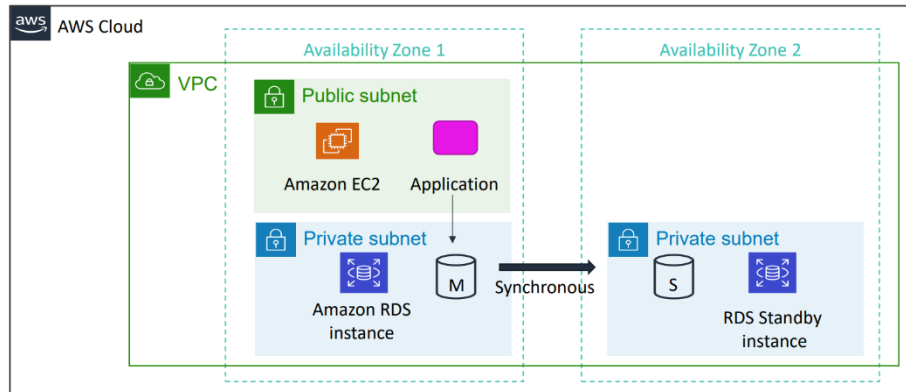
دي من أهم مميزات Amazon RDS، ومن أكثر المواضيع اللي بيجي عليها أسئلة في امتحان AWS، فلازم تفهمها كويس.

يعني إيه High Availability؟

ال (HA) High Availability يعني إن قاعدة البيانات تفضل شغالة حتى لو حصل عطل.

يعني لو السيرفر الأساسي وقع التطبيق ميوقفش.

وده هدف ال Multi-AZ.



يعني إيه Multi-AZ؟

لما تفعل Multi-AZ Deployment، AWS مش بتشغل قاعدة بيانات واحدة.

لا، بتعمل نسخة تانية (Standby) من قاعدة البيانات.

لكن النسخة دي بتكون في Availability Zone مختلفة، داخل نفس ال Region ونفس ال VPC.

بنستخدم Multi-AZ علشان لو حصل مشكلة في ال Primary Database.

زي:

- السيرفر وقع
- الهاردوير باظ
- ال Availability Zone حصل فيها مشكلة
- أثناء أعمال الصيانة

AWS تعمل Automatic Failover.

إيه اللي بيحصل بعد ما AWS تعمل النسخة؟

بعد ما AWS تنشئ ال Standby Database، أي عملية تحصل على ال Primary بتنتقل فوراً لـ Standby.

بنسميها Transactions are synchronously replicated.

يعني إيه Synchronous Replication؟

يعني أي تعديل يحصل يكتب في الإثنين في نفس اللحظة.
يعني النسختين دائماً مُتطابقتين.

Planned Maintenance

مش لازم يحصل Crash.

أوقات AWS نفسها تعمل:

- تحديثات
- صيانة
- Security Patches

بدل ما توقف قاعدة البيانات تنقل التشغيل لل Standby، وبعد ما تخلص تكمل الخدمة عادي.

هل ال Standby بيستقبل Requests؟

لا، وده سؤال مشهور جداً.

ال Standby:

- مش بيستقبل Queries
- مش بيستقبل Read Requests
- مش بيستقبل Write Requests

هو موجود فقط للطوارئ.

وهل أقدر أستخدمه لل Read Scaling؟

لا، لو عايز نسخة للقراءة فقط هنستخدم Read Replica.

وده موضوع مُختلف تماماً هنتكلم عنه بعدين.

❖ High availability with Multi-AZ deployment (2 of 2)

في الشرح اللي فات عرفنا إن AWS بتعمل نسخة Standby Database في Availability Zone ثانية.

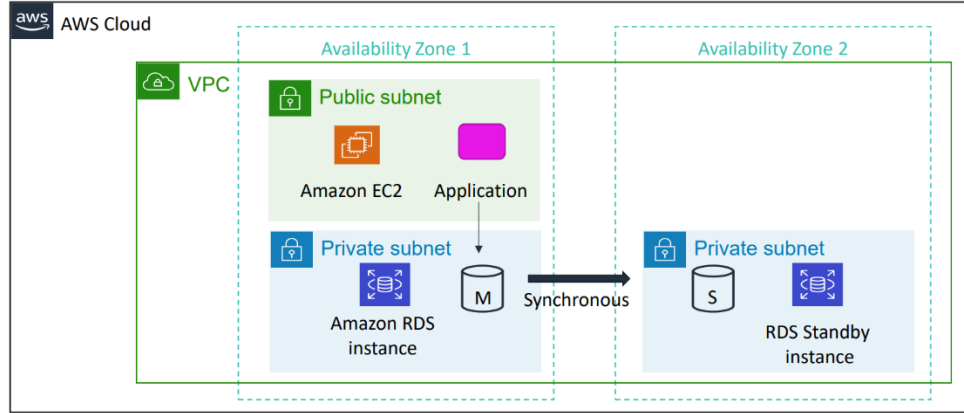
دلوقتي السؤال.. طيب لو ال Primary Database وقعت، إيه اللي هيحصل؟

لو حصل أي مُشكلة في ال Primary Database، زي:

- السيرفر وقع، الهاردوير باظ
- ال Availability Zone حصل فيها عطل
- نظام التشغيل وقف
- قاعدة البيانات نفسها وقفت

AWS مش هتستنى تدخل منك، هتعمل تلقائياً Automatic Failover (يعني ال Standby يتحول مُباشرةً إلى ال Primary الجديد).

وبما إن كل عملية كتابة كانت بتتنسخ في نفس اللحظة على ال Standby فالنسختين كانوا مُتطابقين تقريباً. لذلك لما يحصل Failover احتمال فقدان البيانات بيكون قليل جداً. وده من أكبر مُميزات ال Multi-AZ.



هل التطبيق هيعرف إن قاعدة البيانات اتغيرت لو حصل Automatic Failover؟

الإجابة: لا.

ودي من أهم مُميزات Multi-AZ Deployment.

التطبيق مش بيتصل بال Primary Database مُباشرةً، لكنه بيتصل دائماً بعنوان ثابت اسمه Amazon RDS DNS Endpoint.

مثلاً: mydatabase.abc123.us-east-1.rds.amazonaws.com

التطبيق بيستخدم ال Endpoint ده في كل مرة يتصل فيها بقاعدة البيانات.

ولو حصل Automatic Failover وانتقلت قاعدة البيانات من ال Primary إلى ال Standby، فإن AWS هي اللي بتحدث ال DNS داخلياً بحيث يُشير إلى ال Primary الجديد.

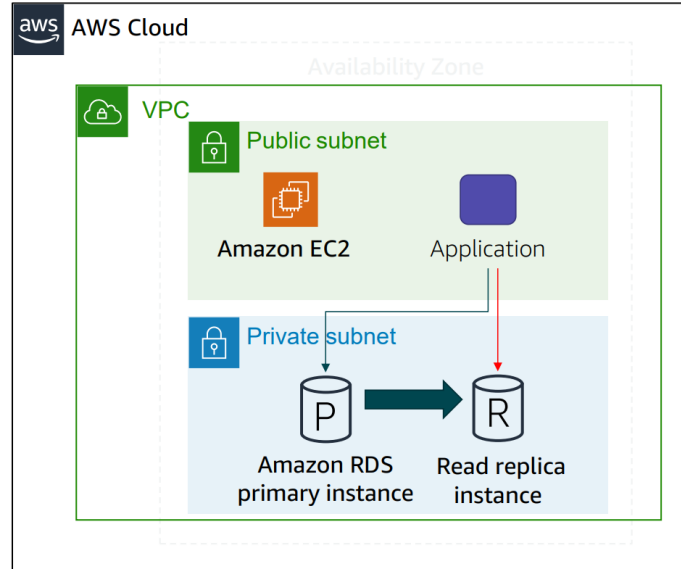
يعني:

- ال Endpoint يفضل نفسه.
- التطبيق يفضل يستخدم نفس العنوان.
- مش هتحتاج تعدل أي سطر كود أو تغير إعدادات الإتصال بقاعدة البيانات.

وده هو السبب إن عملية ال Failover بتحصل بشكل شبه شفاف (Transparent) بالنسبة للتطبيق.

Amazon RDS read replicas ❖

لحد دلوقتي اتكلمنا عن Multi-AZ وقلنا إن هدفه الأساسي هو High Availability لكن في ميزة ثانية في RDS اسمها Read Replica. ودي ناس كثير بتتلخبط بينها وبين Multi-AZ.



يعني إيه Read Replica؟

ال Read Replica هي نسخة من قاعدة البيانات الأساسية (Primary Database)، لكن الهدف منها مش High Availability. هدفها الأساسي هو زيادة سرعة عمليات القراءة (Read Scaling).

ليه محتاج Read Replica؟

تخيل عندك موقع E-Commerce فيه:

- 100 عملية شراء في الدقيقة (Write).
 - 50,000 مُستخدم بيتصفحوا المُنتجات (Read).
- يبقى أغلب الضغط على قاعدة البيانات جاي من عمليات القراءة (SELECT). لو كل ال Reads راحت لل Primary هيبقى عليها حمل كبير جداً.

الحل AWS تعمل Read Replica.

وبيبقى

- عمليات Write تفضل تروح لل Primary
- عمليات Read تروح لل Read Replica

وبالتالي الضغط يقل على ال Primary.

البيانات بتتنقل إزاي؟

AWS بتقول "Updates are asynchronously copied".

يعني النسخة مش بتتحدث في نفس اللحظة، اسمها Asynchronous Replication.

يعني لو المُستخدم عمل Update.

البيانات بتتكتب أولاً في ال Primary DB وبعدها بثوانٍ أو أجزاء من الثانية، بتتنسخ إلى Read Replica.

فببقي فيه Delay بسيط، وده الطبيعي.

طب ليه استخدموا Asynchronous؟

علشان الأداء، لو AWS كانت مستنية كل Replica تتحدث قبل ما ترد على المُستخدم، الأداء هيبقى أبطأ.

علشان كده اختاروا Asynchronous Replication، وده مُناسب جداً لعمليات القراءة.

هل Read Replica ينفع يبقى Primary؟

ينفع لكن مش تلقائي.

لو عايز تحول ال Read Replica إلى Primary لازم أنت تعمل Manual Promotion، وAWS مش هتعملها لوحدها.

وهي مش Automatic لأن ال Replication هنا Asynchronous، وده معناه إن البيانات مش بتتنسخ في نفس اللحظة، لكن بعد فترة بسيطة جداً (ثوانٍ أو أجزاء من الثانية).

وفي الحالة دي، ال Read Replica لسه ما استقبلتش آخر تحديث.

فلو AWS حولتها تلقائياً إلى Primary، آخر البيانات ممكن تضيع.

علشان كده AWS ما بتعملش Automatic Failover مع Read Replicas، ولازم يكون التحويل Manual بعد ما تتأكد إن النسخة مُناسبة للإستخدام.

Cross-Region Read Replica

ودي ميزة جميلة جداً.

ال Read Replica مش لازم تكون في نفس ال Region، ممكن تعملها في Region تانية.

وده بيفيد في حاجتين:

1. تقليل ال Latency

لو المُستخدمين في أوروبا بدل ما كل Read تروح أمريكا، تروح لل Replica الموجودة في أوروبا، فتكون أسرع.

2. Disaster Recovery

لو حصلت مُشكلة كبيرة في Region كاملة، يبقى عندك نسخة في Region تانية.

وده بيساعد في خطة التعافي من الكوارث (Disaster Recovery).

Use Cases ❖

Amazon RDS مُناسب في استخدامات كثير، خاصةً لما تكون محتاج قاعدة بيانات Relational من غير ما تشيل هم إدارتها وصيانتها.

أولاً: Web & Mobile Applications

دي تطبيقات زي:

- مواقع الإنترنت
- تطبيقات الموبايل
- أنظمة SaaS

ليه Amazon RDS مُناسب ليها؟

High Throughput: يعني يقدر يتعامل مع عدد كبير من عمليات القراءة والكتابة في نفس الوقت.

Massive Storage Scalability: لو حجم البيانات زاد مع الوقت تقدر تكبر مساحة قاعدة البيانات بسهولة من غير ما تغير النظام كله.

High Availability: تقدر تستخدم Multi-AZ Deployment بحيث لو حصل عطل في قاعدة البيانات، AWS تعمل Automatic Failover ويكمل التطبيق شغله.

ثانياً: Ecommerce Applications

زي:

- Amazon
- Noon
- Jumia
- Shopify

أو أي موقع بيع أونلاين

ليه Amazon RDS مُناسب ليها؟

Low-Cost Database: بدل ما تشتري سيرفرات وتديرها بنفسك، AWS بتوفرلك قاعدة بيانات جاهزة، وبتدفع على قد استخدامك.

Data Security: AWS بتوفر مميزات أمان كثير زي:

- Encryption
- IAM
- Security Groups
- Automatic Backups

وده مُهم جداً لأن مواقع التجارة الإلكترونية فيها بيانات حساسة زي بيانات العملاء والطلبات.

Fully Managed Solution : AWS هي اللي بتدير:

- التحديثات (Patches)
- النسخ الاحتياطية (Backups)
- الصيانة
- ال High Availability

وأنت تركز على تطوير الموقع نفسه.

ثالثاً: Mobile & Online Games

ألعاب الموبايل أو الألعاب الأونلاين يكون عندها عدد لاعبين كبير جداً.

ليه Amazon RDS مناسب ليها؟

Rapidly Grow Capacity: لو اللعبة انتشرت فجأة وعدد اللاعبين زاد، تقدر تكبر موارد قاعدة البيانات بسهولة.

AWS :Automatic Scaling: تساعدك في زيادة الموارد عند الحاجة، وبالتالي تقدر تستوعب الزيادة في عدد المستخدمين.

AWS :Database Monitoring: توفر أدوات Monitoring لمُتابعة:

- استخدام ال CPU
- الذاكرة
- عدد الإتصالات
- أداء قاعدة البيانات

وده بيساعدك تكتشف أي مُشكلة قبل ما تأثر على اللاعبين.

ملحوظة مُهمّة

كل الأمثلة دي فيها شيء مشترك، إن المطور مش عايز يضيع وقته في إدارة قاعدة البيانات.

هو عايز يركز على تطوير التطبيق أو الموقع أو اللعبة.

وعلشان كده بيستخدم Amazon RDS كخدمة Fully Managed، بحيث AWS تتولى معظم مهام إدارة قاعدة البيانات.

❖ When to Use Amazon RDS

مش كل التطبيقات ينفع تستخدم فيها Amazon RDS. في حالات بيكون هو أفضل اختيار، وحالات تانية الأفضل تستخدم خدمة مُختلفة زي DynamoDB أو تشغل قاعدة البيانات بنفسك على EC2.

امتى أستخدم Amazon RDS؟

استخدم Amazon RDS لو تطبيقك محتاج الآتي:

1. Complex Queries أو Complex Transactions

لو تطبيقك بيعتمد على:

- مُعاملات مُعقدة (Transactions)
- استعلامات SQL مُعقدة (Complex Queries)
- JOIN بين أكثر من جدول
- Foreign Keys
- Stored Procedures

يبقى RDS هو الاختيار المُناسب لأنه قاعدة بيانات Relational.

2. Medium to High Query / Write Rate

يعني عندك عدد كبير من عمليات:

- القراءة (Read)
- الكتابة (Write)

لكن في حدود قدرة قاعدة بيانات واحدة، AWS بتقول تقريباً لحد 30,000 IOPS.

يعني تقريباً:

- Read IOPS 15,000
- Write IOPS 15,000

وده مُناسب لمعظم التطبيقات التقليدية.

معلومة:

IOPS = Input/Output Operations Per Second

يعني عدد عمليات القراءة والكتابة اللي الديسك يقدر ينفذها في الثانية.

3. No More Than a Single Worker Node or Shard

يعني قاعدة البيانات كلها تقدر تشتغل على Server واحد، مش محتاج تقسم البيانات على أكثر من Database Server.

لو Database واحدة تكفي، يبقى RDS مُناسب جداً.

إمتى ما استخدمش Amazon RDS؟

في بعض الحالات RDS مش هيكون مُناسب.

1. Massive Read/Write Rates

لو التطبيق بيستقبل عدد ضخّم جداً من العمليات، مثلاً 150,000 عملية كتابة في الثانية. ده أكبر بكثير من اللي قاعدة بيانات RDS واحدة تقدر تستحمله. يبقى في الحالة دي الأفضل تستخدم حلول ثانية زي:

- DynamoDB
- أو قواعد بيانات موزعة (Distributed Databases)

2. تحتاج Sharding

ال Sharding يعني تقسيم البيانات على أكثر من Database Server. وده بيستخدم لما حجم البيانات يبقى ضخّم جداً، أو عدد المُستخدمين يبقى كبير جداً. وAmazon RDS مش معمول لإدارة ال Sharding.

3. Simple GET / PUT Requests

لو التطبيق بيعمل عمليات بسيطة جداً. زي:

- اقرأ قيمة
- احفظ قيمة

من غير:

- JOIN
- Transactions
- علاقات بين الجداول

يبقى قاعدة بيانات NoSQL زي Amazon DynamoDB هتبقى أسرع وأفضل.

4. تحتاج تعديلات خاصة على ال Database Engine

لو محتاج تتحكم بنفسك في:

- إعدادات نظام التشغيل
- إعدادات قاعدة البيانات الداخلية
- تثبيت Plugins خاصة
- تعديلات مش بيدعمها Amazon RDS

يبقى الأفضل تشغل قاعدة البيانات بنفسك على Amazon EC2 علشان يبقى عندك تحكم كامل.

أستخدم إيه بدل Amazon RDS؟

حسب احتياجك.

لو محتاج:

- بيانات NoSQL
- سرعة عالية جدًا
- ملايين الطلبات في الثانية

استخدم **Amazon DynamoDB**.

لو محتاج:

- تحكم كامل في قاعدة البيانات
- تعديلات خاصة
- تثبيت أي Software أو Plugins

شغل قاعدة البيانات على **Amazon EC2**.

ملحوظة مهمة

اختيار الخدمة يعتمد على احتياجات التطبيق:

لو محتاج قاعدة بيانات Relational مُدارة، استخدم Amazon RDS.

لو محتاج NoSQL وسرعة هائلة، استخدم Amazon DynamoDB.

لو محتاج تحكم كامل وتخصيص عالي، شغل قاعدة البيانات على Amazon EC2.

❖ Amazon RDS: Clock-hour billing and database characteristics

علشان تحسب تكلفة Amazon RDS، لازم تركز على حاجتين أساسيين:

- Clock-hour Billing
- Database Characteristics

أولاً: Clock-hour Billing

يعني ببساطة أنت بتدفع مُقابل الوقت اللي قاعدة البيانات شغالة فيه.

بمُجرد ما تعمل Launch DB Instance هيبداً الحساب.

ولما تعمل Terminate DB Instance الحساب هيقف.

يعني أنت مش بتدفع على عدد الإستعلامات أو عدد المُستخدمين، لكن بتدفع على مُدة تشغيل الـ DB Instance.

ثانياً: Database Characteristics

السعر مش بيعتمد على وقت التشغيل بس، لكن كمان بيعتمد على مواصفات قاعدة البيانات اللي اخترتها.

1. Database Engine

نوع قاعدة البيانات بيأثر على التكلفة.

مثلاً:

- MySQL
- PostgreSQL
- MariaDB
- Oracle
- SQL Server
- IBM Db2

بعض ال Engines (زي Oracle أو SQL Server) ممكن تكون أغلى بسبب تراخيصها (Licensing).

2. Database Size

كل ما مساحة التخزين (Storage) تكبر، التكلفة هتزيد.

3. Memory Class (Instance Class)

دي مواصفات السيرفر اللي هيشغل قاعدة البيانات.

يعني:

- حجم ال RAM
- عدد ال vCPUs
- قوة المُعالج

كل ما تختار Instance أكبر، السعر هيزيد.

ملحوظة مُهمّة

تكلفة Amazon RDS بتعتمد على أكثر من عامل، أهمهم:

- مُدة تشغيل ال Database (Clock Hours)
- حجم التخزين (Storage Size)
- نوع مُحرك قاعدة البيانات (Database Engine)
- مواصفات ال Instance (Memory/CPU)

كل ما تزود الموارد أو تشغل قاعدة البيانات لفترة أطول، التكلفة هتزيد.